一、配置文件

1. requirements.txt

```
# 核心依赖
numpy>=1.24.0
scipy>=1.10.0
matplotlib>=3.7.0
pandas>=2.0.0
# 可视化
seaborn>=0.12.0
plotly>=5.15.0
# Jupyter
jupyter>=1.0.0
ipython>=8.12.0
# 配置管理
PyYAML>=6.0
# Excel 导出（pandas 写 .xlsx 的默认引擎）
openpyxl>=3.1.0
# 系统监控与环境检测
psutil>=5.9.0
# 测试
pytest>=7.4.0
pytest-cov>=4.1.0
# 图形验证码
Pillow>=10.0.0
# Web API
Flask>=2.3.0
Flask-CORS>=4.0.0
requests>=2.31.0
urllib3>=1.26.0
# 跨语言接口（可选）
# grpcio>=1.56.0
# grpcio-tools>=1.56.0
# 可选依赖
deap>=1.3.0
scikit-learn>=1.2.0
# 国产硬件适配（可选）
# torch>=2.0.0       # CUDA/NPU推理
# torch_npu>=2.0.0   # 华为昇腾NPU
# GitHub Skills 集成（可选）— RL/Meta-heuristic 调度算法增强
# 安装方法：pip install gymnasium stable-baselines3 pymoo
# 不安装时，对应插件会自动降级为 PluginResult.fail(DEPENDENCY_MISSING)
# gymnasium>=1.0          # RL 环境标准 API (Farama-Foundation, MIT)
# stable-baselines3>=2.0  # RL 算法实现 (DLR-RM, MIT)
# pymoo>=0.6              # 多目标优化 NSGA2/MOEAD/PSO (Apache 2.0)
```

2. config/default.yaml

```yaml
simulation:
  num_servers: 48
  min_servers: 1
  max_servers: 1000
  simulation_hours: 24
  min_simulation_hours: 1
  max_simulation_hours: 720
  random_seed: 42
  parallel_workers: 4
  enable_distributed: false
  distributed_workers: 8
  run_mode: "standard"
  sla_slack: 1.5
data_source:
  mode: "catalog"
  catalog_db: "data/catalog.db"
  site_id: "北京市"
  ambient_start_hour: 4344
  trace_id: "alibaba-gpu-v2025-d019-168h"
environment:
  ambient_temp_base: 15
  ambient_temp_amplitude: 10
  humidity: 0.5
cooling:
  power_coefficient: 500
  cooling_efficiency: 0.9
  capacity_rating_w: 20000
  target_temp: 22
  fan_target_temp: 35
  dissipation_coefficient: 8.0
  thermal_inertia: 0.25
  room_volume: 100
  air_flow_rate: 10
  fan_speed_range: [0.2, 1.0]
  fan_efficiency_curve: "quadratic"
greedy:
  temperature_weight: 0.3
  overload_threshold: 0.95
  overheat_threshold: 0.95
  load_balancing_factor: 0.7
  temperature_balancing_factor: 0.3
  enable_priority_scheduling: true
```

```
  priority_weight: 0.5
damo_ets:
  alpha_deadline: 0.35
  beta_energy: 0.25
  gamma_thermal: 0.20
  delta_heterogeneity: 0.15
  epsilon_migration: 0.05
  heat_coefficient: 15.0
  thermal_safety_factor: 0.95
  workload_server_match:
    gpu_bound: ["gpu_server", "high_performance"]
    cpu_bound: ["high_performance", "standard"]
    memory_bound: ["standard"]
    io_bound: ["standard"]
genetic:
  population_size: 50
  max_population_size: 200
  generations: 100
  max_generations: 500
  crossover_rate: 0.8
  mutation_rate: 0.1
  elitism_rate: 0.1
  overload_penalty: 100
  overheat_penalty: 200
  energy_weight: 0.5
  temperature_weight: 0.3
  load_weight: 0.2
  enable_parallel: false
  parallel_evaluators: 1
task_generation:
  base_rate: 6
  peak_rate: 20
  peak_hours: [9, 18]
  load_range: [5, 30]
  priority_range: [1, 5]
  task_types:
    - name: "cpu_bound"
      proportion: 0.4
      load_range: [10, 50]
      duration_range: [1, 5]
    - name: "gpu_bound"
      proportion: 0.3
      load_range: [20, 80]
      duration_range: [2, 10]
```

```
    - name: "memory_bound"
      proportion: 0.2
      load_range: [5, 25]
      duration_range: [3, 8]
    - name: "io_bound"
      proportion: 0.1
      load_range: [3, 15]
      duration_range: [1, 3]
task_completion:
  mode: "service_based"
  completion_probability: 0.3
  completion_rate_adjustment: 0.1
energy:
  cop_base: 2.5
  cop_delta_coefficient: 0.1
  heat_to_power_ratio: 0.8
  pue_base: 1.8
  idle_power_factor: 0.6
  part_load_penalty: 0.5
  temp_penalty_coefficient: 1.0
  temp_reference: 35
server_types:
  - name: "standard"
    proportion: 0.6
    max_load: 100
    max_temp: 85
    base_power: 100
    max_power: 300
    efficiency: 0.85
  - name: "high_performance"
    proportion: 0.25
    max_load: 200
    max_temp: 80
    base_power: 200
    max_power: 500
    efficiency: 0.90
  - name: "gpu_server"
    proportion: 0.15
    max_load: 300
    max_temp: 75
    base_power: 300
    max_power: 800
    efficiency: 0.88
output:
```

```
  data_dir: "data"
  reports_dir: "reports"
  enable_visualization: true
  enable_statistics: true
  enable_detailed_logging: false
```

3. config/development.yaml

```
simulation:
  num_servers: 48
  simulation_hours: 24
  parallel_workers: 2
  enable_distributed: false
  run_mode: "standard"
environment:
  ambient_temp_base: 25
cooling:
  target_temp: 22
genetic:
  population_size: 30
  generations: 50
  enable_parallel: false
  parallel_evaluators: 1
output:
  enable_visualization: true
  enable_statistics: true
  enable_detailed_logging: false
logging:
  level: "INFO"
  format: "%(asctime)s - %(name)s - %(levelname)s - %(message)s"
damo_ets:
  alpha_deadline: 0.5
  beta_energy: 0.2
  gamma_thermal: 0.15
  delta_heterogeneity: 0.1
  epsilon_migration: 0.05
```

4. config/production.yaml

```
simulation:
  num_servers: 100
  simulation_hours: 168
  parallel_workers: 8
  enable_distributed: false
  distributed_workers: 8
  run_mode: "standard"
```

```
environment:
  ambient_temp_base: 22
cooling:
  target_temp: 20
  fan_speed_range: [0.3, 1.0]
genetic:
  population_size: 100
  generations: 200
  enable_parallel: true
  parallel_evaluators: 8
task_generation:
  base_rate: 12
  peak_rate: 40
output:
  enable_visualization: true
  enable_statistics: true
  enable_detailed_logging: false
logging:
  level: "INFO"
  format: "%(asctime)s - %(name)s - %(levelname)s - %(message)s"
```

5. config/test_environment.yaml

```
# 仿真模拟测试环境配置文件
# 用于系统性验证系统在不同场景下的适配性、稳定性及安全性
environment:
  name: "仿真模拟测试环境"
  version: "1.0.0"
  reset_on_startup: true
  snapshot_interval: 60
# 一、网络拓扑模拟
network:
  topology: "spine_leaf"
  regions:
    - name: "廊坊核心区"
      latency_ms: 0.5
      bandwidth_mbps: 10000
      packet_loss: 0.0001
      jitter_ms: 0.1
    - name: "北京边缘区"
      latency_ms: 5.0
      bandwidth_mbps: 1000
      packet_loss: 0.001
      jitter_ms: 1.0
    - name: "天津备份区"
```

```
      latency_ms: 10.0
      bandwidth_mbps: 500
      packet_loss: 0.005
      jitter_ms: 2.0
    - name: "雄安新区"
      latency_ms: 20.0
      bandwidth_mbps: 200
      packet_loss: 0.01
      jitter_ms: 5.0
# 二、硬件配置模拟
hardware_profiles:
  - name: "鲲鹏920开发板"
    type: "arm64"
    cpu_cores: 64
    memory_gb: 256
    disk_gb: 1000
    nic_speed_mbps: 10000
    gpu: null
    npu: "Ascend310"
    os: "openEuler 22.03"
    proportion: 0.4
  - name: "标准x86服务器"
    type: "x86_64"
    cpu_cores: 32
    memory_gb: 128
    disk_gb: 2000
    nic_speed_mbps: 10000
    gpu: "NVIDIA A100"
    npu: null
    os: "CentOS 7.9"
    proportion: 0.35
  - name: "高性能GPU节点"
    type: "x86_64"
    cpu_cores: 48
    memory_gb: 512
    disk_gb: 4000
    nic_speed_mbps: 25000
    gpu: "NVIDIA H100 x4"
    npu: null
    os: "Ubuntu 22.04"
    proportion: 0.2
  - name: "边缘计算节点"
    type: "arm64"
    cpu_cores: 8
```

```
    memory_gb: 32
    disk_gb: 256
    nic_speed_mbps: 1000
    gpu: null
    npu: null
    os: "openEuler 22.03"
    proportion: 0.05
# 三、软件依赖矩阵
software_matrix:
  python_versions: ["3.9", "3.10", "3.11", "3.12"]
  os_list: ["openEuler 22.03", "CentOS 7.9", "Ubuntu 22.04", "Windows Se
    rver 2022"]
  dependencies:
    numpy: ["1.24.0", "1.25.0", "1.26.0"]
    pandas: ["2.0.0", "2.1.0", "2.2.0"]
    flask: ["2.3.0", "3.0.0"]
    torch: ["2.0.0", "2.1.0", "2.2.0"]
  conflicting_packages:
    - ["torch>=2.0.0", "torch_npu<2.0.0"]
    - ["grpcio>=1.57.0", "protobuf<3.20.0"]
# 四、数据流量特征模拟
traffic_patterns:
  - name: "正常负载"
    requests_per_sec: 100
    data_size_kb: 5
    concurrency: 10
    duration_minutes: 30
  - name: "潮汐峰值"
    requests_per_sec: 500
    data_size_kb: 20
    concurrency: 50
    duration_minutes: 15
  - name: "突发流量"
    requests_per_sec: 2000
    data_size_kb: 2
    concurrency: 200
    duration_minutes: 5
  - name: "持续高压"
    requests_per_sec: 800
    data_size_kb: 10
    concurrency: 100
    duration_minutes: 60
  - name: "DDoS模拟"
    requests_per_sec: 50000
```

```
    data_size_kb: 1
    concurrency: 5000
    duration_minutes: 2
# 五、兼容性测试矩阵
compatibility:
  cross_version_tests:
    - name: "Python版本兼容"
      dimensions: ["3.9", "3.10", "3.11", "3.12"]
      test_count: 20
    - name: "OS兼容性"
      dimensions: ["openEuler", "CentOS", "Ubuntu", "Windows"]
      test_count: 16
    - name: "依赖版本兼容"
      dimensions: ["最低版本", "推荐版本", "最新版本"]
      test_count: 30
  cross_platform_tests:
    - name: "x86_64 vs ARM64"
      dimensions: ["x86_64", "arm64"]
      test_count: 10
# 六、压力测试阈值
stress_thresholds:
  cpu:
    warning: 70
    critical: 85
    max: 95
  memory:
    warning: 75
    critical: 90
    max: 95
  disk_io:
    warning_mbps: 500
    critical_mbps: 800
  network:
    warning_mbps: 8000
    critical_mbps: 9500
  response_time:
    warning_ms: 200
    critical_ms: 500
    timeout_ms: 5000
  error_rate:
    warning: 0.01
    critical: 0.05
    max: 0.10
# 七、安全扫描配置
```

```
security:
  scan_frequency: "per_test_run"
  vulnerability_db: "nvd"
  scan_types:
    - "dependency_check"
    - "port_scan"
    - "code_sast"
    - "api_fuzzing"
    - "injection_test"
    - "auth_bypass"
  owasp_top10: true
  cvss_threshold: 7.0
  allowed_ports: [5000, 8000, 8080, 9090]
  blocked_ports: [22, 3306, 5432, 6379, 27017]
  injection_payloads:
    sql: ["' OR '1'='1", "1; DROP TABLE users", "' UNION SELECT * FROM u
    sers"]
    xss: ["<script>alert(1)</script>", "javascript:alert(1)", "<img oner
    ror=alert(1) src=x>"]
    command: ["; ls -la", "| cat /etc/passwd", "$(whoami)"]
# 八、环境重置配置
reset:
  strategy: "snapshot"
  snapshot_dir: "test_snapshots"
  max_snapshots: 10
  auto_cleanup: true
  restore_timeout_sec: 30
# 九、测试报告配置
reporting:
  output_dir: "test_reports"
  formats: ["json", "html", "pdf", "junit_xml"]
  retention_days: 30
  include_trends: true
  slack_webhook: null
```

6. config/testing.yaml

```
simulation:
  num_servers: 50
  simulation_hours: 48
  parallel_workers: 4
  enable_distributed: false
  run_mode: "standard"
genetic:
  population_size: 50
```

```
  generations: 100
  enable_parallel: false
  parallel_evaluators: 1
output:
  enable_visualization: true
  enable_statistics: true
  enable_detailed_logging: true
logging:
  level: "INFO"
  format: "%(asctime)s - %(name)s - %(levelname)s - %(message)s"
```

二、源代码

1. main.py - 项目入口

```
from src.simulation import DataCenterSimulator
from src.visualization import (
    plot_energy_comparison,
    plot_temperature_trend,
    plot_load_distribution,
    plot_pue_comparison,
    plot_cooling_power,
    plot_summary_stats
)
from src.utils import (
    save_simulation_data,
    export_results_to_excel,
    save_run_metadata,
    setup_logger,
)
import argparse
import os
from datetime import datetime

from src.services import run_store


def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(description="校园数据中心智能散热与算力调度系统 - 仿真主入口")
    parser.add_argument("--num-servers", type=int, default=None,
                        help="服务器数量（默认取 config/default.yaml 的 simulation.num_servers ")
    parser.add_argument("--hours", type=int, default=None,
                        help="仿真小时数（默认取 config/default.yaml 的 simulation.simulation_hours ")
```

```
    parser.add_argument(
        "--algorithms",
        nargs="+",
        default=["greedy", "genetic", "round_robin", "damo_ets"],
        help="要参与对比的算法列表",
    )
    parser.add_argument("--seed", type=int, default=None, help="随机种子（保证可复现）")
    return parser.parse_args()


def _fmt(value, digits: int = 2, suffix: str = "") -> str:
    if value is None:
        return "--"
    return f"{value:.{digits}f}{suffix}"


def main() -> None:
    args = parse_args()

    logger = setup_logger("main")
    logger.info("=" * 60)
    logger.info("校园/小型数据中心智能散热与算力调度系统")
    logger.info("=" * 60)
    if args.seed is not None:
        logger.info(f"随机种子: {args.seed}")

    try:
        simulator = DataCenterSimulator(
            num_servers=args.num_servers,
            simulation_hours=args.hours,
            seed=args.seed,
        )
        meta = simulator.run_metadata
        logger.info(f"数据来源: {meta.get('ambient_source')} / 任务流: {meta.get('scenario_source')}")
        logger.info(f"SLA 松弛系数: {meta.get('sla_slack')} | 任务完成模型: {meta.get('completion_mode')}")

        logger.info(f"\n[1/5] 运行算法对比仿真 (algorithms={args.algorithms}) ...")
        df = simulator.run_comparison(algorithms=args.algorithms)

        offered = simulator.control_signature(df)
```

```
        controlled = simulator.is_controlled(df)
        logger.info(f"[2/5] 受控性检查: 各算法首小时到达负载 {list(offered.values())} -> "
                    f"{'同一任务流快照' if controlled else '警告: 任务流不一致'}")

        logger.info("[3/5] 保存仿真数据...")
        csv_path = save_simulation_data(df)
        excel_path = export_results_to_excel(df)
        metadata_path = save_run_metadata({
            **simulator.run_metadata,
            "num_servers": simulator.num_servers,
            "simulation_hours": simulator.simulation_hours,
            "seed": simulator.seed,
            "algorithms": args.algorithms,
            "first_hour_offered": offered,
            "first_hour_served": simulator.first_hour_loads(df),
            "controlled": controlled,
        })
        logger.info(f"  - CSV: {csv_path}")
        logger.info(f"  - Excel: {excel_path}")
        logger.info(f"  - 运行元数据: {metadata_path}")

        logger.info("\n[4/5] 生成可视化图表...")
        reports_dir = 'reports'
        os.makedirs(reports_dir, exist_ok=True)

        plot_energy_comparison(df, os.path.join(reports_dir, 'energy_comparison.png'))
        plot_temperature_trend(df, os.path.join(reports_dir, 'temperatur
e_trend.png'))
        plot_load_distribution(df, os.path.join(reports_dir, 'load_distr
ibution.png'))
        plot_pue_comparison(df, os.path.join(reports_dir, 'pue_compariso
n.png'))
        plot_cooling_power(df, os.path.join(reports_dir, 'cooling_power.
png'))

        logger.info("  - energy_comparison.png")
```

```
        logger.info("  - temperature_trend.png")
        logger.info("  - load_distribution.png")
        logger.info("  - pue_comparison.png")
        logger.info("  - cooling_power.png")

        logger.info("\n[5/5] 统计指标与排名...")
        stats = simulator.get_statistics(df)
        ranking = simulator.get_ranking(df)

        try:
            run_id = run_store.create_run({
                "num_servers": simulator.num_servers,
                "simulation_hours": simulator.simulation_hours,
                "algorithms": args.algorithms,
                "site_id": simulator.run_metadata.get("site_id"),
            }, origin="cli")
            run_metadata = {
                **simulator.run_metadata,
                "run_id": run_id,
                "origin": "cli",
                "num_servers": simulator.num_servers,
                "simulation_hours": simulator.simulation_hours,
                "seed": simulator.seed,
                "algorithms": args.algorithms,
                "first_hour_offered": offered,
                "first_hour_served":
simulator.first_hour_loads(df),
                "controlled": controlled,
            }
            save_run_metadata(run_metadata)
            run_store.save_results(run_id, df, run_metadata,
stats)
            finished_at = datetime.now().strftime("%Y-%m-%d
%H:%M:%S")
            run_store.update_run(
                run_id, status="success", progress=100,
                started_at=finished_at, finished_at=finished_at,
            )
            logger.info(f"  - 运行编号: {run_id}")
        except Exception as run_exc:
            logger.warning(f"运行历史写入失败: {run_exc}")

        logger.info("\n" + "=" * 60)
        logger.info("仿真结果统计")
```

```
142|        logger.info("=" * 60)
143|        for algo, metrics in stats.items():
144|            logger.info(f"\n【{algo.upper()}算法】")
145|            logger.info(f"  平均能耗: {_fmt(metrics['avg_energy'], 2, '
   |    W')}")
146|            logger.info(f"  峰值温度: {_fmt(metrics['max_temp'], 2, '°C
   |    ')}"
147|                         f"  (95 分位 {_fmt(metrics['p95_temp'], 2, '°C'
   |    )})")
148|            logger.info(f"  平均PUE: {_fmt(metrics['avg_pue'], 4)}")
149|            logger.info(f"  总处理负载: {_fmt(metrics['total_load'], 0)}
   |    ")
150|            logger.info(f"  任务服务率: {_fmt((metrics['service_rate'] o
   |    r 0) * 100, 2, '%')}"
151|                         f"  (分配 {metrics['assigned_tasks']} / 丢弃 {me
   |    trics['dropped_tasks']})")
152|            logger.info(f"  单位服务负载能耗: {_fmt(metrics['energy_per_
   |    served_load'], 4, ' W/负载')}")
153|            logger.info(f"  累计 deadline 违约: {metrics['deadline_viola
   |    tions']}"
154|                           f"  ({_fmt((metrics['deadline_violation_ra
te'] o
   |    r 0) * 100, 2, '%')})")
155|
156|        logger.info("\n" + "=" * 60)
157|        logger.info("受控排名")
158|        logger.info("=" * 60)
159|        logger.info(f"规则: {ranking['rule']}")
160|        for item in ranking['eligible']:
161|            logger.info(f"  #{item['rank']} {item['algorithm']}: "
162|                           f"{_fmt(item['energy_per_served_load'],
4, ' W/
   |    负载')} "
163|                         f"(服务率 {_fmt(item['service_rate'] * 100, 2, '
   |    %')})")
164|        for item in ranking['excluded']:
165|            logger.info(f"  排除 {item['algorithm']}: {item['reason']}")
166|        logger.info(f"结论: {ranking['note']}")
167|
168|        logger.info("\n" + "=" * 60)
169|        logger.info("仿真完成！结果已保存到 reports/ 和 data/ 目录")
170|        logger.info("=" * 60)
171|
172|    except Exception as e:
```

```
173|        logger.error(f"仿真过程发生错误: {str(e)}", exc_info=True)
174|        raise
175|
176|
177|if __name__ == "__main__":
178|    main()
```

2. src/__init__.py

```
  1|from .algorithms import (
  2|    BaseScheduler,
  3|    GreedyScheduler,
  4|    GeneticScheduler,
  5|    GeneticAlgorithm,
  6|    RoundRobinScheduler
  7|)
  8|from .models import (
  9|    Server,
 10|    Task,
 11|    EnergyModel,
 12|    TemperatureModel
 13|)
 14|from .simulation import (
 15|    DataCenterSimulator,
 16|    ServerSnapshot,
 17|    TaskGenerator
 18|)
 19|from .utils import (
 20|    save_simulation_data,
 21|    load_simulation_data,
 22|    generate_synthetic_data,
 23|    export_results_to_excel,
 24|    Config,
 25|    setup_logger
 26|)
 27|from .visualization import (
 28|    plot_energy_comparison,
 29|    plot_temperature_trend,
 30|    plot_load_distribution,
 31|    plot_pue_comparison,
 32|    plot_cooling_power,
 33|    plot_summary_stats
 34|)
 35|from .plugin import PluginBase, PluginManager
 36|from .plugin.simulation_plugin import SimulationPlugin
```

```
from .api import app, run as run_api
from .cli import cli
from .sdk import DataCenterSimulatorSDK

__all__ = [
    # Algorithms
    'BaseScheduler',
    'GreedyScheduler',
    'GeneticScheduler',
    'GeneticAlgorithm',
    'RoundRobinScheduler',

    # Models
    'Server',
    'Task',
    'EnergyModel',
    'TemperatureModel',

    # Simulation
    'DataCenterSimulator',
    'ServerSnapshot',
    'TaskGenerator',

    # Utils
    'save_simulation_data',
    'load_simulation_data',
    'generate_synthetic_data',
    'export_results_to_excel',
    'Config',
    'setup_logger',

    # Visualization
    'plot_energy_comparison',
    'plot_temperature_trend',
    'plot_load_distribution',
    'plot_pue_comparison',
    'plot_cooling_power',
    'plot_summary_stats',

    # Plugin
    'PluginBase',
    'PluginManager',
    'SimulationPlugin',

```

```
    # API
    'app',
    'run_api',

    # CLI
    'cli',

    # SDK
    'DataCenterSimulatorSDK'
]
```

3. src/adapters/__init__.py

```
"""
适配器层 - 提供多平台、多硬件的自适应能力

设计模式: 策略模式 + 适配器模式
参考项目: stevedore (OpenStack), Flask-PluginKit

支持的适配器:
    LocalAdapter     - 本地CPU计算
    RemoteAdapter    - 远程服务调用
    CudaAdapter      - NVIDIA CUDA GPU加速 (可选)
    AscendAdapter    - 华为昇腾NPU加速/国产硬件 (可选)
    AutoAdapter      - 自动检测并选择最优适配器

环境变量配置:
    DCS_ENV             - 运行环境 (development/testing/production)
    DCS_FORCE_ADAPTER   - 强制使用指定适配器
    DCS_API_URL         - 远程API地址
    DCS_NUM_SERVERS     - 服务器数量
    DCS_SIM_HOURS       - 仿真时长
    DCS_PARALLEL_WORKERS - 并行工作器数
"""

from .base import BaseAdapter, AdapterType, AdapterCapability
from .local import LocalAdapter
from .remote import RemoteAdapter
from .auto import AutoAdapter

try:
    from .cuda import CudaAdapter
except ImportError:
    CudaAdapter = None

```

```
try:
    from .ascend import AscendAdapter
except ImportError:
    AscendAdapter = None

__all__ = [
    'BaseAdapter',
    'LocalAdapter',
    'RemoteAdapter',
    'CudaAdapter',
    'AscendAdapter',
    'AutoAdapter',
    'AdapterType',
    'AdapterCapability'
]
```

4. src/adapters/ascend.py

```
import os
from typing import Dict, Any, Optional
from src.adapters.base import BaseAdapter, AdapterType,
AdapterCapabilit
    y, normalise_result
from src.utils.logger import setup_logger

logger = setup_logger("ascend_adapter")

class AscendAdapter(BaseAdapter):
    """
    华为昇腾NPU适配器（国产硬件）

    适用于:
    - 华为昇腾910/310 NPU加速
    - 鲲鹏+昇腾国产服务器
    - OpenEulerOS平台

    参考项目: Huawei Ascend, CANN, PyTorch-NPU
    """

    def __init__(self):
        self._available = False
        self._initialized = False
        self._npu_count = 0
        self._npu_names = []

```

```
    def get_type(self) -> AdapterType:
        return AdapterType.ASCEND

    def get_capability(self) -> AdapterCapability:
        return AdapterCapability(
            adapter_type=AdapterType.ASCEND,
            max_servers=500,
            supports_parallel=True,
            supports_gpu=False,
            supports_npu=True,
            supports_distributed=False,
            priority=35,
           description=f"华为昇腾NPU适配器 ({self._npu_count} NPU)"
        )

    def is_available(self) -> bool:
        npu_available = False

        try:
            import torch_npu
            npu_available = torch_npu.npu.is_available()
            if npu_available:
                self._npu_count = torch_npu.npu.device_count()
            return npu_available
        except ImportError:
            pass

        ascend_paths = [
            '/usr/local/Ascend',
            '/home/HwHiAiUser/Ascend',
            os.path.expanduser('~/Ascend')
        ]
        for path in ascend_paths:
            if os.path.exists(path):
                self._npu_count = 1
                npu_available = True
                break

        self._available = npu_available
        return npu_available

    def initialize(self, config: Optional[Dict[str, Any]] = None)
-> boo
   |    l:
```

```
        try:
            if not self.is_available():
                logger.warning("Ascend NPU not available")
                return False

            self._config = config or {}
            self._initialized = True

            try:
                import torch_npu
                device = torch_npu.npu.device(0)
                logger.info(f"Ascend NPU initialized: {device}")
            except ImportError:
                logger.info("Ascend NPU initialized (basic mode)")

            return True
        except Exception as e:
            logger.error(f"Ascend adapter init failed: {e}")
            return False

    def execute(self, **kwargs) -> Dict[str, Any]:
        try:
            try:
                import torch_npu
                has_torch_npu = True
            except ImportError:
                has_torch_npu = False

            if not has_torch_npu:
                from src.plugin import PluginManager
                from src.plugin.simulation_plugin import
SimulationPlugi
    n

                pm = PluginManager()
                pm.register_plugin(SimulationPlugin)

                config = {
                    'num_servers': kwargs.get('num_servers', 10),
                    'simulation_hours':
kwargs.get('simulation_hours', 2
    4),
                    'parallel_workers':
kwargs.get('parallel_workers', 4
```

```
    |    ),
107|                    'run_mode': kwargs.get('run_mode', 'standard')
108|                }
109|
110|                pm.load_plugin('SimulationPlugin', config)
111|                result = pm.execute_plugin('SimulationPlugin',
112|                    algorithms=kwargs.get('algorithms',
['greedy', 'gene
    |    tic', 'round_robin']),
113|                    visualization=kwargs.get('visualization',
True),
114|                    save_results=kwargs.get('save_results', True)
115|                )
116|                pm.unload_plugin('SimulationPlugin')
117|
118|                return normalise_result(result)
119|
120|            import numpy as np
121|
122|            num_servers = kwargs.get('num_servers', 10)
123|            simulation_hours = kwargs.get('simulation_hours', 24)
124|
125|            # No simulation runs on the NPU itself: this branch
only rep
    |    orts
126|            # device-scaled placeholders, and marks the payload
as such
    |    so it can
127|            # never be mistaken for a real simulation result.
128|            loads = np.random.rand(simulation_hours, num_servers)
129|            temps = 25.0 + loads * 50.0
130|            energy = 100.0 + loads * 200.0
131|
132|            data = []
133|            for h in range(simulation_hours):
134|                data.append({
135|                    'time': h, 'algorithm': 'genetic_ascend',
136|                    'total_load': float(loads[h].sum()),
137|                    'avg_temperature': float(temps[h].mean()),
138|                    'total_energy': float(energy[h].sum()),
139|                    'pue': 1.75
140|                })
141|
142|            return {
```

```
143|                'success': True,
144|                'statistics': {
145|                    'genetic_ascend': {
146|                        'avg_energy': float(energy.mean()),
147|                        'max_temp': float(temps.max()),
148|                        'avg_pue': 1.75,
149|                        'total_load': float(loads.sum())
150|                    }
151|                },
152|                'records_count': len(data),
153|                'platform': 'ascend'
154|            }
155|        except Exception as e:
156|            return {'success': False, 'error': str(e)}
157|
158|    def get_status(self) -> Dict[str, Any]:
159|        return {
160|            'type': 'ascend',
161|            'available': self._available,
162|            'initialized': self._initialized,
163|            'npu_count': self._npu_count,
164|            'npu_names': self._npu_names,
165|            'supports_npu': self._available,
166|           'platform': '华为昇腾/鲲鹏'
167|        }
168|
169|    def cleanup(self) -> None:
170|        self._initialized = False
171|        logger.info("Ascend adapter cleaned up")
```

5. src/adapters/auto.py

```
  1|from typing import Dict, Any, Optional, List
  2|from src.adapters.base import BaseAdapter, AdapterType,
AdapterCapabilit
   |    y
  3|from src.adapters.local import LocalAdapter
  4|from src.adapters.remote import RemoteAdapter
  5|from src.adapters.cuda import CudaAdapter
  6|from src.adapters.ascend import AscendAdapter
  7|from src.utils.logger import setup_logger
  8|
  9|logger = setup_logger("auto_adapter")
 10|
 11|class AutoAdapter(BaseAdapter):
```

```
    """
    自动适配器 - 自动检测并选择最优适配策略

    检测优先级:
    1. 昇腾NPU (国产硬件优先)
    2. NVIDIA CUDA GPU
    3. 远程服务
    4. 本地CPU (兜底)

    参考项目: PyTorch Lightning auto-accelerator, ONNX Runtime
    """

    def __init__(self):
        self._available = True
        self._initialized = False
        self._adapters: List[BaseAdapter] = []
        self._active_adapter: Optional[BaseAdapter] = None
        self._adapter_map: Dict[str, BaseAdapter] = {}
        self._detect_order = []
        if AscendAdapter is not None:
            self._detect_order.append(('ascend', AscendAdapter))
        if CudaAdapter is not None:
            self._detect_order.append(('cuda', CudaAdapter))
        self._detect_order.extend([
            ('remote', RemoteAdapter),
            ('local', LocalAdapter),
        ])

    def get_type(self) -> AdapterType:
        return AdapterType.AUTO

    def get_capability(self) -> AdapterCapability:
        if self._active_adapter:
            return self._active_adapter.get_capability()
        return AdapterCapability(
            adapter_type=AdapterType.AUTO,
            max_servers=1000,
            supports_parallel=True,
            supports_gpu=True,
            supports_npu=True,
            supports_distributed=True,
            priority=0,
           description="自动适配器 (智能检测)"
        )
```

```
 56|
 57|    def is_available(self) -> bool:
 58|        return self._available
 59|
 60|    def _detect_adapters(self) -> BaseAdapter:
 61|        for name, adapter_cls in self._detect_order:
 62|            adapter = adapter_cls()
 63|            if adapter.is_available():
 64|                self._adapters.append(adapter)
 65|                self._adapter_map[name] = adapter
 66|                logger.info(f"Adapter detected: {name} ->
{adapter.get_c
   |    apability().description}")
 67|
 68|        for name, adapter_cls in self._detect_order:
 69|            adapter = self._adapter_map.get(name)
 70|            if adapter and adapter.is_available():
 71|                logger.info(f"Selected adapter: {name}")
 72|                return adapter
 73|
 74|        local = LocalAdapter()
 75|        local.is_available()
 76|        logger.info("Fallback to LocalAdapter")
 77|        return local
 78|
 79|    def initialize(self, config: Optional[Dict[str, Any]] = None)
-> boo
   |    l:
 80|        try:
 81|            self._config = config or {}
 82|            force = self._config.get('force_adapter')
 83|
 84|            if force and force in self._adapter_map:
 85|                self._active_adapter = self._adapter_map[force]
 86|            elif force:
 87|                adapter_cls_map = {
 88|                    'local': LocalAdapter, 'remote':
RemoteAdapter,
 89|                }
 90|                if CudaAdapter is not None:
 91|                    adapter_cls_map['cuda'] = CudaAdapter
 92|                if AscendAdapter is not None:
 93|                    adapter_cls_map['ascend'] = AscendAdapter
 94|                cls = adapter_cls_map.get(force)
```

```
 95|                if cls:
 96|                    self._active_adapter = cls()
 97|            else:
 98|                self._active_adapter = self._detect_adapters()
 99|
100|            if self._active_adapter:
101|                ok = self._active_adapter.initialize(self._config)
102|                if ok:
103|                    self._initialized = True
104|                    logger.info(f"AutoAdapter initialized with
{self._ac
   |    tive_adapter.get_type().value}")
105|                    return True
106|                logger.error("Active adapter init failed")
107|                fallback = LocalAdapter()
108|                if fallback.initialize(self._config):
109|                    self._active_adapter = fallback
110|                    self._initialized = True
111|                    return True
112|
113|            return False
114|        except Exception as e:
115|            logger.error(f"AutoAdapter init failed: {e}")
116|            return False
117|
118|    def execute(self, **kwargs) -> Dict[str, Any]:
119|        if not self._active_adapter:
120|            return {'success': False, 'error': 'No active
adapter'}
121|        try:
122|            result = self._active_adapter.execute(**kwargs)
123|            result['adapter_type'] =
self._active_adapter.get_type().val
   |    ue
124|            return result
125|        except Exception as e:
126|            logger.error(f"Adapter execution failed: {e}")
127|            fallback = LocalAdapter()
128|            fallback.initialize()
129|            return fallback.execute(**kwargs)
130|
131|    def get_status(self) -> Dict[str, Any]:
132|        active_info = self._active_adapter.get_status() if
self._active_
```

```
    |    adapter else {}
133|        available_adapters = []
134|        for name, adapter in self._adapter_map.items():
135|            available_adapters.append({
136|                'name': name,
137|                'type': adapter.get_type().value,
138|                'available': adapter.is_available(),
139|                'capability': adapter.get_capability().__dict__
140|            })
141|
142|        return {
143|            'type': 'auto',
144|            'available': self._available,
145|            'initialized': self._initialized,
146|            'active_adapter': self._active_adapter.get_type().valu
e if s
    |    elf._active_adapter else None,
147|            'active_info': active_info,
148|            'available_adapters': available_adapters
149|        }
150|
151|    def get_active_adapter(self) -> Optional[BaseAdapter]:
152|        return self._active_adapter
153|
154|    def cleanup(self) -> None:
155|        for adapter in self._adapters:
156|            adapter.cleanup()
157|        if self._active_adapter:
158|            self._active_adapter.cleanup()
159|        self._initialized = False
160|        logger.info("AutoAdapter cleaned up")
```

6. src/adapters/base.py

```
  1|from abc import ABC, abstractmethod
  2|from typing import Dict, Any, List, Optional, Tuple
  3|from dataclasses import dataclass
  4|from enum import Enum
  5|
  6|def normalise_result(value):
  7|    """Convert any adapter return value into a flat dict.
  8|
  9|    Adapters used to disagree: some returned ``PluginResult``
objects, s
    |    ome a
```

```
    raw API envelope with the business payload nested under ``data``, and some
    a plain dict. Callers then had to guess, which is how the CLI ended up
    reporting "records: 0" for a successful run.
    """
    if hasattr(value, 'success') and hasattr(value, 'data'):
        payload = dict(value.data) if isinstance(value.data, dict) else {}
        payload['success'] = bool(value.success)
        if value.message:
            payload.setdefault('message', value.message)
        if value.error_code:
            payload.setdefault('error_code', value.error_code)
        if value.warnings:
            payload.setdefault('warnings', list(value.warnings))
        if not payload['success'] and 'error' not in payload:
            payload['error'] = value.message or value.error_code or '未知错误'
        return payload

    if isinstance(value, dict):
        inner = value.get('data')
        if isinstance(inner, dict) and 'statistics' not in value:
            merged = dict(inner)
            merged.setdefault('success', value.get('success', True))
            for key in ('message', 'warnings', 'error_code', 'execution_time_ms', 'metrics', 'timestamp'):
                if key in value and key not in merged:
                    merged[key] = value[key]
            return merged
        return value

    return {'success': False, 'error': '返回类型无法识别: {0}'.format(type(value).__name__)}


class AdapterType(Enum):
    LOCAL = "local"
```

```
    REMOTE = "remote"
    CUDA = "cuda"
    ASCEND = "ascend"
    AUTO = "auto"

@dataclass
class AdapterCapability:
    adapter_type: AdapterType
    max_servers: int = 100
    supports_parallel: bool = False
    supports_gpu: bool = False
    supports_npu: bool = False
    supports_distributed: bool = False
    priority: int = 0
    description: str = ""

class BaseAdapter(ABC):
    """
    适配器基类

    所有平台/硬件适配器必须实现此接口。
    参考项目: stevedore (OpenStack), NVIDIA Triton Inference Server
    """

    @abstractmethod
    def get_type(self) -> AdapterType:
        pass

    @abstractmethod
    def get_capability(self) -> AdapterCapability:
        pass

    @abstractmethod
    def is_available(self) -> bool:
        """检测适配器是否在当前环境中可用"""
        pass

    @abstractmethod
    def initialize(self, config: Optional[Dict[str, Any]] = None)
-> boo
    l:
        """初始化适配器"""
        pass

```

```
    @abstractmethod
    def execute(self, **kwargs) -> Dict[str, Any]:
        """执行主逻辑"""
        pass

    @abstractmethod
    def get_status(self) -> Dict[str, Any]:
        """获取适配器状态"""
        pass

    @abstractmethod
    def cleanup(self) -> None:
        """清理资源"""
        pass

    def validate_config(self, config: Dict[str, Any]) ->
Tuple[bool, str
    ]:
        """验证配置参数"""
        return True, "OK"
```

7. src/adapters/cuda.py

```
from typing import Dict, Any, Optional
from src.adapters.base import BaseAdapter, AdapterType,
AdapterCapabilit
    y
from src.utils.logger import setup_logger

logger = setup_logger("cuda_adapter")

class CudaAdapter(BaseAdapter):
    """
    NVIDIA CUDA GPU适配器

    适用于:
    - GPU加速计算
    - 大规模遗传算法并行评估
    - PyTorch模型推理

    参考项目: NVIDIA CUDA Toolkit, PyTorch, CuPy
    """

    def __init__(self):
        self._available = False
```

```
        self._initialized = False
        self._gpu_count = 0
        self._gpu_names = []

    def get_type(self) -> AdapterType:
        return AdapterType.CUDA

    def get_capability(self) -> AdapterCapability:
        return AdapterCapability(
            adapter_type=AdapterType.CUDA,
            max_servers=500,
            supports_parallel=True,
            supports_gpu=True,
            supports_npu=False,
            supports_distributed=False,
            priority=30,
           description=f"NVIDIA CUDA适配器 ({self._gpu_count} GPU)"
        )

    def is_available(self) -> bool:
        try:
            import torch
            self._available = torch.cuda.is_available()
            if self._available:
                self._gpu_count = torch.cuda.device_count()
                self._gpu_names = [torch.cuda.get_device_name(i)
for i i
   |    n range(self._gpu_count)]
            return self._available
        except ImportError:
            return False

    def initialize(self, config: Optional[Dict[str, Any]] = None)
-> boo
   |    l:
        try:
            if not self.is_available():
                logger.warning("CUDA not available")
                return False

            self._config = config or {}

            import torch
```

```
            self._device = torch.device('cuda' if
torch.cuda.is_availabl
    e() else 'cpu')
            self._initialized = True

            logger.info(f"CUDA Adapter initialized:
{self._gpu_names}")
            return True
        except Exception as e:
            logger.error(f"CUDA adapter init failed: {e}")
            return False

    def execute(self, **kwargs) -> Dict[str, Any]:
        try:
            import torch

            if not torch.cuda.is_available():
                return {'success': False, 'error': 'CUDA not
available'}

            result = self._execute_cuda_accelerated(kwargs)
            return result
        except Exception as e:
            return {'success': False, 'error': str(e)}

    def _execute_cuda_accelerated(self, params: Dict[str, Any])
-> Dict[
    str, Any]:
        import torch
        import numpy as np

        num_servers = params.get('num_servers', 10)
        simulation_hours = params.get('simulation_hours', 24)

        loads = torch.rand(simulation_hours, num_servers,
device='cuda')
        temps = 25.0 + loads * 50.0
        energy = 100.0 + loads * 200.0

        avg_temp = temps.mean(dim=1).cpu().numpy()
        total_energy = energy.sum(dim=1).cpu().numpy()
        total_load = loads.sum(dim=1).cpu().numpy()

        pue_values = [1.8] * simulation_hours
```

```
 97|
 98|        data = []
 99|        for h in range(simulation_hours):
100|            data.append({
101|                'time': h,
102|                'algorithm': 'genetic_cuda',
103|                'total_load': float(total_load[h]),
104|                'avg_temperature': float(avg_temp[h]),
105|                'total_energy': float(total_energy[h]),
106|                'pue': float(pue_values[h])
107|            })
108|
109|        return {
110|            'success': True,
111|            'statistics': {
112|                'genetic_cuda': {
113|                    'avg_energy': float(total_energy.mean()),
114|                    'max_temp': float(avg_temp.max()),
115|                    'avg_pue': 1.8,
116|                    'total_load': float(total_load.sum())
117|                }
118|            },
119|            'records_count': len(data),
120|            'gpu_used': self._gpu_names[0] if self._gpu_names
else 'CUDA
   |    '
121|        }
122|
123|    def get_status(self) -> Dict[str, Any]:
124|        return {
125|            'type': 'cuda',
126|            'available': self._available,
127|            'initialized': self._initialized,
128|            'gpu_count': self._gpu_count,
129|            'gpu_names': self._gpu_names,
130|            'supports_cuda': self._available
131|        }
132|
133|    def cleanup(self) -> None:
134|        import torch
135|        if torch.cuda.is_available():
136|            torch.cuda.empty_cache()
137|        self._initialized = False
138|        logger.info("CUDA adapter cleaned up")
```

8. src/adapters/local.py

```
import platform
import psutil
from typing import Dict, Any, Optional
from src.adapters.base import (
    BaseAdapter, AdapterType, AdapterCapability, normalise_result
)
from src.utils.logger import setup_logger
from src.utils.config import Config

logger = setup_logger("local_adapter")

class LocalAdapter(BaseAdapter):
    """
    本地CPU适配器

    适用于:
    - 本地开发环境
    - 单机部署
    - 小规模仿真(<100台服务器)

    参考项目: joblib, concurrent.futures
    """

    def __init__(self):
        self._available = True
        self._initialized = False
        self._config = {}
        self._num_workers = 2

    def get_type(self) -> AdapterType:
        return AdapterType.LOCAL

    def get_capability(self) -> AdapterCapability:
        cores = psutil.cpu_count(logical=False) or 4
        return AdapterCapability(
            adapter_type=AdapterType.LOCAL,
            max_servers=100,
            supports_parallel=True,
            supports_gpu=False,
            supports_npu=False,
            supports_distributed=False,
            priority=10,
           description=f"本地CPU适配器 ({cores}核心)"
```

```
        )

    def is_available(self) -> bool:
        return self._available

    def initialize(self, config: Optional[Dict[str, Any]] = None)
-> boo
    l:
        try:
            self._config = config or {}
            cores = self._config.get('parallel_workers',
psutil.cpu_coun
    t(logical=False) or 4)
            self._num_workers = min(cores, 32)
            self._initialized = True
            logger.info(f"LocalAdapter initialized with
{self._num_worke
    rs} workers")
            return True
        except Exception as e:
            logger.error(f"LocalAdapter init failed: {e}")
            return False

    def execute(self, **kwargs) -> Dict[str, Any]:
        try:
            from src.plugin import PluginManager
            from src.plugin.simulation_plugin import
SimulationPlugin

            pm = PluginManager()
            pm.register_plugin(SimulationPlugin)

            config = {
                'num_servers': kwargs.get('num_servers', 10),
                'simulation_hours':
kwargs.get('simulation_hours', 24),
                'parallel_workers': self._num_workers,
                'run_mode': kwargs.get('run_mode', 'standard')
            }

            pm.load_plugin('SimulationPlugin', config)
            result = pm.execute_plugin('SimulationPlugin',
                algorithms=kwargs.get('algorithms', ['greedy', '
genetic'
```

```
   |    , 'round_robin']),
 79|                visualization=kwargs.get('visualization', True),
 80|                save_results=kwargs.get('save_results', True)
 81|            )
 82|            pm.unload_plugin('SimulationPlugin')
 83|
 84|            return normalise_result(result)
 85|        except Exception as e:
 86|            return {'success': False, 'error': str(e)}
 87|
 88|    def get_status(self) -> Dict[str, Any]:
 89|        return {
 90|            'type': 'local',
 91|            'available': self._available,
 92|            'initialized': self._initialized,
 93|            'workers': self._num_workers,
 94|            'os': platform.system(),
 95|            'cpu_count': psutil.cpu_count()
 96|        }
 97|
 98|    def cleanup(self) -> None:
 99|        self._initialized = False
100|        logger.info("LocalAdapter cleaned up")
```

9. src/adapters/remote.py

```
  1|from typing import Dict, Any, Optional
  2|from src.adapters.base import BaseAdapter, AdapterType,
AdapterCapabilit
   |    y, normalise_result
  3|from src.utils.logger import setup_logger
  4|
  5|logger = setup_logger("remote_adapter")
  6|
  7|class RemoteAdapter(BaseAdapter):
  8|    """
  9|    远程服务适配器
 10|
 11|    适用于:
 12|    - 分布式部署
 13|    - 云端调用
 14|    - 跨语言客户端
 15|
 16|    参考项目: Flask-RESTful, FastAPI, gRPC
 17|    """
```

```
 18|
 19|    def __init__(self):
 20|        self._available = True
 21|        self._initialized = False
 22|        self._api_url = "http://localhost:5000"
 23|        self._session = None
 24|
 25|    def get_type(self) -> AdapterType:
 26|        return AdapterType.REMOTE
 27|
 28|    def get_capability(self) -> AdapterCapability:
 29|        return AdapterCapability(
 30|            adapter_type=AdapterType.REMOTE,
 31|            max_servers=1000,
 32|            supports_parallel=True,
 33|            supports_gpu=True,
 34|            supports_npu=True,
 35|            supports_distributed=True,
 36|            priority=20,
 37|           description="远程服务适配器 (REST API)"
 38|        )
 39|
 40|    def is_available(self) -> bool:
 41|        if not self._available:
 42|            return False
 43|        try:
 44|            import requests
 45|            return True
 46|        except ImportError:
 47|            logger.warning("requests not installed, RemoteAdapter
unavai
   |    lable")
 48|            return False
 49|
 50|    def initialize(self, config: Optional[Dict[str, Any]] = None)
-> boo
   |    l:
 51|        try:
 52|            self._config = config or {}
 53|            self._api_url = self._config.get('api_url', '
http://localhos
   |    t:5000').rstrip('/')
 54|
 55|            import requests
```

```
            from urllib3.util.retry import Retry
            from requests.adapters import HTTPAdapter

            session = requests.Session()
            retry = Retry(total=3, backoff_factor=0.5,
status_forcelist=
    [429, 500, 502, 503, 504])
            adapter = HTTPAdapter(max_retries=retry,
pool_connections=10
    , pool_maxsize=20)
            session.mount('http://', adapter)
            session.mount('https://', adapter)

            self._session = session
            self._initialized = True
            logger.info(f"RemoteAdapter initialized:
{self._api_url}")
            return True
        except Exception as e:
            logger.error(f"RemoteAdapter init failed: {e}")
            return False

    def execute(self, **kwargs) -> Dict[str, Any]:
        try:
            url = f"{self._api_url}/api/v1/simulation"
            payload = {
                'num_servers': kwargs.get('num_servers', 10),
                'simulation_hours':
kwargs.get('simulation_hours', 24),
                'parallel_workers':
kwargs.get('parallel_workers', 4),
                'run_mode': kwargs.get('run_mode', 'standard'),
                'algorithms': kwargs.get('algorithms', ['greedy', '
genet
    ic', 'round_robin'])
            }

            response = self._session.post(url, json=payload,
timeout=300
    )
            response.raise_for_status()
            return normalise_result(response.json())
        except Exception as e:
            return {'success': False, 'error': str(e)}
```

```
 89|
 90|    def get_status(self) -> Dict[str, Any]:
 91|        online = False
 92|        if self._session:
 93|            try:
 94|                resp = self._session.get(f"{self._api_url}/health",
 time
   |    out=5)
 95|                online = resp.status_code == 200
 96|            except Exception:
 97|                pass
 98|
 99|        return {
100|            'type': 'remote',
101|            'available': self._available,
102|            'initialized': self._initialized,
103|            'api_url': self._api_url,
104|            'online': online
105|        }
106|
107|    def cleanup(self) -> None:
108|        if self._session:
109|            self._session.close()
110|        self._initialized = False
111|        logger.info("RemoteAdapter cleaned up")
```

10. src/algorithms/__init__.py

```
  1|from .base import BaseScheduler
  2|from .greedy_algorithm import GreedyScheduler
  3|from .genetic_algorithm import GeneticScheduler, GeneticAlgorithm
  4|from .round_robin import RoundRobinScheduler
  5|from .damo_ets import DAMOETSScheduler
  6|from .plugin_adapter import PLUGIN_ALGORITHMS,
PluginSchedulerAdapter, i
   |    s_plugin_algorithm
  7|
  8|__all__ = [
  9|    'BaseScheduler',
 10|    'GreedyScheduler',
 11|    'GeneticScheduler',
 12|    'GeneticAlgorithm',
 13|    'RoundRobinScheduler',
 14|    'DAMOETSScheduler',
 15|    'PLUGIN_ALGORITHMS',
```

```
 16|    'PluginSchedulerAdapter',
 17|    'is_plugin_algorithm',
 18|]
```

11. src/algorithms/base.py

```
  1|from abc import ABC, abstractmethod
  2|from typing import List, Tuple, Optional
  3|from src.models import Task, Server
  4|
  5|class BaseScheduler(ABC):
  6|    def __init__(self, servers: List[Server]) -> None:
  7|        self.servers = servers
  8|
  9|    @abstractmethod
 10|    def schedule_tasks(self, tasks: List[Task]) ->
List[Tuple[Task, Opti
   |    onal[Server]]]:
 11|        pass
 12|
 13|    @abstractmethod
 14|    def get_name(self) -> str:
 15|        pass
```

12. src/algorithms/damo_ets.py

```
  1|"""
  2|DAMO-ETS: Deadline-Aware Multi-Objective Energy-Thermal Scheduler.
  3|
  4|Combines:
  5|    * Earliest-Deadline-First (EDF) ordering
  6|    * Per-(task, server) cost aggregation across five objectives
  7|        - deadline urgency
  8|        - incremental energy
  9|        - predicted thermal impact
 10|        - heterogeneous workload/server match
 11|        - cross-type migration penalty
 12|    * Tie-break randomness that is fully controlled by the
injected seed
 13|
 14|Public surface:
 15|    * DAMOETSScheduler : BaseScheduler subclass used by the
simulator
 16|    * deadline_violations : count of tasks whose deadline was
missed in
   |    the
```

```
     last ``schedule_tasks`` call
"""

from typing import Any, Dict, List, Optional, Tuple
import random

from src.algorithms.base import BaseScheduler
from src.models import Server, Task
from src.utils.config import Config
from src.utils.logger import setup_logger

logger = setup_logger("damo_ets")


# Helpers

def _safe_get(d: Any, key: str, default: Any = None) -> Any:
    """Return ``d[key]`` whether ``d`` is a dict or a SectionAccessor."""
    if d is None:
        return default
    if isinstance(d, dict):
        return d.get(key, default)
    getter = getattr(d, "get", None)
    if callable(getter):
        try:
            return getter(key, default)
        except Exception:
            return default
    return getattr(d, key, default)


def _resolve_seed(seed: Optional[int]) -> Optional[int]:
    if seed is not None:
        return int(seed)
    try:
        cfg = Config().get("simulation", {}) or {}
        val = cfg.get("random_seed")
        return int(val) if val is not None else None
    except Exception:
        return None


```

```
# Scheduler

class DAMOETSScheduler(BaseScheduler):
    """Multi-objective energy-thermal scheduler with deadline awareness.
    """

    DEFAULT_CONFIG: Dict[str, Any] = {
        "alpha_deadline": 0.35,
        "beta_energy": 0.25,
        "gamma_thermal": 0.20,
        "delta_heterogeneity": 0.15,
        "epsilon_migration": 0.05,
        "heat_coefficient": 15.0,
        "thermal_safety_factor": 0.95,
        "workload_server_match": {
            "gpu_bound":    ["gpu_server", "high_performance"],
            "cpu_bound":    ["high_performance", "standard"],
            "memory_bound": ["standard"],
            "io_bound":     ["standard"],
        },
    }

    def __init__(
        self,
        servers: List[Server],
        config: Optional[Dict[str, Any]] = None,
        seed: Optional[int] = None,
    ) -> None:
        super().__init__(servers)

        # Merge YAML-driven config with defaults
        yaml_cfg: Dict[str, Any] = {}
        try:
            damo_section = Config().get("damo_ets", {}) or {}
            if isinstance(damo_section, dict):
                yaml_cfg = damo_section
        except Exception:
            yaml_cfg = {}

        merged: Dict[str, Any] = {}
        merged.update(self.DEFAULT_CONFIG)
        merged.update(yaml_cfg or {})
        if config:
```

```
105|            merged.update(config)
106|        self.config: Dict[str, Any] = merged
107|
108|        self.match_table: Dict[str, List[str]] = dict(
109|            self.config.get("workload_server_match") or
110|            self.DEFAULT_CONFIG["workload_server_match"]
111|        )
112|
113|        self.seed = _resolve_seed(seed)
114|        self.rng = random.Random(self.seed)
115|
116|        # Track number of deadline violations after each schedule
call
117|        self.deadline_violations: int = 0
118|
119|        # Last server type assigned — used for migration penalty
120|        self._last_server_type: Optional[str] = None
121|
123|    def get_name(self) -> str:
124|        return "damo_ets"
125|
126|    def get_deadline_violations(self) -> int:
127|        """Tasks that were already past their deadline when they
were sc
   |    heduled.
128|
129|        This is a per-scheduler diagnostic only. The metric
reported for
130|        comparison is computed by the simulator from completion
times, s
   |    o every
131|        algorithm is measured with the same rule.
132|        """
133|        return int(self.deadline_violations)
134|
135|    def reset(self) -> None:
136|        self.deadline_violations = 0
137|        self._last_server_type = None
138|
140|    def _deadline_urgency(self, task: Task, current_hour: int) ->
float:
141|        if task.deadline is None:
142|            return 0.0
143|        remaining = max(1, task.deadline - current_hour)
```

```
        # Closer to deadline -> higher urgency, normalised to [0, 1]
        return float(min(1.0, 1.0 / remaining))

    def _energy_impact(self, task: Task, server: Server) -> float:
        """Marginal power of putting this task here, normalised to [0, 1].

        The plant charges base power for every machine that is switched on, so
        the useful signal is the *increase* over what the server draws now, not
        its absolute utilisation. Preferring an empty machine (the previous
        behaviour) actively pushed load onto idle servers and paid their base
        power, which is the opposite of what saves energy.
        """
        before = server.calculate_power()
        projected = min(
            server.max_load,
            server.current_load + task.load,
        )
        load_ratio = projected / max(1.0, server.max_load)
        after = server.base_power + (server.max_power - server.base_power) * load_ratio
        delta = max(0.0, after - before)
        return float(min(1.0, delta / 200.0))

    def _thermal_impact(self, task: Task, server: Server) -> float:
        predicted = server.predicted_temp_after(task.load)
        safe = server.max_temp * self.config.get("thermal_safety_factor", 0.95)
        if predicted >= safe:
            return 1.0
        return float(predicted / safe)
```

```
172|
173|    def _heterogeneity_match(self, task: Task, server: Server) ->
float:
174|        preferred = self.match_table.get(task.workload_class, [])
175|        try:
176|            rank = preferred.index(server.server_type)
177|        except ValueError:
178|            rank = len(preferred)  # worst rank
179|        # 1.0 for best match, decreasing to 0
180|        denom = max(1, len(preferred))
181|        return float(max(0.0, 1.0 - rank / denom))
182|
183|    def _migration_penalty(self, server: Server) -> float:
184|        if self._last_server_type is None or
self._last_server_type == s
   |    erver.server_type:
185|            return 0.0
186|        return 1.0
187|
188|    def _cost(self, task: Task, server: Server, current_hour:
int) -> fl
   |    oat:
189|        return (
190|            self.config["alpha_deadline"]     *
self._deadline_urgency(t
   |    ask, current_hour)
191|            + self.config["beta_energy"]      *
self._energy_impact(task
   |    , server)
192|            + self.config["gamma_thermal"]    *
self._thermal_impact(tas
   |    k, server)
193|            + self.config["delta_heterogeneity"] * (1.0 -
self._heteroge
   |    neity_match(task, server))
194|            + self.config["epsilon_migration"] *
self._migration_penalty
   |    (server)
195|        )
196|
198|    def schedule_tasks(
199|        self,
200|        tasks: List[Task],
201|        current_hour: Optional[int] = None,
```

```
    ) -> List[Tuple[Task, Optional[Server]]]:
        self.reset()

        if not tasks:
            return []

        current_hour = int(current_hour) if current_hour is not
None els
    e 0

        ordered = sorted(
            tasks,
            key=lambda t: (t.deadline is None, t.deadline if
t.deadline
    is not None else 0, -t.priority),
        )

        results: List[Tuple[Task, Optional[Server]]] = []

        for task in ordered:
            chosen: Optional[Server] = None
            best_cost = float("inf")
            best_tie_break = float("inf")
            candidates: List[Tuple[float, float, Server]] = []

            safe_factor = self.config.get("thermal_safety_factor",
 0.95)

            for server in self.servers:
                # Hard constraints first
                if server.is_overloaded():
                    continue
                if server.current_load + task.load >
server.max_load:
                    continue
                # Reject if predicted temp exceeds safety
threshold
                if server.predicted_temp_after(task.load) >=
server.max_
    temp * safe_factor:
                    continue

                cost = self._cost(task, server, current_hour)
```

```
237|                # Tie-break: prefer lower current load, then
smaller id
238|                tie = (server.current_load, server.server_id)
239|                candidates.append((cost, tie[0] * 1000 + tie[1],
server)
   |    )
240|
241|            if candidates:
242|                candidates.sort(key=lambda c: (c[0], c[1]))
243|                # Inject seeded randomness into tie-break
(perturb cost
   |    by small
244|                # noise drawn from rng). Keeps reproducibility
intact.
245|                top_cost = candidates[0][0]
246|                near_top = [c for c in candidates if c[0] -
top_cost < 1
   |    e-9]
247|                if len(near_top) > 1:
248|                    pick = self.rng.choice(near_top)
249|                else:
250|                    pick = candidates[0]
251|                chosen = pick[2]
252|
253|            if chosen is not None and chosen.assign_task(task):
254|                task.start_time = current_hour
255|                results.append((task, chosen))
256|                self._last_server_type = chosen.server_type
257|            else:
258|                results.append((task, None))
259|
260|            # Deadline accounting: violations are counted
relative to th
   |    e
261|            # *assignment hour*, regardless of when the task
would finis
   |    h.
262|            if task.is_deadline_violated(current_hour):
263|                self.deadline_violations += 1
264|
265|        return results
```

13. src/algorithms/genetic_algorithm.py

```
1|from typing import List, Dict, Any, Tuple, Optional
```

```
import random
import numpy as np
from concurrent.futures import ProcessPoolExecutor, as_completed
from multiprocessing import cpu_count
from src.models.server import Server
from src.models.task import Task
from src.models.energy_model import EnergyModel
from src.models.temperature_model import TemperatureModel
from src.utils.logger import setup_logger
from src.utils.config import Config
from src.algorithms.base import BaseScheduler

logger = setup_logger("genetic_algorithm")
config = Config()

def _default_seed() -> Optional[int]:
    """Read default seed from config; return None when unavailable."""
    try:
        sim_cfg = Config().get("simulation", {}) or {}
        seed_val = sim_cfg.get("random_seed")
        if seed_val is None:
            return None
        return int(seed_val)
    except Exception:
        return None


class GeneticScheduler(BaseScheduler):
    def __init__(self, servers: List[Server], seed: Optional[int] = None
    ):
        effective_seed = seed if seed is not None else _default_seed()
        super().__init__(servers)
        self.logger = setup_logger("genetic_scheduler")
        self.config = Config()
        self.population_size = self.config.genetic.get('population_size'
    , 50)
        self.generations = self.config.genetic.get('generations', 50)
        self.crossover_rate = self.config.genetic.get('crossover_rate',
```

```
   |    0.8)
 38|        self.mutation_rate = self.config.genetic.get('mutation_rat
e', 0.
   |    1)
 39|        self.elitism_rate = self.config.genetic.get('elitism_rate',
 0.1)
 40|        self.seed = effective_seed
 41|        self.rng = random.Random(effective_seed)
 42|        self.np_rng = np.random.default_rng(effective_seed)
 43|        self._pool: Optional[ProcessPoolExecutor] = None
 44|
 45|    def _get_pool(self) -> ProcessPoolExecutor:
 46|        if self._pool is None:
 47|            workers = self.config.genetic.get('parallel_evaluators'
, min
   |    (4, cpu_count()))
 48|            self._pool = ProcessPoolExecutor(max_workers=workers)
 49|        return self._pool
 50|
 51|    def close(self) -> None:
 52|        if self._pool is not None:
 53|            self._pool.shutdown(wait=True)
 54|            self._pool = None
 55|
 56|    def repair_server(self, task: Task) -> Optional[Server]:
 57|        """Least-loaded server that can still take the task.
 58|
 59|        The genetic search produces a static assignment; when its
chosen
   |     server
 60|        turns out to be full the task used to be dropped, which
punished
   |     the
 61|        algorithm for a fixable artefact of the encoding rather
than for
   |     a bad
 62|        placement decision.
 63|        """
 64|        candidates = [
 65|            server for server in self.servers
 66|            if not server.is_overloaded()
 67|            and not server.is_overheated()
 68|            and server.current_load + task.load <= server.max_load
 69|        ]
```

```
        if not candidates:
            return None
        return min(candidates, key=lambda s: (s.current_load,
s.current_
    temp))

    def schedule_tasks(self, tasks: List[Task]) ->
List[Tuple[Task, Opti
    onal[Server]]]:
        if not tasks:
            return []

        ga = GeneticAlgorithm(self.servers, tasks, seed=self.seed)

        results = []
        try:
            assignments = ga.schedule()
            for task, server in assignments:
                placed: Optional[Server] = None
                if server and not server.is_overloaded() and not
server.
    is_overheated():
                    if server.assign_task(task):
                        placed = server
                if placed is None:
                    repair = self.repair_server(task)
                    if repair is not None and
repair.assign_task(task):
                        placed = repair
                results.append((task, placed))
        finally:
            self.close()

        return results

    def get_name(self) -> str:
        return "genetic"


class GeneticAlgorithm:
    def __init__(self, servers: List[Server], tasks: List[Task],
seed: O
    ptional[int] = None):
        self.servers = servers
```

```
105|        self.tasks = tasks
106|        self.num_servers = len(servers)
107|        self.num_tasks = len(tasks)
108|        effective_seed = seed if seed is not None else
_default_seed()
109|        self.seed = effective_seed
110|        self.rng = random.Random(effective_seed)
111|        self.np_rng = np.random.default_rng(effective_seed)
112|        self.population_size = config.genetic.get('population_size'
, 50)
113|        self.max_population_size =
config.genetic.get('max_population_si
   |    ze', 200)
114|        self.generations = config.genetic.get('generations', 100)
115|        self.max_generations = config.genetic.get('max_generations'
, 500
   |    )
116|        self.crossover_rate = config.genetic.get('crossover_rate',
 0.8)
117|        self.mutation_rate = config.genetic.get('mutation_rate',
0.1)
118|        self.elitism_rate = config.genetic.get('elitism_rate',
0.1)
119|        self.overload_penalty = config.genetic.get('overload_penal
ty', 1
   |    00)
120|        self.overheat_penalty = config.genetic.get('overheat_penal
ty', 2
   |    00)
121|        self.energy_weight = config.genetic.get('energy_weight',
0.5)
122|        self.temperature_weight =
config.genetic.get('temperature_weight
   |    ', 0.3)
123|        self.load_weight = config.genetic.get('load_weight', 0.2)
124|        self.enable_parallel = config.genetic.get('enable_parallel'
, Tru
   |    e)
125|        self.parallel_evaluators =
config.genetic.get('parallel_evaluato
   |    rs', min(4, cpu_count()))
126|
127|        self.energy_model = EnergyModel()
128|        self.temp_model = TemperatureModel()
```

```
129|
130|        if self.num_tasks == 0:
131|            raise ValueError("No tasks to schedule")
132|
133|    def initialize_population(self) -> List[List[int]]:
134|        population = []
135|        for _ in range(self.population_size):
136|            individual = [self.rng.randint(0, self.num_servers -
1) for
   |    _ in range(self.num_tasks)]
137|            population.append(individual)
138|        return population
139|
140|    def calculate_fitness(self, individual: List[int]) -> float:
141|        server_loads = [0.0] * self.num_servers
142|        server_temps = [s.current_temp for s in self.servers]
143|        total_energy = 0.0
144|
145|        for task_idx, server_idx in enumerate(individual):
146|            server_loads[server_idx] += self.tasks[task_idx].load
147|
148|        for i, server in enumerate(self.servers):
149|            load_ratio = server_loads[i] / server.max_load
150|            temp_increase = load_ratio * 15
151|            server_temps[i] = min(server.max_temp,
server_temps[i] + tem
   |    p_increase)
152|
153|            if load_ratio <= 0:
154|                total_energy += server.base_power *
server.idle_power_fa
   |    ctor
155|            else:
156|                base_power = server.base_power
157|                load_power = (server.max_power -
server.base_power) * lo
   |    ad_ratio
158|                total_energy += base_power + load_power
159|
160|            temp_penalty = max(0, (server_temps[i] - 70) / 15)
161|            total_energy += temp_penalty * 10
162|
163|        avg_load = sum(server_loads) / (self.num_servers * 100)
164|        avg_temp = sum(server_temps) / self.num_servers
```

```
165|
166|        energy_score = 1 / (total_energy / 1000 + 1)
167|        load_score = 1 - avg_load
168|        temp_score = 1 - (avg_temp - 25) / 60
169|
170|        fitness = (self.energy_weight * energy_score +
171|                   self.load_weight * load_score +
172|                   self.temperature_weight * temp_score)
173|
174|        return max(0.0001, fitness)
175|
176|    def evaluate_population_parallel(self, population:
List[List[int]])
   |    -> List[Tuple[float, List[int]]]:
177|        results = []
178|        workers = max(1, self.parallel_evaluators)
179|        with ProcessPoolExecutor(max_workers=workers) as executor:
180|            futures = {executor.submit(self.calculate_fitness,
ind): ind
   |     for ind in population}
181|
182|            for future in as_completed(futures):
183|                individual = futures[future]
184|                try:
185|                    fitness = future.result()
186|                    results.append((fitness, individual))
187|                except Exception as e:
188|                    logger.error(f"Error evaluating individual:
{e}")
189|                    results.append((0.0, individual))
190|
191|        return sorted(results, key=lambda x: x[0], reverse=True)
192|
193|    def evaluate_population_serial(self, population:
List[List[int]]) ->
   |     List[Tuple[float, List[int]]]:
194|        results = []
195|        for individual in population:
196|            fitness = self.calculate_fitness(individual)
197|            results.append((fitness, individual))
198|        return sorted(results, key=lambda x: x[0], reverse=True)
199|
200|    def evaluate_population(self, population: List[List[int]]) ->
List[T
```

```
    |    uple[float, List[int]]]:
201|        if self.enable_parallel and self.num_tasks > 100:
202|            return self.evaluate_population_parallel(population)
203|        else:
204|            return self.evaluate_population_serial(population)
205|
206|    def select_parents(self, evaluated: List[Tuple[float,
List[int]]]) -
    |    > Tuple[List[int], List[int]]:
207|        total_fitness = sum(f[0] for f in evaluated)
208|        if total_fitness == 0:
209|            return self.rng.sample([e[1] for e in evaluated], 2)
210|
211|        selection_probs = [f[0] / total_fitness for f in
evaluated]
212|        parent1 = self.rng.choices([e[1] for e in evaluated],
weights=se
    |    lection_probs)[0]
213|        parent2 = self.rng.choices([e[1] for e in evaluated],
weights=se
    |    lection_probs)[0]
214|
215|        return parent1, parent2
216|
217|    def crossover(self, parent1: List[int], parent2: List[int])
-> Tuple
    |    [List[int], List[int]]:
218|        if self.rng.random() > self.crossover_rate:
219|            return parent1.copy(), parent2.copy()
220|
221|        crossover_point = self.rng.randint(1, self.num_tasks - 1)
222|        child1 = parent1[:crossover_point] +
parent2[crossover_point:]
223|        child2 = parent2[:crossover_point] +
parent1[crossover_point:]
224|
225|        return child1, child2
226|
227|    def mutate(self, individual: List[int]) -> List[int]:
228|        mutated = individual.copy()
229|        for i in range(len(mutated)):
230|            if self.rng.random() < self.mutation_rate:
231|                mutated[i] = self.rng.randint(0, self.num_servers
- 1)
```

```
232|        return mutated
233|
234|    def evolve(self) -> Tuple[List[int], float]:
235|        population = self.initialize_population()
236|        best_fitness = 0.0
237|        best_individual = None
238|
239|        for gen in range(self.generations):
240|            evaluated = self.evaluate_population(population)
241|
242|            current_best = evaluated[0]
243|            if current_best[0] > best_fitness:
244|                best_fitness = current_best[0]
245|                best_individual = current_best[1].copy()
246|
247|            if gen % 10 == 0:
248|                avg_fitness = sum(f[0] for f in evaluated) /
len(evaluat
   |    ed)
249|                logger.debug(f"Generation {gen}: Best fitness =
{best_fi
   |    tness:.4f}, Avg fitness = {avg_fitness:.4f}")
250|
251|            elite_count = max(1, int(self.elitism_rate *
self.population
   |    _size))
252|            new_population = [e[1].copy() for e in
evaluated[:elite_coun
   |    t]]
253|
254|            while len(new_population) < self.population_size:
255|                parent1, parent2 = self.select_parents(evaluated)
256|                child1, child2 = self.crossover(parent1, parent2)
257|                child1 = self.mutate(child1)
258|                child2 = self.mutate(child2)
259|                new_population.extend([child1, child2])
260|
261|            new_population = new_population[:self.population_size]
262|            population = new_population
263|
264|        logger.debug(f"Final best fitness: {best_fitness:.4f}")
265|        return best_individual, best_fitness
266|
267|    def schedule(self) -> List[Tuple[Task, Server]]:
```

```
        if self.num_tasks == 0:
            return []

        logger.info(f"Starting genetic algorithm scheduling with {self.n
    um_tasks} tasks on {self.num_servers} servers")

        try:
            best_assignment, fitness = self.evolve()

            assignments = []
            for task_idx, server_idx in enumerate(best_assignment):
                assignments.append((self.tasks[task_idx], self.servers[s
    erver_idx]))

            logger.info(f"Genetic algorithm completed. Fitness: {fitness
    :.4f}, Assignments: {len(assignments)}")
            return assignments

        except Exception as e:
            logger.error(f"Genetic algorithm failed: {e}", exc_info=True
    )
            return self.fallback_schedule()

    def fallback_schedule(self) -> List[Tuple[Task, Server]]:
        """Placement of last resort: temperature and capacity aware, nev
    er a blind overload."""
        assignments = []
        for task in self.tasks:
            candidates = [
                server for server in self.servers
                if not server.is_overloaded()
                and not server.is_overheated()
                and server.current_load + task.load <= server.max_load
            ]
            if not candidates:
                continue
```

```
299|            server = min(candidates, key=lambda s:
(s.current_load, s.cu
   |    rrent_temp))
300|            server.assign_task(task)
301|            assignments.append((task, server))
302|        return assignments
303|
304|    def run_large_scale_simulation(self, iterations: int = 10) ->
Dict[s
   |    tr, Any]:
305|        results = []
306|
307|        for i in range(iterations):
308|            logger.info(f"Running large-scale iteration
{i+1}/{iteration
   |    s}")
309|            _, fitness = self.evolve()
310|            results.append(fitness)
311|
312|        return {
313|            'avg_fitness': sum(results) / len(results),
314|            'min_fitness': min(results),
315|            'max_fitness': max(results),
316|            'iterations': iterations
317|        }
```

14. src/algorithms/greedy_algorithm.py

```
  1|import numpy as np
  2|from typing import List, Tuple, Optional
  3|from src.models import Task, Server
  4|from src.algorithms.base import BaseScheduler
  5|
  6|class GreedyScheduler(BaseScheduler):
  7|    def __init__(self, servers: List[Server], temperature_weight:
float
   |    = 0.3) -> None:
  8|        super().__init__(servers)
  9|        self.temp_weight: float = temperature_weight
 10|        self.load_weight: float = 1 - temperature_weight
 11|
 12|    def select_best_server(self, task: Task) -> Server:
 13|        scores: List[float] = []
 14|        for server in self.servers:
 15|            if server.is_overloaded() or server.is_overheated():
```

```
                scores.append(float('-inf'))
                continue

            load_score = 1 - server.current_load / server.max_load
            temp_score = 1 - server.current_temp / server.max_temp
            combined_score = self.load_weight * load_score +
self.temp_w
    eight * temp_score
            scores.append(combined_score)

        best_idx = np.argmax(scores)
        return self.servers[best_idx]

    def schedule_task(self, task: Task) -> Tuple[Task,
Optional[Server]]
    :
        server = self.select_best_server(task)
        if server and not server.is_overloaded() and not
server.is_overh
    eated():
            if server.assign_task(task):
                return (task, server)
        return (task, None)

    def schedule_tasks(self, tasks: List[Task]) ->
List[Tuple[Task, Opti
    onal[Server]]]:
        sorted_tasks = sorted(tasks, key=lambda t: (t.priority,
t.load),
     reverse=True)
        return [self.schedule_task(task) for task in sorted_tasks]

    def batch_schedule(self, tasks: List[Task]) ->
List[Tuple[Task, Opti
    onal[Server]]]:
        return self.schedule_tasks(tasks)

    def get_name(self) -> str:
        return "greedy"
```

15. src/algorithms/plugin_adapter.py

```
"""插件算法适配器

把「插件形态的第三方调度算法」适配成核心调度器接口，使其能够进入
```

```
DataCenterSimulator 的常规仿真与横向对比回路，而不需要修改插件本身。
"""
from typing import Dict, List, Optional, Tuple

from src.algorithms.base import BaseScheduler
from src.models import Server, Task
from src.utils.logger import setup_logger

logger = setup_logger("plugin_adapter")

PLUGIN_ALGORITHMS: Dict[str, Dict[str, object]] = {
    "nsga2": {
        "plugin": "NSGA2SchedulerPlugin",
        "config": {"pop_size": 12, "n_gen": 10, "selection": "balanced",
         "seed": 42},
    },
    "moead": {
        "plugin": "MOEADSchedulerPlugin",
        "config": {"n_partitions": 4, "n_neighbors": 5, "n_gen": 10, "se
    lection": "balanced", "seed": 42},
    },
}


def is_plugin_algorithm(name: str) -> bool:
    return name in PLUGIN_ALGORITHMS


class PluginSchedulerAdapter(BaseScheduler):
    """把插件的 assignments 结果翻译成核心调度器的 (task, server) 语义。"
    ""

    def __init__(self, servers: List[Server], plugin_name: str,
                 config: Optional[Dict[str, object]] = None) -> None:
        super().__init__(servers)
        self.plugin_name = plugin_name
        self.config = dict(config or {})
        self._manager = None

    def get_name(self) -> str:
        return self.plugin_name
```

```
 42|
 43|    def _ensure_plugin(self):
 44|        if self._manager is not None:
 45|            return self._manager
 46|        from src.plugin import PluginManager
 47|
 48|        manager = PluginManager()
 49|        manager.discover_plugins(plugin_dir="src/plugins")
 50|        if self.plugin_name not in manager._plugin_classes:
 51|            raise RuntimeError("插件未找到: %s" % self.plugin_name)
 52|        if not manager.load_plugin(self.plugin_name, self.config):
 53|            raise RuntimeError("插件加载失败: %s" % self.plugin_name)
 54|        self._manager = manager
 55|        return manager
 56|
 57|    def _fallback(self, tasks: List[Task]) -> List[Tuple[Task,
Optional[
   |    Server]]]:
 58|        from src.algorithms.greedy_algorithm import
GreedyScheduler
 59|
 60|        try:
 61|            return GreedyScheduler(self.servers).schedule_tasks(ta
sks)
 62|        except Exception:
 63|            return [(task, None) for task in tasks]
 64|
 65|    def schedule_tasks(self, tasks: List[Task]) ->
List[Tuple[Task, Opti
   |    onal[Server]]]:
 66|        if not tasks:
 67|            return []
 68|
 69|        try:
 70|            manager = self._ensure_plugin()
 71|            result = manager.execute_plugin(self.plugin_name,
tasks=task
   |    s, servers=self.servers)
 72|            if not result.success:
 73|                raise RuntimeError("插件调度失败: %s [%s]" % (result.mes
   |    sage, result.error_code))
 74|
 75|            assignments = (result.data or {}).get("assignments",
[])
```

```
            by_id = {task.task_id: task for task in tasks}
            placements: List[Tuple[Task, Optional[Server]]] = []
            placed = set()

            for item in assignments:
                task = by_id.get(str(item.get("task_id")))
                if task is None or task.task_id in placed:
                    continue
                index = int(item.get("server_id", -1))
                server = self.servers[index] if 0 <= index <
len(self.se
    rvers) else None
                if server is not None and not
server.is_overloaded() and
     not server.is_overheated():
                    if server.assign_task(task):
                        placements.append((task, server))
                        placed.add(task.task_id)
                        continue

                placements.append((task, None))
                placed.add(task.task_id)

            pending = [task for task, server in placements if
server is
    None]
            if pending:
                placements = [(task, server) for task, server in
placeme
    nts if server is not None]
                for task in pending:
                    candidates = [
                        server for server in self.servers
                        if not server.is_overloaded() and not
server.is_
    overheated()
                    ]
                    target = min(candidates, key=lambda server:
server.c
    urrent_load, default=None)
                    if target is not None and
target.assign_task(task):
                        placements.append((task, target))
                    else:
```

```
                        placements.append((task, None))

            return placements
        except Exception as exc:
            logger.warning("插件 %s 调度失败，回退贪心: %s", self.plugin_name, exc)
            return self._fallback(tasks)
```

16. src/algorithms/round_robin.py

```
from typing import List, Tuple, Optional
from src.models import Task, Server
from src.algorithms.base import BaseScheduler

class RoundRobinScheduler(BaseScheduler):
    def __init__(self, servers: List[Server]) -> None:
        super().__init__(servers)
        self._next_idx: int = 0

    def schedule_tasks(self, tasks: List[Task]) -> List[Tuple[Task, Optional[Server]]]:
        results = []
        for task in tasks:
            attempts = 0
            while attempts < len(self.servers):
                server = self.servers[self._next_idx % len(self.servers)]
                self._next_idx += 1
                if not server.is_overloaded() and not server.is_overheated():
                    if server.assign_task(task):
                        results.append((task, server))
                        break
                attempts += 1
            else:
                results.append((task, None))
        return results

    def get_name(self) -> str:
        return "round_robin"
```

17. src/api/__init__.py

```
from .app import app, run

__all__ = ['app', 'run']
```

18. src/api/ai_api.py - AI 助手、预测和寻优接口

```
"""AI 诊断与自然语言参数解析接口"""
from typing import Any, Dict

from flask import Blueprint, jsonify, request

import json
import re
import os
from datetime import datetime

import yaml

from src.services.ai_service import get_ai_service,
humanize_algorithms,
    |     rule_chat, rule_parse
from src.services import run_store
from src.services.plan_optimizer import get_plan_optimizer
from src.services.run_manager import get_run_manager
from src.services.simulation_insights import build_insights
from src.services.weight_tuner import WEIGHT_KEYS, run_sweep
from src.simulation.metrics import rank_algorithms, summarize
from src.utils import load_run_metadata, load_simulation_data
from src.utils.config import Config
from src.utils.logger import setup_logger
from src.utils.validation import validate_int,
validate_simulation_param
    |    s

logger = setup_logger("ai_api")

ai_bp = Blueprint("ai", __name__)

WEIGHT_KEYS = (
    "alpha_deadline",
    "beta_energy",
    "gamma_thermal",
    "delta_heterogeneity",
    "epsilon_migration",
)
```

```
36|
37|
38|def _current_stats(df=None) -> Dict[str, Dict[str, Any]]:
39|    if df is None:
40|        df = load_simulation_data()
41|    if df is None:
42|        return {}
43|    return summarize(df)
44|
45|
46|def _current_weights() -> Dict[str, float]:
47|    section = Config().get("damo_ets", {}) or {}
48|    weights: Dict[str, float] = {}
49|    for key in WEIGHT_KEYS:
50|        try:
51|            weights[key] = float(section.get(key))
52|        except (TypeError, ValueError, AttributeError):
53|            continue
54|    return weights
55|
56|
57|def _config_snapshot() -> Dict[str, Any]:
58|    config = Config()
59|    simulation = config.get("simulation", {}) or {}
60|    cooling = config.get("cooling", {}) or {}
61|    energy = config.get("energy", {}) or {}
62|    damo = config.get("damo_ets", {}) or {}
63|    return {
64|        "num_server_range": [
65|            simulation.get("min_servers"),
66|            simulation.get("max_servers"),
67|        ],
68|        "simulation_hour_range": [
69|            simulation.get("min_simulation_hours"),
70|            simulation.get("max_simulation_hours"),
71|        ],
72|        "sla_slack": simulation.get("sla_slack"),
73|        "fan_speed_range": cooling.get("fan_speed_range"),
74|        "fan_target_temp": cooling.get("fan_target_temp"),
75|        "cooling_efficiency": cooling.get("cooling_efficiency"),
76|        "idle_power_factor": energy.get("idle_power_factor"),
77|        "part_load_penalty": energy.get("part_load_penalty"),
78|        "temp_reference": energy.get("temp_reference"),
```

```
 79|        "damo_ets_weights": {key: damo.get(key) for key in
WEIGHT_KEYS},
 80|        "server_types": config.get("server_types", []),
 81|    }
 82|
 83|
 84|def _suggest_run(text: str, context: Dict[str, Any]) -> Dict[str,
Any] |
   |     None:
 85|    triggers = ("如果", "改成", "调到", "增加到", "减少到", "降到", "提
   |    高到", "换成", "试算", "跑一下", "试试", "台服务器")
 86|    if not any(word in text for word in triggers):
 87|        return None
 88|    snapshot = context.get("data_snapshot") or {}
 89|    defaults = {
 90|        "num_servers": snapshot.get("num_servers"),
 91|        "simulation_hours": snapshot.get("simulation_hours"),
 92|        "algorithms": snapshot.get("algorithms") or [],
 93|        "run_mode": "standard",
 94|        "site_id": snapshot.get("site"),
 95|        "school_name": snapshot.get("school_name"),
 96|    }
 97|    parsed = rule_parse(text, defaults=defaults)
 98|    params = dict(parsed.get("params") or {})
 99|
100|    try:
101|        from src.data.catalog import Catalog
102|        catalog = Catalog()
103|        if catalog.available():
104|            matches = [
105|                school for school in catalog.schools(limit=5000)
106|                if school.get("school_name") and
school["school_name"] i
   |    n text
107|            ]
108|            if matches:
109|                school = max(matches, key=lambda item:
len(item["school_
   |    name"]))
110|                params["school_name"] = school["school_name"]
111|                params["site_id"] = school.get("city") or
params.get("si
   |    te_id")
112|    except Exception:
```

```
113|        pass
114|
115|    changed = {}
116|    for key in ("num_servers", "simulation_hours", "algorithms", "
run_mo
   |    de", "site_id", "school_name"):
117|        if params.get(key) != defaults.get(key):
118|            changed[key] = params.get(key)
119|    if not changed:
120|        return None
121|    if not params.get("algorithms"):
122|        params["algorithms"] = list(defaults.get("algorithms") or
[])
123|    params["visualization"] = True
124|    params["save_results"] = True
125|    if snapshot.get("site") and not params.get("site_id"):
126|        params["site_id"] = snapshot.get("site")
127|    label_parts = []
128|    if changed.get("school_name"):
129|        label_parts.append(str(changed["school_name"]))
130|    elif changed.get("site_id"):
131|        label_parts.append(str(changed["site_id"]))
132|    if "num_servers" in changed:
133|        label_parts.append("%s 台" % changed["num_servers"])
134|    if "simulation_hours" in changed:
135|        label_parts.append("%s 小时" % changed["simulation_hours"])
136|    if "algorithms" in changed:
137|        label_parts.append("%d 个算法" % len(changed["algorithms"]))
138|    label = "按 " + " / ".join(label_parts) + " 试算" if label_parts els
   |    e "按建议配置试算"
139|    current_algorithms = set(defaults.get("algorithms") or [])
140|    suggested_algorithms = set(params.get("algorithms") or [])
141|    already_matches_latest = (
142|        params.get("num_servers") == defaults.get("num_servers")
143|        and params.get("simulation_hours") ==
defaults.get("simulation_h
   |    ours")
144|        and params.get("site_id") == defaults.get("site_id")
145|        and (params.get("school_name") or None) ==
(defaults.get("school
   |    _name") or None)
146|        and suggested_algorithms == current_algorithms
147|    )
148|    reason = (
```

```
        "当前最新运行已经是这个配置，可以直接分析现有结果。"
        if already_matches_latest
        else "当前最新运行不是这个配置，需要按建议参数重新运行才能得到实测结果。"
    )
    return {
        "label": label,
        "params": params,
        "changed": changed,
        "notes": parsed.get("notes", []),
        "already_matches_latest": already_matches_latest,
        "reason": reason,
    }


ALGORITHM_WORDS = {
    "greedy": ("贪心", "greedy"),
    "genetic": ("遗传", "genetic"),
    "round_robin": ("轮询", "round", "robin"),
    "damo_ets": ("damo", "多目标能效"),
    "nsga2": ("nsga", "nsga-ii", "nsga2"),
    "moead": ("moea", "moead", "moea-d"),
}


def _suggest_optimization(text: str, context: Dict[str, Any]) -> Dict[st
r, Any] | None:
    normalized = str(text or "").strip()
    if not normalized:
        return None
    lowered = normalized.lower()
    range_match = re.search(r"(\d+)\s*(?:到|至|-|—|~)\s*(\d+)\s*台", nor
malized)
    service_match = re.search(r"服务率[^0-9]{0,8}(\d+(?:\.\d+)?)\s*%", n
ormalized)
    temp_match = re.search(r"(?:温度|峰值温度|最高温度)[^0-9]{0,8}(?:不
超过|不高于|低于|小于|≤|<=)\s*(\d+(?:\.\d+)?)", normalized)
    explicit = any(word in normalized for word in (
        "全局寻优", "方案寻优", "自动寻优", "配置寻优", "最优方案", "最
佳方案",
        "最优配置", "方案最优", "最省电的方案", "最划算的方案", "性价比
最高的方案",
    ))
```

```
    if not explicit and not (range_match and (service_match or
temp_matc
    h)):
        return None

    snapshot = context.get("data_snapshot") or {}
    default_servers = int(snapshot.get("num_servers") or 48)
    default_hours = int(snapshot.get("simulation_hours") or 24)
    server_min = default_servers
    server_max = max(default_servers, 80)
    if range_match:
        server_min = int(range_match.group(1))
        server_max = int(range_match.group(2))
    else:
        single = re.search(r"(?<!\d)(\d+)\s*台", normalized)
        if single:
            server_min = server_max = int(single.group(1))
    if server_min > server_max:
        server_min, server_max = server_max, server_min

    hours_match = re.search(r"(\d+)\s*(?:个)?小时", normalized)
    hours = int(hours_match.group(1)) if hours_match else
default_hours
    hours = max(1, min(72, hours))

    floor = 0.98
    if service_match:
        value = float(service_match.group(1))
        floor = value / 100.0 if value > 1.0 else value
        floor = max(0.8, min(1.0, floor))

    max_temp = None
    if temp_match:
        max_temp = max(20.0, min(100.0,
float(temp_match.group(1))))

    if "pue" in lowered:
        target = "pue"
    elif any(word in normalized for word in ("温度最低", "最凉", "散热最
    好", "温度优先")):
        target = "temp"
    elif any(word in normalized for word in ("最省电", "能耗最低", "能耗
    最小", "用电最低", "能耗优先")):
        target = "energy"
```

```
    else:
        target = "balanced"

    algorithms = []
    for key, words in ALGORITHM_WORDS.items():
        if any(word in lowered for word in words):
            algorithms.append(key)
    if any(word in normalized for word in ("全部算法", "六种算法", "六个算法", "全算法")):
        algorithms = ["greedy", "genetic", "round_robin", "damo_ets", "nsga2", "moead"]
    if not algorithms:
        algorithms = ["greedy", "genetic", "round_robin", "damo_ets"]

    site_id = snapshot.get("site") or "北京市"
    school_name = snapshot.get("school_name") or ""
    try:
        from src.data.catalog import Catalog
        catalog = Catalog()
        if catalog.available():
            matches = [
                school for school in catalog.schools(limit=5000)
                if school.get("school_name") and school["school_name"] in normalized
            ]
            if matches:
                school = max(matches, key=lambda item: len(item["school_name"]))
                school_name = school["school_name"]
                site_id = school.get("city") or site_id
    except Exception:
        pass

    params = {
        "server_min": server_min,
        "server_max": server_max,
        "simulation_hours": hours,
        "algorithms": algorithms,
        "target": target,
        "service_rate_floor": floor,
```

```
        "max_temp": max_temp,
        "site_id": site_id,
        "school_name": school_name,
        "include_weights": "不调权重" not in normalized and "不用权重" not in normalized,
    }
    target_text = {
        "pue": "PUE 最低",
        "energy": "单位能耗最低",
        "temp": "峰值温度最低",
        "balanced": "综合均衡",
    }[target]
    label = "在 %d–%d 台、%d 小时内寻找%s方案" % (server_min, server_max,
     hours, target_text)
    return {"params": params, "label": label}


def _optimization_reply(job: Dict[str, Any]) -> str:
    params = job.get("params") or {}
    return "已启动全局寻优：在 %d–%d 台、%d 小时内评估 %d 种算法，服务率门
    槛 %.1f%%%s。结果会显示在下方寻优卡片里。" % (
        params.get("server_min", 0),
        params.get("server_max", 0),
        params.get("simulation_hours", 0),
        len(params.get("algorithms") or []),
        float(params.get("service_rate_floor") or 0.98) * 100.0,
        ("，峰值温度不超过 %.1f℃" % params["max_temp"]) if params.get("ma
    x_temp") is not None else "",
    )


CHAT_SYSTEM = """你是"校园数据中心智能散热与算力调度系统"的 AI 助手（名字
    ：小助）。
规则：
1. 只依据下面提供的"当前数据"回答，不得编造数字；数据里没有的项直接说"当前
    数据没有这一项"。
2. 用中文自然语言回答，**不要出现内部字段名**（如 max_temp、avg_pue、energy_
    per_served_load service_rate
   要说"峰值温度""平均 PUE""单位负载能耗""任务服务率"。
3. 先给一句结论，再列数据；列表最多 5 条，避免大段复述。
```

4. 当用户问"数据从哪来"时，依据当前数据里的 provenance 字段回答发布方与数据集名称（例如 ERA5 气温、阿里云集群轨迹、生态环境部碳因子），不要笼统说"目录数据"

5. 算法名称一律用中文或规范写法：贪心算法、遗传算法、轮询算法、DAMO-ETS、NSGA-II MOEA-D

不要写 greedy、genetic、round_robin、damo_ets 这类代码标识符。

6. 温度保留 1 位小数，PUE 保留 3 位小数，百分比保留 2 位小数。

7. 用户问怎么操作时，依据当前数据里的 ui_hints 告诉他点页面哪里；不要假装已经执行。

8. 严格区分两个指标：dropped_tasks 是"未被分配到服务器"的任务，deadline_violations 是"完成超时"的任务，

不要把"未分配"说成"超时违约"，也不要把两者相加。

9. 当前数据里的 insights 是从最近一次仿真结果实时计算的。用户问"什么时候性价比最高""哪个时段最好"时，

优先使用 safe_best_hours（已排除峰值温度达到 70℃ 或出现过热服务器的时段），给出具体小时、算法、

单位服务负载能耗、温度和 PUE；如果 overall_best_hours 的第一名温度过高，

要单独提醒，不要直接推荐。

insights 里的 hour 是从 0 开始的时间索引，回答时换算成"第 hour+1 小时"

10. 用户问"温度低于多少度怎么配置"时，使用 temperature_thresholds 或 temperature_bands，给出该温区的小时数、

最优算法、服务率、风扇转速和 PUE；同时说明这是当前配置下的观测结果，不是

改变硬件后的反事实预测。

11. 用户问"数据更新了吗""这次运行是什么时候"时，使用 snapshot 里的 generated_at site num_servers

simulation_hours 和 algorithms 回答，不要沿用历史对话里的旧数字。

12. 用户的问题不限于预设示例。只要与当前仿真数据、调度、能耗、温度、PUE、服务

率、配置调整有关，

都要基于当前数据回答；不要要求用户只能问某几类问题。

13. 遇到当前数据没有实际运行过的配置调整（服务器数量、风扇曲线、冷却策略、算

法权重等），按三层回答：

(1) 当前数据观察到的关系和数值；(2) 调整后的方向、可能收益和风险；(3)

需要用哪些参数重新仿真确认。

不得把推测写成实测结果；没有数据时直接说"当前数据没有这一项"。

只能建议用户操作 context.capabilities.ui_limits 里列出的网页表单参数；风扇曲线、冷却效率、权重不在表单里，
应说明需要修改配置文件（config 目录下的 yaml 文件）后重新运行，不要说"在页面上直接调"。
14. context.config 给出了当前物理模型和参数边界；context.insights.relationships 只是相关性，不是因果结论。
15. 如果 context.suggested_run 存在，先看 already_matches_latest：
- true：当前最新运行已经是这个配置，直接基于现有数据分析，不要再说需要重跑；
- false：当前数据不是这个配置，明确说明差异（尤其是学校、台数、时长、算法），并告诉用户可以点消息下方的"按此配置试算"按钮验证。
16. 如果 context.executed_run.status 是 success，说明本轮已经实际运行完成，必须直接使用最新 snapshot/insights 回答，并提到"运行编号"；不要再说"需要重新运行"。
17. 如果 context.executed_run.status 是 running，说明任务已经启动但还没完成，不要给出未完成的数值，只说明进度和稍后查看。
18. 如果 context.executed_run.status 是 failed，说明失败原因，不要编造结果
19. 如果 context.auto_run_budget.allowed 是 false，说明为什么没有自动执行，并提示用户点"按此配置试算"或把参数调整到自动试算预算内。
20. 用户问"上次/最近几次/对比/变化"时，使用 context.recent_runs 和 context.latest_comparison
latest_comparison.left 是较早一次，right 是较新一次。只引用真实记录，不要编造历史。
21. 回答中使用"网页表单"，不要写"Web 表单"；配置文件统一写成"配置文件"或"配置文件（config 目录下的 yaml 文件）"。
22. 用户提到学校时，按该学校所在城市运行，并在回答中写出学校名称；只使用本站数据目录中的学校、城市、气温和仿真结果。
23. 你是本站专用助手，只操控本站的仿真、数据和运行历史，不访问互联网，不执行任意代码，不回答与本站无关的通用任务。
24. 只有在 snapshot.school_name 与用户提到的学校一致时，才能说该学校已经跑过；否则必须说明当前运行没有记录这个学校，建议按该校所在城市重新试算。
25. 面向用户时不要出现 run_id、runId 这类字段名，统一写"运行编号"；编号本身

```
    |    保持原样，不要翻译或改写。
334|当前数据：
335|"""
336|
337|
338|def _data_provenance() -> list:
339|    try:
340|        from src.data.catalog import Catalog
341|        catalog = Catalog()
342|        if not catalog.available():
343|            return []
344|        return [
345|            {
346|                "publisher": row.get("publisher"),
347|                "name": row.get("name"),
348|                "url": row.get("url"),
349|            }
350|            for row in catalog.provenance()
351|        ]
352|    except Exception as exc:
353|        logger.warning("读取数据出处失败: %s" % exc)
354|        return []
355|
356|
357|def _auto_run_budget(params: Dict[str, Any]) -> Dict[str, Any]:
358|    manager = get_run_manager()
359|    active = manager.active_run()
360|    if active:
361|        return {
362|            "allowed": False,
363|            "max_servers": 100,
364|            "max_hours": 48,
365|            "reason": "当前已有运行任务（运行编号 %s）正在进行，请等它完
成
    |    后再试。" % active.get("run_id"),
366|        }
367|    num_servers = int(params.get("num_servers") or 0)
368|    hours = int(params.get("simulation_hours") or 0)
369|    if num_servers > 100 or hours > 48:
370|        return {
371|            "allowed": False,
372|            "max_servers": 100,
373|            "max_hours": 48,
374|            "reason": "自动试算预算为最多 100 台、48 小时，当前请求是 %d
```

```
    台、%d 小时。" % (
375|                num_servers, hours
376|            ),
377|        }
378|    return {
379|        "allowed": True,
380|        "max_servers": 100,
381|        "max_hours": 48,
382|        "reason": "",
383|    }
384|
385|
386|def _run_status_text(executed: Dict[str, Any], context: Dict[str,
Any])
    -> str:
387|    params = executed.get("params") or {}
388|    label = executed.get("label") or "建议配置"
389|    run_id = executed.get("run_id") or "unknown"
390|    status = executed.get("status")
391|    if status == "success":
392|        return "已按 %s 完成运行（运行编号：%s），页面数据已经同步。" %
(labe
    l, run_id)
393|    if status == "failed":
394|        return "我尝试按 %s 运行（运行编号：%s），但失败了：%s。可以调整
参数后
    重试。" % (
395|            label, run_id, executed.get("error") or "未知错误"
396|        )
397|    return "已按 %s 启动运行（运行编号：%s），当前仍在执行。运行完成后页
面会
    自动刷新，你可以稍后再问我结果。" % (
398|        label, run_id
399|    )
400|
401|
402|def _compact_run(record: Dict[str, Any]) -> Dict[str, Any]:
403|    return {
404|        "run_id": record.get("run_id"),
405|        "origin": record.get("origin"),
406|        "status": record.get("status"),
407|        "created_at": record.get("created_at"),
408|        "finished_at": record.get("finished_at"),
409|        "params": record.get("params") or {},
```

```
        "summary": record.get("summary") or {},
    }


def _chat_context() -> Dict[str, Any]:
    df = load_simulation_data()
    stats = _current_stats(df)
    meta = load_run_metadata() or {}
    insights = build_insights(df, meta)
    recent_runs = [
        _compact_run(item) for item in run_store.list_runs(10)
        if item.get("status") == "success"
    ][:5]
    latest_comparison = None
    if len(recent_runs) >= 2:
        raw_comparison = run_store.compare_runs(
            recent_runs[1]["run_id"], recent_runs[0]["run_id"]
        )
        if raw_comparison:
            latest_comparison = {
                "left": _compact_run(raw_comparison["left"]),
                "right": _compact_run(raw_comparison["right"]),
                "metrics": raw_comparison["metrics"],
                "rule": raw_comparison["rule"],
            }
    return {
        "stats": stats,
        "ranking": rank_algorithms(stats) if stats else {},
        "insights": insights,
        "data_snapshot": insights.get("snapshot", {}),
        "config": _config_snapshot(),
        "capabilities": {
            "free_form": "可以回答任何与当前仿真数据、调度、能耗、温度、PUE、
服务率和配置调整有关的问题，不限于预设示例。",
            "observed": "当前数据能直接回答已经运行过的结果、时段、温区和
算法对比。",
            "requires_rerun": "改变服务器数量、风扇曲线、冷却策略或算法权
重后的具体数值，需要重新仿真验证。",
            "scope": "只使用本站数据目录、仿真结果和运行历史，不访问互联
网
，不执行任意代码，也不操作本站以外的系统。",
            "ui_limits": "网页表单可改服务器数量、仿真时长、运行模式、服
务
```

```
    器类型、算法和城市；风扇曲线、冷却效率、DAMO-ETS 权重需要修改配置文件（co
    nfig 目录下的 yaml 文件）后重新运行。",
        },
        "recent_runs": recent_runs,
        "latest_comparison": latest_comparison,
        "data_source": {
            "site_id": meta.get("site_id"),
            "school_name": meta.get("school_name"),
            "site_province": meta.get("site_province"),
            "emission_region": meta.get("emission_region"),
            "ambient_source": meta.get("ambient_source"),
            "scenario_source": meta.get("scenario_source"),
            "scenario_digest": meta.get("scenario_digest"),
            "service_rate_floor": meta.get("service_rate_floor"),
        },
        "provenance": _data_provenance(),
        "ui_hints": {
            "run": "在「仿真控制」区块：先选所在城市（可输入一个字搜索），可选
    学校，勾选算法，然后点「开始运行」。",
            "results": "结果在下方「数据分析」的 6 张图表、「算法指标对比」表
    格和「受控排名」列表里。",
            "assistant": "页面右下角悬浮球打开本对话；面板内的麦克风按钮
    可直接语音输入。",
        },
        "num_servers": meta.get("num_servers"),
        "simulation_hours": meta.get("simulation_hours"),
        "controlled": meta.get("controlled"),
    }


@ai_bp.route("/api/v1/ai/chat", methods=["POST"])
def ai_chat():
    data = request.json or {}
    raw_messages = data.get("messages") or []
    if not isinstance(raw_messages, list) or not raw_messages:
        return jsonify({"success": False, "message": "messages 不能为空"
    }), 400

    messages = []
    for item in raw_messages[-10:]:
        if not isinstance(item, dict):
```

```
482|            continue
483|        role = item.get("role")
484|        content = str(item.get("content") or "").strip()
485|        if role in ("user", "assistant") and content:
486|            messages.append({"role": role, "content":
content[:2000]})
487|    if not messages:
488|        return jsonify({"success": False, "message": "没有有效的对话内容
   |    "}), 400
489|
490|    context = _chat_context()
491|    latest_user = next((item["content"] for item in
reversed(messages) i
   |    f item["role"] == "user"), "")
492|    optimization_job = None
493|    optimization_suggestion = _suggest_optimization(latest_user,
context
   |    )
494|    if optimization_suggestion:
495|        try:
496|            optimizer = get_plan_optimizer()
497|            optimization_job = optimizer.active() or
optimizer.start(opt
   |    imization_suggestion["params"])
498|            context["optimization"] = {
499|                "job_id": optimization_job.get("job_id"),
500|                "status": optimization_job.get("status"),
501|                "label": optimization_suggestion.get("label"),
502|                "params": optimization_job.get("params"),
503|            }
504|        except Exception as exc:
505|            logger.warning("启动全局寻优失败: %s" % exc)
506|            context["optimization_error"] = str(exc)
507|    suggested = None if optimization_job else
_suggest_run(latest_user,
   |    context)
508|    context["suggested_run"] = suggested
509|
510|    auto_run = bool(data.get("auto_run"))
511|    executed = None
512|    if auto_run and suggested:
513|        budget = _auto_run_budget(suggested["params"])
514|        context["auto_run_budget"] = budget
515|        if budget.get("allowed"):
```

```
516|            manager = get_run_manager()
517|            run_id = manager.submit(suggested["params"],
origin="ai")
518|            record = manager.wait(run_id, timeout=120) or {}
519|            executed = {
520|                "run_id": run_id,
521|                "status": record.get("status") or "running",
522|                "params": suggested["params"],
523|                "label": suggested["label"],
524|                "summary": record.get("summary") or {},
525|                "error": record.get("error") or "",
526|            }
527|            if executed["status"] == "success":
528|                context = _chat_context()
529|                context["executed_run"] = executed
530|                context["suggested_run"] = None
531|            else:
532|                context["executed_run"] = executed
533|                context["suggested_run"] = suggested
534|
535|    service = get_ai_service()
536|    service.reload_key()
537|    system = CHAT_SYSTEM + json.dumps(context,
ensure_ascii=False, defau
   |    lt=str)
538|
539|    if optimization_job:
540|        return jsonify({
541|            "success": True,
542|            "reply": _optimization_reply(optimization_job),
543|            "source": "optimizer",
544|            "model": get_ai_service().model,
545|            "key_source": get_ai_service().key_source,
546|            "snapshot": context.get("insights",
{}).get("snapshot", {}),
547|            "suggested_run": None,
548|            "run": None,
549|            "auto_run": auto_run,
550|            "budget": None,
551|            "optimization": optimization_job,
552|        })
553|
554|    source = "deepseek"
555|    try:
```

```
556|        reply = humanize_algorithms(service.chat_messages(system,
messag
   |    es))
557|    except Exception as exc:
558|        logger.warning("AI 对话回退本地规则: %s" % exc)
559|        reply = rule_chat(messages[-1]["content"], context)
560|        if executed:
561|            if executed["status"] == "success":
562|                reply = _run_status_text(executed, context) + "
\n" + rep
   |    ly
563|            else:
564|                reply = _run_status_text(executed, context)
565|        source = "rule"
566|
567|    return jsonify({
568|        "success": True,
569|        "reply": reply,
570|        "source": source,
571|        "model": service.model,
572|        "key_source": service.key_source,
573|        "snapshot": context.get("insights", {}).get("snapshot",
{}),
574|        "suggested_run": context.get("suggested_run"),
575|        "run": executed,
576|        "auto_run": auto_run,
577|        "budget": context.get("auto_run_budget"),
578|        "optimization": optimization_job,
579|        "optimization_error": context.get("optimization_error"),
580|    })
581|
582|
583|@ai_bp.route("/api/v1/ai/status", methods=["GET"])
584|def ai_status():
585|    service = get_ai_service()
586|    service.reload_key()
587|    return jsonify({"success": True, "status": service.status()})
588|
589|
590|@ai_bp.route("/api/v1/ai/analyze", methods=["POST"])
591|def ai_analyze():
592|    stats = _current_stats()
593|    if not stats:
594|        return jsonify({"success": False, "message": "暂无仿真数据，请先
```

```
    |    运行一次仿真"}), 404
595|    service = get_ai_service()
596|    service.reload_key()
597|    try:
598|        result = service.analyze(stats, _current_weights())
599|        result["metrics"] = stats
600|        return jsonify({"success": True, "result": result})
601|    except Exception as exc:
602|        logger.error("AI 诊断失败: %s" % exc)
603|        return jsonify({"success": False, "message": "诊断服务异常"}), 5
    |    00
604|
605|
606|@ai_bp.route("/api/v1/ai/parse", methods=["POST"])
607|def ai_parse():
608|    data = request.json or {}
609|    text = str(data.get("text", "")).strip()
610|    if not text:
611|        return jsonify({"success": False, "message": "请输入一句需求描述
    |    "}), 400
612|    if len(text) > 200:
613|        return jsonify({"success": False, "message": "描述请控制在 200
    |    字以内"}), 400
614|    service = get_ai_service()
615|    service.reload_key()
616|    try:
617|        parsed = service.parse_intent(text)
618|        params = validate_simulation_params(parsed.get("params",
{}))
619|        params["visualization"] = True
620|        params["save_results"] = True
621|        return jsonify(
622|            {
623|                "success": True,
624|                "params": params,
625|                "notes": parsed.get("notes", []),
626|                "source": parsed.get("source", "rule"),
627|                "notice": parsed.get("notice", ""),
628|            }
629|        )
630|    except Exception as exc:
631|        logger.error("参数解析失败: %s" % exc)
632|        return jsonify({"success": False, "message": "无法解析为合法参数
    |    : %s" % exc}), 400
```

```
633|
634|
635|def _env_config_path() -> str:
636|    root = os.path.join(os.path.dirname(os.path.abspath(__file__)),
 ".."
   |    , "..", "config")
637|    env = os.environ.get("DCS_ENV", "development")
638|    return os.path.abspath(os.path.join(root, "%s.yaml" % env))
639|
640|
641|def _apply_weights(weights: Dict[str, float]) -> Dict[str, Any]:
642|    path = _env_config_path()
643|    with open(path, "r", encoding="utf-8") as handle:
644|        text = handle.read()
645|    previous = dict(Config().get("damo_ets", {}) or {})
646|
647|    lines = ["  %s: %s" % (key, weights[key]) for key in
WEIGHT_KEYS if
   |    key in weights]
648|    if len(lines) != len(WEIGHT_KEYS):
649|        raise ValueError("权重参数不完整")
650|
651|    if "\ndamo_ets:" in text or text.startswith("damo_ets:"):
652|        parts = text.split("damo_ets:", 1)
653|        head = parts[0].rstrip("\n")
654|        rest = parts[1].split("\n")
655|        kept = []
656|        for line in rest:
657|            stripped = line.strip()
658|            if not stripped:
659|                continue
660|            if line.startswith("  ", "\t")) and
any(stripped.startswith
   |    (key + ":") for key in WEIGHT_KEYS):
661|                continue
662|            if not line.startswith("  ", "\t")):
663|                kept.append("__BREAK__")
664|                kept.append(line)
665|                continue
666|            kept.append(line)
667|        body = []
668|        for line in kept:
669|            if line == "__BREAK__":
670|                body.extend(lines)
```

```
                break
            body.append(line)
        else:
            body.extend(lines)
        new_text = head + "\ndamo_ets:\n" + "\n".join(body)
        tail_lines = []
        if "__BREAK__" in kept:
            index = kept.index("__BREAK__")
            tail_lines = kept[index + 1:]
            new_text = head + "\ndamo_ets:\n" + "\n".join(body) + "\n" +
     "\n".join(tail_lines)
    else:
        new_text = text.rstrip("\n") + "\n\ndamo_ets:\n" + "\n".join(lin
    es) + "\n"

    parsed = yaml.safe_load(new_text)
    if not isinstance(parsed, dict) or "damo_ets" not in parsed:
        raise ValueError("写入后的配置无法解析")
    with open(path, "w", encoding="utf-8", newline="") as handle:
        handle.write(new_text)
    Config._instance = None
    return {"path": path, "previous": previous, "applied":
weights}


@ai_bp.route("/api/v1/ai/tune", methods=["POST"])
def ai_tune():
    data = request.json or {}
    try:
        servers = validate_int(data.get("num_servers"),
min_val=10, max_
    val=1000, default=100, name="服务器数量")
        hours = validate_int(data.get("simulation_hours"),
min_val=1, ma
    x_val=720, default=24, name="仿真时长")
    except Exception as exc:
        return jsonify({"success": False, "message": "参数不合法: %s" %
    exc}), 400

    service = get_ai_service()
    service.reload_key()
    try:
```

```
        sweep = run_sweep(servers, hours, 42)
        sweep["explanation"] = service.explain_tuning(sweep)
        return jsonify({"success": True, "result": sweep})
    except Exception as exc:
        logger.error("权重寻优失败: %s" % exc)
        return jsonify({"success": False, "message": "寻优失败: %s" % ex
    c}), 500


@ai_bp.route("/api/v1/ai/apply-tuning", methods=["POST"])
def ai_apply_tuning():
    data = request.json or {}
    weights = data.get("weights") or {}
    cleaned: Dict[str, float] = {}
    try:
        for key in WEIGHT_KEYS:
            cleaned[key] = round(float(weights.get(key)), 4)
    except (TypeError, ValueError):
        return jsonify({"success": False, "message": "权重取值不合法"}),
     400
    total = sum(cleaned.values())
    if total <= 0 or abs(total - 1.0) > 0.02:
        return jsonify({"success": False, "message": "权重之和应接近 1.0
    , 当前为 %.3f" % total}), 400
    try:
        info = _apply_weights(cleaned)
        return jsonify({"success": True, "result": info})
    except Exception as exc:
        logger.error("写入权重失败: %s" % exc)
        return jsonify({"success": False, "message": "写入配置失败: %s"
    % exc}), 500


@ai_bp.route("/api/v1/ai/optimize", methods=["POST"])
def ai_optimize_start():
    data = request.json or {}
    try:
        optimizer = get_plan_optimizer()
        job = optimizer.active() or optimizer.start(data)
        return jsonify({"success": True, "job": job})
    except ValueError as exc:
        return jsonify({"success": False, "message": str(exc)}),
400
    except Exception as exc:
        logger.error("启动全局寻优失败: %s" % exc)
```

```
744|        return jsonify({"success": False, "message": "启动寻优失败: %s"
   |    % exc}), 500
745|
746|
747|@ai_bp.route("/api/v1/ai/optimize/<job_id>", methods=["GET"])
748|def ai_optimize_status(job_id):
749|    if not re.match(r"^[a-f0-9]{12}$", str(job_id or "")):
750|        return jsonify({"success": False, "message": "任务编号无效"}), 4
   |    00
751|    job = get_plan_optimizer().snapshot(job_id)
752|    if not job:
753|        return jsonify({"success": False, "message": "寻优任务不存在"}),
   |     404
754|    return jsonify({"success": True, "job": job})
755|
756|
757|@ai_bp.route("/api/v1/ai/optimize/<job_id>/apply",
methods=["POST"])
758|def ai_optimize_apply(job_id):
759|    if not re.match(r"^[a-f0-9]{12}$", str(job_id or "")):
760|        return jsonify({"success": False, "message": "任务编号无效"}), 4
   |    00
761|    try:
762|        result = get_plan_optimizer().apply(job_id)
763|        return jsonify({"success": True, "result": result})
764|    except ValueError as exc:
765|        return jsonify({"success": False, "message": str(exc)}),
400
766|    except Exception as exc:
767|        logger.error("应用寻优方案失败: %s" % exc)
768|        return jsonify({"success": False, "message": "应用方案失败: %s"
   |    % exc}), 500
769|
770|def _prediction_series(records):
771|    fields = (
772|        "pue", "total_energy", "peak_temperature", "
avg_temperature",
773|        "total_load", "served_load", "offered_load", "assigned", "
droppe
   |    d",
774|        "ambient_temperature", "co2_kg",
775|    )
776|    grouped: Dict[str, list] = {}
777|    for row in records or []:
```

```
        algo = str(row.get("algorithm") or "unknown")
        item = {"time": int(row.get("time", 0) or 0)}
        for field in fields:
            value = row.get(field)
            try:
                item[field] = float(value)
            except (TypeError, ValueError):
                item[field] = None
        grouped.setdefault(algo, []).append(item)
    for series in grouped.values():
        series.sort(key=lambda item: item["time"])
    return grouped


def _prediction_explanation(stats, series, best):
    if best and best in stats:
        item = stats[best]
        avg_pue = float(item.get("avg_pue") or 0)
        max_temp = float(item.get("max_temp") or 0)
        service_rate = float(item.get("service_rate") or 0) * 100
        text = (
            "%s 的综合表现最好：平均 PUE %.2f，峰值温度 %.1f° C，服务率 %.1f
    %% "
             % (best, avg_pue, max_temp, service_rate)
        )
        seq = series.get(best) or []
        if seq:
            peak = max(seq, key=lambda row:
row.get("peak_temperature")
    or 0)
            text += " 温度最高出现在第 %d 小时，建议在高温时段前提高制冷
    冗余，或把非紧急任务错峰安排。" % peak["time"]
        if service_rate >= 98:
            text += " 当前配置可以满足服务率门槛。"
        else:
            text += " 当前配置未达到 98% 服务率门槛，建议增加服务器或降低
    负载后复测。"
        return text
    return "当前配置下没有算法达到 98% 服务率门槛，建议增加服务器、降低负
    载或调整算法权重后重新预测。"


@ai_bp.route("/api/v1/ai/predict", methods=["POST"])
```

```
def ai_predict():
    data = request.get_json(silent=True) or {}
    try:
        horizon = validate_int(
            data.get("horizon_hours",
data.get("simulation_hours")),
            min_val=6, max_val=48, default=24, name="预测时长",
        )
        params = validate_simulation_params({
            "num_servers": data.get("num_servers"),
            "simulation_hours": horizon,
            "parallel_workers": data.get("parallel_workers", 1),
            "run_mode": "standard",
            "server_type": data.get("server_type", "mixed"),
            "algorithms": data.get("algorithms") or ["greedy", "
genetic"
    , "round_robin", "damo_ets", "nsga2", "moead"],
        })
    except Exception as exc:
        return jsonify({"success": False, "message": "参数不合法: %s" %
    exc}), 400

    site_id = str(data.get("site_id") or "").strip()
    school_name = str(data.get("school_name") or "").strip()
    if site_id:
        params["site_id"] = site_id
    if school_name:
        params["school_name"] = school_name

    try:
        _, result = get_run_manager().run_prediction(params)
        if not result.success:
            return jsonify({"success": False, "message":
result.message
    or "预测失败"}), 500
        payload = result.data or {}
        stats = payload.get("statistics") or {}
        ranking = payload.get("ranking") or {}
        best = ranking.get("best")
        series = _prediction_series(payload.get("series"))
        assumptions = [
            "使用所选学校所在城市的 2025 年历史气象作为典型日情景",
            "任务流来自 2025 年阿里云 GPU 集群公开轨迹，按 0.3 缩放、1h
粒
```

```
    |    度",
852|            "预测时长 %d 小时，服务器 %d 台，服务器类型 %s" % (
853|                params["simulation_hours"],
params["num_servers"], param
    |    s["server_type"],
854|            ),
855|            "结果为受控仿真，不是学校机房实时预测",
856|        ]
857|        return jsonify({
858|            "success": True,
859|            "scenario": {
860|              "site_id": params.get("site_id") or "北京市",
861|                "school_name": params.get("school_name") or "",
862|                "horizon_hours": params["simulation_hours"],
863|                "num_servers": params["num_servers"],
864|                "server_type": params["server_type"],
865|                "algorithms": params["algorithms"],
866|                "generated_at": datetime.now().strftime("%Y-%m-%d
%H:%M:
    |    %S"),
867|            },
868|            "series": series,
869|            "summary": stats,
870|            "ranking": ranking,
871|            "best_algorithm": best,
872|            "explanation": _prediction_explanation(stats, series,
best),
873|            "assumptions": assumptions,
874|            "disclaimer": "这是基于历史气象与公开任务流的受控情景预测，不
    |    是学校机房实时预测。",
875|            "data_source": payload.get("data_source") or {},
876|        })
877|    except Exception as exc:
878|        logger.error("AI prediction failed: %s" % exc)
879|        return jsonify({"success": False, "message": "预测失败: %s" % ex
    |    c}), 500
880|
881|
882|
883|
884|
885|
886|
887|
```

```
888|
889|
```

19. src/api/app.py - Flask 主应用

```
  1|from flask import Flask, request, jsonify, send_from_directory,
redirect
   |    , session
  2|from flask_cors import CORS
  3|from src.plugin import PluginManager
  4|from src.plugin.simulation_plugin import SimulationPlugin
  5|from src.utils import load_simulation_data
  6|from src.utils.logger import setup_logger
  7|from src.utils.validation import (
  8|    validate_int, validate_str, validate_algorithms,
  9|    validate_simulation_params, security_headers
 10|)
 11|from src.api.download_api import download_bp
 12|from src.api.speech_api import speech_bp
 13|from src.api.ai_api import ai_bp
 14|from src.api.run_api import run_bp
 15|from src.api.auth_api import auth_bp
 16|from src.api.map_api import map_bp
 17|from src.services.run_manager import get_run_manager
 18|from src.algorithms import PLUGIN_ALGORITHMS
 19|from src.simulation.metrics import RANKING_RULE,
SERVICE_RATE_FLOOR, ran
   |    k_algorithms, summarize
 20|from src.utils import load_run_metadata
 21|import os
 22|import mimetypes
 23|import secrets
 24|
 26|# Windows MIME 修正
 27|# Windows 注册表 HKLM\SOFTWARE\Classes\.js 常把 .js 注册为 text/plain,
 28|# 而本项目设置了 X-Content-Type-Options: nosniff, 浏览器会拒绝执行
 29|# Content-Type 为 text/plain 的脚本 -> 前端 JS 全部不执行 (指标卡片停在
   |    --,
 30|# 数据分析图表空白, 插件列表卡在"正在加载")。
 31|# 这里显式注册 Web 资源 MIME, 不依赖操作系统注册表。
 33|mimetypes.add_type('text/javascript', '.js')
 34|mimetypes.add_type('text/css', '.css')
 35|mimetypes.add_type('text/html', '.html')
 36|mimetypes.add_type('application/json', '.json')
 37|mimetypes.add_type('application/manifest+json', '.webmanifest')
```

```
 38|mimetypes.add_type('image/svg+xml', '.svg')
 39|mimetypes.add_type('font/woff2', '.woff2')
 40|mimetypes.add_type('font/woff', '.woff')
 41|mimetypes.add_type('font/ttf', '.ttf')
 43|# 路径解析 (#3: 基于 __file__ 的绝对路径, 不依赖 CWD)
 45|_API_DIR = os.path.dirname(os.path.abspath(__file__))           # .
../sr
   |    c/api/
 46|_SRC_DIR = os.path.dirname(_API_DIR)                           # .
../sr
   |    c/
 47|PROJECT_ROOT = os.path.dirname(_SRC_DIR)                       # .
../Ca
   |    mpusDataCenterScheduler/
 48|FRONTEND_DIR = os.path.join(PROJECT_ROOT, 'frontend')          # .
../fr
   |    ontend
 49|
 50|
 51|def _load_session_secret():
 52|    env_secret = os.environ.get('DCS_SECRET_KEY')
 53|    if env_secret:
 54|        return env_secret
 55|    secret_path = os.path.join(PROJECT_ROOT, 'data', '
.session_secret')
 56|    try:
 57|        if os.path.isfile(secret_path):
 58|            with open(secret_path, 'r', encoding='utf-8') as
handle:
 59|                value = handle.read().strip()
 60|                if value:
 61|                    return value
 62|        os.makedirs(os.path.dirname(secret_path), exist_ok=True)
 63|        value = secrets.token_hex(32)
 64|        with open(secret_path, 'w', encoding='utf-8') as handle:
 65|            handle.write(value)
 66|        return value
 67|    except OSError:
 68|        return 'campus-datacenter-local-session-secret'
 69|
 70|app = Flask(__name__, static_folder=FRONTEND_DIR,
template_folder=FRONTE
   |    ND_DIR)
 71|app.secret_key = _load_session_secret()
```

```
app.config.update(
    SESSION_COOKIE_NAME='dcs_session',
    SESSION_COOKIE_HTTPONLY=True,
    SESSION_COOKIE_SAMESITE='Lax',
    SESSION_COOKIE_SECURE=False,
)
CORS(app)

app.register_blueprint(download_bp)
app.register_blueprint(speech_bp)
app.register_blueprint(ai_bp)
app.register_blueprint(run_bp)
app.register_blueprint(auth_bp)
app.register_blueprint(map_bp)

def algorithm_catalog():
    """One source of truth for the algorithm list exposed to clients."""
    catalog = [
        {'name': 'greedy', 'description': '贪心调度算法', 'type': '实时调度', 'core': True},
        {'name': 'genetic', 'description': '遗传优化算法', 'type': '全局优化', 'core': True},
        {'name': 'round_robin', 'description': '轮询调度算法', 'type': '基准算法', 'core': True},
        {'name': 'damo_ets', 'description': 'DAMO-ETS 多目标能效调度（能耗+温度+时延+异构匹配）',
         'type': '多目标优化', 'core': True},
    ]
    for name, spec in PLUGIN_ALGORITHMS.items():
        catalog.append({
            'name': name,
           'description': '{0} 插件算法'.format(spec.get('plugin')),
           'type': '插件算法',
            'plugin': spec.get('plugin'),
            'core': False,
        })
    return catalog


plugin_manager = PluginManager()
plugin_manager.register_plugin(SimulationPlugin)
plugin_manager.discover_plugins(plugin_dir='src/plugins')

```

```
logger = setup_logger("api")

@app.after_request
def add_security_headers(response):
    for key, value in security_headers().items():
        response.headers.setdefault(key, value)
    return response

@app.route('/')
def index():
    if not session.get('user_id'):
        return redirect('/login.html')
    return send_from_directory(app.static_folder, 'index.html')


@app.route('/index.html')
def index_html():
    if not session.get('user_id'):
        return redirect('/login.html')
    return send_from_directory(app.static_folder, 'index.html')


@app.route('/login.html')
def login_page():
    return send_from_directory(app.static_folder, 'login.html')


@app.route('/register.html')
def register_page():
    return send_from_directory(app.static_folder, 'register.html')


@app.route('/auth.css')
def auth_css():
    return send_from_directory(app.static_folder, 'auth.css')


@app.route('/auth.js')
def auth_js():
    return send_from_directory(app.static_folder, 'auth.js')


@app.route('/map.html')
def map_page():
```

```
155|    if not session.get('user_id'):
156|        return redirect('/login.html')
157|    return send_from_directory(app.static_folder, 'map.html')
158|
159|
160|@app.route('/map.css')
161|def map_css():
162|    return send_from_directory(app.static_folder, 'map.css')
163|
164|
165|@app.route('/map.js')
166|def map_js():
167|    return send_from_directory(app.static_folder, 'map.js')
168|
169|
170|@app.route('/echarts.min.js')
171|def echarts_js():
172|    return send_from_directory(app.static_folder, '
echarts.min.js')
173|
174|
175|@app.route('/static/maps/<path:filename>')
176|def static_maps(filename):
177|    return send_from_directory(os.path.join(app.static_folder, '
static',
   |     'maps'), filename)
178|
179|
180|@app.route('/profile.html')
181|def profile_page():
182|    if not session.get('user_id'):
183|        return redirect('/login.html')
184|    return send_from_directory(app.static_folder, 'profile.html')
185|
186|
187|@app.route('/profile.css')
188|def profile_css():
189|    return send_from_directory(app.static_folder, 'profile.css')
190|
191|
192|@app.route('/profile.js')
193|def profile_js():
194|    return send_from_directory(app.static_folder, 'profile.js')
195|
```

```
@app.route('/download.html')
def download_page():
    return send_from_directory(app.static_folder, 'download.html')

@app.route('/speech_to_text.html')
def speech_page():
    return send_from_directory(app.static_folder, '
speech_to_text.html')

@app.route('/download.js')
def download_js():
    return send_from_directory(app.static_folder, 'download.js')

@app.route('/speech_to_text.js')
def speech_js():
    return send_from_directory(app.static_folder, '
speech_to_text.js')

@app.route('/app.js')
def app_js():
    return send_from_directory(app.static_folder, 'app.js')


@app.route('/assistant.js')
def assistant_js():
    return send_from_directory(app.static_folder, 'assistant.js')

@app.route('/prediction.js')
def prediction_js():
    return send_from_directory(app.static_folder, 'prediction.js')

@app.route('/cross_platform.css')
def cross_platform_css():
    return send_from_directory(app.static_folder, '
cross_platform.css')


@app.route('/theme.css')
def theme_css():
    return send_from_directory(app.static_folder, 'theme.css')


@app.route('/logo.png')
def logo_png():
```

```
237|    return send_from_directory(app.static_folder, 'logo.png')
238|
239|@app.route('/manifest.json')
240|def manifest():
241|    return send_from_directory(app.static_folder, 'manifest.json')
242|
243|@app.route('/service-worker.js')
244|def service_worker():
245|    return send_from_directory(app.static_folder, '
service-worker.js')
246|
247|@app.route('/sw-register.js')
248|def sw_register():
249|    return send_from_directory(app.static_folder, '
sw-register.js')
250|
251|@app.route('/chart.js')
252|def chart_js():
253|    return send_from_directory(app.static_folder, 'chart.js')
254|
255|@app.route('/perf-benchmark.js')
256|def perf_benchmark_js():
257|    return send_from_directory(app.static_folder, '
perf-benchmark.js')
258|
259|@app.route('/platform-utils.js')
260|def platform_utils_js():
261|    return send_from_directory(app.static_folder, '
platform-utils.js')
262|
263|@app.route('/compat-tester.js')
264|def compat_tester_js():
265|    return send_from_directory(app.static_folder, '
compat-tester.js')
266|
267|@app.route('/bootstrap.min.css')
268|def bootstrap_css():
269|    return send_from_directory(app.static_folder, '
bootstrap.min.css')
270|
271|@app.route('/font-awesome.min.css')
272|def font_awesome_css():
273|    return send_from_directory(app.static_folder, '
font-awesome.min.css'
```

```
    |    )
274|
275|@app.route('/bootstrap.bundle.min.js')
276|def bootstrap_js():
277|    return send_from_directory(app.static_folder, '
bootstrap.bundle.min.
    |    js')
278|
279|@app.route('/webfonts/<path:filename>')
280|def webfonts(filename):
281|    return send_from_directory(os.path.join(app.static_folder, '
webfonts
    |    '), filename)
282|
283|# PWA 图标: 前端与 manifest.json 使用 /static/icons/... , 但 Flask 静态
    |    根是
284|# frontend/ , 真实文件在 frontend/static/icons/ , 需显式映射 (否则 404)
285|@app.route('/static/assistant/<path:filename>')
286|def static_assistant(filename):
287|    return send_from_directory(os.path.join(app.static_folder, '
static',
    |     'assistant'), filename)
288|
289|
290|@app.route('/static/icons/<path:filename>')
291|def static_icons(filename):
292|    return send_from_directory(os.path.join(app.static_folder, '
static',
    |     'icons'), filename)
293|@app.route('/install.html')
294|def install_page():
295|    """Legacy mobile-install link: that page was never shipped,
so send
    |    the
296|    visitor to the download page instead of a bare 404."""
297|    return redirect('/download.html', code=302)
298|
299|
300|@app.route('/api/v1/plugins', methods=['GET'])
301|def list_plugins():
302|    try:
303|        plugins = []
304|        for plugin_name, plugin_class in
plugin_manager._plugin_classes.
```

```
    items():
305|            plugin_instance = plugin_manager._plugin_instances.get
(plugi
    n_name)
306|            try:
307|                manifest = plugin_instance.get_manifest() if
plugin_inst
    ance else plugin_class().get_manifest()
308|                plugins.append({
309|                    'name': manifest.name,
310|                    'version': manifest.version,
311|                    'description': "
,".join(manifest.supported_operation
    s),
312|                    'category': manifest.category,
313|                    'loaded': plugin_name in
plugin_manager._plugin_inst
    ances
314|                })
315|            except Exception as e:
316|                logger.warning(f"Failed to get metadata for
plugin {plug
    in_name}: {e}")
317|                plugins.append({
318|                    'name': plugin_name,
319|                    'version': 'unknown',
320|                    'description': '',
321|                    'category': 'algorithm',
322|                    'loaded': plugin_name in
plugin_manager._plugin_inst
    ances
323|                })
324|        return jsonify({'success': True, 'plugins': plugins})
325|    except Exception as e:
326|        logger.error(f"Error listing plugins: {e}")
327|        return jsonify({'success': False, 'message': 'Failed to
list plu
    gins'}), 500
328|
329|@app.route('/api/v1/plugins/<plugin_name>/load', methods=['POST'])
330|def load_plugin(plugin_name):
331|    try:
332|        plugin_name = validate_str(plugin_name, max_len=64,
pattern=r'^[
```

```
   |    a-zA-Z_][a-zA-Z0-9_]*$',
333|                                    name='plugin_name')
334|        config = request.json or {}
335|        success = plugin_manager.load_plugin(plugin_name, config)
336|        if success:
337|            info = plugin_manager.get_plugin_info(plugin_name)
338|            metadata = info.to_dict() if info else {}
339|            return jsonify({'success': True, 'message': 'Plugin
loaded',
   |     'metadata': metadata})
340|        return jsonify({'success': False, 'message': 'Failed to
load plu
   |    gin'}), 500
341|    except ValueError as ve:
342|        return jsonify({'success': False, 'message': str(ve)}),
400
343|    except Exception as e:
344|        logger.error(f"Error loading plugin {plugin_name}: {e}")
345|        return jsonify({'success': False, 'message': 'Internal
server er
   |    ror'}), 500
346|
347|@app.route('/api/v1/plugins/<plugin_name>/unload',
methods=['POST'])
348|def unload_plugin(plugin_name):
349|    try:
350|        plugin_name = validate_str(plugin_name, max_len=64,
pattern=r'^[
   |    a-zA-Z_][a-zA-Z0-9_]*$',
351|                                    name='plugin_name')
352|        plugin_manager.unload_plugin(plugin_name)
353|        return jsonify({'success': True, 'message': 'Plugin
unloaded'})
354|    except ValueError as ve:
355|        return jsonify({'success': False, 'message': str(ve)}),
400
356|    except Exception as e:
357|        logger.error(f"Error unloading plugin {plugin_name}: {e}")
358|        return jsonify({'success': False, 'message': 'Internal
server er
   |    ror'}), 500
359|
360|@app.route('/api/v1/plugins/<plugin_name>/execute',
methods=['POST'])
```

```
def execute_plugin(plugin_name):
    try:
        plugin_name = validate_str(plugin_name, max_len=64,
pattern=r'^[
    a-zA-Z_][a-zA-Z0-9_]*$',
                                   name='plugin_name')
        params = request.json or {}
        result = plugin_manager.execute_plugin(plugin_name,
**params)
        if result is None:
            return jsonify({'success': False, 'message': 'Failed
to exec
    ute plugin'}), 500
        return jsonify(result)
    except ValueError as ve:
        return jsonify({'success': False, 'message': str(ve)}),
400
    except Exception as e:
        logger.error(f"Error executing plugin {plugin_name}: {e}")
        return jsonify({'success': False, 'message': 'Internal
server er
    ror'}), 500

@app.route('/api/v1/plugins/<plugin_name>/config',
methods=['GET'])
def get_plugin_config(plugin_name):
    try:
        plugin_name = validate_str(plugin_name, max_len=64,
pattern=r'^[
    a-zA-Z_][a-zA-Z0-9_]*$',
                                   name='plugin_name')
        plugin = plugin_manager._plugin_instances.get(plugin_name)
        if plugin:
            manifest = plugin.get_manifest()
            config = {
                'name': manifest.name,
                'version': manifest.version,
                'description': ",".join(manifest.supported_operati
ons)
            }
            if hasattr(plugin, 'num_servers'):
                config['num_servers'] = plugin.num_servers
            if hasattr(plugin, 'simulation_hours'):
```

```
392|                config['simulation_hours'] =
plugin.simulation_hours
393|            return jsonify({'success': True, 'config': config})
394|        return jsonify({'success': False, 'message': 'Plugin not
loaded'
   |    }), 404
395|    except ValueError as ve:
396|        return jsonify({'success': False, 'message': str(ve)}),
400
397|    except Exception as e:
398|        logger.error(f"Error getting plugin config {plugin_name}:
{e}")
399|        return jsonify({'success': False, 'message': 'Internal
server er
   |    ror'}), 500
400|
401|@app.route('/api/v1/plugins/<plugin_name>/config',
methods=['PUT'])
402|def update_plugin_config(plugin_name):
403|    try:
404|        plugin_name = validate_str(plugin_name, max_len=64,
pattern=r'^[
   |    a-zA-Z_][a-zA-Z0-9_]*$',
405|                                    name='plugin_name')
406|        plugin = plugin_manager._plugin_instances.get(plugin_name)
407|        if not plugin:
408|            return jsonify({'success': False, 'message': 'Plugin
not loa
   |    ded'}), 404
409|
410|        config = request.json or {}
411|        if 'num_servers' in config:
412|            plugin.num_servers =
validate_int(config['num_servers'], min
   |    _val=1, max_val=1000,
413|
default=plugin.num_servers
   |    , name='num_servers')
414|        if 'simulation_hours' in config:
415|            plugin.simulation_hours =
validate_int(config['simulation_ho
   |    urs'], min_val=1, max_val=720,
416|
default=plugin.simula
```

```
   |    tion_hours, name='simulation_hours')
417|
418|        return jsonify({'success': True, 'message': 'Config
updated'})
419|    except ValueError as ve:
420|        return jsonify({'success': False, 'message': str(ve)}),
400
421|    except Exception as e:
422|        logger.error(f"Error updating plugin config
{plugin_name}: {e}")
423|        return jsonify({'success': False, 'message': 'Internal
server er
   |    ror'}), 500
424|
425|@app.route('/api/v1/simulation', methods=['POST'])
426|def run_simulation():
427|    try:
428|        params = request.json or {}
429|        run_id, result = get_run_manager().run_sync(params,
origin="web"
   |    )
430|        payload = result.to_dict()
431|        payload["run_id"] = run_id
432|        return jsonify(payload)
433|    except ValueError as ve:
434|        return jsonify({'success': False, 'message': str(ve)}),
400
435|    except Exception as e:
436|        logger.error(f"Error running simulation: {str(e)}")
437|        return jsonify({'success': False, 'message': 'Simulation
failed'
   |    }), 500
438|
439|def _current_data_source():
440|    meta = load_run_metadata() or {}
441|    return {
442|        'data_mode': meta.get('data_mode', 'synthetic'),
443|        'ambient_source': meta.get('ambient_source', '
synthetic-sine'),
444|        'scenario_source': meta.get('scenario_source'),
445|        'scenario_digest': meta.get('scenario_digest'),
446|        'sla_slack': meta.get('sla_slack'),
447|        'completion_mode': meta.get('completion_mode'),
448|        'emission_source': meta.get('emission_source'),
```

```
        'site_id': meta.get('site_id'),
        'school_name': meta.get('school_name'),
        'site_province': meta.get('site_province'),
        'emission_region': meta.get('emission_region'),
        'service_rate_floor': meta.get('service_rate_floor',
SERVICE_RAT
    E_FLOOR),
        'ranking_rule': RANKING_RULE,
    }


def _current_controlled():
    meta = load_run_metadata() or {}
    return meta.get('controlled')


@app.route('/api/v1/metrics', methods=['GET'])
def get_metrics():
    try:
        df = load_simulation_data()
        if df is None:
            return jsonify({'success': False, 'message': 'No
simulation
    data found'}), 404

        result = summarize(df)
        ranking = rank_algorithms(result)
        ranks = {item['algorithm']: item['rank'] for item in
ranking['el
    igible']}
        eligible = set(ranks)

        for algo, metrics in result.items():
            metrics['eligible'] = algo in eligible
            metrics['rank'] = ranks.get(algo)

        return jsonify({
            'success': True,
            'data': result,
            'ranking': ranking,
            'data_source': _current_data_source(),
            'controlled': _current_controlled(),
        })
    except Exception as e:
```

```
        logger.error(f"Error getting metrics: {str(e)}")
        return jsonify({'success': False, 'error': str(e)}), 500


@app.route('/api/v1/cities', methods=['GET'])
def list_cities():
    try:
        from src.data.catalog import Catalog
        catalog = Catalog()
        if not catalog.available():
            return jsonify({'success': False, 'message': '数据目录不可用'}), 503
        data = catalog.cities()
        return jsonify({'success': True, 'data': data, 'total': len(data)})
    except Exception as e:
        logger.error(f"Error listing cities: {str(e)}")
        return jsonify({'success': False, 'error': str(e)}), 500


@app.route('/api/v1/schools', methods=['GET'])
def list_schools():
    try:
        query = (request.args.get('q') or '').strip()
        city = (request.args.get('city') or '').strip() or None
        try:
            limit = int(request.args.get('limit', 50))
        except ValueError:
            limit = 50
        limit = max(1, min(limit, 200))

        from src.data.catalog import Catalog
        catalog = Catalog()
        if not catalog.available():
            return jsonify({'success': False, 'message': '数据目录不可用'}), 503
        return jsonify({
            'success': True,
            'data': catalog.schools(query or None, city=city, limit=limit),
            'total': catalog.count_schools(query or None, city=city),
```

```
        })
    except Exception as e:
        logger.error(f"Error listing schools: {str(e)}")
        return jsonify({'success': False, 'error': str(e)}), 500


@app.route('/api/v1/data-source', methods=['GET'])
def get_data_source():
    try:
        payload = dict(_current_data_source())
        meta = load_run_metadata() or {}
        payload['first_hour_offered'] =
meta.get('first_hour_offered')
        payload['first_hour_served'] =
meta.get('first_hour_served')
        payload['controlled'] = meta.get('controlled')
        payload['generated_at'] = meta.get('generated_at')
        payload['catalog'] = None

        try:
            from src.data.catalog import Catalog
            catalog = Catalog()
            if catalog.available():
                payload['catalog'] = {
                    'db_path': catalog.db_path,
                    'sources': catalog.provenance(),
                    'ambient_hours': len(catalog.ambient_series()
or [])
    ,
                }
        except Exception as exc:
            logger.warning(f"Catalog unavailable: {exc}")

        return jsonify({'success': True, 'data': payload})
    except Exception as e:
        logger.error(f"Error getting data source: {str(e)}")
        return jsonify({'success': False, 'error': str(e)}), 500


@app.route('/api/v1/data', methods=['GET'])
def get_data():
    try:
        df = load_simulation_data()
        if df is None:
```

```
563|            return jsonify({'success': False, 'message': 'No
simulation
   |    data found'}), 404
564|
565|        return jsonify({
566|            'success': True,
567|            'data': df.to_dict('records')
568|        })
569|    except Exception as e:
570|        logger.error(f"Error getting data: {str(e)}")
571|        return jsonify({'success': False, 'error': str(e)}), 500
572|
573|@app.route('/api/v1/algorithms', methods=['GET'])
574|def list_algorithms():
575|    return jsonify({'algorithms': algorithm_catalog()})
576|
577|@app.route('/api/v1/system/status', methods=['GET'])
578|def get_system_status():
579|    loaded_plugins = len(plugin_manager._plugin_instances)
580|    registered_plugins = len(plugin_manager._plugin_classes)
581|
582|    status = {
583|        'loaded_plugins': loaded_plugins,
584|        'registered_plugins': registered_plugins,
585|        'status': 'healthy'
586|    }
587|    return jsonify({'success': True, 'status': status})
588|
589|@app.route('/health', methods=['GET'])
590|def health_check():
591|    return jsonify({'status': 'healthy', 'service': '
datacenter-simulati
   |    on-api'})
592|
593|def run(host: str = '0.0.0.0', port: int = 5000, debug: bool =
False):
594|    logger.info(f"Starting API server on {host}:{port}")
595|    app.run(host=host, port=port, debug=debug)
596|
597|if __name__ == '__main__':
598|    run()
```

20. src/api/auth_api.py – 登录注册和验证码

```
1|import base64
```

```
import io
import os
import random
import re
import secrets
import time

from flask import Blueprint, jsonify, request, session
from PIL import Image, ImageDraw, ImageFilter, ImageFont

from src.services.user_store import get_user_store
from src.utils.logger import setup_logger
from src.utils.validation import RateLimiter

auth_bp = Blueprint('auth', __name__)
logger = setup_logger('auth_api')

_CAPTCHA_TTL = 120
_CAPTCHA_CHARS = 'ABCDEFGHJKLMNPQRSTUVWXYZ23456789'
_login_limiter = RateLimiter(max_requests=12, window_sec=60.0)
_USERNAME_RE = re.compile(r'^[A-Za-z0-9_\u4e00-\u9fa5]{3,32}$')


def _captcha_font(size=30):
    font_dir = os.path.join(os.environ.get('WINDIR',
r'C:\Windows'), 'Fo
    nts')
    for name in ('arialbd.ttf', 'arial.ttf', 'segoeuib.ttf', '
segoeui.tt
    f', 'calibrib.ttf'):
        path = os.path.join(font_dir, name)
        if os.path.isfile(path):
            try:
                return ImageFont.truetype(path, size)
            except OSError:
                pass
    try:
        return ImageFont.truetype('DejaVuSans-Bold.ttf', size)
    except OSError:
        return ImageFont.load_default()


def _render_captcha(code):
    width, height = 132, 44
```

```
    image = Image.new('RGB', (width, height), (238, 242, 247))
    draw = ImageDraw.Draw(image)

    for _ in range(5):
        draw.line(
            [
                (random.randint(0, width), random.randint(0,
height)),
                (random.randint(0, width), random.randint(0,
height)),
            ],
            fill=(random.randint(155, 205), random.randint(165,
215), ra
   |    ndom.randint(175, 225)),
            width=1,
        )

    for _ in range(90):
        x = random.randint(0, width - 1)
        y = random.randint(0, height - 1)
        color = (random.randint(120, 190), random.randint(130,
195), ran
   |    dom.randint(145, 210))
        draw.point((x, y), fill=color)

    font = _captcha_font(29)
    colors = [
        (30, 64, 175),
        (22, 104, 220),
        (15, 118, 110),
        (124, 45, 18),
        (91, 33, 182),
    ]
    for index, char in enumerate(code):
        x = 12 + index * 29 + random.randint(-2, 2)
        y = random.randint(1, 8)
        draw.text((x, y), char, font=font,
fill=random.choice(colors))

    image = image.filter(ImageFilter.SMOOTH)
    buffer = io.BytesIO()
    image.save(buffer, format='PNG', optimize=True)
    return base64.b64encode(buffer.getvalue()).decode('ascii')

```

```
 79|
 80|def _consume_captcha(value):
 81|    expected = session.pop('captcha_code', None)
 82|    expires = session.pop('captcha_expires', 0)
 83|    if not expected or time.time() > float(expires or 0):
 84|        return False
 85|    return (value or '').strip().upper() == expected.upper()
 86|
 87|
 88|def _public_user(row):
 89|    data = {'id': row['id'], 'username': row['username']}
 90|    for key in ('created_at', 'last_login'):
 91|        try:
 92|            data[key] = row[key]
 93|        except (IndexError, KeyError):
 94|            pass
 95|    return data
 96|
 97|
 98|@auth_bp.route('/api/v1/auth/captcha', methods=['GET'])
 99|def get_captcha():
100|    code = ''.join(secrets.choice(_CAPTCHA_CHARS) for _ in
range(4))
101|    session['captcha_code'] = code
102|    session['captcha_expires'] = time.time() + _CAPTCHA_TTL
103|    image = _render_captcha(code)
104|    return jsonify({
105|        'success': True,
106|        'image': '' + image,
107|        'expires_in': _CAPTCHA_TTL,
108|    })
109|
110|
111|@auth_bp.route('/api/v1/auth/login', methods=['POST'])
112|def login():
113|    client_ip = request.remote_addr or '0.0.0.0'
114|    if not _login_limiter.is_allowed(client_ip):
115|        return jsonify({'success': False, 'message': '尝试过于频繁，请稍
   |    后再试'}), 429
116|
117|    data = request.get_json(silent=True) or {}
118|    username = (data.get('username') or '').strip()
119|    password = data.get('password') or ''
120|    captcha = (data.get('captcha') or '').strip()
```

```
121|
122|    if not username or not password:
123|        return jsonify({'success': False, 'message': '请输入账号和密码'}
   |    ), 400
124|    if not captcha:
125|        return jsonify({'success': False, 'message': '请输入图形验证码'}
   |    ), 400
126|    if not _consume_captcha(captcha):
127|        return jsonify({'success': False, 'message': '验证码错误或已过期
   |    , 请刷新后重试'}), 400
128|
129|    user = get_user_store().verify_user(username, password)
130|    if user is None:
131|        return jsonify({'success': False, 'message': '账号或密码错误'}),
   |     401
132|
133|    session.clear()
134|    session['user_id'] = user['id']
135|    session['username'] = user['username']
136|    session.permanent = False
137|    logger.info('User login: %s', user['username'])
138|    return jsonify({'success': True, 'user': user})
139|
140|
141|@auth_bp.route('/api/v1/auth/register', methods=['POST'])
142|def register():
143|    data = request.get_json(silent=True) or {}
144|    username = (data.get('username') or '').strip()
145|    password = data.get('password') or ''
146|    confirm_password = data.get('confirm_password') or ''
147|
148|    if not _USERNAME_RE.match(username):
149|        return jsonify({'success': False, 'message': '用户名需为 3-32 位
   |    字母、数字、下划线或中文'}), 400
150|    if len(password) < 6 or len(password) > 64:
151|        return jsonify({'success': False, 'message': '密码长度需为 6-64
   |    位'}), 400
152|    if password != confirm_password:
153|        return jsonify({'success': False, 'message': '两次输入的密码不一
   |    致'}), 400
154|
155|    try:
156|        user = get_user_store().create_user(username, password)
157|    except ValueError as exc:
```

```
158|        return jsonify({'success': False, 'message': str(exc)}),
409
159|
160|    logger.info('User registered: %s', user['username'])
161|    return jsonify({'success': True, 'message': '注册成功，请登录', 'user
   |    ': _public_user(user)}), 201
162|
163|
164|@auth_bp.route('/api/v1/auth/me', methods=['GET'])
165|def current_user():
166|    user_id = session.get('user_id')
167|    if not user_id:
168|        return jsonify({'success': True, 'authenticated': False, '
user':
   |     None})
169|
170|    user = get_user_store().get_by_id(user_id)
171|    if user is None:
172|        session.clear()
173|        return jsonify({'success': True, 'authenticated': False, '
user':
   |     None})
174|
175|    return jsonify({'success': True, 'authenticated': True, '
user': _pub
   |    lic_user(user)})
176|
177|
178|@auth_bp.route('/api/v1/auth/change-password', methods=['POST'])
179|def change_password():
180|    user_id = session.get('user_id')
181|    if not user_id:
182|        return jsonify({'success': False, 'message': '请先登录'}), 401
183|
184|    data = request.get_json(silent=True) or {}
185|    current_password = data.get('current_password') or ''
186|    new_password = data.get('new_password') or ''
187|    confirm_password = data.get('confirm_password') or ''
188|
189|    if not current_password:
190|        return jsonify({'success': False, 'message': '请输入当前密码'}),
   |     400
191|    if len(new_password) < 6 or len(new_password) > 64:
192|        return jsonify({'success': False, 'message': '新密码长度需为 6-6
```

```
    |    4 位'}), 400
193|    if new_password != confirm_password:
194|        return jsonify({'success': False, 'message': '两次输入的新密码不
    |    一致'}), 400
195|
196|    if not get_user_store().change_password(user_id,
current_password, n
    |    ew_password):
197|        return jsonify({'success': False, 'message': '当前密码不正确'}),
    |     400
198|
199|    logger.info('Password changed: %s', session.get('username'))
200|    return jsonify({'success': True, 'message': '密码已修改'})
201|
202|
203|@auth_bp.route('/api/v1/auth/logout', methods=['POST'])
204|def logout():
205|    session.clear()
206|    return jsonify({'success': True, 'message': '已退出登录'})
```

21. src/api/download_api.py

```
  1|"""
  2|应用下载API模块
  3|
  4|提供安全下载服务:
  5|- 下载令牌验证 (防刷/防盗链)
  6|- 文件完整性校验 (SHA256)
  7|- 下载统计上报
  8|- 版本信息查询
  9|- 自动更新检测与通知
 10|- 全平台支持: Android / iOS / Windows / macOS
 11|"""
 12|
 13|import os
 14|import hashlib
 15|import secrets
 16|import time
 17|import re
 18|from typing import Dict, Any, Optional, Tuple
 19|from dataclasses import dataclass, field
 20|from datetime import datetime
 21|
 22|from flask import Blueprint, request, jsonify, send_file,
Response, stre
```

```
    |    am_with_context
 23|from src.utils.logger import setup_logger
 24|from src.utils.download_stats import get_download_stats
 25|from src.utils.validation import (
 26|    validate_str, sanitize_filename, security_headers,
RateLimiter, hash
    |    _sha256
 27|)
 28|
 29|logger = setup_logger("download_api")
 30|
 31|download_bp = Blueprint('download', __name__)
 32|
 34|# 路径解析（基于 __file__ 的绝对路径，不依赖 CWD）
 35|#
 36|#   _API_DIR        = .../PC端安装包/output/CampusDataCenterScheduler/sr
    |    c/api/
 37|#   _SRC_DIR        = .../PC端安装包/output/CampusDataCenterScheduler/sr
    |    c/
 38|#   _PROJECT_ROOT   = .../PC端安装包/output/CampusDataCenterScheduler/
 39|#   _OUTPUT_DIR     = .../PC端安装包/output/   （构建产物目录，含 zip/exe
    |    ）
 40|#   _RELEASES_DIR   = .../PC端安装包/releases/ （将来正式发布目录）
 42|_API_DIR = os.path.dirname(os.path.abspath(__file__))
 43|_SRC_DIR = os.path.dirname(_API_DIR)
 44|_PROJECT_ROOT = os.path.dirname(_SRC_DIR)
 45|_OUTPUT_DIR = os.path.dirname(_PROJECT_ROOT)          # .../PC端安装包/ou
    |    tput/
 46|_RELEASES_DIR = os.path.join(_OUTPUT_DIR, '..', 'releases')
 47|_RELEASES_DIR = os.path.abspath(_RELEASES_DIR)
 48|_DOWNLOAD_DIR = os.path.join(_PROJECT_ROOT, 'downloads')
 49|_PORTABLE_BASENAME = 'CampusDataCenterScheduler_v1.0.0_portable.zi
p'
 50|_PORTABLE_ZIP = os.path.join(_OUTPUT_DIR, _PORTABLE_BASENAME)
 51|
 52|
 53|def _portable_zip_path() -> Optional[str]:
 54|    for folder in _search_dirs():
 55|        candidate = os.path.join(folder, _PORTABLE_BASENAME)
 56|        if os.path.isfile(candidate):
 57|            return candidate
 58|    return None
 59|
 60|
```

```
def _search_dirs() -> tuple:
    """安装包可能出现的目录。

    除构建产物目录外，也兼容两种常见摆放：
      * 仓库内 releases/ 与 downloads/（便于随仓库分发占位包）
      * 上一级的 02_启动器与安装包/（提交包里安装程序所在目录）
    """
    return (
        _RELEASES_DIR,
        os.path.join(_PROJECT_ROOT, 'releases'),
        _DOWNLOAD_DIR,
        _OUTPUT_DIR,
        os.path.join(_OUTPUT_DIR, '02_启动器与安装包'),
        os.path.join(_OUTPUT_DIR, 'downloads'),
    )


def _default_app_type(platform: str) -> str:
    return 'desktop' if platform in ('windows', 'windows_arm', 'macos')
    else 'mobile'

# 配置
TOKEN_EXPIRY_SEC = 600
TOKEN_RE = re.compile(r'^[a-f0-9]{64}$')
_active_tokens: Dict[str, Dict[str, Any]] = {}
_rate_limiter = RateLimiter(max_requests=30, window_sec=60.0)

# 文件解析: platform/filename → 实际文件路径
FILE_BASENAMES = {
    'mobile_android':  'CampusDataCenterScheduler_v1.0.0_android.apk',
    'mobile_ios':      'CampusDataCenterScheduler_v1.0.0_ios.ipa',
    'windows_x64':     'CampusDataCenterScheduler_Setup_v1.0.0.exe',
    'windows_arm64':   'CampusDataCenterScheduler_v1.0.0_portable.zip',
    'macos':           'CampusDataCenterScheduler_v1.0.0_macos.dmg',
}

CONTENT_TYPES = {
    '.exe': 'application/x-msdownload',
    '.apk': 'application/vnd.android.package-archive',
```

```
    '.ipa': 'application/octet-stream',
    '.dmg': 'application/x-apple-diskimage',
    '.zip': 'application/zip',
}


def _app_key_for(platform: str, filename: str) -> Optional[str]:
    """根据 platform + filename 解析出 app_key (mobile_android / windows_x64 ...)"""
    fn = filename.lower()
    if platform == 'android' and (fn.endswith('.apk') or fn.endswith('.zip')):
        return 'mobile_android'
    if platform == 'ios' and (fn.endswith('.ipa') or fn.endswith('.app')
     or fn.endswith('.zip')):
        return 'mobile_ios'
    if platform == 'macos' and (fn.endswith('.dmg') or fn.endswith('.zip')):
        return 'macos'
    if platform == 'windows':
        if 'arm' in fn:
            return 'windows_arm64'
        return 'windows_x64'
    if platform == 'windows_arm':
        return 'windows_arm64'
    return None


def _resolve_file_path(platform: str, filename: str) -> Optional[str]:
    """
    将 platform/filename 解析为绝对文件路径.

    查找顺序:
      1. releases/<basename>      (生产构建产物)
      2. output/<basename>        (CI 直接产物)
      3. Windows x64 退化到 portable zip (开发期替代)
    """
    app_key = _app_key_for(platform, filename)
    if not app_key:
        return None
```

```
    basename = FILE_BASENAMES.get(app_key)
    if not basename:
        return None

    for d in _search_dirs():
        candidate = os.path.join(d, basename)
        if os.path.isfile(candidate):
            return candidate

    # 开发期/安装版退化: Windows x64 没有真 .exe 时, 用 portable zip 替代
    if app_key == 'windows_x64':
        portable = _portable_zip_path()
        if portable:
            return portable

    return None


APP_KEY_PLATFORM = {
    'mobile_android': 'android',
    'mobile_ios': 'ios',
    'windows_x64': 'windows',
    'windows_arm64': 'windows_arm',
    'macos': 'macos',
}


def _file_sha256(path: str) -> str:
    digest = hashlib.sha256()
    with open(path, 'rb') as handle:
        for chunk in iter(lambda: handle.read(1024 * 1024), b''):
            digest.update(chunk)
    return digest.hexdigest()


def _file_size_mb(path: str) -> float:
    return round(os.path.getsize(path) / (1024 * 1024), 2)


def _resolved_app_file(app_key: str) -> Optional[str]:
    basename = FILE_BASENAMES.get(app_key)
    if not basename:
        return None
```

```
    platform = APP_KEY_PLATFORM.get(app_key, '')
    return _resolve_file_path(platform, basename)


def _app_metadata(app_key: str) -> Dict[str, Any]:
    item = dict(APP_VERSIONS.get(app_key, {}))
    path = _resolved_app_file(app_key)
    if path:
        item['sha256'] = _file_sha256(path)
        item['size_mb'] = _file_size_mb(path)
    else:
       # 找不到实际文件时不要回落到写死的旧数值，直接不返回这两个字段
        item.pop('sha256', None)
        item.pop('size_mb', None)
    return item


# Token 管理
def _generate_token(platform: str, app_type: str, filename: str,
                    file_path: Optional[str]) -> str:
    token = secrets.token_hex(32)
    _active_tokens[token] = {
        'platform': platform,
        'app_type': app_type,
        'filename': filename,
       'file_path': file_path,           # #4: 绑定具体文件路径，防 token 滥用
        'app_key': _app_key_for(platform, filename),
        'created_at': time.time(),
        'expires_at': time.time() + TOKEN_EXPIRY_SEC,
        'used': False,
        'ip': None,
    }
    return token


def _cleanup_expired_tokens() -> None:
    now = time.time()
    expired = [t for t, d in _active_tokens.items() if d['expires_at'] <
     now]
    for t in expired:
        del _active_tokens[t]

```

```
225|
227|# 路由: 下载令牌验证
229|@download_bp.route('/api/v1/download/verify', methods=['POST'])
230|def verify_download():
231|    """
232|    下载安全验证接口
233|
234|    Request JSON:
235|      { platform, app_type, filename }
236|
237|    Response:
238|      { success, token, expires_in, message, file_available,
file_size }
239|    """
240|    try:
241|        data = request.json or {}
242|        client_ip = request.remote_addr or '0.0.0.0'
243|
244|        if not _rate_limiter.is_allowed(client_ip):
245|            return jsonify({'success': False, 'message': '请求过于频繁'}
   |    ), 429
246|
247|        platform = validate_str(
248|            data.get('platform', ''), max_len=32,
249|            pattern=r'^(android|ios|windows|windows_arm|macos)$',
250|            default='', name='platform'
251|        )
252|        app_type_raw = (data.get('app_type') or '').strip()
253|        if app_type_raw:
254|            app_type = validate_str(
255|                app_type_raw, max_len=32,
256|                pattern=r'^(mobile|desktop|windows)$',
257|                default='', name='app_type'
258|            )
259|        else:
260|            # 前端偶尔只传 platform, 这里按平台推断, 不再当作错误
261|            app_type = _default_app_type(platform)
262|        filename = sanitize_filename(data.get('filename', ''),
max_len=1
   |    28)
263|
264|        if not platform or not filename:
265|            return jsonify({'success': False, 'message': '参数缺失'}), 4
   |    00
```

```
266|
267|        referrer = request.headers.get('Referer', '')
268|        if referrer and 'datacenter' not in referrer.lower():
269|            logger.warning(f"可疑的Referrer: {referrer}")
270|
271|        # #4: 在 verify 时解析出真实文件路径，绑定到 token
272|        file_path = _resolve_file_path(platform, filename)
273|        file_available = file_path is not None
274|        file_size = os.path.getsize(file_path) if file_available
else 0
275|
276|        _cleanup_expired_tokens()
277|        token = _generate_token(platform, app_type, filename,
file_path)
278|        _active_tokens[token]['ip'] = client_ip
279|
280|        logger.info(
281|            f"Download token issued: {platform}/{filename} "
282|            f"file_available={file_available} size={file_size}"
283|        )
284|
285|        return jsonify({
286|            'success': True,
287|            'token': token,
288|            'expires_in': TOKEN_EXPIRY_SEC,
289|            'file_available': file_available,
290|            'file_size': file_size,
291|            'message': '验证通过' if file_available else '验证通过，但安
    |    装包暂未就绪'
292|        })
293|    except ValueError as e:
294|        logger.warning(f"Download verify 参数不合法: {e}")
295|        return jsonify({'success': False, 'message': f'参数不合法: {e}'}
    |    ), 400
296|    except Exception as e:
297|        logger.error(f"Download verify error: {e}")
298|        return jsonify({'success': False, 'message': '服务器错误'}), 500
299|
300|
302|# 路由: 文件下载 (核心修复 #1)
304|@download_bp.route('/api/v1/download/file/<token>',
methods=['GET'])
305|def download_file(token: str):
306|    """
```

```
    通过 token 下载文件（前端 download.js 第 257 行调用）

    安全约束：
      - token 必须是 64 位 hex
      - token 必须存在且未过期
      - token 必须绑定到一个真实存在的文件（#4）
      - token 单次使用，用后失效（防重放）
    """
    try:
        # 1. 格式校验
        if not TOKEN_RE.match(token or ''):
            logger.warning(f"非法 token 格式：{token[:16] if token else 'None'}...")
            return jsonify({'success': False, 'message': '非法 token'}), 400

        # 2. 存在性 + 过期
        _cleanup_expired_tokens()
        record = _active_tokens.get(token)
        if not record:
            return jsonify({'success': False, 'message': 'token 无效或已过期'}), 404

        # 3. 重放保护（无论文件是否存在，已用过的 token 都不能再用）
        if record.get('used'):
            logger.warning(f"token 重放：{token[:16]}...（已用过）")
            return jsonify({'success': False, 'message': 'token 已使用'}), 410

        # 4. 文件存在性（#4：路径由 verify 时绑定，不接受客户端传入路径）
        file_path = record.get('file_path')
        if not file_path or not os.path.isfile(file_path):
            # 文件缺失时，不消耗 token（允许重试到文件就绪）
            logger.warning(f"文件不存在：{file_path}（token={token[:16]}...）")
            return jsonify({
                'success': False,
                'message': '安装包暂未就绪，请稍后再试',
                'platform': record.get('platform'),
                'filename': record.get('filename'),
            }), 404

        # 5. 标记已用（单次有效，防下载链接外泄后被滥用）- 只在文件存在时消耗
```

```
        record['used'] = True

        # 6. 推断 Content-Type
        ext = os.path.splitext(file_path)[1].lower()
        content_type = CONTENT_TYPES.get(ext, '
application/octet-stream'
    )
        download_name = record['filename'] or
os.path.basename(file_path
    )

        logger.info(
            f"文件下载: {download_name} ({os.path.getsize(file_path)} by
    tes) "
             f"token={token[:16]}..."
        )

        # 7. 流式发送（大文件分块，避免 OOM）
        return _send_file_stream(file_path, download_name,
content_type)

    except Exception as e:
        logger.error(f"File download error: {e}")
        return jsonify({'success': False, 'message': '服务器错误'}), 500


def _send_file_stream(file_path: str, download_name: str,
content_type:
    str):
    """分块流式发送文件，支持 Range（大文件断点续传）"""
    file_size = os.path.getsize(file_path)
    range_header = request.headers.get('Range', None)

    # 解析 Range
    start = 0
    end = file_size - 1
    status_code = 200
    headers = {
        'Content-Type': content_type,
        'Content-Disposition': f'attachment;
filename="{download_name}"'
    ,
        'Accept-Ranges': 'bytes',
        'X-Content-Type-Options': 'nosniff',
```

```
    }

    if range_header:
        m = re.match(r'bytes=(\d+)-(\d*)', range_header)
        if m:
            start = int(m.group(1))
            end = int(m.group(2)) if m.group(2) else file_size - 1
            end = min(end, file_size - 1)
            length = end - start + 1
            status_code = 206
            headers['Content-Range'] = f'bytes
{start}-{end}/{file_size}
   |    '
            headers['Content-Length'] = str(length)

    def generate():
        chunk_size = 64 * 1024  # 64KB
        with open(file_path, 'rb') as f:
            f.seek(start)
            remaining = end - start + 1
            while remaining > 0:
                chunk = f.read(min(chunk_size, remaining))
                if not chunk:
                    break
                remaining -= len(chunk)
                yield chunk

    headers['Content-Length'] = headers.get('Content-Length',
str(file_s
   |    ize))
    return Response(
        stream_with_context(generate()),
        status=status_code,
        headers=headers,
    )


# 路由: 静态路径下载 (兼容 APP_VERSIONS 里的 download_url)
@download_bp.route('/api/v1/download/file/<platform>/<arch>',
methods=['
   |    GET'])
def download_file_legacy(platform: str, arch: str):
    """
    兼容 APP_VERSIONS 中的静态 download_url, 例如:
```

```
419|      /api/v1/download/file/windows/x64
420|      /api/v1/download/file/macos/dmg
421|
422|    与 token 路由不同，此类直接走 IP 限流 + 简单 referrer 校验。
423|    """
424|    try:
425|        client_ip = request.remote_addr or '0.0.0.0'
426|        if not _rate_limiter.is_allowed(client_ip):
427|            return jsonify({'success': False, 'message': '请求过于频繁'}
   |    ), 429
428|
429|        # 映射: (platform, arch) → 内部 platform 标识
430|        if platform == 'windows' and arch == 'x64':
431|            real_platform, real_arch = 'windows', 'x64'
432|        elif platform == 'windows' and arch == 'arm64':
433|            real_platform, real_arch = 'windows_arm', 'arm64'
434|        elif platform == 'macos' and arch == 'dmg':
435|            real_platform, real_arch = 'macos', 'dmg'
436|        elif platform == 'android' and arch == 'apk':
437|            real_platform, real_arch = 'android', 'apk'
438|        elif platform == 'ios' and arch in ('ipa', 'app'):
439|            real_platform, real_arch = 'ios', 'ipa'
440|        else:
441|            return jsonify({'success': False, 'message': '不支持的平台/
   |    架构'}), 400
442|
443|        app_key = {
444|            'android': 'mobile_android',
445|            'ios': 'mobile_ios',
446|            'macos': 'macos',
447|            'windows': 'windows_x64',
448|            'windows_arm': 'windows_arm64',
449|        }.get(real_platform)
450|        basename = FILE_BASENAMES.get(app_key) if app_key else
None
451|        if not basename:
452|            return jsonify({'success': False, 'message': '无此安装包'}),
   |     404
453|
454|        file_path = None
455|        for d in _search_dirs():
456|            candidate = os.path.join(d, basename)
457|            if os.path.isfile(candidate):
458|                file_path = candidate
```

```
459|                break
460|        if not file_path and app_key == 'windows_x64':
461|            file_path = _portable_zip_path()
462|
463|        if not file_path:
464|           return jsonify({'success': False, 'message': '安装包暂未就绪
   |    '}), 404
465|
466|        ext = os.path.splitext(file_path)[1].lower()
467|        content_type = CONTENT_TYPES.get(ext, '
application/octet-stream'
   |    )
468|        return _send_file_stream(file_path, basename,
content_type)
469|
470|    except Exception as e:
471|        logger.error(f"Legacy download error: {e}")
472|       return jsonify({'success': False, 'message': '服务器错误'}), 500
473|
474|
476|# 路由: SHA256 校验
478|
479|APP_VERSIONS = {
480|    'mobile_android': {
481|        'version': 'v1.0.0',
482|        'build': '1000',
483|        'size_mb': 0,
484|        'min_os': 'Android 8.0+ / Chrome',
485|        'arch': 'ARM64 / x86_64',
486|        'release_date': '2026.10.19',
487|        'sha256': '',
488|       'release_notes': 'Android 客户端安装包',
489|        'download_url': '/api/v1/download/file/android/apk',
490|        'web_url': None,
491|        'app_store_url': None,
492|        'platform_family': 'mobile',
493|       'display_name': 'Android 安装包',
494|        'icon': 'fab fa-android'
495|    },
496|    'mobile_ios': {
497|        'version': 'v1.0.0',
498|        'build': '1000',
499|        'size_mb': 0,
500|        'min_os': 'iOS 15.0+ / Safari',
```

```
501|         'arch': 'ARM64',
502|         'release_date': '2026.10.19',
503|         'sha256': '',
504|         'release_notes': 'iOS 客户端安装包',
505|         'download_url': '/api/v1/download/file/ios/ipa',
506|         'web_url': None,
507|         'app_store_url': None,
508|         'platform_family': 'mobile',
509|         'display_name': 'iOS 安装包',
510|         'icon': 'fab fa-app-store-ios'
511|     },
512|     'windows_x64': {
513|         'version': 'v1.0.0',
514|         'build': '1000',
515|         'size_mb': 41.23,
516|         'min_os': 'Windows 10 / Windows 11',
517|         'arch': 'x64',
518|         'release_date': '2026.10.18',
519|         'sha256': '',
520|         'release_notes': '当前源码完整打包，包含真实数据、AI 助手、语音对
话
   |     与运行历史',
521|         'download_url': '/api/v1/download/file/windows/x64',
522|         'web_url': None,
523|         'app_store_url': None,
524|         'platform_family': 'desktop',
525|         'display_name': 'Windows x64 安装包',
526|         'icon': 'fab fa-windows'
527|     },
528|     'windows_arm64': {
529|         'version': 'v1.0.0',
530|         'build': '1000',
531|         'size_mb': 33.69,
532|         'min_os': 'Windows 11 ARM64',
533|         'arch': 'ARM64 / Python 3.12',
534|         'release_date': '2026.10.18',
535|         'sha256': '',
536|         'release_notes': 'Windows ARM64 使用便携版，解压后运行 bin/start.
   |     bat',
537|         'download_url': '/api/v1/download/file/windows/arm64',
538|         'web_url': '/',
539|         'app_store_url': None,
540|         'platform_family': 'desktop',
541|         'display_name': 'Windows ARM64 便携版',
```

```
        'icon': 'fab fa-windows'
    },
    'macos': {
        'version': 'v1.0.0',
        'build': '1000',
        'size_mb': 0,
        'min_os': 'macOS 12+ / Safari 或 Chrome',
        'arch': 'Universal / Intel & Apple Silicon',
        'release_date': '2026.10.19',
        'sha256': '',
        'release_notes': 'macOS 桌面客户端安装包',
        'download_url': '/api/v1/download/file/macos/dmg',
        'web_url': None,
        'app_store_url': None,
        'platform_family': 'desktop',
        'display_name': 'macOS 安装包',
        'icon': 'fab fa-apple'
    }
}

# (旧版 _generate_token / _cleanup_expired_tokens / verify_download 已上移并增强,
#  此处不再重复定义)


@download_bp.route('/api/v1/download/checksum', methods=['POST'])
def verify_checksum():
    """
    SHA256完整性校验接口

    客户端下载完成后将本地SHA256发送至服务端比对。
    """
    try:
        data = request.json or {}
        filename = data.get('filename', '')
        client_sha256 = data.get('sha256', '')

        if not filename or not client_sha256:
            return jsonify({'valid': False, 'message': '校验参数缺失'}), 400

        key_map = {
            'CampusDataCenterScheduler_Setup_v1.0.0.exe': 'windows_x64',
```

```
583|            'CampusDataCenterScheduler_v1.0.0_portable.zip': '
windows_ar
   |    m64',
584|            'CampusDataCenterScheduler_v1.0.0_android.apk': '
mobile_andr
   |    oid',
585|            'CampusDataCenterScheduler_v1.0.0_ios.ipa': '
mobile_ios',
586|            'CampusDataCenterScheduler_v1.0.0_macos.dmg': 'macos',
587|        }
588|
589|        app_key = key_map.get(filename, '')
590|        app_info = APP_VERSIONS.get(app_key, None) if app_key
else None
591|
592|        if app_info:
593|            expected = app_info.get('sha256') or ''
594|            if not expected:
595|                path = _resolved_app_file(app_key)
596|                if path:
597|                    expected = _file_sha256(path)
598|            valid = bool(expected) and client_sha256.lower() ==
expected
   |    .lower()
599|            logger.info(f"SHA256 check for {filename}: {'MATCH'
if valid
   |     else 'MISMATCH'}")
600|            return jsonify({'valid': valid, 'message': '校验通过' if val
   |    id else '文件校验失败'})
601|
602|        return jsonify({'valid': True, 'message': '无校验基准'})
603|
604|    except Exception as e:
605|        logger.error(f"Checksum error: {e}")
606|        return jsonify({'valid': False, 'message': str(e)}), 500
607|
608|@download_bp.route('/api/v1/download/versions', methods=['GET'])
609|def get_versions():
610|    """获取所有平台的最新版本信息"""
611|    return jsonify({
612|        'success': True,
613|        'versions': {
614|            'android': _app_metadata('mobile_android'),
615|            'ios': _app_metadata('mobile_ios'),
```

```
            'windows_x64': _app_metadata('windows_x64'),
            'windows_arm64': _app_metadata('windows_arm64'),
            'macos': _app_metadata('macos')
        },
        'latest_version': 'v1.0.0',
        'latest_build': '1000'
    })

@download_bp.route('/api/v1/download/check-update',
methods=['POST'])
def check_update():
    """
    客户端版本更新检测接口

    客户端发送当前版本号和服务端比对，
    返回是否需要更新及最新版本信息。

    Request JSON:
    {
        "platform": "android" | "ios" | "windows" | "windows_arm"
| "mac
   |    os",
        "current_version": "v3.1.0",
        "current_build": "3100"
    }
    """
    try:
        data = request.json or {}
        platform = validate_str(data.get('platform', ''),
max_len=32,
                                pattern=r'^(android|ios|windows|wi
ndows_
   |    arm|macos)$',
                                default='', name='platform')
        current_version = validate_str(data.get('current_version',
 ''),
   |    max_len=16,

pattern=r'^v?\d+\.\d+\.\d+$',
                                       default='',
name='current_version
   |    ')
        current_build = validate_str(data.get('current_build', '
'), max_
```

```
    len=10,
                                        pattern=r'^\d+$',
                                        default='0',
name='current_build')

        platform_key_map = {
            'android': 'mobile_android',
            'ios': 'mobile_ios',
            'windows': 'windows_x64',
            'windows_arm': 'windows_arm64',
            'macos': 'macos'
        }

        app_key = platform_key_map.get(platform)
        if not app_key:
            return jsonify({'success': False, 'message': '不支持的平台'}
    ), 400

        latest_info = APP_VERSIONS.get(app_key, {})
        if not latest_info:
            return jsonify({'success': False, 'message': '版本信息不可用
    '}), 404

        latest_build_num = int(latest_info.get('build', '0'))
        current_build_num = int(current_build) if current_build
else 0
        update_available = latest_build_num > current_build_num

        return jsonify({
            'success': True,
            'update_available': update_available,
            'current_version': current_version or 'unknown',
            'latest_version': latest_info['version'],
            'latest_build': latest_info['build'],
            'size_mb': latest_info['size_mb'],
            'release_date': latest_info['release_date'],
            'release_notes': latest_info['release_notes'],
            'download_url': latest_info.get('download_url'),
            'app_store_url': latest_info.get('app_store_url'),
            'min_os': latest_info['min_os'],
            'message': '发现新版本！' if update_available else '当前已是
    最新版本'
        })
    except ValueError as ve:
```

```
        return jsonify({'success': False, 'message': str(ve)}),
400
    except Exception as e:
        logger.error(f"Update check error: {e}")
        return jsonify({'success': False, 'message': '服务器错误'}), 500

@download_bp.route('/api/v1/download/stats', methods=['GET'])
def get_stats():
    """获取下载统计数据"""
    try:
        stats = get_download_stats()
        return jsonify(stats.get_summary())
    except Exception as e:
        logger.error(f"Download stats error: {e}")
        return jsonify({'success': False, 'message': str(e)}), 500

@download_bp.route('/api/v1/download/stats', methods=['POST'])
def record_stats():
    """上报下载统计"""
    try:
        data = request.json or {}
        platform = data.get('platform', 'unknown')
        app_type = data.get('app_type', 'unknown')
        filename = data.get('filename', '')
        size_mb = float(data.get('size_mb', 0))
        user_agent = data.get('user_agent', '')[:200]
        referrer = data.get('referrer', '')[:200]
        client_ip = request.remote_addr or '0.0.0.0'

        stats = get_download_stats()
        record = stats.record_download(
            platform=platform, app_type=app_type,
            filename=filename, size_mb=size_mb,
            user_agent=user_agent, ip_address=client_ip
        )

        summary = stats.get_summary()
        return jsonify({
            'success': True,
            'record': record,
            'total': summary['total']
        })
    except Exception as e:
        logger.error(f"Download stats recording error: {e}")
```

```
729|        return jsonify({'success': False, 'message': str(e)}), 500
730|
731|@download_bp.route('/api/v1/download/stats/reset',
methods=['POST'])
732|def reset_stats():
733|    """重置下载统计（管理接口）"""
734|    try:
735|        valid_key = os.environ.get('DCS_ADMIN_KEY')
736|        if not valid_key:
737|            logger.warning("DCS_ADMIN_KEY 未配置，统计重置接口未启用")
738|            return jsonify({'success': False, 'message': '管理接口未启用
   |    （未配置 DCS_ADMIN_KEY）'}), 403
739|        api_key = request.headers.get('X-API-Key', '')
740|        if not api_key or api_key != valid_key:
741|            logger.warning(f"Unauthorized stats reset attempt
from {requ
   |    est.remote_addr}")
742|            return jsonify({'success': False, 'message': '未授权'}), 403
743|
744|        stats = get_download_stats()
745|        stats.reset_stats()
746|        return jsonify({'success': True, 'message': '统计已重置'})
747|    except Exception as e:
748|        return jsonify({'success': False, 'message': str(e)}), 500
```

22. src/api/grpc_adapter.py

```
  1|"""
  2|gRPC/WebSocket 适配层
  3|
  4|提供跨语言接口支持:
  5|- gRPC: 高性能RPC（Go/Java/C#/Python客户端）
  6|- WebSocket: 实时数据推送（Web浏览器、实时监控）
  7|
  8|参考项目: gRPC (Google), Socket.IO, Flask-SocketIO
  9|"""
 10|
 11|import json
 12|import time
 13|from typing import Dict, Any, Generator, Optional
 14|from src.utils.logger import setup_logger
 15|from src.utils.config import Config
 16|
 17|logger = setup_logger("grpc_adapter")
 18|config = Config()
```

```
 19|
 20|class GrpcServerAdapter:
 21|    """
 22|    gRPC服务端适配器
 23|
 24|    使系统可通过gRPC协议被跨语言客户端调用。
 25|    Proto定义: proto/scheduler.proto
 26|
 27|    参考项目: gRPC, grpcio
 28|    """
 29|
 30|    def __init__(self, host: str = '0.0.0.0', port: int = 50051):
 31|        self.host = host
 32|        self.port = port
 33|        self._server = None
 34|
 35|    def start(self) -> bool:
 36|        try:
 37|            import grpc
 38|            from concurrent import futures
 39|            import proto.scheduler_pb2 as pb2
 40|            import proto.scheduler_pb2_grpc as pb2_grpc
 41|
 42|            self._server = grpc.server(futures.ThreadPoolExecutor(
max_wo
   |    rkers=10))
 43|            pb2_grpc.add_DataCenterSchedulerServicer_to_server(
 44|                _GrpcServiceImpl(), self._server
 45|            )
 46|            self._server.add_insecure_port(f'{self.host}:{self.por
t}')
 47|            self._server.start()
 48|            logger.info(f"gRPC server started on
{self.host}:{self.port}
   |    ")
 49|            return True
 50|        except ImportError:
 51|            logger.warning("grpcio not installed, gRPC server
unavailabl
   |    e")
 52|            return False
 53|        except Exception as e:
 54|            logger.error(f"gRPC server start failed: {e}")
 55|            return False
```

```
56|
57|    def stop(self) -> None:
58|        if self._server:
59|            self._server.stop(0)
60|            logger.info("gRPC server stopped")
61|
62|    def wait_for_termination(self) -> None:
63|        if self._server:
64|            self._server.wait_for_termination()
65|
66|
67|class _GrpcServiceImpl:
68|    """gRPC服务实现（自动生成client后替换为实际实现）"""
69|
70|    def RunSimulation(self, request, context):
71|        from src.plugin import PluginManager
72|        from src.plugin.simulation_plugin import SimulationPlugin
73|
74|        pm = PluginManager()
75|        pm.register_plugin(SimulationPlugin)
76|
77|        plugin_config = {
78|            'num_servers': request.num_servers,
79|            'simulation_hours': request.simulation_hours,
80|            'parallel_workers': request.parallel_workers,
81|            'run_mode': request.run_mode,
82|            'server_type': request.server_type
83|        }
84|
85|        pm.load_plugin('SimulationPlugin', plugin_config)
86|        result = pm.execute_plugin('SimulationPlugin',
87|            algorithms=list(request.algorithms),
88|            visualization=request.visualization,
89|            save_results=request.save_results
90|        )
91|        pm.unload_plugin('SimulationPlugin')
92|
93|        return result
94|
95|    def RunSimulationStream(self, request, context):
96|        total_hours = request.simulation_hours
97|        for hour in range(total_hours):
98|            result = {
99|                'time': hour,
```

```
                'algorithm': request.algorithms[0] if request.algorithms
    else 'greedy',
                'total_load': 100.0,
                'avg_temperature': 25.0 + 10.0 * (hour % 12) / 12,
                'total_energy': 500.0,
                'pue': 1.75
            }
            yield result
            time.sleep(0.1)

    def GetMetrics(self, request, context):
        from src.utils import load_simulation_data
        df = load_simulation_data()
        if df is None:
            return {'success': False, 'data': {}}
        return {'success': True, 'data': df.to_dict('records')}

    def HealthCheck(self, request, context):
        return {
            'status': 'healthy',
            'service': 'datacenter-scheduler',
            'timestamp': str(time.time())
        }

    def ListAlgorithms(self, request, context):
        return {
            'algorithms': [
                {'name': 'greedy', 'description': '贪心调度算法', 'type': '实时调度'},
                {'name': 'genetic', 'description': '遗传优化算法', 'type': '全局优化'},
                {'name': 'round_robin', 'description': '轮询调度算法', 'type': '基准算法'}
            ]
        }


class WebSocketAdapter:
    """
    WebSocket适配器

    提供实时数据推送能力，适用于:
    - Web可视化面板实时更新
```

```
    - 移动端App数据推送
    - 第三方监控系统集成

    参考项目: Socket.IO, Flask-SocketIO, WebSockets (RFC 6455)
    """

    def __init__(self):
        self._clients = set()
        self._running = False

    def register(self, ws_server) -> None:
        """注册到WebSocket服务器"""
        self._running = True
        logger.info("WebSocket adapter registered")

    async def broadcast_metric_update(self, data: Dict[str, Any])
-> Non
   |    e:
        """广播指标更新到所有客户端"""
        pass

    async def broadcast_simulation_progress(self, progress:
float, messa
   |    ge: str) -> None:
        """广播仿真进度"""
        pass

    async def handle_client_message(self, client_id: str,
message: Dict[
   |    str, Any]) -> Optional[Dict[str, Any]]:
        """处理客户端消息"""
        action = message.get('action', '')

        handlers = {
            'run_simulation': self._handle_run_simulation,
            'get_metrics': self._handle_get_metrics,
            'ping': self._handle_ping
        }

        handler = handlers.get(action)
        if handler:
            return await handler(message)
        return {'error': f'Unknown action: {action}'}

```

```
177|    async def _handle_run_simulation(self, message: Dict[str,
Any]) -> D
   |    ict[str, Any]:
178|        from src.sdk import DataCenterSimulatorSDK
179|        sdk = DataCenterSimulatorSDK(mode='local')
180|        result = sdk.run_simulation(**message.get('params', {}))
181|        return result
182|
183|    async def _handle_get_metrics(self, message: Dict[str, Any])
-> Dict
   |    [str, Any]:
184|        from src.sdk import DataCenterSimulatorSDK
185|        sdk = DataCenterSimulatorSDK(mode='local')
186|        result = sdk.get_metrics()
187|        return result
188|
189|    async def _handle_ping(self, message: Dict[str, Any]) ->
Dict[str, A
   |    ny]:
190|        return {'action': 'pong', 'timestamp': time.time()}
```

23. src/api/map_api.py – 分布图数据接口

```
  1|import json
  2|import os
  3|from datetime import datetime
  4|
  5|from flask import Blueprint, jsonify
  6|
  7|from src.data.catalog import Catalog, region_for_province
  8|from src.services import run_store
  9|from src.simulation.metrics import rank_algorithms
 10|from src.utils.logger import setup_logger
 11|
 12|map_bp = Blueprint('map', __name__)
 13|logger = setup_logger('map_api')
 14|
 15|_API_DIR = os.path.dirname(os.path.abspath(__file__))
 16|_SRC_DIR = os.path.dirname(_API_DIR)
 17|_PROJECT_ROOT = os.path.dirname(_SRC_DIR)
 18|_FRONTEND_DIR = os.path.join(_PROJECT_ROOT, 'frontend')
 19|_COORDS_PATH = os.path.join(_FRONTEND_DIR, 'static', 'maps', '
city_coord
   |    s.json')
 20|
```

```
 21|
 22|def _load_coords():
 23|    with open(_COORDS_PATH, 'r', encoding='utf-8') as handle:
 24|        return json.load(handle)
 25|
 26|
 27|@map_bp.route('/api/v1/map/data', methods=['GET'])
 28|def map_data():
 29|    try:
 30|        catalog = Catalog()
 31|        if not catalog.available():
 32|            return jsonify({'success': False, 'message': '数据目录不可用
   |    '}), 503
 33|
 34|        coords = _load_coords()
 35|        cities = catalog.cities()
 36|        climate = {row['city']: row.get('avg_temp') for row in
catalog.c
   |    ity_climate_summary()}
 37|        emission_cache = {}
 38|
 39|        city_payload = []
 40|        province_acc = {}
 41|        total_schools = 0
 42|
 43|        for row in cities:
 44|            city = row.get('city') or ''
 45|            province = row.get('province') or ''
 46|            school_count = int(row.get('school_count') or 0)
 47|            total_schools += school_count
 48|            region = region_for_province(province)
 49|            if region not in emission_cache:
 50|                emission_cache[region] =
catalog.emission_factor(region)
 51|            carbon = emission_cache.get(region)
 52|            avg_temp = climate.get(city)
 53|
 54|            acc = province_acc.setdefault(province, {
 55|                'name': province,
 56|                'region': region,
 57|                'school_count': 0,
 58|                'temp_sum': 0.0,
 59|                'temp_weight': 0,
 60|                'carbon_factor': carbon,
```

```
            })
            acc['school_count'] += school_count
            if avg_temp is not None:
                acc['temp_sum'] += float(avg_temp) * school_count
                acc['temp_weight'] += school_count

            coord = coords.get(city)
            if not coord:
                continue
            city_payload.append({
                'city': city,
                'province': province,
                'region': region,
                'latitude': coord.get('latitude'),
                'longitude': coord.get('longitude'),
                'school_count': school_count,
                'avg_temp': round(float(avg_temp), 2) if avg_temp
is not
    |     None else None,
                'carbon_factor': carbon,
            })

        provinces = []
        for acc in province_acc.values():
            avg_temp = None
            if acc['temp_weight']:
                avg_temp = round(acc['temp_sum'] /
acc['temp_weight'], 2
    |    )
            provinces.append({
                'name': acc['name'],
                'region': acc['region'],
                'school_count': acc['school_count'],
                'avg_temp': avg_temp,
                'carbon_factor': acc['carbon_factor'],
            })

        city_payload.sort(key=lambda item:
(-item['school_count'], item[
    |    'city']))
        provinces.sort(key=lambda item: (-item['school_count'],
item['na
    |    me']))
        return jsonify({
```

```
 97|            'success': True,
 98|            'year': 2025,
 99|            'totals': {
100|                'schools': total_schools,
101|                'cities': len(city_payload),
102|                'provinces': len(provinces),
103|            },
104|            'provinces': provinces,
105|            'cities': city_payload,
106|            'sources': catalog.provenance(),
107|            'generated_at': datetime.now().strftime('%Y-%m-%d
%H:%M:%S')
   |    ,
108|        })
109|    except Exception as exc:
110|        logger.error('Map data error: %s', exc)
111|        return jsonify({'success': False, 'message': '地图数据加载失败'}
   |    ), 500
112|
113|def _monthly_climate(catalog, city):
114|    daily = catalog.city_daily(city) or []
115|    buckets = {}
116|    for row in daily:
117|        date_text = str(row.get('date') or '')
118|        try:
119|            month = int(date_text[5:7])
120|        except (TypeError, ValueError, IndexError):
121|            continue
122|        item = buckets.setdefault(month, {'tmean': [], 'tmax':
[], 'tmin
   |    ': []})
123|        for key in ('tmean', 'tmax', 'tmin'):
124|            value = row.get(key + '_c')
125|            if value is not None:
126|                item[key].append(float(value))
127|
128|    months = []
129|    for month in range(1, 13):
130|        item = buckets.get(month)
131|        if not item or not item['tmean']:
132|            months.append({'month': month, 'tmean': None, 'tmax':
None,
   |    'tmin': None})
133|            continue
```

```
        months.append({
            'month': month,
            'tmean': round(sum(item['tmean']) /
len(item['tmean']), 2),
            'tmax': round(max(item['tmax']), 2) if item['tmax']
else Non
    |    e,
            'tmin': round(min(item['tmin']), 2) if item['tmin']
else Non
    |    e,
        })
    return months


def _latest_city_run(city):
    for run in run_store.list_runs(100):
        params = run.get('params') or {}
        if run.get('status') != 'success' or
params.get('site_id') != ci
    |    ty:
            continue
        summary = run.get('summary') or {}
        if not summary:
            continue
        ranking = rank_algorithms(summary)
        best = ranking.get('best')
        if not best:
            continue
        metrics = summary.get(best) or {}
        return {
            'run_id': run.get('run_id'),
            'created_at': run.get('created_at'),
            'num_servers': params.get('num_servers'),
            'simulation_hours': params.get('simulation_hours'),
            'algorithm': best,
            'metrics': {
                'avg_pue': metrics.get('avg_pue'),
                'max_temp': metrics.get('max_temp'),
                'service_rate': metrics.get('service_rate'),
                'energy_per_served_load':
metrics.get('energy_per_served
    |    _load'),
            },
        }
```

```
    return None


@map_bp.route('/api/v1/map/city/<path:city>', methods=['GET'])
def map_city(city):
    try:
        catalog = Catalog()
        if not catalog.available():
            return jsonify({'success': False, 'message': '数据目录不可用'}), 503

        city_name = str(city or '').strip()[:64]
        rows = [row for row in catalog.cities() if row.get('city') == city_name]
        if not rows:
            return jsonify({'success': False, 'message': '城市不存在'}), 404

        row = rows[0]
        province = row.get('province') or catalog.city_province(city_name) or ''
        region = region_for_province(province)
        school_count = int(row.get('school_count') or 0)
        climate = {item['city']: item.get('avg_temp') for item in catalog.city_climate_summary()}
        avg_temp = climate.get(city_name)
        carbon = catalog.emission_factor(region)
        latest = _latest_city_run(city_name)

        if latest:
            metrics = latest.get('metrics') or {}
            recommendation = {
                'servers': latest.get('num_servers') or 48,
                'algorithm': latest.get('algorithm'),
                'hours': latest.get('simulation_hours') or 24,
                'pue': metrics.get('avg_pue'),
                'max_temp': metrics.get('max_temp'),
                'service_rate': metrics.get('service_rate'),
                'source': 'latest_run',
                'run_id': latest.get('run_id'),
            }
```

```
205|        else:
206|            recommendation = {
207|                'servers': 48,
208|                'algorithm': 'damo_ets',
209|                'hours': 24,
210|                'pue': None,
211|                'max_temp': None,
212|                'service_rate': None,
213|                'source': 'baseline',
214|                'run_id': None,
215|            }
216|
217|        return jsonify({
218|            'success': True,
219|            'city': city_name,
220|            'province': province,
221|            'region': region,
222|            'school_count': school_count,
223|            'avg_temp': round(float(avg_temp), 2) if avg_temp is
not Non
   |    e else None,
224|            'carbon_factor': carbon,
225|            'months': _monthly_climate(catalog, city_name),
226|            'latest_run': latest,
227|            'recommendation': recommendation,
228|        })
229|    except Exception as exc:
230|        logger.error('Map city error: %s', exc)
231|        return jsonify({'success': False, 'message': '城市详情加载失败'}
   |    ), 500
```

24. src/api/run_api.py

```
  1|"""Run registry, async simulation jobs and live events."""
  2|
  3|import json
  4|from queue import Empty
  5|
  6|import pandas as pd
  7|from flask import Blueprint, Response, jsonify, request,
stream_with_con
   |    text
  8|
  9|from src.services import run_store
 10|from src.services.run_manager import get_run_manager
```

```
 11|
 12|run_bp = Blueprint("runs", __name__)
 13|
 14|
 15|@run_bp.route("/api/v1/runs", methods=["GET"])
 16|def list_runs():
 17|    get_run_manager()
 18|    try:
 19|        limit = int(request.args.get("limit", 20))
 20|    except ValueError:
 21|        limit = 20
 22|    return jsonify({"success": True, "runs":
run_store.list_runs(limit)}
   |    )
 23|
 24|
 25|@run_bp.route("/api/v1/runs", methods=["POST"])
 26|def create_run():
 27|    body = request.get_json(silent=True) or {}
 28|    origin = "ai" if body.get("origin") == "ai" else "web"
 29|    try:
 30|        run_id = get_run_manager().submit(body, origin=origin)
 31|    except ValueError as exc:
 32|        return jsonify({"success": False, "message": str(exc)}),
400
 33|    except Exception as exc:
 34|        return jsonify({"success": False, "message": "创建运行失败: %s"
   |    % exc}), 500
 35|    return jsonify({"success": True, "run_id": run_id, "status": "
queued
   |    "})
 36|
 37|
 38|@run_bp.route("/api/v1/runs/latest", methods=["GET"])
 39|def latest_run():
 40|    get_run_manager()
 41|    record = run_store.latest_run()
 42|    if not record:
 43|        return jsonify({"success": False, "message": "暂无运行记录"}), 4
   |    04
 44|    return jsonify({"success": True, "run": record})
 45|
 46|
 47|@run_bp.route("/api/v1/runs/compare", methods=["GET"])
```

```
def compare_runs():
    left = (request.args.get("left") or "").strip()
    right = (request.args.get("right") or "").strip()
    if not left or not right:
        runs = run_store.list_runs(2)
        if len(runs) < 2:
            return jsonify({"success": False, "message": "至少需要两次运行才能对比"}), 404
        right, left = runs[0]["run_id"], runs[1]["run_id"]
    comparison = run_store.compare_runs(left, right)
    if not comparison:
        return jsonify({"success": False, "message": "运行记录不存在"}), 404
    return jsonify({"success": True, "comparison": comparison})


@run_bp.route("/api/v1/runs/<run_id>", methods=["GET"])
def get_run(run_id: str):
    record = run_store.get_run(run_id)
    if not record:
        return jsonify({"success": False, "message": "运行记录不存在"}), 404
    return jsonify({"success": True, "run": record})


@run_bp.route("/api/v1/runs/<run_id>/summary", methods=["GET"])
def get_run_summary(run_id: str):
    record = run_store.get_run(run_id)
    if not record:
        return jsonify({"success": False, "message": "运行记录不存在"}), 404
    return jsonify({
        "success": True,
        "run_id": run_id,
        "status": record.get("status"),
        "summary": record.get("summary") or {},
        "params": record.get("params") or {},
    })


@run_bp.route("/api/v1/runs/<run_id>/series", methods=["GET"])
def get_run_series(run_id: str):
    record = run_store.get_run(run_id)
    if not record:
```

```
        return jsonify({"success": False, "message": "运行记录不存在"}),
    404
    df = run_store.load_results(run_id)
    if df is None:
        return jsonify({"success": False, "message": "结果文件不存在"}),
    404
    algorithm = (request.args.get("algorithm") or "").strip()
    if algorithm:
        df = df[df["algorithm"] == algorithm]
    df = df.where(pd.notnull(df), None)
    return jsonify({
        "success": True,
        "run_id": run_id,
        "data": df.to_dict("records"),
    })


@run_bp.route("/api/v1/events", methods=["GET"])
def run_events():
    manager = get_run_manager()
    queue = manager.events.subscribe()

    def stream():
        try:
            yield "event: ready\ndata: {}\n\n"
            while True:
                try:
                    event = queue.get(timeout=15)
                except Empty:
                    yield ": ping\n\n"
                    continue
                yield "data: " + json.dumps(event,
ensure_ascii=False, d
    efault=str) + "\n\n"
        finally:
            manager.events.unsubscribe(queue)

    return Response(
        stream_with_context(stream()),
        mimetype="text/event-stream",
        headers={
            "Cache-Control": "no-cache",
            "X-Accel-Buffering": "no",
            "Connection": "keep-alive",
```

```
128|        },
129|    )
```

25. src/api/speech_api.py

```
  1|from flask import Blueprint, request, jsonify
  2|from src.services.speech_service import (
  3|    get_speech_service, SpeechService, RecognitionState,
  4|    SUPPORTED_LANGUAGES
  5|)
  6|from src.utils.logger import setup_logger
  7|from src.utils.validation import validate_str
  8|
  9|logger = setup_logger("speech_api")
 10|speech_bp = Blueprint('speech', __name__)
 11|
 12|@speech_bp.route('/api/v1/speech/session', methods=['POST'])
 13|def create_session():
 14|    """创建语音识别会话"""
 15|    try:
 16|        data = request.json or {}
 17|        language = validate_str(
 18|            data.get('language', 'zh-CN'), max_len=10,
 19|            pattern=r'^[a-z]{2,3}(-[A-Z]{2,3})?$',
 20|            default='zh-CN', name='language'
 21|        )
 22|        svc = get_speech_service()
 23|        session = svc.create_session(language)
 24|
 25|        lang_info = SUPPORTED_LANGUAGES.get(language,
SUPPORTED_LANGUAGE
   |    S['zh-CN'])
 26|
 27|        return jsonify({
 28|            'success': True,
 29|            'session_id': session.session_id,
 30|            'language': language,
 31|            'language_name': lang_info['name'],
 32|            'offline_available': lang_info['offline_available'],
 33|            'web_api_code': lang_info['web_api_code']
 34|        })
 35|    except Exception as e:
 36|        logger.error(f"Speech session creation error: {e}")
 37|        return jsonify({'success': False, 'message': str(e)}), 500
 38|
```

```
@speech_bp.route('/api/v1/speech/languages', methods=['GET'])
def list_languages():
    """获取支持的语言列表"""
    try:
        svc = get_speech_service()
        langs = svc.get_supported_languages()
        return jsonify({'success': True, 'languages': langs})
    except Exception as e:
        return jsonify({'success': False, 'message': str(e)}), 500

@speech_bp.route('/api/v1/speech/result', methods=['POST'])
def submit_result():
    """提交识别结果"""
    try:
        data = request.json or {}
        session_id = validate_str(data.get('session_id', ''), max_len=64
    , name='session_id')
        text = validate_str(data.get('text', ''), max_len=5000, name='te
    xt')
        confidence = float(data.get('confidence', 0.0))
        is_final = bool(data.get('is_final', False))
        engine_used = data.get('engine_used', 'web_api')
        language = data.get('language', 'zh-CN')

        svc = get_speech_service()
        result = svc.add_result(session_id, text, confidence,
                                is_final=is_final,
    engine_used=engine_us
    ed)

        return jsonify({
            'success': True,
            'result': {
                'text': result.text if result else text,
                'confidence': confidence,
                'is_final': is_final
            }
        })
    except Exception as e:
        logger.error(f"Speech result submission error: {e}")
        return jsonify({'success': False, 'message': str(e)}), 500

```

```
@speech_bp.route('/api/v1/speech/session/<session_id>',
methods=['GET'])
def get_session(session_id):
    """获取会话状态和全部文字"""
    try:
        svc = get_speech_service()
        session = svc.get_session(session_id)
        if not session:
            return jsonify({'success': False, 'message': '会话不存在'}),
     404

        all_text = svc.get_all_text(session_id, final_only=False)

        return jsonify({
            'success': True,
            'session_id': session_id,
            'language': session.language,
            'state': session.state.value,
            'text': all_text,
            'result_count': len(session.results)
        })
    except Exception as e:
        return jsonify({'success': False, 'message': str(e)}), 500

@speech_bp.route('/api/v1/speech/session/<session_id>',
methods=['DELETE
    '])
def cleanup_session(session_id):
    """清理会话"""
    try:
        svc = get_speech_service()
        svc.cleanup_session(session_id)
        return jsonify({'success': True, 'message': '会话已清理'})
    except Exception as e:
        return jsonify({'success': False, 'message': str(e)}), 500

@speech_bp.route('/api/v1/speech/health', methods=['GET'])
def speech_health():
    """语音服务健康检查"""
    try:
        svc = get_speech_service()
        return jsonify({
            'success': True,
            'supported_languages': len(SUPPORTED_LANGUAGES),
```

```
117|            'engine': svc._engine.value if hasattr(svc._engine, '
value')
   |     else str(svc._engine),
118|            'active_sessions': len(svc._sessions)
119|        })
120|    except Exception as e:
121|        return jsonify({'success': False, 'message': str(e)}), 500
```

26. src/cli/__init__.py

```
1|from .main import cli
2|
3|__all__ = ['cli']
```

27. src/cli/main.py

```
1|import argparse
2|import json
3|import os
4|from src.plugin import PluginManager
5|from src.plugin.simulation_plugin import SimulationPlugin
6|from src.api.app import run as run_api
7|from src.utils.logger import setup_logger
8|from src.utils.config import Config
9|from src.adapters import AutoAdapter, LocalAdapter, RemoteAdapter
10|try:
11|    from src.adapters import CudaAdapter
12|except ImportError:
13|    CudaAdapter = None
14|try:
15|    from src.adapters import AscendAdapter
16|except ImportError:
17|    AscendAdapter = None
18|import platform
19|import psutil
20|from typing import Dict, Any
21|
22|logger = setup_logger("cli")
23|config = Config()
24|
25|def run_simulation(args):
26|    """运行仿真命令（支持环境自动检测）"""
27|    adapter = _select_adapter(args)
28|
29|    if not adapter.initialize():
30|        logger.error("适配器初始化失败")
```

```
        return
    logger.info(f"使用适配器: {adapter.get_type().value}")
    logger.info(f"  能力: {adapter.get_capability().description}")

    algorithms = args.algorithms.split(',') if args.algorithms
else ['gr
    eedy', 'genetic', 'round_robin']

    params = {
        'num_servers': args.servers,
        'simulation_hours': args.hours,
        'parallel_workers': args.workers,
        'run_mode': args.mode,
        'algorithms': algorithms,
        'visualization': not args.no_plot,
        'save_results': not args.no_save
    }

    logger.info(f"参数: servers={args.servers}, hours={args.hours}, work
    ers={args.workers}, mode={args.mode}")
    logger.info(f"算法: {algorithms}")

    result = adapter.execute(**params)

    _display_result(result)

def _select_adapter(args) -> 'BaseAdapter':
    """根据参数和环境选择适配器"""
    adapter_map = {
        'auto': AutoAdapter,
        'local': LocalAdapter,
        'remote': RemoteAdapter,
    }
    if CudaAdapter is not None:
        adapter_map['cuda'] = CudaAdapter
    if AscendAdapter is not None:
        adapter_map['ascend'] = AscendAdapter

    adapter_type = args.adapter or 'auto'
    adapter_cls = adapter_map.get(adapter_type, AutoAdapter)
    adapter = adapter_cls()

    if adapter_type == 'auto':
```

```
 72|        logger.info("正在检测环境...")
 73|        status = adapter.get_status()
 74|        logger.info(f"可用适配器: {len(status.get('available_adapters',
   |    []))} 个")
 75|        for a in status.get('available_adapters', []):
 76|            logger.info(f"  - {a['name']}: {'可用' if a['available'] els
   |    e '不可用'}")
 77|
 78|    return adapter
 79|
 80|def _display_payload(result: Any) -> Dict[str, Any]:
 81|    """Accept either a PluginResult or a plain dict and expose a
dict.""
   |    "
 82|    if hasattr(result, 'success') and hasattr(result, 'data'):
 83|        payload = dict(result.data) if isinstance(result.data,
dict) els
   |    e {}
 84|        payload['success'] = bool(result.success)
 85|        if not payload.get('success'):
 86|            payload['error'] = result.message or
result.error_code or '
   |    未知错误'
 87|        return payload
 88|    if isinstance(result, dict):
 89|        return result
 90|    return {'success': False, 'error': '返回类型无法识别'}
 91|
 92|
 93|def _display_result(result: Any) -> None:
 94|    """显示仿真结果"""
 95|    result = _display_payload(result)
 96|    if result.get('success'):
 97|        logger.info("=" * 60)
 98|        logger.info("仿真结果统计")
 99|        logger.info("=" * 60)
100|
101|        for algo, metrics in result.get('statistics', {}).items():
102|            logger.info(f"【{algo.upper()}算法】")
103|            logger.info(f"  平均能耗: {metrics.get('avg_energy', 0):.2f}
   |     W")
104|            logger.info(f"  最高温度: {metrics.get('max_temp', 0):.2f} °
   |    C")
105|            logger.info(f"  平均PUE: {metrics.get('avg_pue', 0):.2f}")
```

```
            logger.info(f"  总处理负载: {metrics.get('total_load', 0):.0f}")

        logger.info(f"总记录数: {result.get('records_count', 0)}")

        if 'csv_path' in result:
            logger.info(f"数据文件: {result['csv_path']}")
        if 'excel_path' in result:
            logger.info(f"Excel文件: {result['excel_path']}")
        if 'plots' in result:
            logger.info("图表文件:")
             for name, path in result['plots'].items():
                 logger.info(f"  - {path}")
        if 'adapter_type' in result:
            logger.info(f"适配器: {result['adapter_type']}")
    else:
        logger.error(f"仿真失败: {result.get('error', 'Unknown error')}")

def run_api_server(args):
    """启动API服务器"""
    run_api(host=args.host, port=args.port, debug=args.debug)

def list_algorithms(args):
    """列出可用算法"""
    algorithms = [
        {'name': 'greedy', 'description': '贪心调度算法', 'type': '实时调度'},
        {'name': 'genetic', 'description': '遗传优化算法', 'type': '全局优化'},
        {'name': 'round_robin', 'description': '轮询调度算法', 'type': '基准算法'},
        {'name': 'damo_ets', 'description': 'DAMO-ETS 多目标能效调度', 'type': '多目标优化'},
        # 第三方插件（GitHub Skills 集成）
        {'name': 'nsga2', 'description': 'pymoo NSGA2 多目标调度（能耗+温度+deadline）', 'type': '多目标优化', 'plugin': 'NSGA2SchedulerPlugin', 'source': 'github.com/anyoptimization/pymoo'},
        {'name': 'moead', 'description': 'pymoo MOEAD 分解多目标调度', 'type': '多目标优化', 'plugin': 'MOEADSchedulerPlugin', 'source': 'github.com/anyoptimization/pymoo'},
    ]
    logger.info(json.dumps(algorithms, indent=2, ensure_ascii=False))
```

```
139|
140|def detect_environment(args):
141|    """环境检测命令"""
142|    info = config.get_runtime_info()
143|
144|    logger.info("=" * 60)
145|    logger.info("系统环境检测报告")
146|    logger.info("=" * 60)
147|    logger.info(f"操作系统:    {info['os']} {info['os_release']} ({info[
   |    'arch']})")
148|    logger.info(f"Python版本:  {info['python_version']}")
149|    logger.info(f"CPU核心:     {info['cpu_physical']} 物理 / {info['cpu_
   |    logical']} 逻辑")
150|    logger.info(f"内存:        {info['total_memory_gb']} GB (可用: {info
   |    ['available_memory_gb']} GB)")
151|    logger.info(f"Docker容器:  {'是' if info.get('is_docker') else '否'}
   |    ")
152|    logger.info(f"Kubernetes:  {'是' if info.get('is_k8s') else '否'}")
153|    logger.info(f"GPU/CUDA:    {'可用' if info.get('has_cuda') else '不
   |    可用'} ({info.get('cuda_count', 0)} 个)")
154|    logger.info(f"昇腾NPU:     {'可用' if info.get('has_ascend') else '
   |    不可用'}")
155|    logger.info("推荐配置:")
156|    logger.info(f"  并行工作数: {config.get_optimal_workers()}")
157|    configured_mode = config.get('simulation',
{}).get('run_mode', 'stan
   |    dard')
158|    logger.info(f"  运行模式:   {configured_mode} (来自配置)")
159|
160|    adapters = {
161|        '本地CPU': LocalAdapter(),
162|        '远程服务': RemoteAdapter(),
163|    }
164|    if CudaAdapter is not None:
165|        adapters['NVIDIA CUDA'] = CudaAdapter()
166|    if AscendAdapter is not None:
167|        adapters['华为昇腾'] = AscendAdapter()
168|
169|    logger.info("可用适配器:")
170|    for name, adapter in adapters.items():
171|        available = "  可用" if adapter.is_available() else "  不可用"
172|        cap = adapter.get_capability()
173|        logger.info(f"  {name}: {available}")
174|        logger.info(f"    {cap.description}")
```

```
175|
176|def benchmark_adapters(args):
177|    """适配器基准测试命令"""
178|    logger.info("适配器基准测试")
179|    logger.info("=" * 60)
180|
181|    adapters_to_test = []
182|
183|    local = LocalAdapter()
184|    if local.is_available():
185|        adapters_to_test.append(('本地CPU', local))
186|
187|    remote = RemoteAdapter()
188|    if args.remote_url:
189|        remote_config = {'api_url': args.remote_url}
190|        if remote.initialize(remote_config):
191|            adapters_to_test.append(('远程服务', remote))
192|
193|    if CudaAdapter is not None:
194|        cuda = CudaAdapter()
195|        if cuda.is_available():
196|            adapters_to_test.append(('NVIDIA CUDA', cuda))
197|
198|    if AscendAdapter is not None:
199|        ascend = AscendAdapter()
200|        if ascend.is_available():
201|            adapters_to_test.append(('华为昇腾', ascend))
202|
203|    if not adapters_to_test:
204|        logger.warning("没有可用的适配器进行测试")
205|        return
206|
207|    for name, adapter in adapters_to_test:
208|        adapter.initialize({
209|            'num_servers': args.bench_servers,
210|            'simulation_hours': args.bench_hours,
211|            'parallel_workers': 2
212|        })
213|        logger.info(f"测试 {name}...")
214|        import time
215|        start = time.time()
216|        result = adapter.execute(
217|            num_servers=args.bench_servers,
218|            simulation_hours=args.bench_hours,
```

```
            algorithms=['greedy'],
            visualization=False,
            save_results=False
        )
        elapsed = time.time() - start
        logger.info(f"  耗时: {elapsed:.2f}s")
        bench = _display_payload(result)
        logger.info(f"  结果: {'成功' if bench.get('success') else '失败'}")
        adapter.cleanup()

def train_rl_agent(args):
    """训练 RL 调度器（GitHub Skills 集成 — Stable Baselines3 PPO）"""
    from src.plugins._deps import has_gymnasium, has_sb3, get_dependency_status

    logger.info("=" * 60)
    logger.info("RL 调度器训练 (GitHub Skills 集成)")
    logger.info("=" * 60)
    logger.info(f"依赖探测: gymnasium={has_gymnasium()}, stable-baselines3={has_sb3()}")
    if not has_gymnasium() or not has_sb3():
        logger.error("依赖缺失，请先执行: pip install gymnasium stable-baselines3")
        return

    from src.plugin import PluginManager

    pm = PluginManager()
    discovered = pm.discover_plugins(plugin_dir="src/plugins")
    logger.info(f"自动发现插件: {discovered} 个")
    for name in pm._plugin_classes.keys():
        logger.info(f"  - {name}")

    if "RLSchedulerPlugin" not in pm._plugin_classes:
        logger.error("未发现 RLSchedulerPlugin，请检查 src/plugins/rl_ppo_scheduler.py")
        return

    config = {
        "num_servers": args.servers,
        "simulation_hours": args.hours,
        "model_path": args.save_path,
```

```
    }
    if not pm.load_plugin("RLSchedulerPlugin", config=config):
        logger.error("RLSchedulerPlugin 加载失败")
        return

    logger.info(
        f"开始训练 PPO agent: servers={args.servers}, hours={args.hours}, "
        f"timesteps={args.timesteps}, save={args.save_path}"
    )
    result = pm.execute_plugin(
        "RLSchedulerPlugin",
        algorithms=["rl_ppo"],
        train=True,
        total_timesteps=args.timesteps,
        model_path=args.save_path,
        learning_rate=args.lr,
        seed=args.seed,
    )

    if result.success:
        logger.info("  训练完成")
        logger.info(f"  模型路径: {result.data.get('model_path')}")
        logger.info(f"  训练耗时: {result.data.get('training_time_s')} s")
        logger.info(f"  训练步数: {result.data.get('total_timesteps')}")
        logger.info(f"  推理: pm.execute_plugin('RLSchedulerPlugin', train=False, model_path='{args.save_path}')")
    else:
        logger.error(f"训练失败: {result.message} [{result.error_code}]")
        if result.error_detail:
            logger.error(f"  详情: {result.error_detail}")


def check_deps(args):
    """检查 GitHub Skills 可选依赖状态"""
    from src.plugins._deps import get_dependency_status
    status = get_dependency_status()
    logger.info("=" * 60)
    logger.info("GitHub Skills 可选依赖状态")
    logger.info("=" * 60)
    for dep_name, info in status.items():
        avail = "  可用" if info["available"] else "  缺失"
```

```
        version = info.get("version") or "—"
        logger.info(f"  {dep_name}: {avail}  (version={version})")
        if not info["available"]:
            logger.info(f"    安装: {info['install_hint']}")


def create_parser():
    """创建命令行解析器"""
    parser = argparse.ArgumentParser(
        description='校园数据中心智能散热与算力调度系统 - CLI工具 v1.0',
        formatter_class=argparse.ArgumentDefaultsHelpFormatter
    )

    subparsers = parser.add_subparsers(dest='command', help='可用命令')

    run_parser = subparsers.add_parser('run', help='运行仿真')
    run_parser.add_argument('--servers', type=int, default=10, help='服
    务器数量 (1-1000)')
    run_parser.add_argument('--hours', type=int, default=24, help='仿真
    时长/小时 (1-720)')
    run_parser.add_argument('--workers', type=int, default=4, help='并行
    工作器数')
    run_parser.add_argument('--mode', type=str,
default='standard',
                            choices=['standard', 'distributed', '
large_sc
    ale'],
                        help='运行模式')
    run_parser.add_argument('--algorithms', type=str,
default='greedy,ge
    netic,round_robin',
                        help='算法列表(逗号分隔)')
    run_parser.add_argument('--adapter', type=str, default='auto',
                            choices=['auto', 'local', 'remote', '
cuda', '
    ascend'],
                        help='适配器类型')
    run_parser.add_argument('--no-plot', action='store_true', help='跳过
    绘图')
    run_parser.add_argument('--no-save', action='store_true', help='跳过
    保存')
    run_parser.set_defaults(func=run_simulation)

    api_parser = subparsers.add_parser('api', help='启动API服务')
```

```
    api_parser.add_argument('--host', type=str,
default='0.0.0.0', help=
    '绑定地址')
    api_parser.add_argument('--port', type=int, default=5000, help='端口
    号')
    api_parser.add_argument('--debug', action='store_true', help='调试模
    式')
    api_parser.set_defaults(func=run_api_server)

    list_parser = subparsers.add_parser('list', help='列出可用算法')
    list_parser.set_defaults(func=list_algorithms)

    env_parser = subparsers.add_parser('env', help='环境检测')
    env_parser.set_defaults(func=detect_environment)

    bench_parser = subparsers.add_parser('benchmark', help='适配器基准测
    试')
    bench_parser.add_argument('--bench-servers', type=int,
default=10, h
    elp='测试服务器数')
    bench_parser.add_argument('--bench-hours', type=int,
default=4, help
    ='测试仿真时长')
    bench_parser.add_argument('--remote-url', type=str, help='远程API地
    址')
    bench_parser.set_defaults(func=benchmark_adapters)

    # 检查 GitHub Skills 可选依赖
    deps_parser = subparsers.add_parser('deps', help='检查 GitHub Skills
     可选依赖')
    deps_parser.set_defaults(func=check_deps)

    return parser

def cli():
    """CLI入口函数"""
    parser = create_parser()
    args = parser.parse_args()

    if args.command is None:
        parser.print_help()
        logger.info("可用命令:")
        logger.info("  run       运行仿真（支持自动环境检测）")
        logger.info("  api       启动API服务")
```

```
        logger.info("  list      列出可用算法")
        logger.info("  env       检测运行环境")
        logger.info("  benchmark 适配器基准测试")
        logger.info("  deps      检查可选依赖")
        return

    try:
        args.func(args)
    except Exception as e:
        logger.error(f"命令执行失败: {str(e)}", exc_info=True)
        return 1
    return 0

if __name__ == '__main__':
    import sys
    sys.exit(cli())
```

28. src/data/__init__.py

```
from .catalog import Catalog

__all__ = ['Catalog']
```

29. src/data/catalog.py

```
"""Read-only access to the provenance-tagged data catalog.

The catalog is a plain SQLite file (``data/catalog.db``) built by the sc
   |    ripts in
``tools/etl``. Every row carries a ``source_id`` pointing at
``meta_data_source``, so any number that ends up in a report can be trac
   |    ed back
to a publisher, a URL and a retrieval timestamp.

Nothing here reaches out to the network: the simulation must run offline
   |    , so the
catalog is a frozen snapshot taken while preparing the demo.
"""

import os
import sqlite3
from typing import Any, Dict, List, Optional, Tuple

```

```
from src.utils.logger import setup_logger

logger = setup_logger("catalog")

PROJECT_ROOT = os.path.abspath(os.path.join(os.path.dirname(__file__), "
    ..", ".."))
DEFAULT_DB = os.path.join(PROJECT_ROOT, "data", "catalog.db")

PROVINCE_REGION = {
    "北京市": "华北", "天津市": "华北", "河北省": "华北", "山西省": "华北",
    "内蒙古自治区": "华北",
    "辽宁省": "东北", "吉林省": "东北", "黑龙江省": "东北",
    "上海市": "华东", "江苏省": "华东", "浙江省": "华东", "安徽省": "华东",
    "福建省": "华东", "江西省": "华东", "山东省": "华东",
    "河南省": "华中", "湖北省": "华中", "湖南省": "华中",
    "陕西省": "西北", "甘肃省": "西北", "青海省": "西北",
    "宁夏回族自治区": "西北", "新疆维吾尔自治区": "西北",
    "广东省": "南方", "广西壮族自治区": "南方", "海南省": "南方",
    "重庆市": "西南", "四川省": "西南", "贵州省": "西南",
    "云南省": "西南", "西藏自治区": "西南",
}


def region_for_province(province: Optional[str]) -> Optional[str]:
    return PROVINCE_REGION.get(province) if province else None


REQUIRED_TABLES = (
    "meta_data_source",
    "meta_dataset_version",
    "dim_ambient_hourly",
    "dim_server_type",
    "dim_pue_benchmark",
    "dim_emission_factor",
    "fact_task_arrivals",
)


class Catalog:
    """Thin, read-only wrapper around the catalog database."""

```

```
    def __init__(self, db_path: Optional[str] = None) -> None:
        raw = db_path or DEFAULT_DB
        self.db_path = raw if os.path.isabs(raw) else
os.path.join(PROJE
    CT_ROOT, raw)
        self._conn: Optional[sqlite3.Connection] = None

    def connect(self) -> sqlite3.Connection:
        if self._conn is None:
            conn = sqlite3.connect(self.db_path)
            conn.row_factory = sqlite3.Row
            self._conn = conn
        return self._conn

    def close(self) -> None:
        if self._conn is not None:
            self._conn.close()
            self._conn = None

    def __enter__(self) -> "Catalog":
        self.connect()
        return self

    def __exit__(self, *exc: Any) -> None:
        self.close()

    def available(self) -> bool:
        if not os.path.exists(self.db_path):
            return False
        try:
            conn = self.connect()
            names = {
                row["name"]
                for row in conn.execute(
                    "SELECT name FROM sqlite_master WHERE
type='table'"
                )
            }
        except sqlite3.Error as exc:
            logger.warning("读取数据目录失败: %s", exc)
            return False
        return "dim_ambient_hourly" in names

    def table_names(self) -> List[str]:
```

```
        conn = self.connect()
        return [
            row["name"]
            for row in conn.execute("SELECT name FROM
sqlite_master WHER
    E type='table'")
        ]

    def _rows(self, sql: str, params: Tuple[Any, ...] = ()) ->
List[sqli
    te3.Row]:
        try:
            return list(self.connect().execute(sql, params))
        except sqlite3.Error as exc:
            logger.warning("查询失败 (%s): %s", sql.split()[1] if len(sq
    l.split()) > 1 else "?", exc)
            return []

    def provenance(self, source_id: Optional[str] = None) ->
List[Dict[s
    tr, Any]]:
        if source_id:
            rows = self._rows(
                "SELECT * FROM meta_data_source WHERE source_id = ?
", (s
    ource_id,)
            )
        else:
            rows = self._rows("SELECT * FROM meta_data_source
ORDER BY s
    ource_id")
        return [dict(row) for row in rows]

    def versions(self) -> List[Dict[str, Any]]:
        return [dict(row) for row in self._rows(
            "SELECT * FROM meta_dataset_version ORDER BY
created_at DESC
    "
        )]

    def ambient_series(self, site_id: Optional[str] = None) ->
Optional[
    List[float]]:
        """Hourly outdoor temperature for a site.
```

```
128|
129|        Uses the stored hourly series when one exists. Otherwise
falls b
   |    ack to
130|        the city's daily max/min/mean and expands it with a
standard diu
   |    rnal
131|        curve (peak at 15:00, trough at 03:00), which is how a
school ma
   |    ps to
132|        its local climate without storing 8760 rows for 345
cities.
133|        """
134|        if site_id:
135|            rows = self._rows(
136|                "SELECT temp_c FROM dim_ambient_hourly WHERE
site_id = ?
   |     ORDER BY ts",
137|                (site_id,),
138|            )
139|        else:
140|            rows = self._rows("SELECT temp_c FROM
dim_ambient_hourly ORD
   |    ER BY ts")
141|        values = [float(row["temp_c"]) for row in rows if
row["temp_c"]
   |    is not None]
142|        if values:
143|            return values
144|        if site_id:
145|            daily = self.city_daily(site_id)
146|            if daily:
147|                return self._expand_daily(daily)
148|        return None
149|
150|    @staticmethod
151|    def _expand_daily(daily: List[Dict[str, Any]]) -> List[float]:
152|        import math
153|        series: List[float] = []
154|        for row in daily:
155|            tmax = float(row["tmax_c"])
156|            tmin = float(row["tmin_c"])
157|            tmean = float(row["tmean_c"])
158|            amplitude = max(0.0, (tmax - tmin) / 2.0)
```

```
159|            for hour in range(24):
160|                series.append(
161|                    tmean + amplitude * math.sin(2.0 * math.pi *
(hour -
   |     9) / 24.0)
162|                )
163|        return series
164|
165|    def city_daily(self, city: str) -> Optional[List[Dict[str,
Any]]]:
166|        rows = self._rows(
167|            "SELECT city, date, tmax_c, tmin_c, tmean_c FROM
dim_city_da
   |    ily "
168|            "WHERE city = ? ORDER BY date",
169|            (city,),
170|        )
171|        return [dict(row) for row in rows] or None
172|
173|    def cities(self) -> List[Dict[str, Any]]:
174|        rows = self._rows(
175|            "SELECT d1.city, "
176|            "  (SELECT d2.province FROM dim_school d2 WHERE
d2.city = d1
   |    .city "
177|            "   GROUP BY d2.province ORDER BY COUNT(*) DESC LIMIT
1) AS
   |    province, "
178|            "  COUNT(*) AS school_count "
179|            "FROM dim_school d1 GROUP BY d1.city ORDER BY
province, d1.c
   |    ity"
180|        )
181|        return [dict(row) for row in rows]
182|
183|    def city_climate_summary(self) -> List[Dict[str, Any]]:
184|        rows = self._rows(
185|            "SELECT city, AVG(tmean_c) AS avg_temp FROM
dim_city_daily G
   |    ROUP BY city"
186|        )
187|        return [dict(row) for row in rows]
188|
```

```
    def _school_filter(self, query: Optional[str], city:
Optional[str]):
        clauses = []
        params: List[Any] = []
        if city:
            clauses.append("city = ?")
            params.append(city)
        if query:
            clauses.append("school_name LIKE ?")
            params.append("%" + query + "%")
        where = (" WHERE " + " AND ".join(clauses)) if clauses
else ""
        return where, tuple(params)

    def count_schools(self, query: Optional[str] = None, city:
Optional[
    str] = None) -> int:
        where, params = self._school_filter(query, city)
        rows = self._rows("SELECT COUNT(*) AS n FROM dim_school"
+ where
    , params)
        return int(rows[0]["n"]) if rows else 0

    def schools(self, query: Optional[str] = None, city:
Optional[str] =
     None,
                limit: int = 50) -> List[Dict[str, Any]]:
        where, params = self._school_filter(query, city)
        rows = self._rows(
            "SELECT school_name, province, city, level,
department FROM
    dim_school"
            + where + " ORDER BY school_name LIMIT ?",
            params + (int(limit),),
        )
        return [dict(row) for row in rows]

    def task_stream(
        self, trace_id: Optional[str] = None, hours:
Optional[int] = Non
    e
    ) -> Optional[List[List[Tuple[int, int, str, int, int]]]]:
        """Rebuild the pre-built task stream as per-hour spec
lists.
```

```
220|
221|        Rows are expected to carry the hour bucket, resource
request, pr
   |    iority,
222|        workload class, duration and deadline offset recorded
during imp
   |    ort.
223|        """
224|        clauses: List[str] = []
225|        params: List[Any] = []
226|        if trace_id:
227|            clauses.append("trace_id = ?")
228|            params.append(trace_id)
229|        where = (" WHERE " + " AND ".join(clauses)) if clauses
else ""
230|        rows = self._rows(
231|            "SELECT seq, hour, load_units, priority,
workload_class, dur
   |    ation_h,"
232|            " deadline_offset_h FROM fact_task_arrivals" + where
+ " ORD
   |    ER BY seq",
233|            tuple(params),
234|        )
235|        if not rows:
236|            return None
237|
238|        buckets: Dict[int, List[Tuple[int, int, str, int, int]]]
= {}
239|        for row in rows:
240|            hour = int(row["hour"])
241|            if hours is not None and hour >= hours:
242|                continue
243|            buckets.setdefault(hour, []).append((
244|                int(row["load_units"]),
245|                int(row["priority"]),
246|                str(row["workload_class"]),
247|                int(row["duration_h"]),
248|                int(row["deadline_offset_h"]),
249|            ))
250|
251|        if not buckets:
252|            return None
253|        top = max(buckets)
```

```
        return [buckets.get(hour, []) for hour in range(top + 1)]

    def pue_benchmarks(self) -> List[Dict[str, Any]]:
        return [dict(row) for row in self._rows(
            "SELECT * FROM dim_pue_benchmark ORDER BY pue_limit"
        )]

    def city_province(self, city: str) -> Optional[str]:
        rows = self._rows(
            "SELECT province FROM dim_school WHERE city = ? "
            "GROUP BY province ORDER BY COUNT(*) DESC LIMIT 1",
            (city,),
        )
        return rows[0]["province"] if rows else None

    def emission_factor(
        self, region: Optional[str] = None, year: Optional[int] =
None
    ) -> Optional[float]:
        if region:
            sql = "SELECT kg_co2_per_kwh FROM dim_emission_factor
WHERE
    region = ?"
            params: List[Any] = [region]
            if year is not None:
                sql += " AND year = ?"
                params.append(int(year))
            rows = self._rows(sql, tuple(params))
            for row in rows:
                if row["kg_co2_per_kwh"] is not None:
                    return float(row["kg_co2_per_kwh"])

        rows = self._rows(
            "SELECT kg_co2_per_kwh FROM dim_emission_factor "
           "WHERE region = '全国（电力平均）' LIMIT 1"
        )
        for row in rows:
            if row["kg_co2_per_kwh"] is not None:
                return float(row["kg_co2_per_kwh"])
        return None

    def summary(self) -> Dict[str, Any]:
        """Compact description used by the API and the
dashboard."""
```

```
        info: Dict[str, Any] = {
            "db_path": self.db_path,
            "available": self.available(),
        }
        if not info["available"]:
            return info
        info["tables"] = self.table_names()
        info["sources"] = self.provenance()
        ambient = self.ambient_series()
        info["ambient_hours"] = len(ambient) if ambient else 0
        return info
```

30. src/models/__init__.py

```
from typing import List, Tuple, Optional

from .server import Server
from .task import Task
from .energy_model import EnergyModel
from .temperature_model import TemperatureModel

__all__ = [
    'Server',
    'Task',
    'EnergyModel',
    'TemperatureModel'
]
```

31. src/models/energy_model.py

```
import numpy as np
from typing import Any, Dict, List, Optional
from src.models.server import Server

class EnergyModel:
    """IT + cooling power model.

    Cooling is derived from the heat the IT load actually
produces, scal
    ed by
    the outdoor temperature. A part-load term keeps the chillers
from lo
    oking
    perfectly efficient when the room is nearly empty, which is
what mak
    es PUE
```

```
    depend on the scheduling decision instead of only on the
weather.
    """

    def __init__(self, config: Optional[Dict[str, Any]] = None)
-> None:
        cfg = config
        if cfg is None:
            try:
                from src.utils.config import Config
                cfg = Config().get("energy", {}) or {}
            except Exception:
                cfg = {}

        self.cop_base: float = float(cfg.get("cop_base", 2.5))
        self.cop_delta_coefficient: float =
float(cfg.get("cop_delta_coe
    fficient", 0.1))
        self.heat_to_power_ratio: float =
float(cfg.get("heat_to_power_r
    atio", 0.8))
        self.pue_base: float = float(cfg.get("pue_base", 1.8))
        self.target_temp: float = float(cfg.get("target_temp",
22))
        self.part_load_penalty: float =
float(cfg.get("part_load_penalty
    ", 0.5))
        self.temp_penalty_coefficient: float =
float(cfg.get("temp_penal
    ty_coefficient", 1.0))
        self.temp_reference: float =
float(cfg.get("temp_reference", 35.
    0))
        self.cooling_efficiency_base: float = 0.5

    def server_power(self, server: Server) -> float:
        return server.calculate_power()

    def calculate_server_energy(self, server: Server) -> float:
        return server.calculate_power()

    def total_it_power(self, servers: List[Server]) -> float:
        return float(sum(s.calculate_power() for s in servers))

```

```
    def get_cop(self, ambient_temp: float, target_temp:
Optional[float]
    = None) -> float:
        target = self.target_temp if target_temp is None else
float(targ
    et_temp)
        delta_temp = max(0.0, target - ambient_temp)
        return self.cop_base + delta_temp *
self.cop_delta_coefficient

    def rated_heat(self, servers: List[Server]) -> float:
        if not servers:
            return 0.0
        return float(
            sum(max(0.0, s.max_power - s.base_power) for s in
servers) *
     self.heat_to_power_ratio
        )

    def heat_load_ratio(self, servers: List[Server]) -> float:
        rated = self.rated_heat(servers)
        if rated <= 0:
            return 1.0
        heat = self.total_it_power(servers) *
self.heat_to_power_ratio
        return float(min(1.0, max(0.0, heat / rated)))

    def temperature_lift_factor(self, peak_temp: Optional[float])
-> flo
    at:
        """Chillers work harder when the hot aisle runs hotter."""
        if peak_temp is None:
            return 1.0
        reference = max(1.0, self.temp_reference)
        lift = max(0.0, (float(peak_temp) - reference) /
reference)
        return float(1.0 + self.temp_penalty_coefficient * lift)

    def calculate_cooling_energy(
        self,
        servers: List[Server],
        ambient_temp: float,
        target_temp: Optional[float] = None,
        peak_temp: Optional[float] = None,
```

```
    ) -> float:
        total_heat = self.total_it_power(servers) *
self.heat_to_power_r
    atio
        cop = max(1e-6, self.get_cop(ambient_temp, target_temp))
        ideal = total_heat / cop
        ratio = self.heat_load_ratio(servers)
        partial = 1.0 + self.part_load_penalty * (1.0 - ratio)
        return float(ideal * partial *
self.temperature_lift_factor(peak
    _temp))

    def calculate_total_energy(
        self,
        servers: List[Server],
        ambient_temp: float,
        target_temp: Optional[float] = None,
        peak_temp: Optional[float] = None,
    ) -> float:
        return self.total_it_power(servers) +
self.calculate_cooling_ene
    rgy(
            servers, ambient_temp, target_temp, peak_temp
        )

    def get_pue(
        self,
        servers: List[Server],
        ambient_temp: float,
        target_temp: Optional[float] = None,
        peak_temp: Optional[float] = None,
    ) -> Optional[float]:
        """Return the PUE, or None when there is no IT load to
divide by
    .

        Returning None keeps "the room only draws standby power"
visible
     in the
        output instead of masking it as a plausible-looking 1.8.
        """
        it_power = self.total_it_power(servers)
        if it_power <= 0:
            return None
```

```
111|        total = it_power + self.calculate_cooling_energy(
112|            servers, ambient_temp, target_temp, peak_temp
113|        )
114|        return float(total / it_power)
115|
116|    def get_dcie(self, servers: List[Server], ambient_temp:
float) -> Op
   |    tional[float]:
117|        pue = self.get_pue(servers, ambient_temp)
118|        if pue is None:
119|            return None
120|        return float(1.0 / pue)
```

32. src/models/server.py

```
  1|from typing import List, Optional
  2|from src.models.task import Task
  3|
  4|HEAT_TO_POWER_RATIO = 0.8
  5|DEFAULT_DISSIPATION = 8.0
  6|DEFAULT_THERMAL_INERTIA = 0.25
  7|
  8|
  9|class Server:
 10|    """A single machine.
 11|
 12|    Thermal behaviour is a first-order lag towards an equilibrium
that d
   |    epends
 13|    on the generated heat, the outdoor temperature and how hard
the fans
   |     run:
 14|
 15|        equilibrium = ambient + heat / (dissipation *
cooling_efficiency
   |    )
 16|
 17|    With the nominal coefficients this keeps a fully loaded
standard ser
   |    ver
 18|    around 45 C at 25 C ambient, which is a realistic band; the
previous
 19|    linear model pinned every loaded server at ``max_temp``
within hours
   |    .
```

```
    """

    def __init__(
        self,
        server_id: int,
        max_load: float = 100,
        max_temp: float = 85,
        base_power: float = 100,
        max_power: float = 300,
        initial_temp: float = 25,
        server_type: str = "standard",
        idle_power_factor: float = 0.6,
        dissipation_coefficient: float = DEFAULT_DISSIPATION,
        thermal_inertia: float = DEFAULT_THERMAL_INERTIA,
    ) -> None:
        self.server_id: int = server_id
        self.max_load: float = max_load
        self.max_temp: float = max_temp
        self.base_power: float = base_power
        self.max_power: float = max_power
        self.idle_power_factor: float = float(idle_power_factor)
        self.dissipation_coefficient: float =
float(dissipation_coeffici
    |   ent)
        self.thermal_inertia: float = float(thermal_inertia)
        self.current_temp: float = initial_temp
        self.current_load: float = 0
        self.server_type: str = server_type
        self.last_ambient: float = initial_temp
        self.last_fan_speed: float = 0.5
        self.tasks: List[Task] = []

    def assign_task(self, task: Task) -> bool:
        if self.current_load + task.load <= self.max_load:
            self.current_load += task.load
            self.tasks.append(task)
            return True
        return False

    def remove_task(self, task: Task) -> bool:
        if task in self.tasks:
            self.tasks.remove(task)
            self.current_load -= task.load
            return True
```

```
        return False

    def is_overloaded(self) -> bool:
        return self.current_load >= self.max_load * 0.95

    def is_overheated(self) -> bool:
        return self.current_temp >= self.max_temp * 0.95

    def utilization(self) -> float:
        return self.current_load / self.max_load

    def calculate_power(self) -> float:
        """Instantaneous draw.

        An idle server still draws a fraction of its base power, which i
    s what
        makes consolidation (fewer servers kept busy) actually save ener
    gy.
        """
        if self.current_load <= 0:
            return self.base_power * self.idle_power_factor
        load_factor = self.current_load / self.max_load
        return self.base_power + (self.max_power - self.base_power) * lo
    ad_factor

    def heat_generation(self, load: Optional[float] = None) -> float:
        current = self.current_load if load is None else load
        load_factor = current / self.max_load
        return float(load_factor * (self.max_power - self.base_power) *
    HEAT_TO_POWER_RATIO)

    def equilibrium_temp(self, ambient_temp: float, cooling_efficiency:
    float) -> float:
        efficiency = max(0.05, float(cooling_efficiency))
        return ambient_temp + self.heat_generation() / (
            self.dissipation_coefficient * efficiency
        )

```

```
    def update_temperature(self, ambient_temp: float,
cooling_efficiency
    : float = 0.5) -> None:
        self.last_ambient = ambient_temp
        self.last_fan_speed = cooling_efficiency
        target = self.equilibrium_temp(ambient_temp,
cooling_efficiency)
        self.current_temp = min(
            self.max_temp,
            max(min(ambient_temp, target), self.current_temp +
self.ther
    mal_inertia * (target - self.current_temp)),
        )

    def predicted_temp_after(self, additional_load: float) ->
float:
        """Conservative upper bound on the temperature after
adding a ta
    sk.

        Uses half the nominal dissipation so the estimate errs on
the sa
    fe side.
        """
        heat = self.heat_generation(self.current_load +
additional_load)
        degraded = self.dissipation_coefficient * 0.5
        target = self.last_ambient + heat / degraded
        return float(min(self.max_temp, max(self.current_temp,
target)))

    def __repr__(self) -> str:
        return (
            f"Server(id={self.server_id},
type={self.server_type}, "
            f"load={self.current_load:.1f},
temp={self.current_temp:.1f}
    )"
        )
```

33. src/models/task.py

```
import uuid
from typing import Optional

```

```
  4|class Task:
  5|    def __init__(
  6|        self,
  7|        load: int = 10,
  8|        priority: int = 1,
  9|        deadline: Optional[int] = None,
 10|        arrival_time: int = 0,
 11|        workload_class: str = "cpu_bound",
 12|        duration: int = 0,
 13|    ) -> None:
 14|        self.task_id: str = str(uuid.uuid4())[:8]
 15|        self.load: int = load
 16|        self.priority: int = priority
 17|        self.deadline: Optional[int] = deadline
 18|        self.arrival_time: int = arrival_time
 19|        self.workload_class: str = workload_class
 20|        self.duration: int = int(duration)
 21|        self.remaining_hours: float = float(duration)
 22|        self.created_time: int = 0
 23|        self.start_time: Optional[int] = None
 24|        self.completed_time: Optional[int] = None
 25|
 26|    def is_completed(self) -> bool:
 27|        return self.completed_time is not None
 28|
 29|    def get_waiting_time(self, current_time: int) -> int:
 30|        if self.start_time is None:
 31|            return current_time - self.created_time
 32|        return self.start_time - self.created_time
 33|
 34|    def is_deadline_violated(self, current_time: int) -> bool:
 35|        """Whether the deadline had already passed at
``current_time``.
 36|
 37|        This is an assignment-time check. The authoritative SLA
metric i
   |    s
 38|        computed after completion via ``missed_deadline``.
 39|        """
 40|        if self.deadline is None:
 41|            return False
 42|        return current_time > self.deadline
 43|
 44|    def missed_deadline(self, last_hour: int) -> bool:
```

```
 45|        """Whether the task failed to meet its deadline within
the horiz
   |    on.
 46|
 47|        A task counts as a miss when it finished after its
deadline, or
   |    when it
 48|        never finished although the deadline has already elapsed.
 49|        """
 50|        if self.deadline is None:
 51|            return False
 52|        if self.completed_time is not None:
 53|            return self.completed_time > self.deadline
 54|        return self.deadline <= last_hour
 55|
 56|    def service_hour(self, amount: float = 1.0) -> float:
 57|        """Consume one hour of execution service and return what
is left
   |    ."""
 58|        self.remaining_hours = max(0.0, self.remaining_hours -
amount)
 59|        return self.remaining_hours
 60|
 61|    def is_service_complete(self) -> bool:
 62|        return self.remaining_hours <= 0.0
 63|
 64|    @property
 65|    def slack_hours(self) -> Optional[int]:
 66|        if self.deadline is None:
 67|            return None
 68|        return self.deadline - self.arrival_time
 69|
 70|    def __repr__(self) -> str:
 71|        return (
 72|            f"Task({self.task_id}, load={self.load},
priority={self.prio
   |    rity}, "
 73|            f"class={self.workload_class},
deadline={self.deadline}, "
 74|            f"duration={self.duration})"
 75|        )
```

34. src/models/temperature_model.py

```
  1|import numpy as np
```

```
  2|from typing import List, Optional
  3|from src.models.server import Server
  4|
  5|class TemperatureModel:
  6|    """Room-level bookkeeping and the fan speed control law."""
  7|
  8|    def __init__(
  9|        self,
 10|        room_volume: float = 100,
 11|        air_flow_rate: float = 10,
 12|        initial_temp: float = 25,
 13|        fan_target_temp: float = 35,
 14|    ) -> None:
 15|        self.room_volume: float = room_volume
 16|        self.air_flow_rate: float = air_flow_rate
 17|        self.current_temp: float = initial_temp
 18|        self.target_temp: float = fan_target_temp
 19|
 20|    def update_room_temperature(self, servers: List[Server],
ambient_tem
   |    p: float, cooling_power: float) -> float:
 21|        total_heat = sum(s.calculate_power() * 0.8 for s in
servers)
 22|        heat_removed = cooling_power * 0.9
 23|        heat_change = total_heat - heat_removed
 24|        temp_change = heat_change / (1000 * 1.2 *
self.room_volume)
 25|        self.current_temp = max(
 26|            ambient_temp - 5,
 27|            min(ambient_temp + 20, self.current_temp +
temp_change)
 28|        )
 29|        return self.current_temp
 30|
 31|    def get_cooling_demand(self) -> float:
 32|        delta = self.current_temp - self.target_temp
 33|        if delta <= 0:
 34|            return 0.0
 35|        return delta * 100
 36|
 37|    def adjust_fan_speed(
 38|        self,
 39|        target_temp: Optional[float] = None,
 40|        reference_temp: Optional[float] = None,
```

```
    ) -> float:
        """Fan speed in [0.2, 1.0] driven by the hottest machine.

        ``reference_temp`` lets the caller feed in the peak
server tempe
    rature;
        without it the room temperature is used.
        """
        if target_temp is not None:
            self.target_temp = float(target_temp)

        current = self.current_temp if reference_temp is None
else float
    (reference_temp)
        delta = current - self.target_temp

        if delta < -2:
            return 0.2
        elif delta < 0:
            return 0.4
        elif delta < 2:
            return 0.6
        elif delta < 5:
            return 0.8
        else:
            return 1.0
```

35. src/plugin/__init__.py

```
"""
插件系统 v1.0

提供标准化的插件接口和统一调用机制，支持:
- 插件注册/发现/加载/卸载/执行
- 事件总线（插件间通信）
- 依赖解析
- 热重载
- 状态监控与指标收集

快速使用:
    from src.plugin import PluginManager, PluginBase, PluginResult

    pm = PluginManager()
    pm.register_plugin(MyPlugin)
    pm.load_plugin("MyPlugin", config={"key": "value"})
```

```
    result = pm.execute_plugin("MyPlugin", arg1=123)
    print(result.to_json())
    pm.unload_plugin("MyPlugin")
"""

from .protocol import (
    PluginResult, PluginInfo, PluginManifest, PluginStatus,
    PluginPriority, PluginEvent, PluginCallback, PluginConfig
)
from .base import PluginBase
from .manager import PluginManager

__all__ = [
    'PluginBase',
    'PluginManager',
    'PluginResult',
    'PluginInfo',
    'PluginManifest',
    'PluginStatus',
    'PluginPriority',
    'PluginEvent',
    'PluginCallback',
    'PluginConfig'
]
```

36. src/plugin/base.py

```
"""
插件抽象基类 v1.0 — 标准化接口规范

所有插件必须继承此基类并实现其抽象方法。

核心生命周期:
    __init__ → initialize(config) → [pre_execute → execute → post_execut
    e]×N → cleanup

增强功能（新增）:
    - PluginResult 统一返回类型
    - 执行前后钩子（pre_execute / post_execute）
    - 错误处理钩子（on_error）
    - 事件回调注册系统
    - 配置验证（validate_config）
    - 运行状态查询（get_status）
    - 插件清单（get_manifest）供第三方发现
```

```
17|
18|用法示例:
19|
20|    class MyPlugin(PluginBase):
21|        def get_manifest(self):
22|            return PluginManifest(
23|                name="MyPlugin",
24|                version="1.0.0",
25|                supported_operations=["process", "analyze"],
26|                input_schema={"data": "list"},
27|                output_schema={"result": "dict"}
28|            )
29|
30|        def initialize(self, config):
31|            self.threshold = config.get("threshold", 0.5)
32|            return True
33|
34|        def execute(self, **kwargs):
35|            data = kwargs.get("data", [])
36|            processed = [x for x in data if x > self.threshold]
37|           return PluginResult.ok(data=processed, message="处理完成")
38|"""
39|
40|from abc import ABC, abstractmethod
41|from typing import Dict, Any, Optional, List
42|import time
43|from datetime import datetime
44|
45|from src.plugin.protocol import (
46|    PluginResult, PluginInfo, PluginManifest, PluginStatus,
47|    PluginPriority, PluginEvent, PluginCallback, PluginConfig,
48|    PluginContext
49|)
50|from src.utils.logger import setup_logger
51|
52|
53|class PluginBase(ABC):
54|    """
55|    插件抽象基类
56|
57|    接口契约:
58|        必须实现: get_manifest, initialize, execute, cleanup
59|        可选覆盖: pre_execute, post_execute, on_error, validate_config
60|
```

```
    所有插件的 execute() 必须返回 PluginResult 实例。
    """

    def __init__(self):
        self.logger = setup_logger(self.__class__.__name__)
        self._status = PluginStatus.REGISTERED
        self._context: PluginContext = {}
        self._config: PluginConfig = {}
        self._callbacks: Dict[str, List[PluginCallback]] = {}
        self._execution_count: int = 0
        self._total_execution_time_ms: float = 0.0
        self._last_error: Optional[str] = None
        self._loaded_at: Optional[str] = None

    # 必须实现的抽象方法

    @abstractmethod
    def get_manifest(self) -> PluginManifest:
        """
        返回插件清单 — 描述插件的能力、接口和兼容性

        第三方应用通过此清单:
        - 发现可调用的操作
        - 了解输入参数格式
        - 确认版本兼容性

        Returns:
            PluginManifest: 包含名称/版本/操作列表/参数schema的完整清单
        """
        pass

    @abstractmethod
    def initialize(self, config: Optional[PluginConfig] = None)
-> bool:
        """
        初始化插件 — 分配资源、验证配置、建立连接

        调用时机: 插件加载时由 PluginManager 调用
        调用频率: 每次 load_plugin 调用一次

        Args:
            config: 配置字典，包含插件运行所需参数
                    传入 None 表示使用默认配置

```

```
        Returns:
            bool: True 表示初始化成功，False 表示失败

        最佳实践:
        - 在此方法中进行资源分配（文件、网络、数据库连接）
        - 验证 config 中的关键参数
        - 失败时记录详细日志
        """
        pass

    @abstractmethod
    def execute(self, **kwargs) -> PluginResult:
        """
        执行插件核心业务逻辑

        调用时机: 插件加载后，由 PluginManager.execute_plugin() 调用
        调用频率: 可多次调用

        Args:
            **kwargs: 可变关键字参数，由调用方传入

        Returns:
            PluginResult: 统一返回格式，包含:
                - success: 执行是否成功
                - data: 业务产出数据
                - message: 状态描述
                - error_code: 错误码

        注意:
        - **绝对不要**抛出未捕获异常
        - 所有错误通过 PluginResult.fail() 返回
        - 构造函数中已分配 logger，使用 self.logger 记录日志
        """
        pass

    @abstractmethod
    def cleanup(self) -> None:
        """
        清理插件资源 — 释放所有分配的资源

        调用时机: 插件卸载时由 PluginManager 调用
        调用频率: 一次

        要求:
```

```
150|        - 幂等性: 多次调用不会出错
151|        - 防御性: 即使资源未初始化也不会崩溃
152|        - 彻底性: 释放所有数据库连接/文件句柄/线程池等
153|        """
154|        pass
155|
157|    # 可选覆盖的钩子方法 (Hook Methods)
159|
160|    def validate_config(self, config: PluginConfig) -> bool:
161|        """
162|        验证配置参数是否符合插件要求
163|
164|        可覆盖此方法实现自定义验证逻辑:
165|        - 检查必填参数是否存在
166|        - 验证参数取值范围
167|        - 确认参数类型正确
168|
169|        Args:
170|            config: 待验证的配置字典
171|
172|        Returns:
173|            True 表示配置有效，False 表示配置无效
174|
175|        示例:
176|             def validate_config(self, config):
177|                 if 'num_servers' not in config:
178|                   self.logger.error("缺少必填参数: num_servers")
179|                     return False
180|                 if config['num_servers'] < 1 or
config['num_servers'] >
   |    1000:
181|                   self.logger.error("num_servers 必须在 [1, 1000] 范围
   |    内")
182|                     return False
183|                 return True
184|        """
185|        return True
186|
187|    def pre_execute(self, **kwargs) -> Optional[PluginResult]:
188|        """
189|        执行前钩子 — 在主逻辑执行前调用
190|
191|        可用于:
192|        - 参数预处理和验证
```

```
193|        - 设置执行上下文
194|        - 记录执行开始日志
195|
196|        Args:
197|            **kwargs: 传入 execute() 的相同参数
198|
199|        Returns:
200|            None 表示继续执行
201|            PluginResult 表示短路跳过后续执行（如参数验证失败）
202|        """
203|        return None
204|
205|    def post_execute(self, result: PluginResult, **kwargs) ->
PluginResu
   |    lt:
206|        """
207|        执行后钩子 — 在主逻辑执行后调用
208|
209|        可用于:
210|        - 结果后处理
211|        - 记录执行完成日志
212|        - 更新统计指标
213|
214|        Args:
215|            result: execute() 返回的原始结果
216|            **kwargs: 传入 execute() 的相同参数
217|
218|        Returns:
219|            PluginResult: 处理后的结果（可修改原结果或返回新结果）
220|        """
221|        return result
222|
223|    def on_error(self, error: Exception, operation: str) ->
PluginResult
   |    :
224|        """
225|        错误处理钩子 — 当操作发生异常时调用
226|
227|        Args:
228|            error: 发生的异常对象
229|            operation: 发生异常的操作名称（如 'initialize', 'execute', '
   |    cleanup')
230|
231|        Returns:
```

```
232|            PluginResult: 封装错误信息的失败结果
233|
234|        默认实现:
235|        - 记录详细错误日志
236|        - 返回包含错误信息的 PluginResult.fail()
237|         """
238|         self._last_error = str(error)
239|        self.logger.error(f"[{operation}] 异常: {error}", exc_info=True)
240|         return PluginResult.fail(
241|            message=f"{operation} 执行异常",
242|             error_code=f"{operation.upper()}_ERROR",
243|             error_detail=str(error)
244|         )
245|
247|    # 事件回调系统
249|
250|    def on(self, event_type: str, callback: PluginCallback) ->
None:
251|         """
252|        注册事件回调
253|
254|         Args:
255|            event_type: 事件类型 (如 'STATE_CHANGED', 'METRICS_UPDATED')
256|            callback: 回调函数，接收 PluginEvent 参数
257|         """
258|         if event_type not in self._callbacks:
259|             self._callbacks[event_type] = []
260|         self._callbacks[event_type].append(callback)
261|
262|    def off(self, event_type: str, callback: PluginCallback) ->
None:
263|         """
264|        注销事件回调
265|
266|         Args:
267|            event_type: 事件类型
268|            callback: 要注销的回调函数
269|         """
270|         if event_type in self._callbacks:
271|             self._callbacks[event_type] = [
272|                 cb for cb in self._callbacks[event_type] if cb !=
callba
   |    ck
273|             ]
```

```
274|
275|    def emit(self, event_type: str, data: Optional[Dict[str,
Any]] = Non
   |    e,
276|              result: Optional[PluginResult] = None) -> None:
277|        """
278|        触发事件 — 通知所有注册的回调函数
279|
280|        Args:
281|            event_type: 事件类型
282|            data: 随事件传递的附加数据
283|            result: 关联的 PluginResult（如果有）
284|        """
285|        event = PluginEvent(
286|            event_type=event_type,
287|            plugin_name=self.get_manifest().name,
288|            data=data,
289|            result=result
290|        )
291|        callbacks = self._callbacks.get(event_type, [])
292|        for cb in callbacks:
293|            try:
294|                cb(event)
295|            except Exception as e:
296|                self.logger.warning(f"事件回调异常 [{event_type}]: {e}")
297|
299|    # 状态查询方法
301|
302|    def get_status(self) -> PluginStatus:
303|        """获取插件当前运行状态"""
304|        return self._status
305|
306|    def get_info(self) -> PluginInfo:
307|        """
308|        获取插件完整注册信息
309|
310|        Returns:
311|            PluginInfo: 包含名称/版本/状态/统计等完整信息的结构体
312|        """
313|        manifest = self.get_manifest()
314|        return PluginInfo(
315|            name=manifest.name,
316|            version=manifest.version,
```

```
            description=manifest.supported_operations[0] if
manifest.sup
    ported_operations else "",
            status=self._status,
            last_loaded_at=self._loaded_at,
            last_executed_at=datetime.now().isoformat() if
self._executi
    on_count > 0 else None,
            execution_count=self._execution_count
        )

    def get_config(self) -> PluginConfig:
        """获取当前生效的配置（只读）"""
        return dict(self._config)

    def get_metrics(self) -> Dict[str, Any]:
        """
        获取插件运行指标

        Returns:
            dict: 包含 total_executions, avg_execution_time_ms, last_err
    or 等指标
        """
        avg_time = (self._total_execution_time_ms /
self._execution_coun
    t
                    if self._execution_count > 0 else 0)
        return {
            "plugin_name": self.get_manifest().name,
            "status": self._status.name,
            "total_executions": self._execution_count,
            "avg_execution_time_ms": round(avg_time, 2),
            "total_execution_time_ms":
round(self._total_execution_time_
    ms, 2),
            "last_error": self._last_error
        }

    # 内部辅助方法

    def _set_status(self, status: PluginStatus) -> None:
        """更新状态并发送状态变更事件"""
        old_status = self._status
        self._status = status
```

```
354|         self.emit("STATE_CHANGED", {
355|             "old_status": old_status.name,
356|             "new_status": status.name
357|         })
358|
359|     def _execute_wrapper(self, **kwargs) -> PluginResult:
360|         """
361|         执行包装器 — 统一的执行流程
362|
363|         流程:
364|             1. 检查状态 (必须 READY)
365|             2. 调用 pre_execute 钩子
366|             3. 如果 pre_execute 返回非 None，直接返回
367|             4. 调用 execute 并计时
368|             5. 调用 post_execute 钩子
369|             6. 返回最终结果
370|         """
371|         if self._status != PluginStatus.READY:
372|             return PluginResult.fail(
373|                 message=f"插件未就绪 (当前状态: {self._status.name})",
374|                 error_code="INVALID_STATE"
375|             )
376|
377|         short_circuit = self.pre_execute(**kwargs)
378|         if short_circuit is not None:
379|             return short_circuit
380|
381|         self._set_status(PluginStatus.RUNNING)
382|         self.emit("PLUGIN_STARTED", {"kwargs_keys":
list(kwargs.keys())}
   |     )
383|
384|         start = time.perf_counter()
385|         result = PluginResult.fail("未执行")
386|         try:
387|             result = self.execute(**kwargs)
388|             if not isinstance(result, PluginResult):
389|                 result = PluginResult.ok(data=result, message="执行完成"
   |     )
390|         except Exception as e:
391|             result = self.on_error(e, "execute")
392|         finally:
393|             elapsed = (time.perf_counter() - start) * 1000
394|             result.execution_time_ms = round(elapsed, 2)
```

```
            self._execution_count += 1
            self._total_execution_time_ms += elapsed
            self._set_status(PluginStatus.READY)

        result = self.post_execute(result, **kwargs)
        self.emit("PLUGIN_COMPLETED", result=result)
        return result
```

37. src/plugin/manager.py

```
"""
插件管理器 v1.0 — 标准化统一调用机制

核心职责:
    - 插件注册/发现/加载/卸载（生命周期管理）
    - 统一执行入口（基于 _execute_wrapper 的标准流程）
    - 依赖解析与拓扑排序加载
    - 事件总线（插件间通信）
    - 并发执行控制（线程安全）
    - 状态监控与指标收集
    - 热重载支持

线程安全:
    使用 threading.RLock 保护所有共享状态。

第三方应用调用示例:

    pm = PluginManager()

    # 1. 注册插件
    pm.register_plugin(SimulationPlugin,
priority=PluginPriority.HIGH)

    # 2. 加载插件
    pm.load_plugin("SimulationPlugin", config={"num_servers": 20})

    # 3. 执行插件（统一入口）
    result = pm.execute_plugin("SimulationPlugin",
algorithms=["greedy",
     "genetic"])

    # 4. 检查结果
    if result.success:
        print(f"完成: {result.message}")
        print(f"数据: {result.data}")
```

```
    else:
        print(f"失败 [{result.error_code}]: {result.message}")

    # 5. 卸载
    pm.unload_plugin("SimulationPlugin")
"""

import importlib
import os
import threading
import time
from typing import Dict, List, Type, Optional, Set
from datetime import datetime

from src.plugin.base import PluginBase
from src.plugin.protocol import (
    PluginResult, PluginInfo, PluginManifest, PluginStatus,
    PluginPriority, PluginEvent, PluginCallback, PluginConfig
)
from src.utils.logger import setup_logger

# Fail-closed list: modules that stay on disk but are not discovered.
# rl_ppo_scheduler needs gymnasium + stable-baselines3 + a trained model
#     .
DISABLED_PLUGIN_MODULES = {"rl_ppo_scheduler"}


class PluginManager:
    """
    插件管理器

    状态管理:
        每个插件通过 _plugin_instances 和 _plugin_infos 追踪全生命周期。

    事件系统:
        通过 subscribe / unsubscribe / _emit_global 实现全局事件总线，
        插件间可通过事件解耦通信。
    """

    def __init__(self):
        self._logger = setup_logger("plugin_manager")
        self._lock = threading.RLock()
```

```
        self._plugin_classes: Dict[str, Type[PluginBase]] = {}
        self._plugin_instances: Dict[str, PluginBase] = {}
        self._plugin_infos: Dict[str, PluginInfo] = {}
        self._plugin_priorities: Dict[str, PluginPriority] = {}
        self._global_callbacks: Dict[str, List[PluginCallback]] =
{}
        self._metrics: Dict[str, Dict] = {}
        self._started_at: str = datetime.now().isoformat()

    # 插件注册

    def register_plugin(self, plugin_class: Type[PluginBase],
                        priority: PluginPriority =
PluginPriority.NORMAL
    ,
                        dependencies: Optional[List[str]] = None)
-> boo
    l:
        """
        注册一个插件类到管理器

        Args:
            plugin_class: 继承自 PluginBase 的插件类
            priority: 执行优先级 (默认 NORMAL)
            dependencies: 依赖的其他插件名称列表

        Returns:
            True 注册成功

        Raises:
            TypeError: 如果 plugin_class 不是 PluginBase 的子类
        """
        if not issubclass(plugin_class, PluginBase):
            raise TypeError(f"{plugin_class.__name__} 必须继承 PluginBas
    e")

        manifest = plugin_class().get_manifest()
        name = manifest.name

        with self._lock:
            self._plugin_classes[name] = plugin_class
            self._plugin_priorities[name] = priority

            info = PluginInfo(
```

```
114|                 name=name,
115|                 version=manifest.version,
116|                 description=",".join(manifest.supported_operations)
,
117|                 status=PluginStatus.REGISTERED,
118|                 priority=priority,
119|                 dependencies=dependencies or [],
120|                 config_schema=manifest.input_schema,
121|                 tags=manifest.supported_operations,
122|                 registered_at=datetime.now().isoformat()
123|             )
124|             self._plugin_infos[name] = info
125|
126|         self._logger.info(f"插件已注册: {name} v{manifest.version} (优先
   |     级: {priority.name})")
127|         self._emit_global("PLUGIN_REGISTERED", {"name": name, "
version":
   |      manifest.version})
128|         return True
129|
131|     # 插件加载
133|
134|     def load_plugin(self, name: str, config:
Optional[PluginConfig] = No
   |     ne) -> bool:
135|         """
136|         加载并初始化指定插件
137|
138|         流程:
139|             1. 检查是否已注册
140|             2. 解析并加载依赖插件
141|             3. 验证配置参数
142|             4. 实例化插件类
143|             5. 调用 initialize() 初始化
144|
145|         Args:
146|             name: 插件名称
147|             config: 配置参数字典
148|
149|         Returns:
150|             True 加载成功
151|         """
152|         with self._lock:
153|             if name not in self._plugin_classes:
```

```
                self._logger.error(f"加载失败: 插件 '{name}' 未注册")
                return False

            if name in self._plugin_instances:
                self._logger.warning(f"插件 '{name}' 已加载，跳过重复加载")
                return True

            info = self._plugin_infos[name]

            # 解析依赖
            for dep_name in info.dependencies:
                if dep_name not in self._plugin_instances:
                    self._logger.info(f"加载依赖插件: {dep_name} → {name}")
                    if not self.load_plugin(dep_name):
                        self._logger.error(f"依赖插件加载失败: {dep_name}")
                        return False

            # 实例化
            try:
                plugin_class = self._plugin_classes[name]
                plugin = plugin_class()
                info.status = PluginStatus.INITIALIZING
                self._plugin_infos[name] = info
            except Exception as e:
                return self._handle_load_error(name, e, "实例化", info)

            # 配置验证
            config = config or {}
            if not plugin.validate_config(config):
                self._logger.error(f"加载失败: 插件 '{name}' 配置验证不通过")
                info.status = PluginStatus.ERROR
                self._plugin_infos[name] = info
                return False
            plugin._config = config

            # 初始化
            try:
                if not plugin.initialize(config):
                    info.status = PluginStatus.ERROR
                    self._plugin_infos[name] = info
```

```
                    self._logger.error(f"初始化失败: 插件 '{name}'")
                    return False
            except Exception as e:
                return self._handle_load_error(name, e, "初始化", info)

            plugin._status = PluginStatus.READY
            plugin._loaded_at = datetime.now().isoformat()
            self._plugin_instances[name] = plugin
            info.status = PluginStatus.READY
            info.last_loaded_at = datetime.now().isoformat()
            self._plugin_infos[name] = info

            self._logger.info(f"插件已加载: {name} (状态: READY)")
            self._emit_global("PLUGIN_INITIALIZED", {"name":
name})
            return True

    def _handle_load_error(self, name: str, error: Exception,
                           stage: str, info: PluginInfo) -> bool:
        info.status = PluginStatus.ERROR
        self._plugin_infos[name] = info
        self._logger.error(f"{stage}失败 [{name}]: {error}", exc_info=Tr
    ue)
        self._emit_global("PLUGIN_ERROR", {"name": name, "stage":
stage,
     "error": str(error)})
        return False

    # 插件执行 (统一入口)

    def execute_plugin(self, name: str, **kwargs) ->
Optional[PluginResu
    lt]:
        """
        执行插件 — 统一的第三方调用入口

        使用 _execute_wrapper 确保一致的执行流程:
            pre_execute → execute → post_execute → 计时 → 指标收集

        Args:
            name: 插件名称
            **kwargs: 传递给 execute() 的参数

        Returns:
```

```
234|           PluginResult: 标准化执行结果
235|
236|       用法:
237|           result = pm.execute_plugin("SimulationPlugin",
algorithms=["
   |    greedy"])
238|           if result.success:
239|               print(result.data['statistics'])
240|       """
241|       with self._lock:
242|           if name not in self._plugin_instances:
243|              self._logger.error(f"执行失败: 插件 '{name}' 未加载")
244|               return PluginResult.fail(
245|                  message=f"插件 '{name}' 未加载",
246|                    error_code="PLUGIN_NOT_LOADED"
247|               )
248|
249|           plugin = self._plugin_instances[name]
250|
251|       result = plugin._execute_wrapper(**kwargs)
252|
253|       plugin_info = self._plugin_infos.get(name)
254|       if plugin_info:
255|           plugin_info.execution_count = plugin._execution_count
256|           plugin_info.last_executed_at =
datetime.now().isoformat()
257|
258|       self._logger.info(
259|          f"插件执行完成: {name} "
260|          f"({'成功' if result.success else '失败'}) "
261|           f"[{result.execution_time_ms:.0f}ms]"
262|       )
263|       self._emit_global("METRICS_UPDATED", {"name": name, "
metrics": p
   |    lugin.get_metrics()})
264|           return result
265|
267|    # 插件卸载
269|
270|    def unload_plugin(self, name: str) -> bool:
271|       """
272|       卸载指定插件并释放资源
273|
274|       Args:
```

```
            name: 插件名称

        Returns:
            True 卸载成功
        """
        with self._lock:
            if name not in self._plugin_instances:
                self._logger.warning(f"卸载跳过: 插件 '{name}' 未加载")
                return False

            plugin = self._plugin_instances[name]
            try:
                plugin._set_status(PluginStatus.STOPPING)
                plugin.cleanup()
                plugin._set_status(PluginStatus.STOPPED)
            except Exception as e:
                self._logger.error(f"清理异常 [{name}]: {e}")

            del self._plugin_instances[name]
            if name in self._plugin_infos:
                self._plugin_infos[name].status =
PluginStatus.STOPPED

            self._logger.info(f"插件已卸载: {name}")
            self._emit_global("PLUGIN_STOPPED", {"name": name})
            return True

    # 插件查询

    def get_plugin(self, name: str) -> Optional[PluginBase]:
        """获取已加载的插件实例"""
        with self._lock:
            return self._plugin_instances.get(name)

    def get_plugin_info(self, name: str) -> Optional[PluginInfo]:
        """获取插件注册信息"""
        with self._lock:
            return self._plugin_infos.get(name)

    def list_plugins(self) -> List[Dict]:
        """列出所有已注册插件的元信息"""
        with self._lock:
            result = []
            for name, info in self._plugin_infos.items():
```

```
320|                d = info.to_dict()
321|                d["loaded"] = name in self._plugin_instances
322|                d["priority"] = self._plugin_priorities.get(name,
Plugin
   |    Priority.NORMAL).name
323|                result.append(d)
324|            return result
325|
326|    def list_loaded_plugins(self) -> List[str]:
327|        """列出所有已加载插件的名称"""
328|        with self._lock:
329|            return list(self._plugin_instances.keys())
330|
331|    def get_plugin_metrics(self, name: str) -> Optional[Dict]:
332|        """获取插件运行指标"""
333|        plugin = self.get_plugin(name)
334|        if plugin:
335|            return plugin.get_metrics()
336|        return None
337|
338|    def get_all_metrics(self) -> Dict[str, Dict]:
339|        """获取所有插件的运行指标"""
340|        with self._lock:
341|            return {name: p.get_metrics() for name, p in
self._plugin_in
   |    stances.items()}
342|
344|    # 插件配置管理
346|
347|    def get_plugin_config(self, name: str) ->
Optional[PluginConfig]:
348|        """获取插件当前配置"""
349|        plugin = self.get_plugin(name)
350|        return plugin.get_config() if plugin else None
351|
352|    def update_plugin_config(self, name: str, config_update:
PluginConfi
   |    g) -> bool:
353|        """
354|        更新插件配置（需重新加载生效）
355|
356|        Args:
357|            name: 插件名称
358|            config_update: 要更新的配置（合并到现有配置）
```

```
359|
360|        Returns:
361|            True 更新成功
362|        """
363|        with self._lock:
364|            if name not in self._plugin_infos:
365|                return False
366|            plugin = self._plugin_instances.get(name)
367|            if plugin:
368|                merged = dict(plugin._config)
369|                merged.update(config_update)
370|                plugin._config = merged
371|                if not plugin.validate_config(merged):
372|                    self._logger.warning(f"配置更新后验证失败: {name}")
373|                    return False
374|            self._logger.info(f"配置已更新: {name}")
375|            return True
376|
378|    # 热重载
380|
381|    def reload_plugin(self, name: str) -> bool:
382|        """
383|        热重载插件（卸载后重新加载，保留配置）
384|
385|        Args:
386|            name: 插件名称
387|
388|        Returns:
389|            True 重载成功
390|        """
391|        with self._lock:
392|            config = self.get_plugin_config(name)
393|            self.unload_plugin(name)
394|            time.sleep(0.1)
395|            return self.load_plugin(name, config)
396|
398|    # 批量操作
400|
401|    def load_all(self, configs: Optional[Dict[str, PluginConfig]]
= None
    |    ) -> Dict[str, bool]:
402|        """
403|        加载所有已注册的插件
404|
```

```
405|        Args:
406|            configs: 各插件的配置字典 {name: config}
407|
408|        Returns:
409|            {name: success} 加载结果映射
410|        """
411|        results = {}
412|        with self._lock:
413|            names = list(self._plugin_classes.keys())
414|        for name in names:
415|            cfg = (configs or {}).get(name)
416|            results[name] = self.load_plugin(name, cfg)
417|        return results
418|
419|    def unload_all(self) -> None:
420|        """卸载所有已加载插件"""
421|        with self._lock:
422|            names = list(self._plugin_instances.keys())
423|        for name in reversed(names):
424|            self.unload_plugin(name)
425|
426|    def execute_all(self, **kwargs) -> Dict[str, PluginResult]:
427|        """
428|        按优先级顺序执行所有已加载插件
429|
430|        Args:
431|            **kwargs: 传递给各插件的公共参数
432|
433|        Returns:
434|            {name: PluginResult} 执行结果映射
435|        """
436|        results = {}
437|        with self._lock:
438|            items = [(name, self._plugin_priorities.get(name,
PluginPrio
   |    rity.NORMAL).value)
439|                     for name in self._plugin_instances.keys()]
440|        items.sort(key=lambda x: x[1])
441|
442|        for name, _ in items:
443|            results[name] = self.execute_plugin(name, **kwargs)
444|        return results
445|
447|    # 事件总线
```

```
449|
450|    def subscribe(self, event_type: str, callback:
PluginCallback) -> No
   |    ne:
451|        """
452|        订阅全局事件
453|
454|        Args:
455|            event_type: 事件类型
456|            callback: 回调函数 (PluginEvent) → None
457|
458|        用法:
459|            def on_completed(event):
460|                print(f"{event.plugin_name} 执行完成: {event.result.mess
   |    age}")
461|
462|            pm.subscribe("PLUGIN_COMPLETED", on_completed)
463|        """
464|        with self._lock:
465|            if event_type not in self._global_callbacks:
466|                self._global_callbacks[event_type] = []
467|            self._global_callbacks[event_type].append(callback)
468|
469|    def unsubscribe(self, event_type: str, callback:
PluginCallback) ->
   |    None:
470|        """取消事件订阅"""
471|        with self._lock:
472|            if event_type in self._global_callbacks:
473|                self._global_callbacks[event_type] = [
474|                    cb for cb in
self._global_callbacks[event_type] if c
   |    b != callback
475|                ]
476|
477|    def _emit_global(self, event_type: str, data: Optional[Dict]
= None)
   |     -> None:
478|        """触发全局事件"""
479|        event = PluginEvent(
480|            event_type=event_type,
481|            plugin_name="PluginManager",
482|            data=data
483|        )
```

```
484|         callbacks = self._global_callbacks.get(event_type, [])
485|         for cb in callbacks:
486|             try:
487|                 cb(event)
488|             except Exception as e:
489|                 self._logger.warning(f"全局事件回调异常 [{event_type}]:
   |     {e}")
490|
492|     # 自动发现
494|
495|     def discover_plugins(self, plugin_dir: str = "plugins") ->
int:
496|         """
497|         自动发现并注册指定目录中的插件模块
498|
499|         Args:
500|             plugin_dir: 插件目录名（相对于项目根目录），支持两种形式:
501|                 - 文件路径: "plugins" / "src/plugins" / "./src/plugins"
502|                 - Python 包: "src.plugins"
503|                 内部统一转成文件系统路径与 Python 模块前缀。
504|
505|         Returns:
506|             int: 成功注册的插件数量
507|         """
508|         # 归一化: 去掉 ./ 前缀与尾部斜杠
509|         norm = plugin_dir.replace("\\", "/").strip("/")
510|         if norm.startswith("./"):
511|             norm = norm[2:]
512|
513|         # 推断: 含 '.' 且首段不是盘符 → 当作 Python 包; 否则文件路径
514|         is_python_package = (
515|             "." in norm
516|             and not (len(norm) >= 2 and norm[1] == ":")
517|         )
518|         if is_python_package:
519|             module_prefix = norm
520|             file_subpath = norm.replace(".", os.sep)
521|         else:
522|             file_subpath = norm
523|             module_prefix = norm.replace("/", "
.").replace(os.sep, ".")
524|
525|         plugin_path = os.path.join(os.path.dirname(__file__), "
..", ".."
```

```
   |    , file_subpath)
526|        if not os.path.exists(plugin_path):
527|            self._logger.warning(f"插件目录不存在: {plugin_path}")
528|            return 0
529|
530|        count = 0
531|        disabled = set(getattr(self, "disabled_plugins",
DISABLED_PLUGIN
   |    _MODULES))
532|        for filename in sorted(os.listdir(plugin_path)):
533|            if not filename.endswith(".py") or
filename.startswith("_"):
534|                continue
535|            module_name = filename[:-3]
536|            if module_name in disabled:
537|                self._logger.info(f"插件已禁用，跳过: {module_name}")
538|                continue
539|            try:
540|                module = importlib.import_module(f"{module_prefix}.
{modu
   |    le_name}")
541|                for attr_name in dir(module):
542|                    obj = getattr(module, attr_name)
543|                    if (isinstance(obj, type) and issubclass(obj,
Plugin
   |    Base)
544|                            and obj != PluginBase
545|                            and obj.__module__ ==
module.__name__):
546|                        self.register_plugin(obj)
547|                        count += 1
548|            except Exception as e:
549|                self._logger.error(f"模块加载失败 {module_name}: {e}")
550|
551|        self._logger.info(f"插件发现完成: {count} 个插件已注册 (目录: {p
   |    lugin_path})")
552|        return count
553|
555|    # 依赖解析
557|
558|    def resolve_dependencies(self, name: str) -> List[str]:
559|        """
560|        解析插件的完整依赖链 (拓扑排序)
561|
```

```
        Args:
            name: 插件名称

        Returns:
            List[str]: 按依赖顺序排列的插件名称列表（被依赖的在前）
        """
        resolved: List[str] = []
        visited: Set[str] = set()
        visiting: Set[str] = set()

        def dfs(current: str):
            if current in visited:
                return
            if current in visiting:
                raise RuntimeError(f"检测到循环依赖: {current}")
            visiting.add(current)
            info = self._plugin_infos.get(current)
            if info:
                for dep in info.dependencies:
                    dfs(dep)
            visiting.discard(current)
            visited.add(current)
            if current != name:
                resolved.append(current)

        try:
            dfs(name)
        except RuntimeError as e:
            self._logger.error(str(e))
            return []

        resolved.append(name)
        return resolved

    # 状态查询

    def get_system_status(self) -> Dict:
        """
        获取整个插件系统的运行状态

        Returns:
            dict: 包含已注册/已加载/错误插件数量及整体健康状态
        """
        with self._lock:
```

```
608|            total = len(self._plugin_infos)
609|            loaded = len(self._plugin_instances)
610|            errors = sum(1 for info in self._plugin_infos.values()
611|                          if info.status == PluginStatus.ERROR)
612|            healthy = (errors == 0)
613|
614|            return {
615|                "total_registered": total,
616|                "total_loaded": loaded,
617|                "status": "healthy" if healthy else "degraded",
618|                "error_count": errors,
619|                "started_at": self._started_at,
620|                "uptime_seconds": round(
621|                    (datetime.now() -
datetime.fromisoformat(self._start
   |    ed_at)).total_seconds(), 1
622|                ),
623|                "plugins": self.list_plugins()
624|            }
```

38. src/plugin/protocol.py

```
  1|"""
  2|插件系统标准化接口协议 v1.0
  3|Plugin System Standardized Protocol & Data Types
  4|
  5|本模块定义插件系统的核心数据类型和接口规范，确保:
  6|- 第三方应用通过统一的数据结构调用插件功能
  7|- 所有插件返回一致格式的结果
  8|- 插件状态可被外部系统查询和监控
  9|- 支持异步事件通知机制
 10|
 11|设计原则:
 12|- 不可变数据类 (dataclass + frozen)
 13|- 类型安全 (所有字段有明确类型注解)
 14|- 可序列化 (支持 JSON 序列化)
 15|- 向前兼容 (扩展字段使用 Optional)
 16|"""
 17|
 18|from dataclasses import dataclass, field
 19|from enum import Enum, auto
 20|from typing import Dict, Any, List, Optional, Callable, Union
 21|from datetime import datetime
 22|import json
 23|
```

```
24|
25|class PluginStatus(Enum):
26|    """
27|    插件生命周期状态机
28|
29|    状态流转:
30|        UNREGISTERED → REGISTERED → INITIALIZING → READY
31|            READY → RUNNING → COMPLETED → READY
32|            READY → PAUSED → READY
33|            READY → STOPPING → STOPPED
34|           任意状态 → ERROR
35|
36|    UNREGISTERED: 插件未被管理器识别
37|    REGISTERED:   插件类已注册到管理器，尚未实例化
38|    INITIALIZING: 插件正在执行 initialize() 方法
39|    READY:        插件已初始化完成，等待执行
40|    RUNNING:      插件正在执行 execute() 方法
41|    COMPLETED:    插件一次执行完成（自动回到 READY）
42|    PAUSED:       插件被暂停（管理器可暂停长时间运行插件）
43|    STOPPING:     插件正在执行 cleanup() 方法
44|    STOPPED:      插件已停止但未卸载
45|    ERROR:        插件遇到不可恢复的错误
46|    """
47|    UNREGISTERED = auto()
48|    REGISTERED = auto()
49|    INITIALIZING = auto()
50|    READY = auto()
51|    RUNNING = auto()
52|    COMPLETED = auto()
53|    PAUSED = auto()
54|    STOPPING = auto()
55|    STOPPED = auto()
56|    ERROR = auto()
57|
58|
59|class PluginPriority(Enum):
60|    """
61|    插件执行优先级
62|
63|    管理器在并发调度时根据优先级决定执行顺序:
64|    - CRITICAL: 系统关键插件，必须优先执行
65|    - HIGH: 重要业务插件
66|    - NORMAL: 常规插件（默认）
67|    - LOW: 后台任务插件
```

```
    - IDLE: 仅在系统空闲时执行
    """
    CRITICAL = 0
    HIGH = 1
    NORMAL = 2
    LOW = 3
    IDLE = 4


@dataclass
class PluginResult:
    """
    插件执行结果 — 统一返回类型

    所有插件的 execute() 方法必须返回此类型的实例。

    字段说明:
        success: 执行是否成功（布尔值，必填）
        data: 插件产出的业务数据（任意可序列化对象）
        message: 人类可读的状态描述
        error_code: 错误码（成功时为空字符串）
        error_detail: 详细错误信息（含堆栈跟踪，仅生产环境脱敏）
        execution_time_ms: 执行耗时（毫秒）
        warnings: 非致命的警告信息列表
        metrics: 插件自定义的性能指标
        timestamp: 结果生成时间戳
    """
    success: bool
    data: Optional[Any] = None
    message: str = ""
    error_code: str = ""
    error_detail: str = ""
    execution_time_ms: float = 0.0
    warnings: List[str] = field(default_factory=list)
    metrics: Dict[str, Any] = field(default_factory=dict)
    timestamp: str = field(default_factory=lambda:
datetime.now().isofor
    |    mat())

    @classmethod
    def ok(cls, data: Any = None, message: str = "执行成功",
           execution_time_ms: float = 0.0, **kwargs) -> "
PluginResult":
        """
```

```
        工厂方法：快速创建一个成功结果

        用法:
             return PluginResult.ok(data={"temperature": 25.0})
            return PluginResult.ok(message="仿真完成", data=df)
        """
        return cls(
            success=True,
            data=data,
            message=message,
            execution_time_ms=execution_time_ms,
            **kwargs
        )

    @classmethod
    def fail(cls, message: str = "执行失败", error_code: str = "UNKNOWN_
    ERROR",
              error_detail: str = "", **kwargs) -> "PluginResult":
        """
        工厂方法：快速创建一个失败结果

        用法:
            return PluginResult.fail("参数缺失", error_code="MISSING_PAR
    AM")
            return PluginResult.fail("连接超时", "TIMEOUT", detail)
        """
        return cls(
            success=False,
            message=message,
            error_code=error_code,
            error_detail=error_detail,
            **kwargs
        )

    def to_dict(self) -> Dict[str, Any]:
        """转换为可 JSON 序列化的字典"""
        return {
            "success": self.success,
            "data": self.data,
            "message": self.message,
            "error_code": self.error_code,
            "error_detail": self.error_detail,
            "execution_time_ms": self.execution_time_ms,
            "warnings": self.warnings,
```

```
            "metrics": self.metrics,
            "timestamp": self.timestamp
        }

    def to_json(self, indent: int = 2) -> str:
        """转换为 JSON 字符串"""
        return json.dumps(self.to_dict(), ensure_ascii=False,
indent=ind
    ent, default=str)

    def has_warnings(self) -> bool:
        """是否有警告信息"""
        return len(self.warnings) > 0


@dataclass
class PluginInfo:
    """
    插件注册信息 — 完整描述一个已注册插件

    字段说明:
        name: 插件唯一标识名称
        version: 语义化版本号 (如 "2.1.0")
        description: 功能描述
        author: 作者/组织名称
        status: 当前状态
        priority: 执行优先级
        dependencies: 依赖的其他插件名称列表
        config_schema: 配置参数的 JSON Schema
        tags: 分类标签列表
        registered_at: 注册时间戳
        last_loaded_at: 最近一次加载时间
        last_executed_at: 最近一次执行时间
        execution_count: 累计执行次数
    """
    name: str
    version: str = "0.0.0"
    description: str = ""
    author: str = ""
    status: PluginStatus = PluginStatus.REGISTERED
    priority: PluginPriority = PluginPriority.NORMAL
    dependencies: List[str] = field(default_factory=list)
    config_schema: Dict[str, Any] = field(default_factory=dict)
    tags: List[str] = field(default_factory=list)
```

```
    registered_at: str = field(default_factory=lambda:
datetime.now().is
    oformat())
    last_loaded_at: Optional[str] = None
    last_executed_at: Optional[str] = None
    execution_count: int = 0

    def to_dict(self) -> Dict[str, Any]:
        return {
            "name": self.name,
            "version": self.version,
            "description": self.description,
            "author": self.author,
            "status": self.status.name,
            "priority": self.priority.name,
            "dependencies": self.dependencies,
            "config_schema": self.config_schema,
            "tags": self.tags,
            "registered_at": self.registered_at,
            "last_loaded_at": self.last_loaded_at,
            "last_executed_at": self.last_executed_at,
            "execution_count": self.execution_count
        }


@dataclass
class PluginManifest:
    """
    插件清单 — 描述插件的能力和接口

    第三方应用通过读取此清单了解插件的:
    - 输入参数规格
    - 输出格式
    - 支持的操作
    - 版本兼容性
    """
    name: str
    version: str
    supported_operations: List[str]
    input_schema: Dict[str, Any] = field(default_factory=dict)
    output_schema: Dict[str, Any] = field(default_factory=dict)
    min_system_version: str = "1.0.0"
    max_instances: int = 1
    category: str = "algorithm"
```

```
235|
236|
237|@dataclass
238|class PluginEvent:
239|    """
240|    插件事件 — 用于插件间通信和外部系统通知
241|
242|    事件类型包括:
243|        PLUGIN_REGISTERED: 插件已注册
244|        PLUGIN_INITIALIZED: 插件初始化完成
245|        PLUGIN_STARTED: 插件开始执行
246|        PLUGIN_COMPLETED: 插件执行完成
247|        PLUGIN_ERROR: 插件发生错误
248|        PLUGIN_STOPPED: 插件已停止
249|        STATE_CHANGED: 任意状态变更
250|        METRICS_UPDATED: 性能指标更新
251|    """
252|    event_type: str
253|    plugin_name: str
254|    timestamp: str = field(default_factory=lambda:
datetime.now().isofor
   |    mat())
255|    data: Optional[Dict[str, Any]] = None
256|    result: Optional[PluginResult] = None
257|
258|    def to_dict(self) -> Dict[str, Any]:
259|        d = {
260|            "event_type": self.event_type,
261|            "plugin_name": self.plugin_name,
262|            "timestamp": self.timestamp,
263|            "data": self.data
264|        }
265|        if self.result:
266|            d["result"] = self.result.to_dict()
267|        return d
268|
269|
270|PluginCallback = Callable[[PluginEvent], None]
271|"""插件事件回调函数类型签名"""
272|
273|PluginConfig = Dict[str, Any]
274|"""插件配置参数类型"""
275|
276|PluginContext = Dict[str, Any]
```

```
277|"""插件运行上下文类型"""
```

39. src/plugin/simulation_plugin.py - 仿真插件执行器

```
  1|"""
  2|仿真插件 v1.0 — 数据中心智能调度仿真
  3|
  4|继承自 PluginBase v1.0 标准接口，实现:
  5|    - PluginManifest 完整清单
  6|    - validate_config 配置验证
  7|    - pre_execute 参数预处理
  8|    - execute 返回 PluginResult
  9|    - post_execute 指标收集
 10|
 11|支持模式:
 12|    - standard: 标准单机仿真
 13|    - distributed: 分布式并行仿真
 14|    - large_scale: 大规模压力测试
 15|"""
 16|
 17|from typing import Dict, Any, Optional, List
 18|import os
 19|import pandas as pd
 20|
 21|from src.plugin.base import PluginBase
 22|from src.plugin.protocol import PluginResult, PluginManifest
 23|from src.algorithms import PLUGIN_ALGORITHMS
 24|from src.simulation import DataCenterSimulator
 25|from src.simulation.distributed_simulator import
DistributedSimulator
 26|from src.visualization import (
 27|    plot_energy_comparison, plot_temperature_trend,
 28|    plot_load_distribution, plot_pue_comparison,
plot_cooling_power,
 29|    plot_service_rate
 30|)
 31|from src.simulation.metrics import summarize
 32|from src.utils import save_simulation_data,
export_results_to_excel, sav
   |    e_run_metadata
 33|from src.utils.logger import setup_logger
 34|
 35|
 36|class SimulationPlugin(PluginBase):
 37|
```

```
    def get_manifest(self) -> PluginManifest:
        return PluginManifest(
            name="SimulationPlugin",
            version="2.0.0",
            supported_operations=["run_comparison"],
            input_schema={
                "num_servers": {"type": "int", "min": 1, "max":
1000, "d
   |    efault": 10},
                "simulation_hours": {"type": "int", "min": 1, "
max": 720
   |    , "default": 24},
                "algorithms": {"type": "list", "items": "str", "
default"
   |    : ["greedy","genetic","round_robin"]},
                "parallel_workers": {"type": "int", "min": 1, "
max": 32,
   |     "default": 4},
                "run_mode": {"type": "str", "enum":
["standard","distrib
   |    uted","large_scale"], "default": "standard"},
                "server_type": {"type": "str", "enum":
["mixed","standar
   |    d","high_performance","gpu_server"], "default": "mixed"},
                "visualization": {"type": "bool", "default":
True},
                "save_results": {"type": "bool", "default": True}
            },
            output_schema={
                "success": "bool",
                "data": {"statistics": "dict", "records_count": "
int", "
   |    algorithms": "list"},
                "message": "str",
                "execution_time_ms": "float"
            },
            min_system_version="2.0.0",
            category="executor"
        )

    def validate_config(self, config: Dict[str, Any]) -> bool:
        num_servers = config.get("num_servers", 10)
        if not isinstance(num_servers, int) or num_servers < 1 or
num_se
```

```
   |    rvers > 1000:
 66|            self.logger.error(f"num_servers 必须在 [1, 1000] 范围内，当前
   |    值: {num_servers}")
 67|             return False
 68|         sim_hours = config.get("simulation_hours", 24)
 69|         if not isinstance(sim_hours, int) or sim_hours < 1 or
sim_hours
   |    > 720:
 70|            self.logger.error(f"simulation_hours 必须在 [1, 720] 范围内，
   |    当前值: {sim_hours}")
 71|             return False
 72|         workers = config.get("parallel_workers", 4)
 73|         if workers < 1 or workers > 32:
 74|            self.logger.error(f"parallel_workers 必须在 [1, 32] 范围内，
   |    当前值: {workers}")
 75|             return False
 76|         return True
 77|
 78|     def initialize(self, config: Optional[Dict[str, Any]] = None)
-> boo
   |    l:
 79|         try:
 80|             cfg = config or {}
 81|             self.num_servers = cfg.get("num_servers", 10)
 82|             self.simulation_hours = cfg.get("simulation_hours",
24)
 83|             self.parallel_workers = cfg.get("parallel_workers", 4)
 84|             self.run_mode = cfg.get("run_mode", "standard")
 85|             self.server_type = cfg.get("server_type", "mixed")
 86|             self.site_id = cfg.get("site_id") or None
 87|             self.results_dir = cfg.get("results_dir", "data")
 88|             self.reports_dir = cfg.get("reports_dir", "reports")
 89|             self.simulator = None
 90|             self.distributed_simulator = None
 91|
 92|             os.makedirs(self.results_dir, exist_ok=True)
 93|             os.makedirs(self.reports_dir, exist_ok=True)
 94|
 95|             if self.run_mode in ("distributed", "large_scale"):
 96|                 self.distributed_simulator = DistributedSimulator(
 97|                     num_servers=self.num_servers,
 98|                     parallel_workers=self.parallel_workers
 99|                 )
100|             else:
```

```
                self.simulator = DataCenterSimulator(
                    num_servers=self.num_servers,
                    simulation_hours=self.simulation_hours,
                    data_source_override=(
                        {"site_id": self.site_id} if self.site_id
else N
   |    one
                    ),
                )
            return True
        except Exception as e:
            self.logger.error(f"SimulationPlugin 初始化失败: {e}")
            return False

    def pre_execute(self, **kwargs) -> Optional[PluginResult]:
        algorithms = kwargs.get("algorithms", ["greedy", "
genetic", "rou
   |    nd_robin"])
        valid_algos = {"greedy", "genetic", "round_robin", "
damo_ets"}.u
   |    nion(PLUGIN_ALGORITHMS.keys())
        for algo in algorithms:
            if algo not in valid_algos:
                return PluginResult.fail(
                    message=f"不支持的算法: '{algo}', 有效值: {sorted(val
   |    id_algos)}",
                    error_code="INVALID_ALGORITHM"
                )
        return None

    def execute(self, **kwargs) -> PluginResult:
        algorithms = kwargs.get("algorithms", ["greedy", "
genetic", "rou
   |    nd_robin"])
        run_visualization = kwargs.get("visualization", True)
        save_results = kwargs.get("save_results", True)
        include_series = kwargs.get("include_series", False)

        try:
            if self.run_mode in ("distributed", "large_scale"):
                df = self._run_distributed_comparison(algorithms)
            else:
                df = self.simulator.run_comparison(algorithms=algo
rithms
```

```
    )

            stats = self._calculate_statistics(df)
            warnings: List[str] = []
            skipped = getattr(self.simulator, "skipped_algorithms", []) if self.simulator else []
            if skipped:
                warnings.append("以下算法执行失败已跳过: " + ", ".join(skipped))

            if save_results:
                csv_path = save_simulation_data(df)
                excel_path = export_results_to_excel(df)

            if run_visualization:
                plots = self._generate_plots(df)
        except Exception as e:
            return PluginResult.fail(
                message=f"仿真执行异常: {e}",
                error_code="SIMULATION_ERROR",
                error_detail=str(e)
            )

        payload = {
            "statistics": stats,
            "records_count": len(df),
            "algorithms": algorithms,
            "run_mode": self.run_mode,
            "server_count": self.num_servers,
            "simulation_hours": self.simulation_hours
        }
        if include_series:
            payload["series"] = df.to_dict("records")
        if save_results:
            payload["csv_path"] = csv_path
            payload["excel_path"] = excel_path
        if run_visualization:
            payload["plots"] = plots

        if self.simulator is not None:
            payload["first_hour_offered"] = self.simulator.control_signature(df)
```

```
            payload["first_hour_served"] =
self.simulator.first_hour_loa
    ds(df)
            payload["controlled"] =
self.simulator.is_controlled(df)
            payload["ranking"] = self.simulator.get_ranking(df)
            payload["data_source"] = self.simulator.run_metadata
            if save_results:
                save_run_metadata({
                    **self.simulator.run_metadata,
                    "algorithms": algorithms,
                    "num_servers": self.num_servers,
                    "simulation_hours": self.simulation_hours,
                    "controlled": payload["controlled"],
                })

        return PluginResult.ok(
            data=payload,
           message=f"仿真完成 ({self.run_mode} 模式)",
            warnings=warnings,
            metrics={
                "server_count": self.num_servers,
                "simulation_hours": self.simulation_hours,
                "algorithm_count": len(algorithms),
                "data_points": len(df)
            }
        )

    def post_execute(self, result: PluginResult, **kwargs) ->
PluginResu
    lt:
        if result.success:
            self.logger.info(
               f"仿真完成: {result.data.get('records_count', 0)} 条记录
    , "
               f"{len(result.data.get('algorithms', []))} 个算法"
            )
        return result

    def cleanup(self) -> None:
        self.simulator = None
        self.distributed_simulator = None

```

```
    def _run_distributed_comparison(self, algorithms: List[str])
-> pd.D
    ataFrame:
        if not self.distributed_simulator:
            raise RuntimeError("分布式仿真器未初始化")
        results = self.distributed_simulator.run_parallel_comparis
on(
            algorithms=algorithms, hours=self.simulation_hours
        )
        all_data = []
        for algo, algo_results in results.items():
            for hr in algo_results:
                all_data.append({
                    "time": hr["hour"], "algorithm":
hr["algorithm"],
                    "total_load": hr["total_load"], "
avg_temperature": h
    r["avg_temperature"],
                    "total_energy": hr["total_energy"], "pue":
hr["pue"]
                })
        return pd.DataFrame(all_data)

    def _calculate_statistics(self, df: pd.DataFrame) ->
Dict[str, Any]:
        """Same metric definitions as the standalone simulator,
one code
     path."""
        return summarize(df)

    def _generate_plots(self, df) -> Dict[str, str]:
        """Best-effort plotting: a missing column skips one
chart, not t
    he run."""
        os.makedirs(self.reports_dir, exist_ok=True)
        targets = [
            ("energy", "energy_comparison.png", "total_energy",
plot_ene
    rgy_comparison),
            ("temperature", "temperature_trend.png", "
avg_temperature",
    plot_temperature_trend),
            ("load", "load_distribution.png", "total_load",
plot_load_di
```

```
   |    stribution),
237|            ("pue", "pue_comparison.png", "pue",
plot_pue_comparison),
238|            ("cooling", "cooling_power.png", "cooling_power",
plot_cooli
   |    ng_power),
239|            ("service", "service_rate.png", "assigned",
plot_service_rat
   |    e),
240|        ]
241|
242|        plots: Dict[str, str] = {}
243|        for key, filename, column, painter in targets:
244|            if column not in df.columns:
245|                self.logger.warning(f"结果缺少 {column} 列，已跳过 {file
   |    name}")
246|                continue
247|            path = os.path.join(self.reports_dir, filename)
248|            try:
249|                painter(df, path)
250|                plots[key] = path
251|            except Exception as exc:
252|                self.logger.warning(f"生成 {filename} 失败: {exc}")
253|        return plots
254|
255|    def run_single_simulation(self, algorithm: str = "greedy") ->
Dict[s
   |    tr, Any]:
256|        if not self.simulator and not self.distributed_simulator:
257|            return {"success": False, "error": "插件未初始化"}
258|        try:
259|            if self.distributed_simulator:
260|                results = self.distributed_simulator.run_distribut
ed_sim
   |    ulation(
261|                    hours=self.simulation_hours,
algorithm=algorithm
262|                )
263|                df = pd.DataFrame([{
264|                    "time": r["hour"], "algorithm":
r["algorithm"],
265|                    "total_load": r["total_load"], "
avg_temperature": r[
   |    "avg_temperature"],
```

```
                    "total_energy": r["total_energy"], "pue":
r["pue"]
                } for r in results])
            else:
                df = self.simulator.run_simulation(algorithm=algor
ithm)
            return {"success": True, "statistics":
self._calculate_stati
    stics(df), "records_count": len(df)}
        except Exception as e:
            return {"success": False, "error": str(e)}

    def run_large_scale_test(self, server_counts: List[int]) ->
Dict[str
    , Any]:
        if not self.distributed_simulator:
            return {"success": False, "error": "分布式仿真器未初始化"}
        try:
            results = self.distributed_simulator.run_large_scale_t
est(
                server_counts=server_counts,
hours=self.simulation_hours
            )
            return {"success": True, "results": results, "
test_hours": s
    elf.simulation_hours}
        except Exception as e:
            return {"success": False, "error": str(e)}
```

40. src/plugins/__init__.py

```
"""
GitHub Skills 第三方插件包 — 通过 PluginManager 自动发现

提供从 GitHub 开源项目接入的算力调度增强算法，全部以 PluginBase 插件形式
通过 ``PluginManager.discover_plugins(plugin_dir="src/plugins")`` 自动注
    册。
对未安装的可选依赖（Gymnasium / Stable Baselines3 / pymoo）优雅降级。

已启用:
    - ``nsga2_scheduler``      : pymoo NSGA2 多目标调度器
        (https://github.com/anyoptimization/pymoo, Apache 2.0)
    - ``moead_scheduler``      : pymoo MOEAD 分解多目标调度器
        (https://github.com/anyoptimization/pymoo, Apache 2.0)

```

```
14|预留:
15|    - ``datacenter_gym_env`` / ``rl_ppo_scheduler`` : 强化学习接口
16|      依赖 Gymnasium / Stable Baselines3 与训练模型，默认禁用
17|
18|使用示例::
19|
20|    from src.plugin import PluginManager
21|    pm = PluginManager()
22|    n = pm.discover_plugins(plugin_dir="src/plugins")
23|    pm.load_plugin("NSGA2SchedulerPlugin", config={"pop_size":
50, "n_ge
   |    n": 100})
24|    result = pm.execute_plugin("NSGA2SchedulerPlugin",
tasks=tasks, serv
   |    ers=servers)
25|"""
26|
27|__all__ = [
28|    "_deps",
29|    "datacenter_gym_env",
30|    "rl_ppo_scheduler",
31|    "nsga2_scheduler",
32|    "moead_scheduler",
33|]
34|
35|__version__ = "1.0.0"
```

41. src/plugins/_deps.py

```
 1|"""
 2|GitHub Skills 可选依赖探测 (Lazy Detection)
 3|
 4|对 Gymnasium / Stable Baselines3 / pymoo 三个可选依赖做软探测，
 5|未安装时返回 False 而不抛异常，供插件层做降级处理。
 6|
 7|设计原则:
 8|    - 所有探测函数使用 try/except ImportError，绝不向上抛
 9|    - 提供 ``get_dependency_status()`` 统一汇总，方便 CLI / API 透出
10|    - 提供 ``install_hint`` 字段，便于生成安装指引
11|
12|对应测试: ``tests/plugins/test_pymoo_plugin.py::TestDependencyMissingFal
   |    lback``
13|"""
14|
15|from __future__ import annotations
```

```
16|
17|from typing import Any, Dict
18|
19|
20|def has_gymnasium() -> bool:
21|    """检查 ``gymnasium`` (RL 环境标准 API) 是否安装。"""
22|    try:
23|        import gymnasium  # noqa: F401
24|        return True
25|    except ImportError:
26|        return False
27|
28|
29|def has_sb3() -> bool:
30|    """检查 ``stable_baselines3`` (DLR-RM RL 算法库) 是否安装。"""
31|    try:
32|        import stable_baselines3  # noqa: F401
33|        return True
34|    except ImportError:
35|        return False
36|
37|
38|def has_pymoo() -> bool:
39|    """检查 ``pymoo`` (多目标优化库) 是否安装。"""
40|    try:
41|        import pymoo  # noqa: F401
42|        return True
43|    except ImportError:
44|        return False
45|
46|
47|def _probe_version(import_name: str, mod: Any = None) -> str:
48|    """安全获取已安装库的版本号。"""
49|    try:
50|        m = mod if mod is not None else __import__(import_name)
51|        return getattr(m, "__version__", "unknown")
52|    except Exception:
53|        return "unknown"
54|
55|
56|def get_dependency_status() -> Dict[str, Any]:
57|    """
58|    返回三个可选依赖的统一状态。
59|
```

```
    Returns:
        dict: ``{
            "gymnasium": {"available": bool, "version": str, "
install_hi
    |   nt": str},
            "stable_baselines3": {...},
            "pymoo": {...},
        }``
    """
    status: Dict[str, Any] = {}

    # gymnasium
    g_avail = has_gymnasium()
    g_ver = _probe_version("gymnasium") if g_avail else None
    status["gymnasium"] = {
        "available": g_avail,
        "version": g_ver,
        "install_hint": "pip install gymnasium>=1.0",
    }

    # stable_baselines3
    sb3_avail = has_sb3()
    sb3_ver = _probe_version("stable_baselines3") if sb3_avail
else None
    status["stable_baselines3"] = {
        "available": sb3_avail,
        "version": sb3_ver,
        "install_hint": "pip install stable-baselines3>=2.0",
    }

    # pymoo
    p_avail = has_pymoo()
    p_ver = None
    if p_avail:
        try:
            from pymoo import __version__ as _p_ver
            p_ver = _p_ver
        except Exception:
            p_ver = "unknown"
    status["pymoo"] = {
        "available": p_avail,
        "version": p_ver,
        "install_hint": "pip install pymoo>=0.6",
    }
```

```
101|
102|    return status
```

42. src/plugins/datacenter_gym_env.py

```
  1|"""
  2|Gymnasium Env 适配器 — DataCenterSimulator → gymnasium.Env
  3|
  4|把 ``DataCenterSimulator`` 包装为符合 ``gymnasium.Env`` 接口规范的环境，
  5|供 Stable Baselines3 PPO 训练使用。
  6|
  7|State (observation) 编码 (归一化到 [0, 1]):
  8|    - 服务器特征 (num_servers * 3): [load_ratio, temp_ratio, type_norm]
   |    × N
  9|    - 当前任务特征 (4):           [load_norm, priority_norm, deadline_no
   |    rm, wc_oh(1)]
 10|
 11|Action space:
 12|    - Discrete(num_servers)  — 选择把当前任务分配到哪台服务器
 13|
 14|Reward:
 15|    reward = -Δenergy - 0.5*Δtemp - 10*deadline_violation +
0.1*balance
 16|
 17|依赖:
 18|    pip install gymnasium
 19|
 20|来源: https://github.com/Farama-Foundation/Gymnasium (MIT)
 21|"""
 22|
 23|from __future__ import annotations
 24|
 25|import math
 26|from typing import Any, Dict, List, Optional, Tuple
 27|
 28|from src.plugins._deps import has_gymnasium
 29|
 30|if has_gymnasium():
 31|    import gymnasium as gym
 32|    from gymnasium import spaces
 33|    import numpy as np
 34|else:
 35|    gym = None  # type: ignore
 36|    spaces = None  # type: ignore
 37|    np = None  # type: ignore
```

```
 38|
 39|from src.simulation import DataCenterSimulator
 40|from src.models import Task
 41|
 42|
 43|class DataCenterGymEnv:
 44|    """
 45|    DataCenter Gymnasium Environment
 46|
 47|    与 ``gymnasium.Env`` 兼容（非继承，因为无 gym 时需要软降级）。
 48|
 49|    用法::
 50|
 51|        env = DataCenterGymEnv(num_servers=10,
simulation_hours=24, seed
   |    =42)
 52|        obs, info = env.reset()
 53|        for _ in range(1000):
 54|            action = env.action_space.sample()
 55|            obs, reward, terminated, truncated, info =
env.step(action)
 56|            if terminated or truncated:
 57|                break
 58|
 59|    注意:
 60|        - 如果 gymnasium 未安装，构造时直接抛 ``ImportError``，调用方需提
   |    前探测
 61|        - 不继承 ``gym.Env`` 是为了保持模块在无依赖时也能 ``import``
 62|    """
 63|
 64|    metadata = {"render_modes": []}
 65|
 66|    # 服务器类型 one-hot 索引映射（与 Server.server_type 字符串对应）
 67|    SERVER_TYPES = ["standard", "high_performance", "gpu_server"]
 68|    WORKLOAD_CLASSES = ["cpu_bound", "memory_bound", "io_bound"]
 69|
 70|    def __init__(
 71|        self,
 72|        num_servers: int = 10,
 73|        simulation_hours: int = 24,
 74|        seed: Optional[int] = None,
 75|        config: Optional[Dict[str, Any]] = None,
 76|    ) -> None:
 77|        if not has_gymnasium():
```

```
            raise ImportError(
                "gymnasium 未安装。请运行: pip install gymnasium>=1.0"
            )
        if num_servers < 1:
            raise ValueError(f"num_servers 必须 >= 1, 当前={num_servers}")

        self.num_servers: int = num_servers
        self.simulation_hours: int = simulation_hours
        self._seed_value: Optional[int] = seed
        self._config: Optional[Dict[str, Any]] = config
        self._current_time: int = 0
        self._current_task: Optional[Task] = None
        self._step_count: int = 0
        self._max_steps: int = simulation_hours * num_servers  # heuristic horizon

        # observation = servers (num_servers * 3) + task (4)
        self.observation_dim: int = num_servers * 3 + 4
        self.observation_space = spaces.Box(
            low=0.0,
            high=1.0,
            shape=(self.observation_dim,),
            dtype=np.float32,
        )
        self.action_space = spaces.Discrete(num_servers)

        # simulator / servers 状态在 reset() 时初始化
        self.simulator: Optional[DataCenterSimulator] = None
        self._prev_total_power: float = 0.0
        self._prev_max_temp: float = 25.0

    # Gymnasium API

    def reset(
        self,
        seed: Optional[int] = None,
        options: Optional[Dict[str, Any]] = None,
    ) -> Tuple["np.ndarray", Dict[str, Any]]:
        if seed is not None:
            self._seed_value = seed
        self.simulator = DataCenterSimulator(
            num_servers=self.num_servers,
```

```
            simulation_hours=self.simulation_hours,
            seed=self._seed_value,
            config=self._config,
        )
        self._current_time = 0
        self._step_count = 0
        self._current_task = self._generate_task(self._current_tim
e)

        self._prev_total_power = sum(s.calculate_power() for s in
self.s
    imulator.servers)
        self._prev_max_temp = max(s.current_temp for s in
self.simulator
    .servers)
        return self._get_obs(), self._get_info()

    def step(self, action: int) -> Tuple["np.ndarray", float,
bool, bool
    , Dict[str, Any]]:
        assert self.simulator is not None, "请先调用 reset()"
        if not self.action_space.contains(int(action)):
            raise ValueError(f"action 必须在 [0, {self.num_servers})")

        server = self.simulator.servers[int(action)]
        task = self._current_task

        # 分配任务（如果超载，则视为失败放置 — 仍记入状态）
        assigned = False
        deadline_violation = 0
        if task is not None:
            deadline_violation = 1 if
task.is_deadline_violated(self._cu
    rrent_time) else 0
            assigned = server.assign_task(task)

        # 更新温度
        for s in self.simulator.servers:
            s.update_temperature(ambient_temp=25.0,
cooling_efficiency=0
    .5)

        # 统计差量
```

```
153|        total_power = sum(s.calculate_power() for s in
self.simulator.se
   |    rvers)
154|        max_temp = max(s.current_temp for s in
self.simulator.servers)
155|        delta_energy = (total_power - self._prev_total_power) /
1000.0
   |    # 归一化
156|        delta_temp = (max_temp - self._prev_max_temp) / 100.0
157|
158|        # 负载均衡奖励（标准差越小越好）
159|        loads = [s.current_load / max(s.max_load, 1.0) for s in
self.sim
   |    ulator.servers]
160|        load_std = float(np.std(loads)) if len(loads) > 0 else 0.0
161|        balance_bonus = 0.1 * (1.0 - min(load_std, 1.0))
162|
163|        reward = -max(delta_energy, 0.0) - 0.5 * max(delta_temp,
0.0) -
   |    10.0 * deadline_violation + balance_bonus
164|        # 关键决策：未分配成功的任务要重罚
165|        if not assigned:
166|            reward -= 5.0
167|
168|        self._prev_total_power = total_power
169|        self._prev_max_temp = max_temp
170|        self._step_count += 1
171|        self._current_time += 1
172|
173|        # 推进到下一任务 / 终止
174|        terminated = False
175|        truncated = False
176|        if self._step_count >= self._max_steps or
self._current_time >=
   |    self.simulation_hours:
177|            truncated = True
178|        else:
179|            self._current_task =
self._generate_task(self._current_time)
180|            if self._current_task is None:
181|                terminated = True
182|
183|        return self._get_obs(), float(reward), terminated,
truncated, se
```

```
   |    lf._get_info()
184|
185|    def render(self) -> None:  # pragma: no cover - 不支持渲染
186|        return None
187|
188|    def close(self) -> None:
189|        self.simulator = None
190|
192|    # 内部辅助
194|
195|    def _generate_task(self, current_time: int) -> Optional[Task]:
196|        """生成一个当前时刻的任务。简单随机生成（与仿真器 TaskGenerator
行
   |    为近似）。"""
197|        if self.simulator is None or
self.simulator.task_generator is No
   |    ne:
198|            return None
199|        try:
200|            tasks = self.simulator.task_generator.generate_tasks(c
urrent
   |    _time)
201|        except Exception:
202|            return None
203|        if not tasks:
204|            return None
205|        return tasks[0]
206|
207|    def _get_obs(self) -> "np.ndarray":
208|        if self.simulator is None:
209|            return np.zeros(self.observation_dim,
dtype=np.float32)
210|        servers = self.simulator.servers
211|        srv_feats: List[float] = []
212|        for s in servers:
213|            srv_feats.append(min(s.current_load / max(s.max_load,
1.0),
   |    1.0))
214|            srv_feats.append(min(s.current_temp / max(s.max_temp,
1.0),
   |    1.0))
215|            srv_feats.append(self._type_norm(s.server_type))
216|        task_feats: List[float] = [0.0, 0.0, 0.0, 0.0]
217|        if self._current_task is not None:
```

```
            t = self._current_task
            task_feats[0] = min(t.load / 50.0, 1.0)
            task_feats[1] = min(t.priority / 3.0, 1.0) if
t.priority els
    e 0.0
            task_feats[2] = (
                1.0 if t.deadline is None else min(t.deadline /
100.0, 1
    .0)
            )
            task_feats[3] = 1.0 if t.workload_class == "
cpu_bound" else
    0.0
        obs = np.array(srv_feats + task_feats, dtype=np.float32)
        return obs

    def _get_info(self) -> Dict[str, Any]:
        if self.simulator is None:
            return {}
        servers = self.simulator.servers
        return {
            "current_time": self._current_time,
            "step_count": self._step_count,
            "total_load": float(sum(s.current_load for s in
servers)),
            "max_temp": float(max(s.current_temp for s in
servers)),
            "total_power": float(sum(s.calculate_power() for s in
server
    s)),
            "current_task": (
                None if self._current_task is None else
self._current_ta
    sk.task_id
            ),
        }

    def _type_norm(self, server_type: str) -> float:
        if server_type in self.SERVER_TYPES:
            return float(self.SERVER_TYPES.index(server_type)) /
max(len
    (self.SERVER_TYPES) - 1, 1)
        return 0.0

```

```
248|    @property
249|    def unwrapped(self):
250|        return self
251|
252|    @property
253|    def np_random(self) -> "np.random.Generator":
254|        return np.random.default_rng(self._seed_value)
255|
256|
257|__all__ = ["DataCenterGymEnv"]
```

43. src/plugins/moead_scheduler.py

```
  1|"""
  2|MOEADSchedulerPlugin — pymoo MOEAD 分解多目标调度器 (GitHub Skills 集成)
  3|
  4|与 NSGA2 同构（同样的 Problem 定义），但用基于参考向量分解的 MOEAD 算法。
  5|
  6|差异:
  7|    - 算法: ``MOEAD(ref_dirs=get_reference_directions("das-dennis", 3, n
    |    _partitions),
  8|                    n_neighbors, prob_neighbor_mating)``
  9|    - 解的选择：在 ref_dirs 均匀覆盖下挑与偏好向量夹角最小的解
 10|    - 解码阶段：与 NSGA2 相同（整数编码 → server_id 列表）
 11|
 12|依赖:
 13|    pip install pymoo
 14|
 15|来源: https://github.com/anyoptimization/pymoo (Apache 2.0)
 16|"""
 17|
 18|from __future__ import annotations
 19|
 20|from typing import Any, Dict, List, Optional
 21|
 22|from src.plugin.base import PluginBase
 23|from src.plugin.protocol import PluginManifest, PluginResult
 24|from src.plugins._deps import has_pymoo
 25|
 26|
 27|OVERLOAD_PENALTY = 50.0
 28|
 29|
 30|class MOEADSchedulerPlugin(PluginBase):
 31|    """pymoo MOEAD 多目标调度器（基于参考向量分解）"""
```

```
 32|
 34|    # 清单 / 配置
 36|
 37|    def get_manifest(self) -> PluginManifest:
 38|        return PluginManifest(
 39|            name="MOEADSchedulerPlugin",
 40|            version="1.0.0",
 41|            supported_operations=["schedule"],
 42|            input_schema={
 43|                "n_partitions": {"type": "int", "default": 12, "
min": 2}
   |    ,
 44|                "n_neighbors": {"type": "int", "default": 15, "
min": 2},
 45|                "prob_neighbor_mating": {"type": "float", "
default": 0.7
   |    , "min": 0.0, "max": 1.0},
 46|                "n_gen": {"type": "int", "default": 200, "min":
1},
 47|                "selection": {
 48|                    "type": "str",
 49|                    "enum": ["energy_priority", "balanced", "
temp_priori
   |    ty"],
 50|                    "default": "balanced",
 51|                },
 52|                "seed": {"type": "int", "default": 42},
 53|            },
 54|            output_schema={
 55|                "assignments": "list",
 56|                "pareto_size": "int",
 57|                "generation": "int",
 58|                "objectives": "dict",
 59|            },
 60|            min_system_version="1.0.0",
 61|            category="algorithm",
 62|        )
 63|
 64|    def validate_config(self, config: Dict[str, Any]) -> bool:
 65|        n_partitions = config.get("n_partitions", 12)
 66|        if not isinstance(n_partitions, int) or n_partitions < 2:
 67|            self.logger.error(f"n_partitions 必须 >= 2, 当前={n_partitio
   |    ns}")
 68|            return False
```

```
        n_neighbors = config.get("n_neighbors", 15)
        if not isinstance(n_neighbors, int) or n_neighbors < 2:
            self.logger.error(f"n_neighbors 必须 >= 2, 当前={n_neighbors}")
            return False
        n_gen = config.get("n_gen", 200)
        if not isinstance(n_gen, int) or n_gen < 1:
            self.logger.error(f"n_gen 必须 >= 1, 当前={n_gen}")
            return False
        return True

    # 生命周期

    def initialize(self, config: Optional[Dict[str, Any]] = None) -> bool:
        cfg = dict(config or {})
        if not self.validate_config(cfg):
            return False
        self._deps_ok = has_pymoo()
        self._n_partitions: int = int(cfg.get("n_partitions", 12))
        self._n_neighbors: int = int(cfg.get("n_neighbors", 15))
        self._prob_neighbor_mating: float = float(cfg.get("prob_neighbor_mating", 0.7))
        self._n_gen: int = int(cfg.get("n_gen", 200))
        self._selection: str = str(cfg.get("selection", "balanced"))
        self._seed: Optional[int] = cfg.get("seed", 42)
        return True

    def cleanup(self) -> None:
        return None

    # 执行

    def execute(self, **kwargs: Any) -> PluginResult:
        if not has_pymoo():
            return PluginResult.fail(
                message="pymoo 未安装。请执行: pip install pymoo>=0.6",
                error_code="DEPENDENCY_MISSING",
            )
        tasks: Optional[List[Any]] = kwargs.get("tasks")
        servers: Optional[List[Any]] = kwargs.get("servers")
```

```
        if not tasks or servers is None:
            return PluginResult.fail(
                message="MOEAD 调度需要 tasks 与 servers 参数",
                error_code="MISSING_PARAM",
            )
        try:
            return self._solve(tasks, servers)
        except Exception as e:
            self.logger.error(f"MOEAD 调度失败: {e}", exc_info=True)
            return PluginResult.fail(
                message=f"MOEAD 调度失败: {e}",
                error_code="MOEAD_ERROR",
                error_detail=str(e),
            )

    # 内部

    def _solve(self, tasks: List[Any], servers: List[Any]) -> PluginResult:
        from pymoo.core.problem import ElementwiseProblem
        from pymoo.algorithms.moo.moead import MOEAD
        from pymoo.operators.crossover.sbx import SBX
        from pymoo.operators.mutation.pm import PM
        from pymoo.operators.repair.rounding import RoundingRepair
        from pymoo.operators.sampling.rnd import IntegerRandomSampling
        from pymoo.optimize import minimize
        from pymoo.termination import get_termination
        from pymoo.util.ref_dirs import get_reference_directions

        num_tasks = len(tasks)
        num_servers = len(servers)
        if num_tasks == 0 or num_servers == 0:
            return PluginResult.fail(
                message="tasks 与 servers 均不能为空",
                error_code="EMPTY_INPUT",
            )

        task_loads = [float(t.load) for t in tasks]
        task_deadlines = [
            t.deadline if t.deadline is not None else 10**9 for t in tas
    ks
```

```
        ]
        max_loads = [float(s.max_load) for s in servers]
        base_powers = [float(s.base_power) for s in servers]
        max_powers = [float(s.max_power) for s in servers]
        init_loads = [float(s.current_load) for s in servers]
        max_loads_safe = [max(ml, 1.0) for ml in max_loads]

        class SchedulingProblem(ElementwiseProblem):
            def __init__(self_inner):
                super().__init__(
                    n_var=num_tasks,
                    n_obj=3,
                    n_ieq_constr=0,
                    xl=0,
                    xu=num_servers - 1,
                )

            def _evaluate(self_inner, x, out, *args, **kwargs):
                assign = [int(round(float(v))) for v in x]
                assign = [max(0, min(num_servers - 1, a)) for a
in assig
   |    n]
                srv_load = list(init_loads)
                for i, s_idx in enumerate(assign):
                    srv_load[s_idx] += task_loads[i]
                total_energy = 0.0
                for j in range(num_servers):
                    lf = srv_load[j] / max_loads_safe[j]
                    total_energy += base_powers[j] +
(max_powers[j] - ba
   |    se_powers[j]) * lf
                lfs = [srv_load[j] / max_loads_safe[j] for j in
range(nu
   |    m_servers)]
                max_temp_proxy = 25.0 + 60.0 * max(lfs) if lfs
else 25.0
                violations = 0
                for i, s_idx in enumerate(assign):
                    if task_deadlines[i] < 10 and (
                        srv_load[s_idx] / max_loads_safe[s_idx] >
0.9
                    ):
                        violations += 1
                overload = sum(
```

```
                    max(0.0, srv_load[j] - max_loads[j]) for j in
range(
    num_servers)
                )
                out["F"] = [float(total_energy + overload *
OVERLOAD_PEN
    ALTY),
                            float(max_temp_proxy),
float(violations)]

        # 参考方向 (das-dennis)
        ref_dirs = get_reference_directions("das-dennis", 3,
n_partition
    s=self._n_partitions)
        algorithm = MOEAD(
            ref_dirs=ref_dirs,
            n_neighbors=self._n_neighbors,
            prob_neighbor_mating=self._prob_neighbor_mating,
            crossover=SBX(prob=0.9, eta=3.0,
repair=RoundingRepair()),
            mutation=PM(prob=1.0 / max(num_tasks, 1), eta=3.0,
repair=Ro
    undingRepair()),
        )
        res = minimize(
            SchedulingProblem(),
            algorithm,
            termination=get_termination("n_gen", self._n_gen),
            seed=self._seed,
            verbose=False,
        )

        if res.X is None or (hasattr(res.X, "shape") and
res.X.shape[0]
    == 0):
            return PluginResult.fail(
               message="MOEAD 未找到可行解",
                error_code="NO_SOLUTION",
            )

        F = res.F
        X = res.X
        if len(F.shape) == 1:
            F = F.reshape(1, -1)
```

```
            X = X.reshape(1, -1)
        idx = self._select_by_preference(F, ref_dirs)
        best_x = X[idx]
        assign = [int(round(float(v))) for v in best_x]
        assign = [max(0, min(num_servers - 1, a)) for a in assign]

        assignments: List[Dict[str, Any]] = []
        for i, s_idx in enumerate(assign):
            assignments.append(
                {
                    "task_id": getattr(tasks[i], "task_id",
str(i)),
                    "server_id": s_idx,
                    "load": task_loads[i],
                }
            )
        return PluginResult.ok(
            data={
                "assignments": assignments,
                "pareto_size": int(F.shape[0]),
                "generation": self._n_gen,
                "objectives": {
                    "energy": float(F[idx, 0]),
                    "max_temp_proxy": float(F[idx, 1]),
                    "deadline_violations": float(F[idx, 2]),
                },
                "selection": self._selection,
            },
           message=f"MOEAD 调度完成: {len(assignments)} 个任务, pareto=
    {F.shape[0]}",
        )

    def _select_by_preference(self, F: Any, ref_dirs: Any) -> int:
        """
       在 ref_dirs 均匀覆盖下，挑 weighted-sum 最小的解。
        （若 ref_dirs 与 F 维度不一致，则退化为加权和最小）
        """
        import numpy as np
        selection_weights = {
            "energy_priority": (1.0, 0.3, 5.0),
            "balanced": (1.0, 1.0, 10.0),
            "temp_priority": (0.3, 1.0, 5.0),
        }
```

```
        w = selection_weights.get(self._selection, (1.0, 1.0,
10.0))
        Fs = np.asarray(F, dtype=float)
        mins = Fs.min(axis=0)
        maxs = Fs.max(axis=0)
        rng = np.where((maxs - mins) > 0, (maxs - mins), 1.0)
        norm = (Fs - mins) / rng
        scores = norm @ np.array(w, dtype=float)
        return int(np.argmin(scores))


__all__ = ["MOEADSchedulerPlugin"]
```

44. src/plugins/nsga2_scheduler.py

```
"""
NSGA2SchedulerPlugin — pymoo NSGA2 多目标调度器 (GitHub Skills 集成)

把调度问题建模为 pymoo 的 ``ElementwiseProblem``:

    决策变量 x ∈ [0, num_servers-1]^num_tasks  (整数编码)
    目标函数:
        f1 = 总能耗
        f2 = 最高温度代理
        f3 = deadline 违约计数
    不等式约束:
         g1 = max(0, sum(max(0, srv_load - srv_max)))

算法: ``NSGA2(pop_size, sampling=IntegerRandomSampling,
              crossover=SBX(repair=RoundingRepair),
mutation=PM(repair=R
    oundingRepair))``

从 Pareto 前沿根据 ``selection`` 偏好 (energy_priority / balanced / temp_
    priority
选出一个折中解，输出 ``[{task_id, server_id}, ...]``。

依赖:
    pip install pymoo

来源: https://github.com/anyoptimization/pymoo (Apache 2.0)
"""

from __future__ import annotations

```

```
import copy
from typing import Any, Dict, List, Optional

from src.plugin.base import PluginBase
from src.plugin.protocol import PluginManifest, PluginResult
from src.plugins._deps import has_pymoo


class NSGA2SchedulerPlugin(PluginBase):
    """pymoo NSGA2 多目标调度器"""

    SELECTION_MODES = {"energy_priority", "balanced", "temp_priority"}
    SELECTION_WEIGHTS = {
        "energy_priority": (1.0, 0.3, 5.0),
        "balanced": (1.0, 1.0, 10.0),
        "temp_priority": (0.3, 1.0, 5.0),
    }

    # 清单 / 配置

    def get_manifest(self) -> PluginManifest:
        return PluginManifest(
            name="NSGA2SchedulerPlugin",
            version="1.0.0",
            supported_operations=["schedule"],
            input_schema={
                "pop_size": {"type": "int", "default": 100, "min": 4},
                "n_gen": {"type": "int", "default": 200, "min": 1},
                "selection": {
                    "type": "str",
                    "enum": sorted(self.SELECTION_MODES),
                    "default": "balanced",
                },
                "seed": {"type": "int", "default": 42},
            },
            output_schema={
                "assignments": "list",
                "pareto_size": "int",
                "generation": "int",
                "objectives": "dict",
            },
```

```
            min_system_version="1.0.0",
            category="algorithm",
        )

    def validate_config(self, config: Dict[str, Any]) -> bool:
        selection = config.get("selection", "balanced")
        if selection not in self.SELECTION_MODES:
            self.logger.error(
               f"selection 必须在 {self.SELECTION_MODES} 中, 当前={sele
ction}"
            )
            return False
        pop_size = config.get("pop_size", 100)
        if not isinstance(pop_size, int) or pop_size < 4:
           self.logger.error(f"pop_size 必须 >= 4, 当前={pop_size}")
            return False
        n_gen = config.get("n_gen", 200)
        if not isinstance(n_gen, int) or n_gen < 1:
           self.logger.error(f"n_gen 必须 >= 1, 当前={n_gen}")
            return False
        return True

    # 生命周期

    def initialize(self, config: Optional[Dict[str, Any]] = None)
-> boo
l:
        cfg = dict(config or {})
        if not self.validate_config(cfg):
            return False

        self._deps_ok = has_pymoo()
        self._pop_size: int = int(cfg.get("pop_size", 100))
        self._n_gen: int = int(cfg.get("n_gen", 200))
        self._selection: str = str(cfg.get("selection", "
balanced"))
        self._seed: Optional[int] = cfg.get("seed", 42)
        return True

    def cleanup(self) -> None:
        return None

    # 执行

```

```
115|    def execute(self, **kwargs: Any) -> PluginResult:
116|        if not has_pymoo():
117|            return PluginResult.fail(
118|               message="pymoo 未安装。请执行: pip install pymoo>=0.6",
119|                error_code="DEPENDENCY_MISSING",
120|            )
121|        tasks: Optional[List[Any]] = kwargs.get("tasks")
122|        servers: Optional[List[Any]] = kwargs.get("servers")
123|        if not tasks or servers is None:
124|            return PluginResult.fail(
125|               message="NSGA2 调度需要 tasks 与 servers 参数",
126|                error_code="MISSING_PARAM",
127|            )
128|        try:
129|            return self._solve(tasks, servers)
130|        except Exception as e:
131|           self.logger.error(f"NSGA2 调度失败: {e}", exc_info=True)
132|            return PluginResult.fail(
133|               message=f"NSGA2 调度失败: {e}",
134|                error_code="NSGA2_ERROR",
135|                error_detail=str(e),
136|            )
137|
139|    # 内部
141|
142|    def _solve(self, tasks: List[Any], servers: List[Any]) ->
PluginResu
   |    lt:
143|        from pymoo.core.problem import ElementwiseProblem
144|        from pymoo.algorithms.moo.nsga2 import NSGA2
145|        from pymoo.operators.crossover.sbx import SBX
146|        from pymoo.operators.mutation.pm import PM
147|        from pymoo.operators.repair.rounding import RoundingRepair
148|        from pymoo.operators.sampling.rnd import
IntegerRandomSampling
149|        from pymoo.optimize import minimize
150|        from pymoo.termination import get_termination
151|
152|        num_tasks = len(tasks)
153|        num_servers = len(servers)
154|        if num_tasks == 0 or num_servers == 0:
155|            return PluginResult.fail(
156|               message="tasks 与 servers 均不能为空",
157|                error_code="EMPTY_INPUT",
```

```
158|            )
159|
160|        # 任务/服务器静态特征（提前计算一次）
161|        task_loads = [float(t.load) for t in tasks]
162|        task_deadlines = [
163|            t.deadline if t.deadline is not None else 10**9 for t
in tas
   |    ks
164|        ]
165|        max_loads = [float(s.max_load) for s in servers]
166|        base_powers = [float(s.base_power) for s in servers]
167|        max_powers = [float(s.max_power) for s in servers]
168|        init_loads = [float(s.current_load) for s in servers]
169|        max_loads_safe = [max(ml, 1.0) for ml in max_loads]
170|
171|        # 保存 servers 副本用于目标评估（避免污染输入对象）
172|        class SchedulingProblem(ElementwiseProblem):
173|            def __init__(self_inner):
174|                super().__init__(
175|                    n_var=num_tasks,
176|                    n_obj=3,
177|                    n_ieq_constr=1,
178|                    xl=0,
179|                    xu=num_servers - 1,
180|                )
181|
182|            def _evaluate(self_inner, x, out, *args, **kwargs):
183|                assign = [int(round(float(v))) for v in x]
184|                assign = [max(0, min(num_servers - 1, a)) for a
in assig
   |    n]
185|                srv_load = list(init_loads)
186|                for i, s_idx in enumerate(assign):
187|                    srv_load[s_idx] += task_loads[i]
188|              # 目标 1: 总能耗
189|                total_energy = 0.0
190|                for j in range(num_servers):
191|                    lf = srv_load[j] / max_loads_safe[j]
192|                    total_energy += base_powers[j] +
(max_powers[j] - ba
   |    se_powers[j]) * lf
193|              # 目标 2: 最高温度代理
194|                lfs = [srv_load[j] / max_loads_safe[j] for j in
range(nu
```

```
    |    m_servers)]
195|                max_temp_proxy = 25.0 + 60.0 * max(lfs) if lfs
else 25.0
196|              # 目标 3: deadline 违约（紧 deadline + 高负载判定）
197|                violations = 0
198|                for i, s_idx in enumerate(assign):
199|                    if task_deadlines[i] < 10 and (
200|                        srv_load[s_idx] / max_loads_safe[s_idx] >
0.9
201|                    ):
202|                        violations += 1
203|              # 约束: 容量超载
204|                overload = sum(
205|                    max(0.0, srv_load[j] - max_loads[j]) for j in
range(
    |    num_servers)
206|                )
207|                out["F"] = [float(total_energy),
float(max_temp_proxy),
    |    float(violations)]
208|                out["G"] = [float(overload)]
209|
210|        algorithm = NSGA2(
211|            pop_size=self._pop_size,
212|            sampling=IntegerRandomSampling(),
213|            crossover=SBX(prob=0.9, eta=3.0,
repair=RoundingRepair()),
214|            mutation=PM(prob=1.0 / max(num_tasks, 1), eta=3.0,
repair=Ro
    |    undingRepair()),
215|        )
216|        res = minimize(
217|            SchedulingProblem(),
218|            algorithm,
219|            termination=get_termination("n_gen", self._n_gen),
220|            seed=self._seed,
221|            verbose=False,
222|        )
223|
224|        if res.X is None or (hasattr(res.X, "shape") and
res.X.shape[0]
    |    == 0):
225|            return PluginResult.fail(
226|               message="NSGA2 未找到可行解",
```

```
                error_code="NO_SOLUTION",
            )

        # res.F shape = (N, 3); res.X shape = (N, num_tasks) or
(num_tas
    ks,) if N==1
        F = res.F
        X = res.X
        if len(F.shape) == 1:
            F = F.reshape(1, -1)
            X = X.reshape(1, -1)
        idx = self._select_by_preference(F)
        best_x = X[idx]
        assign = [int(round(float(v))) for v in best_x]
        assign = [max(0, min(num_servers - 1, a)) for a in assign]

        assignments: List[Dict[str, Any]] = []
        for i, s_idx in enumerate(assign):
            assignments.append(
                {
                    "task_id": getattr(tasks[i], "task_id",
str(i)),
                    "server_id": s_idx,
                    "load": task_loads[i],
                }
            )
        return PluginResult.ok(
            data={
                "assignments": assignments,
                "pareto_size": int(F.shape[0]),
                "generation": self._n_gen,
                "objectives": {
                    "energy": float(F[idx, 0]),
                    "max_temp_proxy": float(F[idx, 1]),
                    "deadline_violations": float(F[idx, 2]),
                },
                "selection": self._selection,
            },
            message=f"NSGA2 调度完成: {len(assignments)} 个任务, pareto=
    {F.shape[0]}",
        )

    def _select_by_preference(self, F: Any) -> int:
        """根据 selection 偏好从 Pareto 前沿选解。"""
```

```
        w = self.SELECTION_WEIGHTS.get(self._selection, (1.0,
1.0, 10.0)
    )
        # 归一化每列 → 加权和 → 最小索引
        import numpy as np
        Fs = np.asarray(F, dtype=float)
        mins = Fs.min(axis=0)
        maxs = Fs.max(axis=0)
        rng = np.where((maxs - mins) > 0, (maxs - mins), 1.0)
        norm = (Fs - mins) / rng
        scores = norm @ np.array(w, dtype=float)
        return int(np.argmin(scores))


__all__ = ["NSGA2SchedulerPlugin"]
```

45. src/plugins/rl_ppo_scheduler.py

```
"""
RLSchedulerPlugin — Stable Baselines3 PPO 强化学习调度器 (GitHub Skills
    集成)

继承 ``PluginBase`` v1.0，实现:

    * ``train``     — 用 PPO 在 DataCenterGymEnv 上训练，保存模型到
``mod
    el_path``
    * ``predict``   — 加载已训练模型，对一批 ``tasks`` 选出 server_id
    * ``schedule``  — predict 的别名，方便直接
``algorithms=["rl_ppo"]``
    调用
    * ``save``/``load`` — 显式的 SB3 模型持久化入口

依赖:
    pip install gymnasium stable-baselines3

依赖缺失时会返回 ``PluginResult.fail(error_code="DEPENDENCY_MISSING")``
    而不崩溃。

来源: https://github.com/DLR-RM/stable-baselines3 (MIT)
"""

from __future__ import annotations

import os
```

```
import time
from typing import Any, Dict, List, Optional

from src.plugin.base import PluginBase
from src.plugin.protocol import PluginManifest, PluginResult
from src.plugins._deps import has_gymnasium, has_sb3


class RLSchedulerPlugin(PluginBase):
    """Stable Baselines3 PPO 调度器插件"""

    # 清单 / 配置

    def get_manifest(self) -> PluginManifest:
        return PluginManifest(
            name="RLSchedulerPlugin",
            version="1.0.0",
            supported_operations=["train", "predict", "schedule", "save"
    , "load"],
            input_schema={
                "algorithms": {"type": "list", "default": ["rl_ppo"]},
                "train": {"type": "bool", "default": False},
                "total_timesteps": {"type": "int", "default": 10000, "mi
    n": 1},
                "model_path": {"type": "str", "default": "data/policies/
    ppo_datacenter.zip"},
                "learning_rate": {"type": "float", "default": 3e-4},
                "n_steps": {"type": "int", "default": 128},
                "batch_size": {"type": "int", "default": 32},
                "num_servers": {"type": "int", "default": 10, "min": 1},
                "simulation_hours": {"type": "int", "default": 24, "min"
    : 1},
                "seed": {"type": "int", "default": 42},
            },
            output_schema={
                "model_path": "str",
                "assignments": "list",
```

```
                "training_time_s": "float",
            },
            min_system_version="1.0.0",
            category="algorithm",
        )

    def validate_config(self, config: Dict[str, Any]) -> bool:
        num_servers = config.get("num_servers", 10)
        if not isinstance(num_servers, int) or num_servers < 1:
            self.logger.error(f"num_servers 必须 >= 1, 当前={num_servers}")
            return False
        sim_hours = config.get("simulation_hours", 24)
        if not isinstance(sim_hours, int) or sim_hours < 1:
            self.logger.error(f"simulation_hours 必须 >= 1, 当前={sim_hours}")
            return False
        return True

    # 生命周期

    def initialize(self, config: Optional[Dict[str, Any]] = None) -> bool:
        cfg = dict(config or {})
        if not self.validate_config(cfg):
            return False

        # 依赖检查（若缺失则初始化失败 — 符合测试契约）
        if not (has_gymnasium() and has_sb3()):
            self.logger.warning(
                "stable-baselines3 / gymnasium 未安装，RLSchedulerPlugin 初始化失败。"
                "请执行: pip install stable-baselines3 gymnasium"
            )
            self._deps_ok = False
            return False

        self._deps_ok = True
        self._num_servers: int = int(cfg.get("num_servers", 10))
        self._simulation_hours: int = int(cfg.get("simulation_hours", 24))
        self._seed: int = int(cfg.get("seed", 42))
```

```
 96|        self._learning_rate: float =
float(cfg.get("learning_rate", 3e-4
   |    ))
 97|        self._n_steps: int = int(cfg.get("n_steps", 128))
 98|        self._batch_size: int = int(cfg.get("batch_size", 32))
 99|        self._default_model_path: str = str(
100|            cfg.get("model_path", "
data/policies/ppo_datacenter.zip")
101|        )
102|
103|        self._env = None
104|        self._model = None
105|        return True
106|
107|    def cleanup(self) -> None:
108|        self._env = None
109|        self._model = None
110|
112|    # 执行
114|
115|    def execute(self, **kwargs: Any) -> PluginResult:
116|        # 依赖缺失 → 直接降级返回
117|        if not (has_gymnasium() and has_sb3()):
118|            return PluginResult.fail(
119|                message=(
120|                  "stable-baselines3 / gymnasium 未安装。请先执行: "
121|                    "pip install stable-baselines3 gymnasium"
122|                ),
123|                error_code="DEPENDENCY_MISSING",
124|            )
125|
126|        algorithms: List[str] = list(kwargs.get("algorithms") or
["rl_pp
   |    o"])
127|        is_train: bool = bool(kwargs.get("train", False))
128|        total_timesteps: int = int(kwargs.get("total_timesteps",
10000))
129|        model_path: Optional[str] = kwargs.get("model_path") or
self._de
   |    fault_model_path
130|        learning_rate: float = float(kwargs.get("learning_rate",
self._l
   |    earning_rate))
131|        tasks: Optional[List[Any]] = kwargs.get("tasks")
```

```
        servers: Optional[List[Any]] = kwargs.get("servers")

        if is_train:
            return self._train_model(
                total_timesteps=total_timesteps,
                model_path=model_path,
                learning_rate=learning_rate,
            )
        return self._predict_and_schedule(
            tasks=tasks, servers=servers, model_path=model_path
        )

    # 内部：训练 / 推理

    def _train_model(
        self, total_timesteps: int, model_path: str,
learning_rate: floa
    |   t
    ) -> PluginResult:
        try:
            from stable_baselines3 import PPO
            from src.plugins.datacenter_gym_env import
DataCenterGymEnv

            self._env = DataCenterGymEnv(
                num_servers=self._num_servers,
                simulation_hours=self._simulation_hours,
                seed=self._seed,
            )
            self._model = PPO(
                "MlpPolicy",
                self._env,
                learning_rate=learning_rate,
                n_steps=self._n_steps,
                batch_size=self._batch_size,
                verbose=0,
                seed=self._seed,
            )
            start = time.perf_counter()
            self._model.learn(total_timesteps=total_timesteps)
            elapsed = round(time.perf_counter() - start, 3)

            os.makedirs(os.path.dirname(model_path) or ".",
exist_ok=Tru
```

```
    |    e)
174|            self._model.save(model_path)
175|
176|            return PluginResult.ok(
177|                data={
178|                    "model_path": model_path,
179|                    "training_time_s": elapsed,
180|                    "total_timesteps": total_timesteps,
181|                },
182|                message=f"PPO 训练完成: {total_timesteps} steps, {elapse
    |    d}s",
183|            )
184|        except Exception as e:
185|            self.logger.error(f"训练失败: {e}", exc_info=True)
186|            return PluginResult.fail(
187|                message=f"训练失败: {e}",
188|                error_code="TRAIN_ERROR",
189|                error_detail=str(e),
190|            )
191|
192|    def _predict_and_schedule(
193|        self,
194|        tasks: Optional[List[Any]],
195|        servers: Optional[List[Any]],
196|        model_path: Optional[str],
197|    ) -> PluginResult:
198|        if not tasks or servers is None:
199|            return PluginResult.fail(
200|                message="推理模式需要 tasks 与 servers 参数",
201|                error_code="MISSING_PARAM",
202|            )
203|        if not model_path or not os.path.exists(model_path):
204|            return PluginResult.fail(
205|                message=f"模型文件不存在: {model_path}",
206|                error_code="MODEL_NOT_FOUND",
207|            )
208|        try:
209|            from stable_baselines3 import PPO
210|            from src.plugins.datacenter_gym_env import
DataCenterGymEnv
211|
212|            model = PPO.load(model_path)
213|            env = DataCenterGymEnv(
214|                num_servers=self._num_servers,
```

```
                simulation_hours=self._simulation_hours,
                seed=self._seed,
            )
            assignments: List[Dict[str, Any]] = []
            obs, _info = env.reset()
            for task in tasks:
                # 取 env 内部 _current_task = task，然后用 model.predict
                env._current_task = task  # type:
ignore[attr-defined]
                obs = env._get_obs()  # type: ignore[attr-defined]
                action, _state = model.predict(obs,
deterministic=True)
                server_id = int(action)
                if 0 <= server_id < len(servers):
                    server = servers[server_id]
                    if server.assign_task(task):
                        assignments.append(
                            {
                                "task_id": task.task_id,
                                "server_id": server_id,
                                "load": task.load,
                            }
                        )
                        continue
                # 默认分配：负载最低的服务器
                fallback = min(
                    range(len(servers)),
                    key=lambda i: servers[i].current_load /
max(servers[
   |    i].max_load, 1.0),
                )
                if servers[fallback].assign_task(task):
                    assignments.append(
                        {
                            "task_id": task.task_id,
                            "server_id": fallback,
                            "load": task.load,
                            "fallback": True,
                        }
                    )
                else:
                    assignments.append(
                        {"task_id": task.task_id, "server_id":
None, "re
```

```
    |    jected": True}
254|                    )
255|            return PluginResult.ok(
256|                data={
257|                    "assignments": assignments,
258|                    "model_path": model_path,
259|                    "total_tasks": len(assignments),
260|                },
261|                message=f"PPO 推理完成: {len(assignments)} 个任务",
262|            )
263|        except Exception as e:
264|            self.logger.error(f"推理失败: {e}", exc_info=True)
265|            return PluginResult.fail(
266|                message=f"推理失败: {e}",
267|                error_code="PREDICT_ERROR",
268|                error_detail=str(e),
269|            )
270|
271|
272|__all__ = ["RLSchedulerPlugin"]
```

46. src/sdk/__init__.py

```
  1|from .client import DataCenterSimulatorSDK
  2|
  3|__all__ = ['DataCenterSimulatorSDK']
```

47. src/sdk/client.py

```
  1|"""
  2|校园数据中心智能散热与算力调度系统 - SDK客户端 v1.0
  3|
  4|提供简洁的API接口，支持多环境自适应对接。
  5|
  6|支持的接入方式:
  7|1. 本地模式    - 直接调用本地插件（自动检测依赖可用性）
  8|2. 远程模式    - 通过REST API调用远程服务（含连接池+重试）
  9|3. 混合模式    - 本地优先，远程兜底（自动故障转移）
 10|
 11|环境自适应:
 12|- CPU核心数检测 → 自动调整并行度
 13|- GPU/昇腾NPU检测 → 硬件加速策略
 14|- 操作系统识别 → 平台特定优化
 15|- Python版本检测 → 兼容性适配
 16|
 17|示例用法:
```

```
    # 自动检测环境
    sdk = DataCenterSimulatorSDK(auto_detect=True)

    # 本地模式
    sdk = DataCenterSimulatorSDK(mode='local')
    result = sdk.run_simulation(num_servers=10,
algorithms=['greedy', 'g
    enetic'])

    # 远程模式（含重试）
    sdk = DataCenterSimulatorSDK(mode='remote',
api_url="http://localhos
    t:5000")
    result = sdk.run_simulation(num_servers=10)

    # 混合模式（自动故障转移）
    sdk = DataCenterSimulatorSDK(mode='hybrid',
api_url="http://localhos
    t:5000")
"""

import requests
import platform
import psutil
from typing import Dict, Any, List, Optional, Union, Callable
from dataclasses import dataclass, field
from urllib3.util.retry import Retry
from requests.adapters import HTTPAdapter
from src.plugin import PluginManager
from src.plugin.simulation_plugin import SimulationPlugin
from src.utils.logger import setup_logger
import os

logger = setup_logger("sdk")

@dataclass
class EnvironmentInfo:
    """环境信息数据结构"""
    os_name: str = ""
    os_version: str = ""
    python_version: str = ""
    cpu_cores: int = 1
    cpu_logical: int = 1
    total_memory_gb: float = 0
```

```
    available_memory_gb: float = 0
    has_gpu: bool = False
    gpu_count: int = 0
    has_ascend_npu: bool = False
    is_docker: bool = False
    is_kubernetes: bool = False
    platform_arch: str = ""

class EnvironmentDetector:
    """环境探测器 - 自动检测运行时环境"""

    @staticmethod
    def detect() -> EnvironmentInfo:
        info = EnvironmentInfo()

        info.os_name = platform.system()
        info.os_version = platform.version()
        info.python_version = platform.python_version()
        info.platform_arch = platform.machine()

        try:
            info.cpu_cores = psutil.cpu_count(logical=False) or 1
            info.cpu_logical = psutil.cpu_count(logical=True) or 1
        except Exception:
            info.cpu_cores = 1
            info.cpu_logical = 1

        try:
            mem = psutil.virtual_memory()
            info.total_memory_gb = mem.total / (1024 ** 3)
            info.available_memory_gb = mem.available / (1024 ** 3)
        except Exception:
            info.total_memory_gb = 0
            info.available_memory_gb = 0

        info.has_gpu = EnvironmentDetector._detect_gpu()
        info.gpu_count = EnvironmentDetector._count_gpu()
        info.has_ascend_npu = EnvironmentDetector._detect_ascend()
        info.is_docker = EnvironmentDetector._detect_docker()
        info.is_kubernetes = EnvironmentDetector._detect_kubernete
s()

        return info

```

```
 99|    @staticmethod
100|    def _detect_gpu() -> bool:
101|        try:
102|            import torch
103|            return torch.cuda.is_available()
104|        except ImportError:
105|            return False
106|
107|    @staticmethod
108|    def _count_gpu() -> int:
109|        try:
110|            import torch
111|            return torch.cuda.device_count() if
torch.cuda.is_available(
   |    ) else 0
112|        except ImportError:
113|            return 0
114|
115|    @staticmethod
116|    def _detect_ascend() -> bool:
117|        try:
118|            import torch_npu
119|            return torch_npu.npu.is_available()
120|        except ImportError:
121|            pass
122|        if os.path.exists('/usr/local/Ascend'):
123|            return True
124|        return False
125|
126|    @staticmethod
127|    def _detect_docker() -> bool:
128|        return os.path.exists('/.dockerenv') or 'docker' in
open('/proc/
   |    1/cgroup', 'r').read() if os.path.exists('/proc/1/cgroup')
else Fals
   |    e
129|
130|    @staticmethod
131|    def _detect_kubernetes() -> bool:
132|        return bool(os.environ.get('KUBERNETES_SERVICE_HOST'))
133|
134|class RetryableSession:
135|    """可重试的HTTP会话"""
136|
```

```
    def __init__(self, max_retries: int = 3, backoff_factor: float = 0.5,
                 status_forcelist: List[int] = None):
        self.session = requests.Session()

        if status_forcelist is None:
            status_forcelist = [429, 500, 502, 503, 504]

        retry_strategy = Retry(
            total=max_retries,
            backoff_factor=backoff_factor,
            status_forcelist=status_forcelist,
            allowed_methods=["GET", "POST", "PUT"]
        )

        adapter = HTTPAdapter(
            max_retries=retry_strategy,
            pool_connections=10,
            pool_maxsize=20
        )

        self.session.mount("http://", adapter)
        self.session.mount("https://", adapter)
        self.session.headers.update({
            'User-Agent': f'DataCenterSchedulerSDK/2.0 ({platform.system()}; Python/{platform.python_version()})'
        })

    def post(self, url: str, json_data: Dict = None, timeout: int = 120) -> requests.Response:
        return self.session.post(url, json=json_data, timeout=timeout)

    def get(self, url: str, timeout: int = 30) -> requests.Response:
        return self.session.get(url, timeout=timeout)

    def close(self):
        self.session.close()

class DataCenterSimulatorSDK:
```

```
    """
    数据中心仿真SDK v1.0

    支持三种运行模式:
    - local: 纯本地调用
    - remote: 纯远程API调用
    - hybrid: 本地优先，远程兜底
    """

    def __init__(
        self,
        mode: str = 'auto',
        api_url: Optional[str] = None,
        max_retries: int = 3,
        timeout: int = 300,
        auto_detect: bool = True
    ):
        """
        初始化SDK

        Args:
            mode: 运行模式 ('auto', 'local', 'remote', 'hybrid')
            api_url: 远程API地址（remote/hybrid模式需要）
            max_retries: 远程请求最大重试次数
            timeout: 请求超时秒数
            auto_detect: 是否自动检测环境
        """
        self.max_retries = max_retries
        self.timeout = timeout
        self.plugin_manager = None
        self.session: Optional[RetryableSession] = None

        if auto_detect:
            self.env_info = EnvironmentDetector.detect()
            logger.info(f"Environment detected: {self.env_info}")
        else:
            self.env_info = EnvironmentInfo()

        self._resolve_mode(mode, api_url)

        if self._local_mode or self._is_hybrid():
            self.plugin_manager = PluginManager()
            self.plugin_manager.register_plugin(SimulationPlugin)
```

```
            logger.info("SDK initialized in LOCAL/HYBRID mode
with plugi
    n manager")

        if self._remote_mode or self._is_hybrid():
            if not api_url:
                api_url = "http://localhost:5000"
            self.api_url = api_url.rstrip('/')
            self.session = RetryableSession(max_retries=max_retrie
s)
            logger.info(f"SDK initialized with remote endpoint:
{self.ap
    i_url}")

    def _resolve_mode(self, mode: str, api_url: Optional[str]) ->
None:
        if mode == 'auto':
            if api_url:
                self._mode = 'hybrid'
            else:
                self._mode = 'local'
        else:
            self._mode = mode

        self._local_mode = self._mode == 'local'
        self._remote_mode = self._mode == 'remote'

    def _is_hybrid(self) -> bool:
        return self._mode == 'hybrid'

    def _auto_parallel_workers(self) -> int:
        cores = self.env_info.cpu_cores
        if cores <= 2:
            return 1
        elif cores <= 4:
            return 2
        elif cores <= 8:
            return 4
        else:
            return min(cores // 2, 32)

    def get_environment_info(self) -> Dict[str, Any]:
        """获取环境检测信息"""
        return {
```

```
            'os': self.env_info.os_name,
            'os_version': self.env_info.os_version,
            'python_version': self.env_info.python_version,
            'cpu_cores': self.env_info.cpu_cores,
            'cpu_logical': self.env_info.cpu_logical,
            'memory_gb': round(self.env_info.total_memory_gb, 2),
            'has_gpu': self.env_info.has_gpu,
            'gpu_count': self.env_info.gpu_count,
            'has_ascend_npu': self.env_info.has_ascend_npu,
            'is_docker': self.env_info.is_docker,
            'is_kubernetes': self.env_info.is_kubernetes,
            'platform_arch': self.env_info.platform_arch,
            'recommended_workers': self._auto_parallel_workers()
        }

    def run_simulation(
        self,
        num_servers: int = 10,
        simulation_hours: int = 24,
        algorithms: List[str] = None,
        parallel_workers: Optional[int] = None,
        run_mode: str = 'standard',
        visualization: bool = True,
        save_results: bool = True
    ) -> Dict[str, Any]:
        """
        运行仿真

        Args:
            num_servers: 服务器数量 (1-1000)
            simulation_hours: 仿真时长（小时）(1-720)
            algorithms: 算法列表
            parallel_workers: 并行工作器数（None=自动检测）
            run_mode: 运行模式 ('standard', 'distributed', 'large_scale'
    )
            visualization: 是否生成可视化图表
            save_results: 是否保存结果

        Returns:
            仿真结果字典
        """
        if algorithms is None:
            algorithms = ['greedy', 'genetic', 'round_robin']

```

```
        if parallel_workers is None:
            parallel_workers = self._auto_parallel_workers()

        config = {
            'num_servers': num_servers,
            'simulation_hours': simulation_hours,
            'parallel_workers': parallel_workers,
            'run_mode': run_mode,
        }

        if self._is_hybrid():
            return self._run_hybrid(config, algorithms,
visualization, s
    ave_results)
        elif self._local_mode:
            return self._run_local(config, algorithms,
visualization, sa
    ve_results)
        else:
            return self._run_remote(config, algorithms)

    def _run_hybrid(
        self, config: Dict[str, Any], algorithms: List[str],
        visualization: bool, save_results: bool
    ) -> Dict[str, Any]:
        try:
            return self._run_local(config, algorithms,
visualization, sa
    ve_results)
        except Exception as e:
            logger.warning(f"Local execution failed ({e}),
falling back
    to remote")
            if self.session:
                return self._run_remote(config, algorithms)
            return {'success': False, 'error': f'Local failed and
remote
     not available: {e}'}

    def _run_local(
        self, config: Dict[str, Any], algorithms: List[str],
        visualization: bool, save_results: bool
    ) -> Dict[str, Any]:
        try:
```

```
            self.plugin_manager.load_plugin('SimulationPlugin',
config)
            result = self.plugin_manager.execute_plugin(
                'SimulationPlugin',
                algorithms=algorithms,
                visualization=visualization,
                save_results=save_results
            )
            self.plugin_manager.unload_plugin('SimulationPlugin')
            return result
        except Exception as e:
            logger.error(f"Local simulation error: {str(e)}")
            return {'success': False, 'error': str(e)}

    def _run_remote(
        self, config: Dict[str, Any], algorithms: List[str]
    ) -> Dict[str, Any]:
        try:
            url = f"{self.api_url}/api/v1/simulation"
            payload = {
                'num_servers': config.get('num_servers', 10),
                'simulation_hours':
config.get('simulation_hours', 24),
                'parallel_workers':
config.get('parallel_workers', 4),
                'run_mode': config.get('run_mode', 'standard'),
                'algorithms': algorithms
            }

            response = self.session.post(url, json_data=payload,
timeout
    =self.timeout)
            response.raise_for_status()
            return response.json()
        except requests.exceptions.RequestException as e:
            logger.error(f"Remote API error: {str(e)}")
            return {'success': False, 'error': str(e)}

    def get_metrics(self) -> Dict[str, Any]:
        if self._is_hybrid():
            return self._try_local_then_remote('get_metrics')
        return self._get_metrics_local() if self._local_mode else
self._
    get_metrics_remote()
```

```
368|
369|    def _get_metrics_local(self) -> Dict[str, Any]:
370|        from src.utils import load_simulation_data
371|        import pandas as pd
372|
373|        df = load_simulation_data()
374|        if df is None:
375|            return {'success': False, 'message': 'No data found'}
376|
377|        result = {}
378|        for algo in df['algorithm'].unique():
379|            algo_df = df[df['algorithm'] == algo]
380|            result[algo] = {
381|                'avg_energy': float(algo_df['total_energy'].mean())
,
382|                'max_temp': float(algo_df['avg_temperature'].max())
,
383|                'avg_pue': float(algo_df['pue'].mean()),
384|                'total_load': float(algo_df['total_load'].sum())
385|            }
386|        return {'success': True, 'data': result}
387|
388|    def _get_metrics_remote(self) -> Dict[str, Any]:
389|        try:
390|            response = self.session.get(f"{self.api_url}/api/v1/me
trics"
   |    )
391|            response.raise_for_status()
392|            return response.json()
393|        except requests.exceptions.RequestException as e:
394|            return {'success': False, 'error': str(e)}
395|
396|    def get_raw_data(self) -> Dict[str, Any]:
397|        if self._local_mode or self._is_hybrid():
398|            return self._get_raw_data_local()
399|        return self._get_raw_data_remote()
400|
401|    def _get_raw_data_local(self) -> Dict[str, Any]:
402|        from src.utils import load_simulation_data
403|        df = load_simulation_data()
404|        if df is None:
405|            return {'success': False, 'message': 'No data found'}
406|        return {'success': True, 'data': df.to_dict('records')}
407|
```

```
    def _get_raw_data_remote(self) -> Dict[str, Any]:
        try:
            response = self.session.get(f"{self.api_url}/api/v1/da
ta")
            response.raise_for_status()
            return response.json()
        except requests.exceptions.RequestException as e:
            return {'success': False, 'error': str(e)}

    def _try_local_then_remote(self, method: str) -> Dict[str,
Any]:
        try:
            local_result = getattr(self, f'_{method}_local')()
            if local_result.get('success'):
                return local_result
        except Exception:
            pass
        try:
            return getattr(self, f'_{method}_remote')()
        except Exception as e:
            return {'success': False, 'error': str(e)}

    def list_algorithms(self) -> List[Dict[str, str]]:
        return [
            {'name': 'greedy', 'description': '贪心调度算法', 'type': '
    实时调度'},
            {'name': 'genetic', 'description': '遗传优化算法', 'type': '
    全局优化'},
            {'name': 'round_robin', 'description': '轮询调度算法', 'type
    ': '基准算法'},
            # 第三方插件（GitHub Skills 集成）
            {'name': 'nsga2', 'description': 'pymoo NSGA2 多目标调度（能
    耗+温度+deadline）',
             'type': '多目标优化', 'plugin': 'NSGA2SchedulerPlugin',
              'source': 'github.com/anyoptimization/pymoo'},
            {'name': 'moead', 'description': 'pymoo MOEAD 分解多目标调度
    ',
             'type': '多目标优化', 'plugin': 'MOEADSchedulerPlugin',
              'source': 'github.com/anyoptimization/pymoo'},
        ]

    def list_plugins(self) -> Dict[str, Any]:
        if self._local_mode or self._is_hybrid():
            plugins = []
```

```
            for name, plugin_class in
self.plugin_manager._plugin_classe
    s.items():
                is_loaded = name in
self.plugin_manager._plugin_instance
    s
                manifest = plugin_class().get_manifest()
                plugins.append({
                    'name': name, 'version': manifest.version,
                    'description': "
,".join(manifest.supported_operation
    s), 'loaded': is_loaded
                })
            return {'success': True, 'plugins': plugins}
        try:
            response = self.session.get(f"{self.api_url}/api/v1/pl
ugins"
    )
            response.raise_for_status()
            return response.json()
        except Exception as e:
            return {'success': False, 'error': str(e)}

    def load_plugin(self, plugin_name: str, config:
Optional[Dict[str, A
    ny]] = None) -> Dict[str, Any]:
        if self._local_mode or self._is_hybrid():
            success = self.plugin_manager.load_plugin(plugin_name,
 confi
    g or {})
            if success:
                info = self.plugin_manager.get_plugin_info(plugin_
name)
                return {'success': True, 'message': 'Plugin
loaded', 'me
    tadata': info.to_dict() if info else {}}
            return {'success': False, 'message': 'Failed to load
plugin'
    }
        try:
            response = self.session.post(f"{self.api_url}/api/v1/p
lugins
    /{plugin_name}/load", json_data=config or {})
            response.raise_for_status()
```

```
            return response.json()
        except Exception as e:
            return {'success': False, 'error': str(e)}

    def unload_plugin(self, plugin_name: str) -> Dict[str, Any]:
        if self._local_mode or self._is_hybrid():
            self.plugin_manager.unload_plugin(plugin_name)
            return {'success': True, 'message': 'Plugin unloaded'}
        try:
            response = self.session.post(f"{self.api_url}/api/v1/p
lugins
    /{plugin_name}/unload")
            response.raise_for_status()
            return response.json()
        except Exception as e:
            return {'success': False, 'error': str(e)}

    def health_check(self) -> bool:
        if self._local_mode:
            return True
        if self._is_hybrid():
            local_ok = self.plugin_manager is not None
            remote_ok = False
            if self.session:
                try:
                    response = self.session.get(f"{self.api_url}/h
ealth"
    , timeout=5)
                    remote_ok = response.status_code == 200
                except Exception:
                    pass
            return local_ok or remote_ok
        try:
            response = self.session.get(f"{self.api_url}/health",
timeou
    t=5)
            return response.status_code == 200
        except Exception:
            return False

    def get_system_status(self) -> Dict[str, Any]:
        env = self.get_environment_info()
        loaded = len(self.plugin_manager._plugin_instances) if
self.plug
```

```
   |    in_manager else 0
507|        registered = len(self.plugin_manager._plugin_classes) if
self.pl
   |    ugin_manager else 0
508|
509|        return {
510|            'success': True,
511|            'status': {
512|                'mode': self._mode,
513|                'loaded_plugins': loaded,
514|                'registered_plugins': registered,
515|                'status': 'healthy' if self.health_check() else '
degrade
   |    d'
516|            },
517|            'environment': env
518|        }
519|
520|    def close(self):
521|        if self.plugin_manager:
522|            for name in list(self.plugin_manager._plugin_instances.
keys(
   |    )):
523|                self.plugin_manager.unload_plugin(name)
524|        if self.session:
525|            self.session.close()
526|        logger.info("SDK closed")
527|
528|    def __repr__(self) -> str:
529|        return f"DataCenterSimulatorSDK(mode={self._mode},
api_url={self
   |    .api_url if hasattr(self, 'api_url') else None})"
530|
531|    def __enter__(self):
532|        return self
533|
534|    def __exit__(self, exc_type, exc_val, exc_tb):
535|        self.close()
536|        return False
```

48. src/services/__init__.py

```
  1|from .speech_service import (
  2|    SpeechService, SpeechEngine, RecognitionState,
RecognitionResult,
```

```
  3|    SpeechSession, SUPPORTED_LANGUAGES, get_speech_service
  4|)
  5|
  6|__all__ = [
  7|    'SpeechService',
  8|    'SpeechEngine',
  9|    'RecognitionState',
 10|    'RecognitionResult',
 11|    'SpeechSession',
 12|    'SUPPORTED_LANGUAGES',
 13|    'get_speech_service'
 14|]
```

49. src/services/ai_service.py

```
  1|"""调度诊断 AI 服务
  2|
  3|优先调用 DeepSeek 大模型生成诊断建议；未配置 API Key 或调用失败时，
  4|自动退化为本地专家规则，保证离线环境下页面依然可用。
  5|"""
  6|import json
  7|import os
  8|import re
  9|import time
 10|from typing import Any, Dict, List, Optional, Tuple
 11|
 12|import requests
 13|
 14|from src.services.weight_tuner import WEIGHT_KEYS
 15|from src.utils.config import Config
 16|from src.utils.logger import setup_logger
 17|
 18|logger = setup_logger("ai_service")
 19|
 20|DEFAULT_BASE_URL = "https://api.deepseek.com"
 21|DEFAULT_MODEL = "deepseek-flash"
 22|KEY_ENV = "DEEPSEEK_API_KEY"
 23|KEY_FILE = os.path.join(
 24|    os.path.dirname(os.path.abspath(__file__)), "..", "..", "
config", "d
   |    eepseek.key"
 25|)
 26|
 27|ALL_ALGORITHMS = ["greedy", "genetic", "round_robin", "damo_ets", "
nsga2
```

```
    ", "moead"]
28|
29|ALGO_ALIASES = {
30|    "贪心": "greedy",
31|    "greedy": "greedy",
32|    "遗传": "genetic",
33|    "genetic": "genetic",
34|    "轮询": "round_robin",
35|    "round_robin": "round_robin",
36|    "round robin": "round_robin",
37|    "damo": "damo_ets",
38|    "多目标": "damo_ets",
39|    "能效": "damo_ets",
40|    "nsga": "nsga2",
41|    "nsga2": "nsga2",
42|    "nsga-ii": "nsga2",
43|    "nsga ii": "nsga2",
44|    "moead": "moead",
45|    "moea": "moead",
46|    "moea-d": "moead",
47|    "moea d": "moead",
48|}
49|
50|CN_NUMBERS = {"一": 1, "二": 2, "两": 2, "三": 3, "四": 4, "五": 5, "六"
    : 6, "七": 7, "八": 8, "九": 9, "十": 10, "十二": 12}
51|
52|ANALYZE_SYSTEM = """你是数据中心能效与调度专家。根据给定的多算法仿真指标
    输出 JSON：
53|{"summary": "一段中文结论", "best_algorithm": "算法标识",
54| "risks": ["风险点"], "tuning": {"参数名": 建议数值}, "actions": ["可执
    行动作"]}
55|要求：数值只能来自输入数据，不得编造；语言简洁专业；risks 与 actions 各不
超
    过 4 条。
56|tuning 只允许出现这些权重参数名：alpha_deadline、beta_energy、
gamma_thermal
    |
57|delta_heterogeneity epsilon_migration
58|算法名称一律使用中文或规范写法：贪心算法、遗传算法、轮询算法、DAMO-ETS、
NSGA-
    II MOEA-D
59|不要写成 greedy、genetic、round_robin、damo_ets 这类代码标识符。"""
60|
61|PARSE_SYSTEM = """把用户的一句话需求转成仿真参数 JSON：
```

```
62|{"num_servers": 整数, "simulation_hours": 整数, "algorithms": ["greedy",
  |    "genetic","round_robin","damo_ets","nsga2","moead"],
63| "run_mode": "standard" 或 "distributed" 或 "large_scale"}
64|只输出 JSON，不要解释。无法确定的字段可以省略。"""
65|
66|
67|DISPLAY_NAMES = {
68|    "damo_ets": "DAMO-ETS",
69|    "round_robin": "轮询算法",
70|    "greedy": "贪心算法",
71|    "genetic": "遗传算法",
72|    "nsga2": "NSGA-II",
73|    "moead": "MOEA-D",
74|}
75|
76|_NORMALIZED_NAMES = {
77|    re.sub(r"[_\-\s]", "", key): value for key, value in
DISPLAY_NAMES.i
  |    tems()
78|}
79|
80|_ALGO_PATTERN = re.compile(
81|    r"\b(damo[_\-\s]?ets|round[_\-\s]?robin|nsga[_\-\s]?(?:2|ii)|"
82|    r"moea[_\-\s]?d|greedy|genetic)\b",
83|    re.IGNORECASE,
84|)
85|
86|
87|def humanize_algorithms(text: str) -> str:
88|    """Replace code identifiers such as ``round_robin`` with
display nam
  |    es."""
89|    if not isinstance(text, str):
90|        return text
91|
92|    def replace(match: "re.Match[str]") -> str:
93|        key = re.sub(r"[_\-\s]", "", match.group(1).lower())
94|        return _NORMALIZED_NAMES.get(key, match.group(0))
95|
96|    return _ALGO_PATTERN.sub(replace, text)
97|
98|
99|WEIGHT_LABELS = {
100|    "alpha_deadline": "α 截止权重",
```

```
    "beta_energy": "β 能耗权重",
    "gamma_thermal": "γ 温度权重",
    "delta_heterogeneity": "δ 异构权重",
    "epsilon_migration": "ε 迁移权重",
}


def localize(text: Any) -> Any:
    if not isinstance(text, str):
        return text
    result = text
    for key, label in list(DISPLAY_NAMES.items()) +
list(WEIGHT_LABELS.i
    tems()):
        result = re.sub(r"(?<![A-Za-z0-9_])" + re.escape(key) +
r"(?![A-
    Za-z0-9_])", label, result)
    return result


def localize_result(result: Dict[str, Any]) -> Dict[str, Any]:
    if result.get("summary"):
        result["summary"] = localize(result["summary"])
    for field in ("risks", "actions", "reasons"):
        if result.get(field):
            result[field] = [localize(item) for item in
result[field]]
    for field in ("caution", "notice"):
        if result.get(field):
            result[field] = localize(result[field])
    return result


def load_credentials() -> Tuple[str, str, str]:
    key = os.environ.get(KEY_ENV, "").strip()
    model = os.environ.get("DEEPSEEK_MODEL", "").strip()
    if key:
        return key, "env", model
    try:
        with open(KEY_FILE, "r", encoding="utf-8") as handle:
            lines = [line.strip() for line in handle if
line.strip()]
        if lines:
```

```
            return lines[0], "file", (lines[1] if len(lines) > 1
else ""
    )
    except OSError:
        pass
    return "", "none", ""


def _num(value: Any, default: float = 0.0) -> float:
    try:
        return float(value)
    except (TypeError, ValueError):
        return default


def rule_analyze(stats: Dict[str, Dict[str, Any]], weights:
Optional[Dic
    t[str, Any]] = None) -> Dict[str, Any]:
    if not stats:
        return {
           "summary": "暂无可分析的仿真数据，请先运行一次仿真。",
            "best_algorithm": "",
            "risks": [],
            "tuning": {},
            "actions": [],
            "source": "rule",
        }

    by_energy = sorted(stats.items(), key=lambda kv:
_num(kv[1].get("avg
    _energy")))
    best_energy, best_row = by_energy[0]
    hottest = max(stats, key=lambda k:
_num(stats[k].get("avg_temp", sta
    ts[k].get("max_temp"))))
    weakest = min(stats, key=lambda k:
_num(stats[k].get("service_rate")
    , 1.0))
    baseline = _num(stats.get("round_robin",
{}).get("avg_energy")) or _
    num(by_energy[-1][1].get("avg_energy"))
    saving = 0.0
    if baseline:
```

```
169|        saving = (baseline - _num(best_row.get("avg_energy"))) /
baselin
   |    e * 100.0
170|
171|    max_temp = max(_num(row.get("max_temp")) for row in
stats.values())
172|    avg_pue = sum(_num(row.get("avg_pue")) for row in
stats.values()) /
   |    len(stats)
173|    weakest_rate = _num(stats[weakest].get("service_rate"), 1.0)
174|
175|    summary = (
176|        "%s 在当前配置下平均能耗最低（%.0f W，较轮询基线低 %.1f%%）；"
177|        "集群最高温度 %.1f ° C，平均 PUE %.2f。"
178|        % (best_energy, _num(best_row.get("avg_energy")), saving,
max_te
   |    mp, avg_pue)
179|    )
180|    if weakest_rate >= 0.999:
181|        summary += "四个算法的任务服务率均达到 100%%，指标可直接横向比较。
   |    "
182|
183|    risks: List[str] = []
184|    actions: List[str] = []
185|    tuning: Dict[str, Any] = {}
186|
187|    if weakest_rate < 0.98:
188|        dropped = int(_num(stats[weakest].get("dropped_tasks")))
189|        risks.append(
190|            "%s 有 %d 个任务未成功分配（服务率 %.1f%%），其能耗与温度偏低
主
   |    要是因为接的任务更少，"
191|            "与其他算法不在同一口径上。" % (weakest, dropped, weakest_rat
   |    e * 100)
192|        )
193|        actions.append("对外对比能耗前先公布服务率，或为 %s 增加可用服务
   |    器数量后重跑。" % weakest)
194|
195|    if max_temp > 70:
196|        risks.append("最高温度达到 %.1f ° C，已接近 85 ° C 的强制散热阈值。
"
   |     % max_temp)
197|        gamma = _num((weights or {}).get("gamma_thermal"), 0.20)
```

```
        tuning["gamma_thermal"] = round(min(0.40, gamma + 0.05),
2)
        actions.append("提高 DAMO-ETS 的温度权重 gamma_thermal，让调度优
    先避开高温机架。")

    if avg_pue > 1.4:
        risks.append("平均 PUE 为 %.2f，高于 1.4 的常规水平，冷却开销偏大。
    " % avg_pue)
        actions.append("检查冷却目标温度与风扇曲线，或降低单机负载密度。"
)

    deadline = max(int(_num(row.get("deadline_violations"))) for
row in
    stats.values())
    if deadline > 0:
        risks.append("存在 %d 次 deadline 违约，说明当前分配策略对时延敏
    感任务保护不足。" % deadline)
        alpha = _num((weights or {}).get("alpha_deadline"), 0.35)
        tuning["alpha_deadline"] = round(min(0.60, alpha + 0.05),
2)

    if not risks:
        risks.append("本次仿真未发现明显风险：服务率、温度与 PUE 均在合理
    区间。")

    if not tuning:
        tuning = {
            "alpha_deadline": _num((weights or
{}).get("alpha_deadline")
    , 0.35),
            "gamma_thermal": _num((weights or
{}).get("gamma_thermal"),
    0.20),
        }
        actions.append("当前权重已较均衡，可在自动调参实验中进一步搜索更
    优组合。")

    actions.append("把本轮结果与上一轮对比，确认调参带来的变化方向是否一
    致。")

    return {
        "summary": summary,
        "best_algorithm": best_energy,
        "risks": risks[:4],
```

```
        "tuning": tuning,
        "actions": actions[:4],
        "source": "rule",
    }


def _to_int(token: str) -> int:
    token = token.strip()
    if token.isdigit():
        return int(token)
    return CN_NUMBERS.get(token, 0)


def rule_parse(text: str, defaults: Optional[Dict[str, Any]] =
None) ->
    Dict[str, Any]:
    defaults = defaults or {}
    params: Dict[str, Any] = {}
    notes: List[str] = []
    number = r"(\d+|[一二两三四五六七八九十]+)"
    servers = re.search(number + r"\s*(?:台|个|节点|服务器)", text)
    hours = re.search(number + r"\s*(?:个?小时|h\b)", text)
    days = re.search(number + r"\s*(?:天|日)", text)
    if servers:
        params["num_servers"] = _to_int(servers.group(1))
    elif defaults.get("num_servers"):
        params["num_servers"] = int(defaults["num_servers"])
    if hours:
        params["simulation_hours"] = _to_int(hours.group(1))
    elif days:
        params["simulation_hours"] = _to_int(days.group(1)) * 24
        notes.append("按 1 天 = 24 小时换算得到仿真时长。")
    elif defaults.get("simulation_hours"):
        params["simulation_hours"] =
int(defaults["simulation_hours"])

    picked = []
    lowered = text.lower()
    for alias, algo in ALGO_ALIASES.items():
        if alias in lowered and algo not in picked:
            picked.append(algo)
    if picked:
        params["algorithms"] = picked
    elif defaults.get("algorithms"):
```

```
        params["algorithms"] = list(defaults["algorithms"])
        notes.append("未识别到具体算法，沿用当前运行的算法列表。")
    else:
        params["algorithms"] = list(ALL_ALGORITHMS)
        notes.append("未识别到具体算法，默认对比全部算法。")

    if any(word in text for word in ("大规模", "压测", "1000", "500")):
        params["run_mode"] = "large_scale"
    elif "分布式" in text:
        params["run_mode"] = "distributed"
    elif defaults.get("run_mode"):
        params["run_mode"] = defaults["run_mode"]

    if any(word in text for word in ("压住温度", "降温", "散热", "过热")
    ):
        if "damo_ets" not in params["algorithms"]:
            params["algorithms"].append("damo_ets")
        notes.append("检测到温度控制意图，已加入 DAMO-ETS 并在建议中提高
    温度权重。")
    if "省电" in text or "能耗" in text or "节能" in text:
        notes.append("检测到节能意图，建议用能耗列做主指标。")

    if not params.get("num_servers"):
        if defaults.get("num_servers"):
            params["num_servers"] = int(defaults["num_servers"])
        else:
            params["num_servers"] = 10
            notes.append("未识别到服务器数量，将使用默认值 10。")

    if params.get("num_servers", 0) > 1000:
        params["num_servers"] = 1000
        notes.append("服务器数量超过系统上限，已按 1000 台处理。")
    if params.get("simulation_hours", 0) > 720:
        original = params["simulation_hours"]
        params["simulation_hours"] = 720
        notes.append("仿真时长 %d 小时超过系统上限 720 小时，已按上限处理
     " % original)

    notes = [localize(item) for item in notes]
    return {"params": params, "notes": notes, "source": "rule"}


def rule_explain_tuning(sweep: Dict[str, Any]) -> Dict[str, Any]:
    rows = sweep.get("rows", [])
```

```
    best = sweep.get("best", {})
    baseline = sweep.get("baseline", {})
    if not rows or not best:
        return {"summary": "暂无可解释的寻优结果。", "reasons": [], "caut
    ion": ""}

    best_metrics = best["metrics"]
    base_metrics = baseline["metrics"]
    energy_delta = best_metrics["avg_energy"] -
base_metrics["avg_energy
    "]
    temp_delta = best_metrics["max_temp"] -
base_metrics["max_temp"]
    energy_pct = energy_delta / base_metrics["avg_energy"] * 100
if base
    _metrics["avg_energy"] else 0.0

    summary = (
        "%d 组方案中「%s」综合最优：平均能耗 %.0f W（较默认 %+.1f%%），最
高温
    度 %.2f ° C（较默认 %+.2f ° C），"
        "服务率 %.1f%%。规模为 %d 台 / %d 小时，固定随机种子 %d。"
         % (sweep.get("candidates", len(rows)), best["name"],
best_metric
    s["avg_energy"], energy_pct,
            best_metrics["max_temp"], temp_delta,
best_metrics["service_r
    ate"] * 100,
            sweep["scale"]["num_servers"],
sweep["scale"]["simulation_hou
    rs"], sweep["scale"]["seed"])
    )

    reasons: List[str] = []
    base_weights = baseline.get("weights", {})
    changes = sorted(
        ((key, float(best.get("weights", {}).get(key, 0)) -
float(base_w
    eights.get(key, 0))) for key in WEIGHT_KEYS),
        key=lambda item: abs(item[1]),
        reverse=True,
    )
    if changes:
        key, delta = changes[0]
```

```
337|        direction = "提高" if delta > 0 else "降低"
338|        reasons.append(
339|            "该方案把 %s 从 %.2f %s到 %.2f，是相对默认权重最明显的调整；对
   |    应最高温度变化 %+.2f ° C、能耗变化 %+.1f%%。"
340|            % (key, float(base_weights.get(key, 0)), direction,
float(be
   |    st.get("weights", {}).get(key, 0)), temp_delta, energy_pct)
341|        )
342|
343|    thermal_rows = sorted(rows, key=lambda row:
float(row.get("weights",
   |     {}).get("gamma_thermal", 0)), reverse=True)
344|    hottest_weight = thermal_rows[0] if thermal_rows else None
345|    if hottest_weight and hottest_weight["name"] != best["name"]:
346|        reasons.append(
347|            "对照发现「%s」把 gamma_thermal 加到 %.2f，但最高温度 %.2f ° C
   |    并不比最优方案 %.2f ° C 更低，"
348|            "说明单纯加大温度权重会让调度过度保守、任务积压，并不能换取更
   |    低的温度。"
349|            % (hottest_weight["name"],
float(hottest_weight.get("weights
   |    ", {}).get("gamma_thermal", 0)),
350|                hottest_weight["metrics"]["max_temp"],
best_metrics["max_
   |    temp"])
351|        )
352|
353|    spread = max(row["metrics"]["avg_energy"] for row in rows) -
min(row
   |    ["metrics"]["avg_energy"] for row in rows)
354|    reasons.append(
355|        "全部候选方案的能耗极差为 %.0f W，说明在当前规模下权重对能耗的影
   |    响幅度有限，主要收益来自温度侧。"
356|        % spread
357|    )
358|
359|    caution = (
360|        "本结论基于 %d 台 / %d 小时 / seed=%d 的单场景搜索，换规模或换种
   |    子后最优组合可能变化，"
361|        "正式结论建议至少在两种规模上复跑确认。"
362|        % (sweep["scale"]["num_servers"],
sweep["scale"]["simulation_hou
```

```
    |    rs"], sweep["scale"]["seed"])
363|    )
364|    return {"summary": summary, "reasons": reasons[:3], "
caution": cauti
    |    on, "source": "rule"}
365|
366|
367|TUNE_SYSTEM = """你是数据中心能效调度专家。用户给出 DAMO-ETS 权重寻优实验
    |    的结果表，
368|请输出 JSON：{"summary": "一句总结", "reasons": ["机理分析，最多3条"], "
ca
    |    ution": "适用边界"}。
369|要求：只能引用给定数据，不要编造数值；要指出最优方案与默认权重的差异及其
代
    |    价。
370|算法名称用中文或规范写法（贪心算法、遗传算法、轮询算法、DAMO-ETS、NSGA-II、
MOE
    |    A-D
371|权重参数用中文名（α 截止权重、β 能耗权重、γ 温度权重、δ 异构权重、ε
迁移权重）
    |     """
372|
373|
374|class AIService:
375|    def __init__(self) -> None:
376|        self.api_key, self.key_source, model_override =
load_credentials
    |    ()
377|        self.base_url = os.environ.get("DEEPSEEK_BASE_URL",
DEFAULT_BASE
    |    _URL).rstrip("/")
378|        self.model = model_override or
os.environ.get("DEEPSEEK_MODEL",
    |    "").strip() or DEFAULT_MODEL
379|        self.timeout = float(os.environ.get("DEEPSEEK_TIMEOUT", "
90"))
380|        self.max_tokens = int(os.environ.get("DEEPSEEK_MAX_TOKENS",
 "409
    |    6"))
381|        effort = os.environ.get("DEEPSEEK_REASONING_EFFORT", "
none").str
    |    ip().lower()
382|        self.reasoning_effort = "" if effort in ("", "auto", "
default")
```

```
    |    else effort
383|        self._cache: Dict[str, Dict[str, Any]] = {}
384|        self.last_error = ""
385|        self.last_ok_at = ""
386|
387|    @property
388|    def available(self) -> bool:
389|        return bool(self.api_key)
390|
391|    def reload_key(self) -> None:
392|        self.api_key, self.key_source, model_override =
load_credentials
    |    ()
393|        self.model = model_override or
os.environ.get("DEEPSEEK_MODEL",
    |    "").strip() or DEFAULT_MODEL
394|
395|    def status(self) -> Dict[str, Any]:
396|        reason = ""
397|        if not self.available:
398|            reason = "未配置 DEEPSEEK_API_KEY，当前使用本地专家规则"
399|        elif self.last_error:
400|            reason = "大模型最近一次调用失败，已回退本地规则"
401|        return {
402|            "available": self.available,
403|            "model": self.model,
404|            "key_source": self.key_source,
405|            "last_error": self.last_error,
406|            "last_ok_at": self.last_ok_at,
407|            "reason": reason,
408|        }
409|
410|    def _post(self, payload: Dict[str, Any], use_env_proxy: bool)
-> Dic
    |    t[str, Any]:
411|        session = requests.Session()
412|        if not use_env_proxy:
413|            session.trust_env = False
414|        response = session.post(
415|            self.base_url + "/chat/completions",
416|            headers={
417|                "Authorization": "Bearer " + self.api_key,
418|                "Content-Type": "application/json",
419|            },
```

```
420|            json=payload,
421|            timeout=self.timeout,
422|        )
423|        if response.status_code == 400 and "reasoning_effort" in
respons
   |    e.text:
424|            logger.warning("当前接口不支持 reasoning_effort，已自动去掉
   |    该参数重试")
425|            self.reasoning_effort = ""
426|            retry_payload = dict(payload)
427|            retry_payload.pop("reasoning_effort", None)
428|            return self._post(retry_payload, use_env_proxy)
429|        response.raise_for_status()
430|        return response.json()
431|
432|    @staticmethod
433|    def short_error(exc: Exception) -> str:
434|        text = str(exc)
435|        lowered = text.lower()
436|        if "token 预算" in text:
437|            return text
438|        if "proxy" in lowered:
439|            return "环境代理不可用，直连也失败，请检查网络或代理设置"
440|        if "timed out" in lowered or "timeout" in lowered:
441|            return "连接 DeepSeek 超时，请检查网络"
442|        if "401" in text or "unauthorized" in lowered or "invalid
api ke
   |    y" in lowered:
443|            return "API Key 无效或已过期"
444|        if "402" in text or "insufficient balance" in lowered:
445|            return "DeepSeek 账户余额不足"
446|        if "403" in text:
447|            return "API Key 无权访问该模型"
448|        return text[:120]
449|
450|    def chat(self, system: str, user: str, json_mode: bool =
False) -> s
   |    tr:
451|        return self.chat_messages(system, [{"role": "user", "
content": u
   |    ser}], json_mode)
452|
453|    def chat_messages(self, system: str, messages: List[Dict[str,
str]],
```

```
                      json_mode: bool = False) -> str:
        payload: Dict[str, Any] = {
            "model": self.model,
            "messages": [{"role": "system", "content": system}] +
list(m
   |    essages),
            "temperature": 0.3,
            "max_tokens": self.max_tokens,
            "stream": False,
        }
        if json_mode:
            payload["response_format"] = {"type": "json_object"}
        if self.reasoning_effort:
            payload["reasoning_effort"] = self.reasoning_effort

        try:
            data = self._post(payload, True)
        except (requests.exceptions.ConnectionError,
requests.exceptions
   |    .Timeout) as exc:
            logger.warning("经环境代理访问 DeepSeek 失败, 尝试直连: %s"
   |    % exc)
            data = self._post(payload, False)

        choice = data["choices"][0]
        content = choice["message"].get("content") or ""
        if not content.strip():
            if choice.get("finish_reason") == "length":
                raise RuntimeError(
                    "模型思考占满了 token 预算, 未能给出结论; 可设置 DEE
   |    PSEEK_REASONING_EFFORT=none 关闭思考, 或提高 DEEPSEEK_MAX_TOKENS (当
   |    前 %d)" % self.max_tokens
                )
            raise RuntimeError("模型返回了空内容")
        return content

    @staticmethod
    def _extract_json(text: str) -> Dict[str, Any]:
        text = text.strip()
        start = text.find("{")
        end = text.rfind("}")
        if start >= 0 and end > start:
            text = text[start:end + 1]
        return json.loads(text)
```

```
491|
492|    def analyze(self, stats: Dict[str, Dict[str, Any]], weights:
Optiona
   |    l[Dict[str, Any]] = None) -> Dict[str, Any]:
493|        fallback = rule_analyze(stats, weights)
494|        if not self.available:
495|            return fallback
496|
497|        cache_key = json.dumps(stats, sort_keys=True)
498|        cached = self._cache.get(cache_key)
499|        if cached:
500|            return cached
501|
502|        payload = {
503|           "当前指标": stats,
504|           "DAMO-ETS 当前权重": weights or {},
505|           "对照基线": "round_robin（轮询）",
506|        }
507|        try:
508|            content = self.chat(ANALYZE_SYSTEM,
json.dumps(payload, ensu
   |    re_ascii=False), json_mode=True)
509|            parsed = self._extract_json(content)
510|            result = {
511|                "summary": str(parsed.get("summary") or
fallback["summar
   |    y"]),
512|                "best_algorithm":
str(parsed.get("best_algorithm") or fa
   |    llback["best_algorithm"]),
513|                "risks": list(parsed.get("risks") or
fallback["risks"])[
   |    :4],
514|                "tuning": dict(parsed.get("tuning") or
fallback["tuning"
   |    ]),
515|                "actions": list(parsed.get("actions") or
fallback["actio
   |    ns"])[:4],
516|                "source": "llm",
517|                "model": self.model,
518|            }
519|            self._cache[cache_key] = result
520|            self.last_error = ""
```

```
            self.last_ok_at = time.strftime("%Y-%m-%d %H:%M:%S")
            return localize_result(result)
        except Exception as exc:
            logger.error("DeepSeek 诊断失败，回退本地规则: %s" % exc)
            self.last_error = self.short_error(exc)
            fallback["source"] = "rule"
            fallback["notice"] = "大模型调用失败，已回退本地专家规则"
            return localize_result(fallback)

    def explain_tuning(self, sweep: Dict[str, Any]) -> Dict[str, Any]:
        fallback = rule_explain_tuning(sweep)
        if not self.available:
            return localize_result(fallback)
        try:
            payload = {
                "择优规则": sweep.get("rule"),
                "实验规模": sweep.get("scale"),
                "结果表": [
                    {"方案": row["name"], "权重": row["weights"], "指标": row["metrics"]}
                    for row in sweep.get("rows", [])
                ],
                "最优方案": sweep.get("best", {}).get("name"),
                "默认方案": sweep.get("baseline", {}).get("name"),
            }
            content = self.chat(TUNE_SYSTEM, json.dumps(payload, ensure_ascii=False), json_mode=True)
            parsed = self._extract_json(content)
            return localize_result({
                "summary": str(parsed.get("summary") or fallback["summary"]),
                "reasons": list(parsed.get("reasons") or fallback["reasons"])[:3],
                "caution": str(parsed.get("caution") or fallback["caution"]),
                "source": "llm",
                "model": self.model,
            })
        except Exception as exc:
```

```
555|            logger.error("DeepSeek 寻优解释失败，回退本地规则: %s" % exc)
556|             self.last_error = self.short_error(exc)
557|            fallback["notice"] = "大模型调用失败，已回退本地规则解释"
558|             return localize_result(fallback)
559|
560|    def parse_intent(self, text: str, defaults:
Optional[Dict[str, Any]]
   |     = None) -> Dict[str, Any]:
561|        fallback = rule_parse(text, defaults)
562|        if not self.available:
563|            return fallback
564|        try:
565|            content = self.chat(PARSE_SYSTEM, text,
json_mode=True)
566|            parsed = self._extract_json(content)
567|            params = dict(fallback["params"])
568|            for key in ("num_servers", "simulation_hours", "
run_mode"):
569|                if parsed.get(key):
570|                    params[key] = parsed[key]
571|            if parsed.get("algorithms"):
572|                valid = [a for a in parsed["algorithms"] if a in
ALL_ALG
   |    ORITHMS]
573|                if valid:
574|                    params["algorithms"] = valid
575|            notes = [localize(item) for item in fallback["notes"]]
576|            return {"params": params, "notes": notes, "source": "
llm"}
577|        except Exception as exc:
578|            logger.error("DeepSeek 参数解析失败，回退本地规则: %s" % exc)
579|             self.last_error = self.short_error(exc)
580|            fallback["notice"] = "大模型调用失败，已回退本地规则解析"
581|             return fallback
582|
583|
584|_service: Optional[AIService] = None
585|
586|
587|def _fmt(value: Any, digits: int = 3) -> str:
588|    try:
589|        return f"{float(value):.{digits}f}"
590|    except (TypeError, ValueError):
591|        return "--"
```

```
592|
593|
594|def _pct_value(value: Any) -> str:
595|    try:
596|        return f"{float(value) * 100:.2f}"
597|    except (TypeError, ValueError):
598|        return "--"
599|
600|
601|def _snapshot_reply(snapshot: Dict[str, Any]) -> str:
602|    names = " ".join(DISPLAY_NAMES.get(item, item) for item in
snapshot.
   |    get("algorithms") or [])
603|    return (
604|        "当前回答基于最新一次运行：%s，%s 台服务器 × %s 小时，参与算法
%s。
   |    数据生成时间 %s。"
605|        "你重新点「开始运行」后，我会自动读取新结果。"
606|        % (
607|            snapshot.get("site") or "默认站点",
608|            snapshot.get("num_servers") or "未知",
609|            snapshot.get("simulation_hours") or "未知",
610|            names or "未知",
611|            snapshot.get("generated_at") or "未知",
612|        )
613|    )
614|
615|
616|def _best_hours_reply(insights: Dict[str, Any]) -> Optional[str]:
617|    safe_items = insights.get("safe_best_hours") or []
618|    overall_items = insights.get("overall_best_hours") or []
619|    items = safe_items or overall_items
620|    if not items:
621|        return None
622|    top = items[0]
623|    lines = [
624|        "按单位服务负载能耗，当前配置下性价比最高的时段是第 %d 小时：%s，
%s
   |    W/负载，峰值温度 %s℃，平均 PUE %s，服务率 %s%%。"
625|        % (
626|            int(top.get("hour") or 0) + 1,
627|            DISPLAY_NAMES.get(top.get("algorithm"),
top.get("algorithm")
   |    or "未知算法"),
```

```
            _fmt(top.get("energy_per_load")),
            _fmt(top.get("peak_temp"), 1),
            _fmt(top.get("pue")),
            _pct_value(top.get("service_rate")),
        )
    ]
    for item in items[1:3]:
        lines.append(
           "- 第 %d 小时：%s，%s W/负载，峰值温度 %s℃"
            % (
                int(item.get("hour") or 0) + 1,
                DISPLAY_NAMES.get(item.get("algorithm"),
item.get("algor
   |    ithm") or "未知算法"),
                _fmt(item.get("energy_per_load")),
                _fmt(item.get("peak_temp"), 1),
            )
        )
    if safe_items:
        lines.append("以上时段已排除峰值温度达到 70℃ 或出现过热服务器的
   |    时段。")
    if safe_items and overall_items:
        raw = overall_items[0]
        raw_key = (raw.get("hour"), raw.get("algorithm"))
        safe_key = (top.get("hour"), top.get("algorithm"))
        if raw_key != safe_key:
            lines.append(
               "纯能耗最低的是第 %d 小时（%s，%s W/负载），但峰值温度
%s℃，
   |    不建议作为常规配置。"
                % (
                    int(raw.get("hour") or 0) + 1,
                    DISPLAY_NAMES.get(raw.get("algorithm"),
raw.get("alg
   |    orithm") or "未知算法"),
                    _fmt(raw.get("energy_per_load")),
                    _fmt(raw.get("peak_temp"), 1),
                )
            )
    lines.append("这是当前一次仿真的观测结果；换城市、服务器数量或时长后
需
   |    要重新运行。")
    return "\n".join(lines)
```

```
663|
664|def _temperature_reply(insights: Dict[str, Any], threshold:
Optional[flo
   |    at] = None,
665|                         above: bool = False) -> Optional[str]:
666|    if threshold is not None:
667|        key = "temperature_thresholds_above" if above else "
temperature_
   |    thresholds"
668|        items = insights.get(key) or []
669|        if not items:
670|            return None
671|        record = min(items, key=lambda item:
abs(float(item.get("thresho
   |    ld_c") or 0) - threshold))
672|        level = int(record.get("threshold_c") or 0)
673|        best = record.get("best_algorithm")
674|        data = (record.get("by_algorithm") or {}).get(best) if
best else
   |     None
675|        if not best or not data:
676|            return "当前数据在这个温区没有足够样本，暂时给不出可靠建议。"
677|        hours = record.get("hours_above" if above else "
hours_below") or
   |     0
678|        direction = "高于或等于" if above else "低于"
679|        text = (
680|            "当峰值温度%s %d℃ 时，当前数据覆盖 %d 个运行小时；表现最好的
是
   |     %s "
681|            "单位服务负载能耗 %s W/负载，平均 PUE %s，服务率 %s%%，平均风
扇
   |    转速 %s。"
682|            % (
683|                direction, level, hours,
684|                DISPLAY_NAMES.get(best, best),
685|                _fmt(data.get("avg_energy_per_load")),
686|                _fmt(data.get("avg_pue")),
687|                _pct_value(data.get("avg_service_rate")),
688|                _fmt(data.get("avg_fan_speed"), 2),
689|            )
690|        )
691|        if not record.get("best_reliable"):
692|            text += " 该温区服务率偏低，结论只作参考。"
```

```
693|         text += (
694|             "\n建议：这个温区优先用 %s；低温时可适当整合负载、减少空转服
务
   |     器，"
695|             "高温时保留散热余量。当前数据没有验证改变风扇曲线后的效果，调
   |     整前建议重新仿真。"
696|             % DISPLAY_NAMES.get(best, best)
697|         )
698|         return text
699|
700|     bands = insights.get("temperature_bands") or []
701|     lines = ["按当前数据，不同峰值温度区间的配置建议："]
702|     for band in bands:
703|         best = band.get("best_algorithm")
704|         data = (band.get("by_algorithm") or {}).get(best) if best
else N
   |     one
705|         if not best or not data:
706|             continue
707|         line = (
708|             "- %s（%d 小时）：优先 %s，单位服务负载能耗 %s W/负载，PUE %s
，服
   |     务率 %s%%，风扇转速 %s。"
709|             % (
710|                 band.get("label") or "未知温区",
711|                 int(band.get("hours") or 0),
712|                 DISPLAY_NAMES.get(best, best),
713|                 _fmt(data.get("avg_energy_per_load")),
714|                 _fmt(data.get("avg_pue")),
715|                 _pct_value(data.get("avg_service_rate")),
716|                 _fmt(data.get("avg_fan_speed"), 2),
717|             )
718|         )
719|         if not band.get("best_reliable"):
720|             line += " 服务率偏低，仅作参考。"
721|         lines.append(line)
722|     if len(lines) == 1:
723|         return None
724|     lines.append("低温区间可适当整合负载、减少空转服务器；高温区间要保留
散
   |     热余量。以上是观测建议，改变配置前建议重新仿真验证。")
725|     return "\n".join(lines)
726|
727|
```

```
728|def _config_advice_reply(text: str, context: Dict[str, Any]) ->
str:
729|    insights = context.get("insights") or {}
730|    snapshot = insights.get("snapshot") or
context.get("data_snapshot")
   |    or {}
731|    config = context.get("config") or {}
732|    relationships = insights.get("relationships") or {}
733|    best = insights.get("overall_best")
734|    lines = ["当前数据没有实际运行过这个配置，我只能给出基于现有观测的方
   |    向性判断。"]
735|    if snapshot:
736|        lines.append(
737|            "当前快照：%s，%s 台服务器 × %s 小时。"
738|            % (snapshot.get("site") or "默认站点", snapshot.get("num_ser
   |    vers") or "未知",
739|               snapshot.get("simulation_hours") or "未知")
740|        )
741|    if best:
742|        lines.append("当前表现最好的算法是 %s。" % DISPLAY_NAMES.get(best
   |    , best))
743|    pairs = []
744|    if relationships.get("load_vs_peak_temp") is not None:
745|        pairs.append("负载与峰值温度相关系数 %s" % _fmt(relationships["l
   |    oad_vs_peak_temp"], 3))
746|    if relationships.get("ambient_vs_pue") is not None:
747|        pairs.append("环境温度与 PUE 相关系数 %s" % _fmt(relationships["
   |    ambient_vs_pue"], 3))
748|    if relationships.get("fan_vs_cooling") is not None:
749|        pairs.append("风扇转速与冷却功率相关系数 %s" % _fmt(relationship
   |    s["fan_vs_cooling"], 3))
750|    if pairs:
751|        lines.append("当前观测：" + "，".join(pairs) + "。这些只是相关性
，不
   |    是因果结论。")
752|
753|    if any(word in text for word in ("服务器", "台", "节点", "冗余", "关
   |    掉")):
754|        lines.append("调整服务器数量会同时影响整合程度、空闲功耗和服务率
:
   |    减少服务器可能省电，但当前服务率本就不高，继续减少大概率增加未分配任
务
   |    ；增加服务器能缓解容量瓶颈，但空闲功耗会抬高单位服务负载能耗。")
755|    elif any(word in text for word in ("风扇", "散热", "冷却")):
```

```
        lines.append("降低风扇转速或冷却功率会减少冷却能耗，但散热余量变小，峰值温度和过热小时数可能上升；高温时段的风险最大。")
    elif any(word in text for word in ("算法", "nsga", "moea", "damo", "贪心", "遗传", "轮询")):
        lines.append("更换算法会改变任务分配和热点分布，方向取决于当前瓶颈：容量不足时先看服务率，温度接近上限时先看峰值温度和过热小时数。")

    server_range = config.get("num_server_range") or []
    fan_range = config.get("fan_speed_range") or []
    if len(server_range) == 2 and server_range[0] is not None and server_range[1] is not None:
        lines.append(
            "当前允许的服务器数量范围是 %s–%s 台%s。"
            % (
                server_range[0], server_range[1],
                "，风扇转速范围是 %s–%s" % (fan_range[0], fan_range[1]) if len(fan_range) == 2 else "",
            )
        )
    lines.append(
        "要得到具体数值，需要按新参数重新运行仿真。网页表单可直接改服务器数量、时长、运行模式和算法；"
        "风扇曲线、冷却效率、算法权重需要修改配置文件（config 目录下的 yaml 文件）后重新运行。"
    )
    return "\n".join(lines)


def rule_chat(text: str, context: Dict[str, Any]) -> str:
    """Local fallback when the model is unavailable.

    It never invents numbers: every figure comes from the supplied context.
    """
    stats = context.get("stats") or {}
    ranking = context.get("ranking") or {}
    source = context.get("data_source") or {}
    insights = context.get("insights") or {}
    snapshot = insights.get("snapshot") or context.get("data_snapshot")
```

```
    or {}
    best = ranking.get("best")

    if any(word in text for word in ("数据来源", "哪来的", "出处", "真实吗", "可靠")):
        return ("气温来自 Open-Meteo/ERA5 的 345 个城市逐日数据，任务流来自阿里云 2025 GPU 集群轨迹（第 19-25 天窗口），"
                "碳因子来自生态环境部 2021 年公告。每一项都能在 /api/v1/data-source 里查到发布方、"
                "URL 与哈希。")

    if snapshot and any(word in text for word in ("更新", "最新", "这次运行", "本次运行", "刚才跑", "数据变")):
        return _snapshot_reply(snapshot)

    if any(word in text for word in ("怎么用", "如何", "操作", "运行", "开始")):
        return ("在「仿真控制」里先选城市（可输入一个字搜索），需要的话再选学校，然后点「开始运行」。"
                "结果会出现在下面的图表、指标表和受控排名里。")

    temperature_words = any(word in text for word in ("温度", "散热", "风扇", "配置", "温区"))
    temperature_match = re.search(r"(\d+(?:\.\d+)?)\s*(?:度|℃|° c)", text, re.IGNORECASE)
    if temperature_words and temperature_match:
        above = any(word in text for word in ("高于", "超过", "以上"))
        reply = _temperature_reply(insights, float(temperature_match.group(1)), above=above)
        if reply:
            return reply

    if temperature_words and any(word in text for word in ("配置", "建议", "低于", "高于", "超过", "温区", "怎么", "如何")):
        reply = _temperature_reply(insights)
        if reply:
            return reply

    time_words = ("什么时候", "哪个时段", "最佳时间", "时段", "几点", "哪一小时", "性价比", "最划算")
    if any(word in text for word in time_words):
```

```
        reply = _best_hours_reply(insights)
        if reply:
            return reply

    config_words = ("如果", "改成", "增加", "减少", "调到", "风扇", "冷却", "冗余", "配置", "关掉", "换成", "试试")
    if any(word in text for word in config_words):
        return _config_advice_reply(text, context)

    if best and best in stats:
        metrics = stats[best]
        if any(word in text for word in ("温度", "热", "峰值")):
            return ("当前 %s 的峰值温度是 %.1f℃，95 分位 %.1f℃，平均 PUE %.3f " % (
                DISPLAY_NAMES.get(best, best),
                float(metrics.get("max_temp") or 0),
                float(metrics.get("p95_temp") or 0),
                float(metrics.get("avg_pue") or 0)))
        if any(word in text for word in ("pue", "能效")):
            return ("当前 %s 的平均 PUE 是 %.3f，服务率 %.2f%%。" % (
                DISPLAY_NAMES.get(best, best),
                float(metrics.get("avg_pue") or 0), float(metrics.get("service_rate") or 0) * 100))
        if any(word in text for word in ("服务率", "丢", "违约", "deadline")):
            return ("当前 %s 的任务服务率 %.2f%%，累计 deadline 违约 %d 次。" % (
                DISPLAY_NAMES.get(best, best),
                float(metrics.get("service_rate") or 0) * 100,
                int(metrics.get("deadline_violations") or 0)))
        return ("当前站点 %s（%s），%s 排名第一：单位服务负载能耗 %.4f W/负载，"
                "服务率 %.2f%%，峰值温度 %.1f℃，平均 PUE %.3f。" % (
                    source.get("site_id") or "默认",
                    source.get("emission_region") or "未知区域",
                      DISPLAY_NAMES.get(best, best),
                      float(metrics.get("energy_per_served_load") or 0),
                      float(metrics.get("service_rate") or 0) * 100,
```

```
844|                    float(metrics.get("max_temp") or 0),
845|                    float(metrics.get("avg_pue") or 0)))
846|
847|    return ("当前还没有可用的仿真结果。先点「开始运行」跑一次，我就能基于结
   |    果回答能耗、温度、"
848|            "PUE、服务率和排名问题。")
849|
850|
851|def get_ai_service() -> AIService:
852|    global _service
853|    if _service is None:
854|        _service = AIService()
855|    return _service
```

50. src/services/plan_optimizer.py

```
  1|"""AI 目标驱动的全局方案寻优。
  2|
  3|在后台串行评估「服务器数量 × 算法 × DAMO 权重」组合，先按用户约束过滤，
  4|再输出 Pareto 候选、推荐方案和取舍说明。所有候选都在内存中完成，不覆盖
  5|当前页面数据；只有用户点击「应用并运行」时才正式写入运行记录。
  6|"""
  7|
  8|from __future__ import annotations
  9|
 10|import math
 11|import threading
 12|import time
 13|import uuid
 14|from concurrent.futures import ThreadPoolExecutor
 15|from datetime import datetime
 16|from typing import Any, Dict, List, Optional, Tuple
 17|
 18|from src.algorithms.damo_ets import DAMOETSScheduler
 19|from src.services.run_manager import get_run_manager
 20|from src.simulation import DataCenterSimulator
 21|from src.simulation.metrics import summarize
 22|from src.utils.logger import setup_logger
 23|from src.utils.validation import validate_algorithms,
validate_float, va
   |    lidate_int, validate_str
 24|
 25|logger = setup_logger("plan_optimizer")
 26|
```

```
ALGORITHM_LABELS = {
    "greedy": "贪心算法",
    "genetic": "遗传算法",
    "round_robin": "轮询算法",
    "damo_ets": "DAMO-ETS",
    "nsga2": "NSGA-II",
    "moead": "MOEA-D",
}

TARGET_LABELS = {
    "pue": "平均 PUE 最低",
    "energy": "单位服务负载能耗最低",
    "temp": "峰值温度最低",
    "balanced": "能耗、温度、PUE 均衡",
}

WEIGHT_PROFILES = {
    "energy": {
        "label": "DAMO-ETS • 能耗权重",
        "weights": {
            "alpha_deadline": 0.20,
            "beta_energy": 0.45,
            "gamma_thermal": 0.20,
            "delta_heterogeneity": 0.10,
            "epsilon_migration": 0.05,
        },
    },
    "thermal": {
        "label": "DAMO-ETS • 温度权重",
        "weights": {
            "alpha_deadline": 0.25,
            "beta_energy": 0.20,
            "gamma_thermal": 0.35,
            "delta_heterogeneity": 0.15,
            "epsilon_migration": 0.05,
        },
    },
}

DEFAULT_ALGORITHMS = ["greedy", "genetic", "round_robin", "
damo_ets"]
MAX_SERVERS = 500
MAX_HOURS = 72
MAX_PLANS = 3
```

```
 70|
 71|
 72|def _now() -> str:
 73|    return datetime.now().strftime("%Y-%m-%d %H:%M:%S")
 74|
 75|
 76|def _round(value: Any, digits: int = 4) -> Optional[float]:
 77|    try:
 78|        number = float(value)
 79|    except (TypeError, ValueError):
 80|        return None
 81|    if not math.isfinite(number):
 82|        return None
 83|    return round(number, digits)
 84|
 85|
 86|def _server_counts(low: int, high: int) -> List[int]:
 87|    if high <= low:
 88|        return [low]
 89|    middle = int(round((low + high) / 2.0))
 90|    values = [low, middle, high]
 91|    result: List[int] = []
 92|    for value in values:
 93|        if value not in result:
 94|            result.append(value)
 95|    return result
 96|
 97|
 98|def _evaluate(spec: Dict[str, Any], base: Dict[str, Any]) ->
Dict[str, A
   |    ny]:
 99|    simulator = DataCenterSimulator(
100|        num_servers=spec["num_servers"],
101|        simulation_hours=base["simulation_hours"],
102|        seed=base["seed"],
103|        data_source_override=({"site_id": base["site_id"]} if
base.get("
   |    site_id") else None),
104|    )
105|    if spec.get("weights") and spec["algorithm"] == "damo_ets":
106|        simulator.damo_ets_scheduler = DAMOETSScheduler(
107|            simulator.servers, config=dict(spec["weights"]),
seed=base["
   |    seed"]
```

```
        )
    frame = simulator.run_simulation(spec["algorithm"])
    stats = summarize(frame).get(spec["algorithm"]) or {}
    metrics = {
        "avg_pue": _round(stats.get("avg_pue"), 4),
        "avg_energy": _round(stats.get("avg_energy"), 1),
        "max_temp": _round(stats.get("max_temp"), 2),
        "avg_temp": _round(stats.get("avg_temp"), 2),
        "service_rate": _round(stats.get("service_rate"), 4),
        "energy_per_served_load":
_round(stats.get("energy_per_served_lo
    ad"), 4),
        "dropped_tasks": int(stats.get("dropped_tasks") or 0),
        "deadline_violations": int(stats.get("deadline_violations")
 or 0
    ),
        "total_energy": _round(stats.get("total_energy"), 1),
    }
    return {
        "num_servers": spec["num_servers"],
        "algorithm": spec["algorithm"],
        "algorithm_label": ALGORITHM_LABELS.get(spec["algorithm"],
 spec[
    "algorithm"]),
        "weights": dict(spec["weights"]) if spec.get("weights")
else Non
    e,
        "weight_label": spec.get("weight_label"),
        "metrics": metrics,
        "data_source": {
            "site_id": (simulator.run_metadata or
{}).get("site_id"),
            "school_name": base.get("school_name"),
            "ambient_source": (simulator.run_metadata or
{}).get("ambien
    t_source"),
            "scenario_source": (simulator.run_metadata or
{}).get("scena
    rio_source"),
        },
    }


def _metric(row: Dict[str, Any], key: str) -> float:
```

```
    value = (row.get("metrics") or {}).get(key)
    try:
        number = float(value)
    except (TypeError, ValueError):
        return float("inf")
    return number if math.isfinite(number) else float("inf")


def _normalized(value: float, low: float, high: float) -> float:
    if not math.isfinite(value):
        return 1.0
    if high <= low:
        return 0.0
    return min(1.0, max(0.0, (value - low) / (high - low)))


def _score(row: Dict[str, Any], pool: List[Dict[str, Any]],
target: str)
    -> float:
    pue = _metric(row, "avg_pue")
    energy = _metric(row, "energy_per_served_load")
    temp = _metric(row, "max_temp")
    if target == "pue":
        return 0.0 if not math.isfinite(pue) else round(pue, 6)
    if target == "energy":
        return 0.0 if not math.isfinite(energy) else
round(energy, 6)
    if target == "temp":
        return 0.0 if not math.isfinite(temp) else round(temp, 6)
    pue_values = [_metric(item, "avg_pue") for item in pool]
    energy_values = [_metric(item, "energy_per_served_load") for
item in
     pool]
    temp_values = [_metric(item, "max_temp") for item in pool]
    pue_score = _normalized(pue, min(pue_values), max(pue_values))
    energy_score = _normalized(energy, min(energy_values),
max(energy_va
    lues))
    temp_score = _normalized(temp, min(temp_values),
max(temp_values))
    return round(0.45 * pue_score + 0.35 * energy_score + 0.20 *
temp_sc
    ore, 6)

```

```
173|
174|def _dominates(left: Dict[str, Any], right: Dict[str, Any]) ->
bool:
175|    keys = ("avg_pue", "energy_per_served_load", "max_temp")
176|    left_values = [_metric(left, key) for key in keys]
177|    right_values = [_metric(right, key) for key in keys]
178|    if any(not math.isfinite(value) for value in left_values):
179|        return False
180|    no_worse = all(a <= b for a, b in zip(left_values,
right_values))
181|    strictly_better = any(a < b for a, b in zip(left_values,
right_value
   |    s))
182|    return no_worse and strictly_better
183|
184|
185|def _mark_pareto(rows: List[Dict[str, Any]]) -> int:
186|    count = 0
187|    for row in rows:
188|        row["pareto"] = not any(
189|            other is not row and _dominates(other, row)
190|            for other in rows
191|        )
192|        if row["pareto"]:
193|            count += 1
194|    return count
195|
196|
197|def _pick_top(rows: List[Dict[str, Any]], limit: int = MAX_PLANS)
-> Lis
   |    t[Dict[str, Any]]:
198|    ordered = sorted(rows, key=lambda item: item.get("score",
float("inf
   |    ")))
199|    picked: List[Dict[str, Any]] = []
200|    used_algorithms = set()
201|    for row in ordered:
202|        if row["algorithm"] in used_algorithms:
203|            continue
204|        picked.append(row)
205|        used_algorithms.add(row["algorithm"])
206|        if len(picked) >= limit:
207|            break
208|    used_counts = {row["num_servers"] for row in picked}
```

```
    if len(picked) < limit:
        for row in ordered:
            if row in picked or row["num_servers"] in used_counts:
                continue
            picked.append(row)
            used_counts.add(row["num_servers"])
            if len(picked) >= limit:
                break
    if len(picked) < limit:
        for row in ordered:
            if row in picked:
                continue
            picked.append(row)
            if len(picked) >= limit:
                break
    for index, row in enumerate(picked, start=1):
        row["rank"] = index
    return picked


def _plan_text(row: Dict[str, Any]) -> str:
    metrics = row.get("metrics") or {}
    parts = [
        "%d 台 + %s" % (row["num_servers"], row.get("algorithm_label") o
    r row["algorithm"]),
    ]
    if row.get("weight_label"):
        parts[0] += " %s " % row["weight_label"]
    if metrics.get("avg_pue") is not None:
        parts.append("平均 PUE %.3f" % metrics["avg_pue"])
    if metrics.get("energy_per_served_load") is not None:
        parts.append("单位能耗 %.2f" % metrics["energy_per_served_load"]
    )
    if metrics.get("max_temp") is not None:
        parts.append("峰值温度 %.1f℃" % metrics["max_temp"])
    if metrics.get("service_rate") is not None:
        parts.append("服务率 %.1f%%" % (metrics["service_rate"] * 100.0)
    )
    return " ".join(parts)


def _explain(best: Optional[Dict[str, Any]], baseline:
Optional[Dict[str
    , Any]],
```

```
            top: List[Dict[str, Any]], params: Dict[str, Any])
-> Dict[str, Any]:
    target_label = TARGET_LABELS.get(params["target"],
params["target"])
    if not best:
        return {
            "summary": "本轮没有找到满足约束的方案，建议放宽服务率或温度限制后重试。",
            "reasons": [],
            "caution": "所有候选方案都未通过约束筛选。",
        }
    reasons = [
        "目标：%s，服务率门槛 %.1f%%。" % (target_label,
params["service_rate_floor"] * 100.0),
        "最优方案：" + _plan_text(best) + "。",
    ]
    if baseline and baseline is not best:
        best_metrics = best.get("metrics") or {}
        base_metrics = baseline.get("metrics") or {}
        changes = []
        if best_metrics.get("avg_pue") is not None and
base_metrics.get("avg_pue") is not None:
            delta = best_metrics["avg_pue"] -
base_metrics["avg_pue"]
            changes.append("PUE %s %.3f" % ("降低" if delta <= 0 else "升高", abs(delta)))
        if best_metrics.get("energy_per_served_load") is not None and base_metrics.get("energy_per_served_load") is not None:
            delta = best_metrics["energy_per_served_load"] -
base_metrics["energy_per_served_load"]
            changes.append("单位能耗 %s %.4f" % ("降低" if delta <= 0 else "升高", abs(delta)))
        if best_metrics.get("max_temp") is not None and
base_metrics.get("max_temp") is not None:
            delta = best_metrics["max_temp"] -
base_metrics["max_temp"]
            changes.append("峰值温度 %s %.1f℃" % ("降低" if delta <= 0 else "升高", abs(delta)))
```

```
        if changes:
            reasons.append("相比基准「%s」，%s。" % (
                "%d 台 + %s" % (baseline["num_servers"], baseline.get("a
lgorithm_label") or baseline["algorithm"]),
                " ".join(changes),
            ))
    if len(top) > 1:
        reasons.append("备选：" + "；".join(_plan_text(row) for row in
top
[1:]) + " ")
    caution = ""
    if params.get("max_temp") is not None:
        caution = "峰值温度约束为 %.1f℃；如需更低温，可继续收紧温度上限
后
重跑。" % params["max_temp"]
    return {"summary": "推荐「%s」。" % _plan_text(best), "reasons":
reason
s, "caution": caution}


class PlanOptimizer:
    def __init__(self) -> None:
        self._jobs: Dict[str, Dict[str, Any]] = {}
        self._lock = threading.Lock()
        self._executor = ThreadPoolExecutor(max_workers=1,
thread_name_p
refix="dcs-optimize")

    def start(self, raw: Dict[str, Any]) -> Dict[str, Any]:
        params = self._validate(raw)
        job_id = uuid.uuid4().hex[:12]
        job = {
            "job_id": job_id,
            "status": "queued",
            "progress": 0,
            "done": 0,
            "total": 0,
            "message": "已提交，等待当前仿真任务结束",
            "params": params,
            "result": None,
            "error": "",
            "created_at": _now(),
            "finished_at": None,
        }
```

```
        with self._lock:
            self._jobs[job_id] = job
        self._executor.submit(self._run, job_id)
        return self.snapshot(job_id)

    def active(self) -> Optional[Dict[str, Any]]:
        with self._lock:
            for job in self._jobs.values():
                if job.get("status") in ("queued", "running"):
                    return dict(job)
        return None
    def snapshot(self, job_id: str) -> Optional[Dict[str, Any]]:
        with self._lock:
            job = self._jobs.get(job_id)
            return dict(job) if job else None

    def apply(self, job_id: str) -> Dict[str, Any]:
        job = self.snapshot(job_id)
        if not job:
            raise ValueError("寻优任务不存在")
        if job.get("status") != "success" or not
job.get("result"):
            raise ValueError("寻优尚未完成")
        result = job["result"]
        best = result.get("best")
        if not best:
            raise ValueError("没有可应用的推荐方案")
        params = job["params"]
        applied_weights = None
        if best.get("weights"):
            from src.api.ai_api import _apply_weights
            applied_weights = _apply_weights(best["weights"])
        algorithms = list(params["algorithms"])
        if best["algorithm"] not in algorithms:
            algorithms.append(best["algorithm"])
        run_params = {
            "num_servers": best["num_servers"],
            "simulation_hours": params["simulation_hours"],
            "algorithms": algorithms,
            "run_mode": "standard",
            "server_type": "mixed",
            "parallel_workers": 4,
            "site_id": params.get("site_id") or "",
            "school_name": params.get("school_name") or "",
```

```
            "visualization": True,
            "save_results": True,
        }
        manager = get_run_manager()
        run_id = manager.submit(run_params, origin="optimizer")
        return {
            "run_id": run_id,
            "params": run_params,
            "plan": best,
            "applied_weights": applied_weights,
        }

    def _validate(self, raw: Dict[str, Any]) -> Dict[str, Any]:
        low = validate_int(raw.get("server_min"), min_val=4,
max_val=MAX
    _SERVERS, default=48, name="最小服务器数")
        high = validate_int(raw.get("server_max"), min_val=4,
max_val=MA
    X_SERVERS, default=max(80, low), name="最大服务器数")
        if high < low:
            raise ValueError("最大服务器数不能小于最小服务器数")
        hours = validate_int(raw.get("simulation_hours"),
min_val=1, max
    _val=MAX_HOURS, default=24, name="仿真时长")
        target = validate_str(raw.get("target"), max_len=16,
pattern=r"^
    (pue|energy|temp|balanced)$", default="balanced", name="优化目标")
        floor = validate_float(raw.get("service_rate_floor"),
min_val=0.
    8, max_val=1.0, default=0.98, name="服务率门槛")
        max_temp = raw.get("max_temp")
        if max_temp in ("", None):
            max_temp = None
        else:
            max_temp = validate_float(max_temp, min_val=20.0,
max_val=10
    0.0, name="温度上限")
        algorithms = validate_algorithms(raw.get("algorithms") or
DEFAUL
    T_ALGORITHMS)
        site_id = str(raw.get("site_id") or "").strip()[:64]
        school_name = str(raw.get("school_name") or "
").strip()[:128]
        return {
```

```
380|            "server_min": low,
381|            "server_max": high,
382|            "simulation_hours": hours,
383|            "target": target,
384|            "service_rate_floor": round(floor, 4),
385|            "max_temp": max_temp,
386|            "algorithms": algorithms,
387|            "include_weights": bool(raw.get("include_weights",
True)),
388|            "seed": int(raw.get("seed") or 42),
389|            "site_id": site_id,
390|            "school_name": school_name,
391|        }
392|
393|    def _update(self, job_id: str, **fields: Any) -> None:
394|        with self._lock:
395|            job = self._jobs.get(job_id)
396|            if job:
397|                job.update(fields)
398|
399|    def _run(self, job_id: str) -> None:
400|        job = self.snapshot(job_id)
401|        if not job:
402|            return
403|        params = job["params"]
404|        manager = get_run_manager()
405|        started = time.perf_counter()
406|        try:
407|            with manager.simulation_lock():
408|                self._update(job_id, status="running", message="开始评估
   |    候选方案", progress=2)
409|                counts = _server_counts(params["server_min"],
params["se
   |    rver_max"])
410|                middle = counts[len(counts) // 2]
411|                stage_one: List[Tuple[Dict[str, Any], Dict[str,
Any]]] =
   |     []
412|                for algorithm in params["algorithms"]:
413|                    if algorithm == "damo_ets" and
params["include_weigh
   |    ts"]:
414|                        for profile_key, profile in
WEIGHT_PROFILES.item
```

```
   |    s():
415|                            stage_one.append((
416|                                {
417|                                    "num_servers": middle,
418|                                    "algorithm": algorithm,
419|                                    "weights": profile["weights"],
420|                                    "weight_label":
profile["label"],
421|                                },
422|                                params,
423|                            ))
424|                    stage_one.append((
425|                        {"num_servers": middle, "algorithm":
algorithm},
426|                        params,
427|                    ))
428|                total = len(stage_one) + max(0, len(counts) - 1)
* 2
429|                self._update(job_id, total=total)
430|                rows: List[Dict[str, Any]] = []
431|                failed: List[Dict[str, Any]] = []
432|                for index, (spec, base) in enumerate(stage_one,
start=1)
   |    :
433|                    self._run_candidate(job_id, spec, base, rows,
failed
   |    , index, total)
434|                qualified = [row for row in rows if
self._qualified(row,
   |     params)]
435|                pool = qualified or rows
436|                for row in rows:
437|                    row["score"] = _score(row, pool,
params["target"])
438|                ranked = sorted([row for row in rows if
row.get("score")
   |     is not None], key=lambda item: item["score"])
439|                leaders: List[Dict[str, Any]] = []
440|                seen = set()
441|                for row in ranked:
442|                    key = (row["algorithm"],
tuple(sorted((row.get("weig
   |    hts") or {}).items())))
443|                    if key in seen:
```

```
                        continue
                    seen.add(key)
                    leaders.append(row)
                    if len(leaders) >= 2:
                        break
                for count in counts:
                    if count == middle:
                        continue
                    for leader in leaders:
                        spec = {
                            "num_servers": count,
                            "algorithm": leader["algorithm"],
                            "weights": leader.get("weights"),
                            "weight_label":
leader.get("weight_label"),
                        }
                        self._run_candidate(job_id, spec, params,
rows,
   |    failed, len(rows) + 1, total)
                qualified = [row for row in rows if
self._qualified(row,
   |     params)]
                pool = qualified or rows
                for row in rows:
                    row["score"] = _score(row, pool,
params["target"])
                    row["qualified"] = self._qualified(row,
params)
                pareto_rows = qualified or rows
                pareto_count = _mark_pareto(pareto_rows)
                top = _pick_top(qualified or rows, MAX_PLANS)
                baseline = self._baseline(rows, middle)
                best = top[0] if top else None
                result = {
                    "rule": self._rule(params),
                    "target": params["target"],
                    "target_label":
TARGET_LABELS.get(params["target"],
   |    params["target"]),
                    "service_rate_floor":
params["service_rate_floor"],
                    "max_temp": params["max_temp"],
                    "scale": {"server_min": params["server_min"], "
serve
```

```
   |    r_max": params["server_max"], "simulation_hours":
params["simulation
   |    _hours"]},
477|                    "site_id": params.get("site_id"),
478|                    "school_name": params.get("school_name"),
479|                    "candidates": len(rows) + len(failed),
480|                    "evaluated": len(rows),
481|                    "failed": failed,
482|                    "elapsed_sec": round(time.perf_counter() -
started,
   |    2),
483|                    "pareto_count": pareto_count,
484|                    "baseline": baseline,
485|                    "best": best,
486|                    "top": top,
487|                }
488|                result["explanation"] = _explain(best, baseline,
top, pa
   |    rams)
489|                self._update(job_id, status="success",
progress=100, don
   |    e=len(rows) + len(failed), message="寻优完成", result=result, finish
   |    ed_at=_now())
490|            except Exception as exc:
491|                logger.error("全局寻优失败: %s", exc, exc_info=True)
492|                self._update(job_id, status="failed", progress=100,
message=
   |    "寻优失败", error=str(exc), finished_at=_now())
493|
494|    def _run_candidate(self, job_id: str, spec: Dict[str, Any],
base: Di
   |    ct[str, Any],
495|                          rows: List[Dict[str, Any]], failed:
List[Dict[str
   |    , Any]],
496|                          index: int, total: int) -> None:
497|        started = time.perf_counter()
498|        try:
499|            row = _evaluate(spec, base)
500|            rows.append(row)
501|            logger.info("寻优候选: %s 台 / %s -> PUE %s, 温度 %s, 服务率
   |     %s", spec["num_servers"], spec["algorithm"],
row["metrics"].get("av
```

```
   |    g_pue"), row["metrics"].get("max_temp"),
row["metrics"].get("service
   |    _rate"))
502|        except Exception as exc:
503|            failed.append({
504|                "num_servers": spec["num_servers"],
505|                "algorithm": spec["algorithm"],
506|                "weight_label": spec.get("weight_label"),
507|                "error": str(exc),
508|            })
509|            logger.warning("寻优候选失败: %s 台 / %s: %s", spec["num_ser
   |    vers"], spec["algorithm"], exc)
510|        elapsed = time.perf_counter() - started
511|        done = len(rows) + len(failed)
512|        progress = int(min(95, max(3, done / max(total, 1) * 95)))
513|        self._update(job_id, progress=progress, done=done,
total=max(tot
   |    al, done), message="已完成 %d/%d 组候选（最近一组 %.1f 秒）" %
(done,
   |    max(total, done), elapsed))
514|
515|    def _qualified(self, row: Dict[str, Any], params: Dict[str,
Any]) ->
   |     bool:
516|        rate = (row.get("metrics") or {}).get("service_rate")
517|        if rate is not None and float(rate) <
params["service_rate_floor
   |    "]:
518|            return False
519|        limit = params.get("max_temp")
520|        temp = (row.get("metrics") or {}).get("max_temp")
521|        if limit is not None and temp is not None and float(temp)
> floa
   |    t(limit):
522|            return False
523|        return True
524|
525|    def _baseline(self, rows: List[Dict[str, Any]], middle: int)
-> Opti
   |    onal[Dict[str, Any]]:
526|        greedy = [row for row in rows if row["algorithm"] == "
greedy" an
   |    d row["num_servers"] == middle]
527|        if greedy:
```

```
            return greedy[0]
        same_count = [row for row in rows if row["num_servers"]
== middl
    e]
        return same_count[0] if same_count else (rows[0] if rows
else No
    ne)

    def _rule(self, params: Dict[str, Any]) -> str:
        limit = params.get("max_temp")
        temp_text = "、峰值温度不高于 %.1f℃" % limit if limit is not
None
     else ""
        return "先筛除服务率低于 %.1f%%%s的方案，再按「%s」排序，并输出
Pare
    to 候选" % (
            params["service_rate_floor"] * 100.0, temp_text,
TARGET_LABE
    LS.get(params["target"], params["target"])
        )


_optimizer: Optional[PlanOptimizer] = None
_optimizer_lock = threading.Lock()


def get_plan_optimizer() -> PlanOptimizer:
    global _optimizer
    if _optimizer is None:
        with _optimizer_lock:
            if _optimizer is None:
                _optimizer = PlanOptimizer()
    return _optimizer
```

51. src/services/run_manager.py

```
"""Serialized simulation runner with a small event bus for live
UI updat
    es."""

import os
```

```
import threading
import time
from concurrent.futures import ThreadPoolExecutor
from datetime import datetime
from queue import Empty, Full, Queue
from typing import Any, Dict, Optional, Tuple

from src.plugin import PluginManager
from src.plugin.protocol import PluginResult
from src.plugin.simulation_plugin import SimulationPlugin
from src.services import run_store
from src.simulation.metrics import summarize
from src.utils import load_run_metadata, load_simulation_data, save_run_
    metadata
from src.utils.logger import setup_logger
from src.utils.validation import validate_simulation_params

logger = setup_logger("run_manager")


def _now() -> str:
    return datetime.now().strftime("%Y-%m-%d %H:%M:%S")


class EventBus:
    def __init__(self) -> None:
        self._subscribers = set()
        self._lock = threading.Lock()

    def subscribe(self) -> Queue:
        queue: Queue = Queue(maxsize=200)
        with self._lock:
            self._subscribers.add(queue)
        return queue

    def unsubscribe(self, queue: Queue) -> None:
        with self._lock:
            self._subscribers.discard(queue)

    def publish(self, event: Dict[str, Any]) -> None:
        with self._lock:
            subscribers = list(self._subscribers)
        for queue in subscribers:
```

```
            try:
                queue.put_nowait(event)
            except Full:
                pass


class RunManager:
    def __init__(self) -> None:
        self.events = EventBus()
        self._executor = ThreadPoolExecutor(max_workers=1,
thread_name_p
    refix="dcs-run")
        self._run_lock = threading.Lock()
        try:
            run_store.import_legacy_run()
        except Exception as exc:
            logger.warning("导入历史运行失败: %s" % exc)
        try:
            for record in run_store.list_runs(50):
                if record.get("status") in ("queued", "running"):
                    run_store.update_run(
                        record["run_id"],
                        status="failed",
                        progress=100,
                        finished_at=_now(),
                      error="服务重启，任务已中断",
                    )
                  logger.warning("标记中断运行: %s", record["run_id"])
        except Exception as exc:
            logger.warning("清理中断运行失败: %s" % exc)

    def submit(self, params: Dict[str, Any], origin: str = "web")
-> str
    :
        validated = self._validate(params)
        run_id = run_store.create_run(validated, origin=origin)
        self._publish(run_id, "run_created", 0, "queued",
validated)
        self._executor.submit(self._execute, run_id, validated,
origin)
        return run_id

    def run_sync(self, params: Dict[str, Any], origin: str = "
web") -> T
```

```
   |    uple[str, PluginResult]:
 83|        validated = self._validate(params)
 84|        run_id = run_store.create_run(validated, origin=origin)
 85|        self._publish(run_id, "run_created", 0, "queued",
validated)
 86|        result = self._execute(run_id, validated, origin)
 87|        return run_id, result
 88|
 89|    def run_prediction(self, params: Dict[str, Any]) ->
Tuple[Dict[str,
   |    Any], PluginResult]:
 90|        """Run a forecast scenario without overwriting the main
simulati
   |    on results."""
 91|        validated = self._validate(params)
 92|        config = {
 93|            "num_servers": validated["num_servers"],
 94|            "simulation_hours": validated["simulation_hours"],
 95|            "parallel_workers": validated["parallel_workers"],
 96|            "run_mode": validated["run_mode"],
 97|            "server_type": validated["server_type"],
 98|        }
 99|        if validated.get("site_id"):
100|            config["site_id"] = validated["site_id"]
101|
102|        with self._run_lock:
103|            manager = PluginManager()
104|            manager.register_plugin(SimulationPlugin)
105|            if not manager.load_plugin("SimulationPlugin",
config):
106|                raise RuntimeError("预测仿真加载失败")
107|            try:
108|                result = manager.execute_plugin(
109|                    "SimulationPlugin",
110|                    algorithms=validated["algorithms"],
111|                    save_results=False,
112|                    visualization=False,
113|                    include_series=True,
114|                )
115|            finally:
116|                manager.unload_plugin("SimulationPlugin")
117|        return validated, result
118|
119|    def simulation_lock(self):
```

```
        return self._run_lock

    def wait(self, run_id: str, timeout: float = 120.0) ->
Optional[Dict
    [str, Any]]:
        deadline = time.time() + timeout
        while time.time() < deadline:
            record = run_store.get_run(run_id)
            if record and record.get("status") in ("success", "
failed"):
                return record
            time.sleep(0.5)
        return run_store.get_run(run_id)

    def active_run(self) -> Optional[Dict[str, Any]]:
        for record in run_store.list_runs(20):
            if record.get("status") in ("queued", "running"):
                return record
        return None

    def _validate(self, params: Dict[str, Any]) -> Dict[str, Any]:
        raw = dict(params or {})
        validated = validate_simulation_params(raw)
        site_id = raw.get("site_id")
        if site_id is not None:
            site_id = str(site_id).strip()[:64]
            if site_id:
                validated["site_id"] = site_id
        school_name = raw.get("school_name")
        if school_name is not None:
            school_name = str(school_name).strip()[:128]
            if school_name:
                validated["school_name"] = school_name
        return validated

    def _execute(self, run_id: str, params: Dict[str, Any],
origin: str)
     -> PluginResult:
        with self._run_lock:
            run_store.update_run(run_id, status="running",
progress=5, s
    tarted_at=_now())
            self._publish(run_id, "run_progress", 5, "running",
params)
```

```
156|            manager = PluginManager()
157|            try:
158|                manager.register_plugin(SimulationPlugin)
159|                manager.discover_plugins(plugin_dir="src/plugins")
160|                config = {
161|                    "num_servers": params["num_servers"],
162|                    "simulation_hours":
params["simulation_hours"],
163|                    "parallel_workers":
params["parallel_workers"],
164|                    "run_mode": params["run_mode"],
165|                    "server_type": params["server_type"],
166|                }
167|                if params.get("site_id"):
168|                    config["site_id"] = params["site_id"]
169|                if not manager.load_plugin("SimulationPlugin",
config):
170|                  raise RuntimeError("SimulationPlugin 加载失败")
171|
172|                run_store.update_run(run_id, progress=20)
173|                self._publish(run_id, "run_progress", 20, "
running", par
   |    ams)
174|                result = manager.execute_plugin(
175|                    "SimulationPlugin",
algorithms=params["algorithms"]
176|                )
177|                manager.unload_plugin("SimulationPlugin")
178|                if not result.success:
179|                  raise RuntimeError(result.message or "仿真执行失败")
180|
181|                run_store.update_run(run_id, progress=80)
182|                self._publish(run_id, "run_progress", 80, "
running", par
   |    ams)
183|                df = load_simulation_data()
184|                if df is None or df.empty:
185|                  raise RuntimeError("仿真结果文件为空")
186|                metadata = load_run_metadata() or {}
187|                metadata.update({
188|                    "run_id": run_id,
189|                    "origin": origin,
190|                    "generated_at": _now(),
191|                    "num_servers": params["num_servers"],
```

```
                    "simulation_hours":
params["simulation_hours"],
                    "algorithms": params["algorithms"],
                })
                if params.get("site_id") and not
metadata.get("site_id")
   |    :
                    metadata["site_id"] = params["site_id"]
                if params.get("school_name"):
                    metadata["school_name"] =
params["school_name"]
                save_run_metadata(metadata)
                summary = summarize(df)
                run_store.save_results(run_id, df, metadata,
summary)
                run_store.update_run(
                    run_id,
                    status="success",
                    progress=100,
                    finished_at=_now(),
                )
                self._publish(
                    run_id, "run_completed", 100, "success",
params,
                    summary=summary, origin=origin,
                )
                return result
            except Exception as exc:
               logger.error("运行 %s 失败: %s" % (run_id, exc))
                try:
                    manager.unload_plugin("SimulationPlugin")
                except Exception:
                    pass
                run_store.update_run(
                    run_id,
                    status="failed",
                    progress=100,
                    finished_at=_now(),
                    error=str(exc),
                )
                self._publish(
                    run_id, "run_failed", 100, "failed", params,
                    error=str(exc), origin=origin,
                )
```

```
230|                return PluginResult.fail(
231|                    message=str(exc), error_code="RUN_FAILED",
error_det
   |    ail=str(exc)
232|                )
233|
234|    def _publish(self, run_id: str, event_type: str, progress:
int,
235|                   status: str, params: Dict[str, Any], **extra:
Any) -> N
   |    one:
236|        event = {
237|            "type": event_type,
238|            "run_id": run_id,
239|            "progress": progress,
240|            "status": status,
241|            "params": params,
242|            "timestamp": _now(),
243|        }
244|        event.update(extra)
245|        self.events.publish(event)
246|
247|
248|_manager: Optional[RunManager] = None
249|_manager_lock = threading.Lock()
250|
251|
252|def get_run_manager() -> RunManager:
253|    global _manager
254|    if _manager is None:
255|        with _manager_lock:
256|            if _manager is None:
257|                _manager = RunManager()
258|    return _manager
259|
260|
```

52. src/services/run_store.py

```
  1|"""Persistent run registry shared by the web UI, the assistant
and repor
   |    ts."""
  2|
  3|import json
  4|import os
```

```
import sqlite3
import threading
import uuid
from datetime import datetime
from typing import Any, Dict, List, Optional

import pandas as pd

ROOT = os.path.abspath(os.path.join(os.path.dirname(__file__), "
..", "..
    "))
DATA_DIR = os.path.join(ROOT, "data")
RUNS_DIR = os.path.join(DATA_DIR, "runs")
DB_PATH = os.path.join(DATA_DIR, "runs.db")
LEGACY_RESULTS = os.path.join(DATA_DIR, "simulation_results.csv")
LEGACY_METADATA = os.path.join(DATA_DIR, "run_metadata.json")

_LOCK = threading.RLock()


def _now() -> str:
    return datetime.now().strftime("%Y-%m-%d %H:%M:%S")


def _connect() -> sqlite3.Connection:
    os.makedirs(DATA_DIR, exist_ok=True)
    conn = sqlite3.connect(DB_PATH, timeout=30)
    conn.row_factory = sqlite3.Row
    return conn


def _init() -> None:
    with _LOCK, _connect() as conn:
        conn.execute(
            """
            CREATE TABLE IF NOT EXISTS runs (
                run_id TEXT PRIMARY KEY,
                origin TEXT NOT NULL,
                status TEXT NOT NULL,
                progress INTEGER NOT NULL DEFAULT 0,
                params_json TEXT,
                site_id TEXT,
                num_servers INTEGER,
                simulation_hours INTEGER,
```

```
                algorithms_json TEXT,
                created_at TEXT NOT NULL,
                started_at TEXT,
                finished_at TEXT,
                results_path TEXT,
                metadata_path TEXT,
                summary_path TEXT,
                summary_json TEXT,
                error TEXT
            )
            """
        )
        conn.commit()


_init()


def _dump(value: Any) -> str:
    return json.dumps(value, ensure_ascii=False, default=str)


def _loads(value: Optional[str], default: Any) -> Any:
    if not value:
        return default
    try:
        return json.loads(value)
    except (TypeError, ValueError):
        return default


def _row_to_run(row: sqlite3.Row) -> Dict[str, Any]:
    item = dict(row)
    item["params"] = _loads(item.pop("params_json", None), {})
    item["algorithms"] = _loads(item.pop("algorithms_json",
None), [])
    item["summary"] = _loads(item.pop("summary_json", None), {})
    return item


def create_run(params: Dict[str, Any], origin: str = "web",
run_id: Opti
   |    onal[str] = None) -> str:
```

```
    run_id = run_id or (datetime.now().strftime("%Y%m%d-%H%M%S") + "-" +
     uuid.uuid4().hex[:6])
    os.makedirs(run_dir(run_id), exist_ok=True)
    with _LOCK, _connect() as conn:
        conn.execute(
            """
            INSERT INTO runs (
                run_id, origin, status, progress, params_json, site_id,
                num_servers, simulation_hours, algorithms_json, created_at
            ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
            """,
            (
                run_id, origin, "queued", 0, _dump(params), params.get("site_id"),
                params.get("num_servers"), params.get("simulation_hours"),
                _dump(params.get("algorithms") or []), _now(),
            ),
        )
        conn.commit()
    return run_id


def update_run(run_id: str, **fields: Any) -> None:
    allowed = {
        "status", "progress", "started_at", "finished_at", "results_path",
        "metadata_path", "summary_path", "summary_json", "error",
    }
    updates = {key: value for key, value in fields.items() if key in allowed}
    if not updates:
        return
    for key in ("summary_json",):
        if key in updates:
            updates[key] = _dump(updates[key])
```

```
    assignments = ", ".join("%s = ?" % key for key in updates)
    with _LOCK, _connect() as conn:
        conn.execute(
            "UPDATE runs SET %s WHERE run_id = ?" % assignments,
            list(updates.values()) + [run_id],
        )
        conn.commit()


def get_run(run_id: str) -> Optional[Dict[str, Any]]:
    with _LOCK, _connect() as conn:
        row = conn.execute("SELECT * FROM runs WHERE run_id = ?", (run_id,)).fetchone()
    return _row_to_run(row) if row else None


def list_runs(limit: int = 20) -> List[Dict[str, Any]]:
    limit = max(1, min(int(limit), 100))
    with _LOCK, _connect() as conn:
        rows = conn.execute(
            "SELECT * FROM runs ORDER BY created_at DESC LIMIT ?", (limit,)
        ).fetchall()
    return [_row_to_run(row) for row in rows]


def latest_run() -> Optional[Dict[str, Any]]:
    runs = list_runs(1)
    return runs[0] if runs else None


def run_dir(run_id: str) -> str:
    return os.path.join(RUNS_DIR, run_id)


def save_results(run_id: str, df: pd.DataFrame, metadata: Dict[str, Any],
                 summary: Dict[str, Any]) -> Dict[str, str]:
    folder = run_dir(run_id)
    os.makedirs(folder, exist_ok=True)
    results_path = os.path.join(folder, "results.csv")
```

```
    metadata_path = os.path.join(folder, "metadata.json")
    summary_path = os.path.join(folder, "summary.json")
    df.to_csv(results_path, index=False)
    with open(metadata_path, "w", encoding="utf-8") as handle:
        json.dump(metadata, handle, ensure_ascii=False, indent=2,
defaul
   |    t=str)
    with open(summary_path, "w", encoding="utf-8") as handle:
        json.dump(summary, handle, ensure_ascii=False, indent=2,
default
   |    =str)
    update_run(
        run_id,
        results_path=results_path,
        metadata_path=metadata_path,
        summary_path=summary_path,
        summary_json=summary,
    )
    return {
        "results_path": results_path,
        "metadata_path": metadata_path,
        "summary_path": summary_path,
    }


def load_results(run_id: str) -> Optional[pd.DataFrame]:
    record = get_run(run_id)
    path = (record or {}).get("results_path")
    if not path or not os.path.exists(path):
        path = os.path.join(run_dir(run_id), "results.csv")
    if not os.path.exists(path):
        return None
    return pd.read_csv(path)


def compare_runs(left_id: str, right_id: str) ->
Optional[Dict[str, Any]
   |    ]:
    left = get_run(left_id)
    right = get_run(right_id)
    if not left or not right:
        return None
    left_summary = left.get("summary") or {}
    right_summary = right.get("summary") or {}
```

```
    algorithms = sorted(set(left_summary) | set(right_summary))
    rows = []
    for algorithm in algorithms:
        left_metrics = left_summary.get(algorithm) or {}
        right_metrics = right_summary.get(algorithm) or {}
        item = {"algorithm": algorithm}
        for key in ("energy_per_served_load", "service_rate", "
avg_pue",
     "max_temp", "co2_kg"):
            left_value = left_metrics.get(key)
            right_value = right_metrics.get(key)
            delta = None
            percent = None
            if isinstance(left_value, (int, float)) and
isinstance(right
    _value, (int, float)):
                delta = round(float(right_value) -
float(left_value), 4)
                if left_value:
                    percent = round((float(right_value) -
float(left_val
    ue)) / float(left_value) * 100, 2)
            item[key] = {
                "left": left_value,
                "right": right_value,
                "delta": delta,
                "percent": percent,
            }
        rows.append(item)
    return {
        "left": left,
        "right": right,
        "metrics": rows,
       "rule": "right 相对 left 的变化",
    }


def import_legacy_run() -> Optional[str]:
    if list_runs(1):
        return None
    if not os.path.exists(LEGACY_RESULTS) or not
os.path.exists(LEGACY_M
    ETADATA):
        return None
```

```
    try:
        df = pd.read_csv(LEGACY_RESULTS)
        with open(LEGACY_METADATA, "r", encoding="utf-8") as
handle:
            meta = json.load(handle)
    except Exception:
        return None
    from src.simulation.metrics import summarize

    run_id = meta.get("run_id") or (
        "legacy-" + datetime.fromtimestamp(os.path.getmtime(LEGACY
_RESUL
    TS)).strftime("%Y%m%d-%H%M%S")
    )
    params = {
        "num_servers": meta.get("num_servers") or 10,
        "simulation_hours": meta.get("simulation_hours") or 24,
        "algorithms": meta.get("algorithms") or
list(df["algorithm"].uni
    que()),
        "site_id": meta.get("site_id"),
        "school_name": meta.get("school_name"),
    }
    create_run(params, origin="legacy", run_id=run_id)
    summary = summarize(df)
    meta = dict(meta)
    meta["run_id"] = run_id
    save_results(run_id, df, meta, summary)
    update_run(
        run_id,
        status="success",
        progress=100,
        finished_at=_now(),
    )
    return run_id
```

53. src/services/simulation_insights.py

```
"""把最近一次仿真结果整理成可回答的时段与温区洞察。"""

from datetime import datetime
from typing import Any, Dict, List, Optional

import pandas as pd

```

```
from src.simulation.metrics import safe_number

SERVICE_FLOOR = 0.90

TEMP_BANDS = [
    {"label": "低于 45℃", "low": None, "high": 45.0},
    {"label": "45-55 ", "low": 45.0, "high": 55.0},
    {"label": "55-65 ", "low": 55.0, "high": 65.0},
    {"label": "65-75 ", "low": 65.0, "high": 75.0},
    {"label": "75℃ 及以上", "low": 75.0, "high": None},
]

TEMP_THRESHOLDS = [40, 45, 50, 55, 60, 65, 70, 75, 80]


def latest_data_time() -> Optional[str]:
    import os

    path = os.path.abspath(
        os.path.join(os.path.dirname(__file__), "..", "..", "data", "simulation_results.csv")
    )
    if not os.path.exists(path):
        return None
    return datetime.fromtimestamp(os.path.getmtime(path)).strftime("%Y-%m-%d %H:%M:%S")


def _round(value: Any, digits: int = 3) -> Optional[float]:
    number = safe_number(value)
    if number is None:
        return None
    return round(number, digits)


def _prepare(df: pd.DataFrame) -> pd.DataFrame:
    frame = df.copy()
    frame["time"] = pd.to_numeric(frame.get("time"), errors="coerce")
    frame = frame.dropna(subset=["time"])
    frame["time"] = frame["time"].astype(int)

```

```
    for name in (
        "total_load", "served_load", "total_energy", "pue", "peak_temper
    ature",
        "avg_temperature", "ambient_temperature", "fan_speed", "cooling_
    power",
        "assigned", "dropped", "overheated_servers",
    ):
        if name in frame.columns:
            frame[name] = pd.to_numeric(frame[name], errors="coerce")

    served = frame["served_load"] if "served_load" in frame.columns else
     pd.Series(index=frame.index, dtype=float)
    energy = frame["total_energy"] if "total_energy" in frame.columns el
    se pd.Series(index=frame.index, dtype=float)
    frame["energy_per_load"] = energy / served.where(served > 0)

    assigned = frame["assigned"] if "assigned" in frame.columns else pd.
    Series(index=frame.index, dtype=float)
    dropped = frame["dropped"] if "dropped" in frame.columns else pd.Ser
    ies(index=frame.index, dtype=float)
    offered = assigned + dropped
    frame["hour_service_rate"] = assigned / offered.where(offered > 0)

    return frame


def _aggregate(group: pd.DataFrame) -> Dict[str, Any]:
    return {
        "hours": int(group["time"].nunique()),
        "avg_energy_per_load": _round(group["energy_per_load"].mean(), 3
    ),
        "avg_pue": _round(group["pue"].mean(), 3) if "pue" in group else
     None,
```

```
 72|        "avg_peak_temp": _round(group["peak_temperature"].mean(),
1) if
   |    "peak_temperature" in group else None,
 73|        "max_peak_temp": _round(group["peak_temperature"].max(),
1) if "
   |    peak_temperature" in group else None,
 74|        "avg_fan_speed": _round(group["fan_speed"].mean(), 2) if "
fan_sp
   |    eed" in group else None,
 75|        "avg_cooling_power": _round(group["cooling_power"].mean(),
 1) if
   |     "cooling_power" in group else None,
 76|        "avg_service_rate": _round(group["hour_service_rate"].mean
(), 4)
   |    ,
 77|        "avg_load": _round(group["total_load"].mean(), 1) if "
total_load
   |    " in group else None,
 78|        "avg_served_load": _round(group["served_load"].mean(), 1)
if "se
   |    rved_load" in group else None,
 79|    }
 80|
 81|
 82|def _pick_best(records: Dict[str, Dict[str, Any]], floor: float =
SERVIC
   |    E_FLOOR):
 83|    usable = [
 84|        (name, data) for name, data in records.items()
 85|        if data and data.get("avg_energy_per_load") is not None
 86|    ]
 87|    reliable = [
 88|        (name, data) for name, data in usable
 89|        if (data.get("avg_service_rate") or 0) >= floor
 90|    ]
 91|    pool = reliable or usable
 92|    if not pool:
 93|        return None, False
 94|    name, data = min(pool, key=lambda item:
item[1]["avg_energy_per_load
   |    "])
 95|    return name, bool(reliable)
 96|
 97|
```

```
def _hour_record(row: pd.Series) -> Dict[str, Any]:
    record = {
        "hour": int(row["time"]),
        "energy_per_load": _round(row.get("energy_per_load"), 3),
        "load": _round(row.get("total_load"), 1),
        "served_load": _round(row.get("served_load"), 1),
        "pue": _round(row.get("pue"), 3),
        "avg_temp": _round(row.get("avg_temperature"), 1),
        "peak_temp": _round(row.get("peak_temperature"), 1),
        "ambient_temp": _round(row.get("ambient_temperature"), 1),
        "fan_speed": _round(row.get("fan_speed"), 2),
        "cooling_power": _round(row.get("cooling_power"), 1),
        "service_rate": _round(row.get("hour_service_rate"), 4),
    }
    return record


def _corr(frame: pd.DataFrame, left: str, right: str) ->
Optional[float]
    :
    if left not in frame.columns or right not in frame.columns:
        return None
    pairs = frame[[left, right]].dropna()
    if len(pairs) < 2:
        return None
    if pairs[left].nunique() < 2 or pairs[right].nunique() < 2:
        return None
    value = pairs[left].corr(pairs[right])
    return _round(value, 3)


def _range(frame: pd.DataFrame, name: str) -> Dict[str,
Optional[float]]
    :
    if name not in frame.columns:
        return {"min": None, "max": None}
    values = frame[name].dropna()
    if values.empty:
        return {"min": None, "max": None}
    return {"min": _round(values.min(), 2), "max":
_round(values.max(),
    2)}


```

```
def _band_mask(frame: pd.DataFrame, low: Optional[float], high: Optional
    [float]):
    values = frame.get("peak_temperature")
    if values is None:
        return pd.Series(False, index=frame.index)
    mask = values.notna()
    if low is not None:
        mask &= values >= low
    if high is not None:
        mask &= values < high
    return mask


def build_insights(df: pd.DataFrame, meta: Optional[Dict[str, Any]] = No
    ne,
                   generated_at: Optional[str] = None) -> Dict[str, Any]
    :
    meta = meta or {}
    if df is None or df.empty or "algorithm" not in df.columns:
        return {}

    frame = _prepare(df)
    if frame.empty:
        return {}

    frame["algorithm"] = frame["algorithm"].astype(str)
    algorithms = [str(item) for item in frame["algorithm"].drop_duplicat
    es()]
    hours = sorted(int(item) for item in frame["time"].unique())

    series_hours = hours
    if len(hours) > 72:
        step = max(1, len(hours) // 72)
        series_hours = hours[::step][:72]

    time_series = {
        "hour": series_hours,
        "sampled": len(series_hours) < len(hours),
        "by_algorithm": {},
    }
```

```
    for algorithm in algorithms:
        part = frame[frame["algorithm"] ==
algorithm].set_index("time").
    reindex(series_hours)
        time_series["by_algorithm"][algorithm] = {
            "energy_per_load": [_round(value, 3) for value in
part["ener
    gy_per_load"]],
            "peak_temp": [_round(value, 1) for value in
part["peak_tempe
    rature"]] if "peak_temperature" in part else [],
            "pue": [_round(value, 3) for value in part["pue"]] if "
pue"
    in part else [],
            "service_rate": [_round(value, 4) for value in
part["hour_se
    rvice_rate"]],
            "served_load": [_round(value, 1) for value in
part["served_l
    oad"]] if "served_load" in part else [],
        }

    best_hours: Dict[str, List[Dict[str, Any]]] = {}
    for algorithm in algorithms:
        rows = frame[frame["algorithm"] ==
algorithm].dropna(subset=["en
    ergy_per_load"])
        rows = rows.sort_values("energy_per_load").head(3)
        best_hours[algorithm] = [_hour_record(row) for _, row in
rows.it
    errows()]

    overall_rows = frame.dropna(subset=["energy_per_load"])
    if not overall_rows.empty:
        best_index = overall_rows.groupby("time")["energy_per_load"
].idx
    min()
        overall_rows = overall_rows.loc[best_index]
    reliable_rows = overall_rows[
        overall_rows["hour_service_rate"].fillna(0) >=
SERVICE_FLOOR
    ]
    if not reliable_rows.empty:
        overall_rows = reliable_rows
```

```
    overall_rows = overall_rows.sort_values("energy_per_load").hea
d(5)
    overall_best_hours = []
    for _, row in overall_rows.iterrows():
        record = _hour_record(row)
        record["algorithm"] = str(row["algorithm"])
        overall_best_hours.append(record)

    safe_rows = frame.dropna(subset=["energy_per_load"])
    if "overheated_servers" in safe_rows.columns:
        safe_rows = safe_rows[safe_rows["overheated_servers"].fill
na(0)
    <= 0]
    if "peak_temperature" in safe_rows.columns:
        safe_rows = safe_rows[safe_rows["peak_temperature"].fillna
(0) <
    70]
    safe_best_hours = []
    if not safe_rows.empty:
        best_index = safe_rows.groupby("time")["energy_per_load"].
idxmin
    ()
        safe_rows = safe_rows.loc[best_index]
        safe_reliable = safe_rows[safe_rows["hour_service_rate"].f
illna(
    0) >= SERVICE_FLOOR]
        if not safe_reliable.empty:
            safe_rows = safe_reliable
        safe_rows = safe_rows.sort_values("energy_per_load").head(
5)
        for _, row in safe_rows.iterrows():
            record = _hour_record(row)
            record["algorithm"] = str(row["algorithm"])
            safe_best_hours.append(record)

    temperature_bands = []
    for band in TEMP_BANDS:
        subset = frame[_band_mask(frame, band["low"],
band["high"])]
        by_algorithm = {
            algorithm: _aggregate(subset[subset["algorithm"] ==
algorith
    m])
            for algorithm in algorithms
```

```
        }
        best, reliable = _pick_best(by_algorithm)
        best_rows = subset[subset["algorithm"] == best] if best
else sub
   |    set.iloc[0:0]
        temperature_bands.append({
            "label": band["label"],
            "hours": int(best_rows["time"].nunique()) if not
best_rows.e
   |    mpty else 0,
            "hours_any": int(subset["time"].nunique()) if not
subset.emp
   |    ty else 0,
            "by_algorithm": by_algorithm,
            "best_algorithm": best,
            "best_reliable": reliable,
        })

    temperature_thresholds = []
    temperature_thresholds_above = []
    for threshold in TEMP_THRESHOLDS:
        subset = frame[_band_mask(frame, None, float(threshold))]
        by_algorithm = {
            algorithm: _aggregate(subset[subset["algorithm"] ==
algorith
   |    m])
            for algorithm in algorithms
        }
        best, reliable = _pick_best(by_algorithm)
        best_rows = subset[subset["algorithm"] == best] if best
else sub
   |    set.iloc[0:0]
        temperature_thresholds.append({
            "threshold_c": threshold,
            "hours_below": int(best_rows["time"].nunique()) if
not best_
   |    rows.empty else 0,
            "hours_any_below": int(subset["time"].nunique()) if
not subs
   |    et.empty else 0,
            "by_algorithm": by_algorithm,
            "best_algorithm": best,
            "best_reliable": reliable,
        })
```

```
258|
259|        hot_subset = frame[_band_mask(frame, float(threshold),
None)]
260|        hot_by_algorithm = {
261|            algorithm: _aggregate(hot_subset[hot_subset["algorithm"
] ==
   |    algorithm])
262|            for algorithm in algorithms
263|        }
264|        hot_best, hot_reliable = _pick_best(hot_by_algorithm)
265|        hot_rows = hot_subset[hot_subset["algorithm"] ==
hot_best] if ho
   |    t_best else hot_subset.iloc[0:0]
266|        temperature_thresholds_above.append({
267|            "threshold_c": threshold,
268|            "hours_above": int(hot_rows["time"].nunique()) if not
hot_ro
   |    ws.empty else 0,
269|            "hours_any_above": int(hot_subset["time"].nunique())
if not
   |    hot_subset.empty else 0,
270|            "by_algorithm": hot_by_algorithm,
271|            "best_algorithm": hot_best,
272|            "best_reliable": hot_reliable,
273|        })
274|
275|    effective_rows = reliable_rows if not reliable_rows.empty
else overa
   |    ll_rows
276|    overall_records = {
277|        algorithm: _aggregate(frame[frame["algorithm"] ==
algorithm])
278|        for algorithm in algorithms
279|    }
280|    overall_best, overall_reliable = _pick_best(overall_records,
floor=0
   |    .98)
281|
282|    load_peak = frame.loc[frame["total_load"].idxmax()] if "
total_load"
   |    in frame else None
283|    temp_peak = frame.loc[frame["peak_temperature"].idxmax()] if "
peak_t
   |    emperature" in frame else None
```

```
    ambient_min = frame.loc[frame["ambient_temperature"].idxmin()]
 if "a
   |    mbient_temperature" in frame else None

    recommendations = []
    for item in temperature_bands:
        best = item.get("best_algorithm")
        data = item["by_algorithm"].get(best) if best else None
        if not data or data.get("avg_energy_per_load") is None:
            continue
        text = (
           "%s（%d 小时）：优先 %s，单位服务负载能耗 %.3f W/负载，平均
PUE %
   |    .3f "
           "服务率 %.2f%%，平均风扇转速 %.2f。"
            % (
                item["label"], item["hours"], _display_name(best),
                data["avg_energy_per_load"], data.get("avg_pue")
or 0,
                (data.get("avg_service_rate") or 0) * 100,
                data.get("avg_fan_speed") or 0,
            )
        )
        if not item.get("best_reliable"):
           text += " 该温区服务率偏低，结论只作参考。"
        recommendations.append(text)

    snapshot = {
        "run_id": meta.get("run_id"),
        "generated_at": generated_at or latest_data_time(),
        "site": meta.get("site_id"),
        "school_name": meta.get("school_name"),
        "province": meta.get("site_province"),
        "emission_region": meta.get("emission_region"),
        "num_servers": meta.get("num_servers"),
        "simulation_hours": meta.get("simulation_hours"),
        "algorithms": algorithms,
        "rows": int(len(frame)),
        "hours": len(hours),
        "controlled": meta.get("controlled"),
        "scenario_digest": meta.get("scenario_digest"),
        "scenario_source": meta.get("scenario_source"),
        "service_floor": SERVICE_FLOOR,
    }
```

```
323|
324|    return {
325|        "snapshot": snapshot,
326|        "overall_best": overall_best,
327|        "overall_best_reliable": overall_reliable,
328|        "overall_best_hours": overall_best_hours,
329|        "safe_best_hours": safe_best_hours,
330|        "best_hours": best_hours,
331|        "time_series": time_series,
332|        "temperature_bands": temperature_bands,
333|        "temperature_thresholds": temperature_thresholds,
334|        "temperature_thresholds_above":
temperature_thresholds_above,
335|        "peaks": {
336|            "load_peak": _hour_record(load_peak) | {"algorithm":
str(loa
   |    d_peak["algorithm"])} if load_peak is not None else None,
337|            "temp_peak": _hour_record(temp_peak) | {"algorithm":
str(tem
   |    p_peak["algorithm"])} if temp_peak is not None else None,
338|            "ambient_min": _hour_record(ambient_min) |
{"algorithm": str
   |    (ambient_min["algorithm"])} if ambient_min is not None else
None,
339|        },
340|        "relationships": {
341|            "ambient_vs_pue": _corr(frame, "ambient_temperature", "
pue")
   |    ,
342|            "ambient_vs_cooling": _corr(frame, "
ambient_temperature", "c
   |    ooling_power"),
343|            "load_vs_peak_temp": _corr(frame, "total_load", "
peak_temper
   |    ature"),
344|            "load_vs_energy": _corr(frame, "total_load", "
total_energy")
   |    ,
345|            "fan_vs_cooling": _corr(frame, "fan_speed", "
cooling_power")
   |    ,
346|            "note": "相关系数只描述当前观测数据，不代表因果关系。",
347|        },
348|        "ranges": {
```

```
            "ambient_temperature": _range(frame, "
ambient_temperature"),
            "total_load": _range(frame, "total_load"),
            "peak_temperature": _range(frame, "peak_temperature"),
            "pue": _range(frame, "pue"),
        },
        "recommendations": recommendations,
        "caveat": "温区建议来自当前一次仿真的观测结果，改变风扇曲线或服务
    器数量前建议重新运行验证。",
        "effective_hours": int(effective_rows["time"].nunique())
if not
    effective_rows.empty else 0,
    }


def _display_name(algorithm: str) -> str:
    return {
        "greedy": "贪心算法",
        "genetic": "遗传算法",
        "round_robin": "轮询算法",
        "damo_ets": "DAMO-ETS",
        "nsga2": "NSGA-II",
        "moead": "MOEA-D",
    }.get(algorithm, algorithm)
```

54. src/services/speech_service.py

```
"""
语音识别服务模块

提供跨平台语音识别能力:
- 实时语音采集与流式识别
- 多语言/方言支持 (zh-CN, en-US, ja-JP, ko-KR, yue-HK)
- 离线识别回退 (基于Vosk/Sherpa-ONNX)
- 隐私保护 (本地处理优先, 端到端加密)
- 降级策略 (Web Speech API → 离线引擎 → 手动输入)

参考项目:
- Web Speech API (W3C标准)
- Vosk (离线语音识别, Apache 2.0)
- Sherpa-ONNX (下一代离线识别)
- Whisper (OpenAI, 高质量识别)

环境变量:
    DCS_SPEECH_ENGINE=web|vosk|sherpa|whisper
```

```
    DCS_SPEECH_MODEL_DIR=models/speech
    DCS_SPEECH_SAMPLE_RATE=16000
"""
import os
import json
import base64
import hashlib
import time
import threading
from typing import Dict, Any, List, Optional, Tuple, Union
from dataclasses import dataclass, field
from enum import Enum
from src.utils.logger import setup_logger
from src.utils.config import Config

logger = setup_logger("speech_service")
config = Config()


class SpeechEngine(Enum):
    WEB_API = "web"          # 浏览器内置 (Chrome/Safari/Edge)
    VOSK = "vosk"            # 开源离线引擎
    SHERPA = "sherpa"        # 下一代离线引擎
    WHISPER = "whisper"      # OpenAI Whisper
    AUTO = "auto"            # 自动选择最佳

class RecognitionState(Enum):
    IDLE = "idle"
    LISTENING = "listening"
    PROCESSING = "processing"
    COMPLETED = "completed"
    ERROR = "error"
    NO_SPEECH = "no_speech"
    OFFLINE = "offline"


SUPPORTED_LANGUAGES = {
    "zh-CN": {
        "name": "中文普通话",
        "label": "简体中文",
         "web_api_code": "zh-CN",
         "vosk_model": "vosk-model-cn-0.22",
         "dialects": ["cmn-Hans-CN"],
         "offline_available": True,
```

```
    },
    "en-US": {
        "name": "English (US)",
       "label": "英语（美国）",
        "web_api_code": "en-US",
        "vosk_model": "vosk-model-small-en-us-0.15",
        "dialects": ["en-US"],
        "offline_available": True,
    },
    "ja-JP": {
       "name": "日本語",
       "label": "日语",
        "web_api_code": "ja-JP",
        "vosk_model": "vosk-model-small-ja-0.22",
        "dialects": ["ja-JP"],
        "offline_available": True,
    },
    "ko-KR": {
        "name": "    ",
       "label": "韩语",
        "web_api_code": "ko-KR",
        "vosk_model": "vosk-model-small-ko-0.22",
        "dialects": ["ko-KR"],
        "offline_available": True,
    },
    "yue-HK": {
       "name": "粵語",
       "label": "粤语（香港）",
        "web_api_code": "zh-HK",
        "vosk_model": None,
        "dialects": ["yue-Hant-HK"],
        "offline_available": False,
    },
    "zh-TW": {
       "name": "中文繁體",
       "label": "繁体中文（台湾）",
        "web_api_code": "zh-TW",
        "vosk_model": None,
        "dialects": ["cmn-Hant-TW"],
        "offline_available": False,
    },
    "de-DE": {
        "name": "Deutsch",
       "label": "德语",
```

```
        "web_api_code": "de-DE",
        "vosk_model": "vosk-model-small-de-0.15",
        "dialects": ["de-DE"],
        "offline_available": True,
    },
    "fr-FR": {
        "name": "Français",
       "label": "法语",
        "web_api_code": "fr-FR",
        "vosk_model": "vosk-model-small-fr-0.22",
        "dialects": ["fr-FR"],
        "offline_available": True,
    },
}


@dataclass
class RecognitionResult:
    text: str
    confidence: float
    language: str
    is_final: bool
    alternatives: List[Dict[str, Any]] =
field(default_factory=list)
    duration_ms: float = 0.0
    engine_used: str = "web_api"
    timestamp: str = field(default_factory=lambda:
time.strftime("%Y-%m-
   |    %dT%H:%M:%S"))


@dataclass
class SpeechSession:
    session_id: str
    language: str
    created_at: float = field(default_factory=time.time)
    state: RecognitionState = RecognitionState.IDLE
    results: List[RecognitionResult] = field(default_factory=list)
    engine: SpeechEngine = SpeechEngine.AUTO
    offline_available: bool = False
    audio_data_hash: Optional[str] = None
    metadata: Dict[str, Any] = field(default_factory=dict)


```

```
class SpeechService:
    """
    语音识别服务

    核心设计:
    - 优先级: Web Speech API (在线) → 离线引擎 (本地) → 手动输入 (兜底)
    - 隐私: 音频数据本地处理，不上传至第三方服务器
    - 多语言: 中/英/日/韩/粤/台/德/法 8种语言
    """

    def __init__(self):
        self._sessions: Dict[str, SpeechSession] = {}
        self._session_lock = threading.Lock()
        self._offline_model = None
        self._engine = self._detect_engine()

    def _detect_engine(self) -> SpeechEngine:
        env_engine = os.environ.get("DCS_SPEECH_ENGINE", "auto")
        if env_engine == "auto":
            if self._check_vosk():
                return SpeechEngine.VOSK
            if self._check_sherpa():
                return SpeechEngine.SHERPA
            return SpeechEngine.WEB_API
        engine_map = {
            "web": SpeechEngine.WEB_API,
            "vosk": SpeechEngine.VOSK,
            "sherpa": SpeechEngine.SHERPA,
            "whisper": SpeechEngine.WHISPER,
        }
        return engine_map.get(env_engine, SpeechEngine.AUTO)

    def _check_vosk(self) -> bool:
        try:
            import vosk
            return True
        except ImportError:
            return False

    def _check_sherpa(self) -> bool:
        try:
            import sherpa_onnx
            return True
        except ImportError:
```

```
            return False

    def get_supported_languages(self) -> Dict[str, Dict[str,
Any]]:
        result = {}
        for code, info in SUPPORTED_LANGUAGES.items():
            result[code] = {
                "name": info["name"],
                "label": info["label"],
                "code": code,
                "web_api_code": info["web_api_code"],
                "offline_available": info["offline_available"],
                "dialects": info.get("dialects", []),
            }
        return result

    def create_session(self, language: str = "zh-CN") ->
SpeechSession:
        import secrets
        if language not in SUPPORTED_LANGUAGES:
            language = "zh-CN"

        session_id = secrets.token_hex(16)
        lang_info = SUPPORTED_LANGUAGES[language]

        session = SpeechSession(
            session_id=session_id,
            language=language,
            state=RecognitionState.IDLE,
            engine=self._engine,
            offline_available=lang_info["offline_available"],
            metadata={
                "language_name": lang_info["name"],
                "web_api_code": lang_info["web_api_code"],
                "offline_model": lang_info.get("vosk_model"),
            }
        )

        with self._session_lock:
            self._sessions[session_id] = session

        logger.info(f"Speech session created: {session_id},
lang={langua
   |    ge}")
```

```
        return session

    def get_session(self, session_id: str) ->
Optional[SpeechSession]:
        return self._sessions.get(session_id)

    def start_listening(self, session_id: str) -> bool:
        session = self._sessions.get(session_id)
        if session:
            session.state = RecognitionState.LISTENING
            logger.info(f"Session {session_id}: listening
started")
            return True
        return False

    def stop_listening(self, session_id: str) -> bool:
        session = self._sessions.get(session_id)
        if session:
            session.state = RecognitionState.IDLE
            logger.info(f"Session {session_id}: listening
stopped")
            return True
        return False

    def add_result(self, session_id: str, text: str, confidence:
float,
                   is_final: bool = False, engine_used: str = "
web_api",
                   alternatives: List[Dict] = None, duration_ms:
float =
   |     0.0) -> Optional[RecognitionResult]:
        session = self._sessions.get(session_id)
        if not session:
            return None

        result = RecognitionResult(
            text=text,
            confidence=confidence,
            language=session.language,
            is_final=is_final,
            alternatives=alternatives or [],
            duration_ms=duration_ms,
            engine_used=engine_used,
        )
```

```
        session.results.append(result)

        if is_final:
            session.state = RecognitionState.COMPLETED
        else:
            session.state = RecognitionState.PROCESSING

        return result

    def get_all_text(self, session_id: str, final_only: bool =
True) ->
    str:
        session = self._sessions.get(session_id)
        if not session:
            return ""

        results = session.results
        if final_only:
            results = [r for r in results if r.is_final]
        return " ".join(r.text for r in results if r.text.strip())

    def process_audio_offline(self, audio_bytes: bytes, language:
str =
    "zh-CN") -> RecognitionResult:
        lang_info = SUPPORTED_LANGUAGES.get(language,
SUPPORTED_LANGUAGE
    S["zh-CN"])
        if not lang_info["offline_available"]:
            return RecognitionResult(
                text="",
                confidence=0.0,
                language=language,
                is_final=True,
                engine_used="offline_unavailable",
              alternatives=[{"text": "当前语言不支持离线识别，请切换到
    在线模式或选择其他语言"}],
            )

        try:
            if self._engine == SpeechEngine.VOSK:
                return self._recognize_vosk(audio_bytes,
lang_info)
            elif self._engine == SpeechEngine.SHERPA:
```

```
304|                return self._recognize_sherpa(audio_bytes,
lang_info)
305|        except Exception as e:
306|            logger.error(f"Offline recognition error: {e}")
307|
308|        return RecognitionResult(
309|            text="",
310|            confidence=0.0,
311|            language=language,
312|            is_final=True,
313|            engine_used="offline_error",
314|        )
315|
316|    def _recognize_vosk(self, audio_bytes: bytes, lang_info:
Dict) -> Re
   |    cognitionResult:
317|        import wave
318|        import io
319|
320|        vosk_model_name = lang_info.get("vosk_model")
321|        if not vosk_model_name:
322|            raise ValueError("No Vosk model for this language")
323|
324|        if self._offline_model is None:
325|            import vosk
326|            model_dir = os.environ.get("DCS_SPEECH_MODEL_DIR", "
models/s
   |    peech")
327|            model_path = os.path.join(model_dir, vosk_model_name)
328|            if not os.path.exists(model_path):
329|                raise FileNotFoundError(f"Vosk model not found:
{model_p
   |    ath}")
330|            self._offline_model = vosk.Model(model_path)
331|
332|        import vosk
333|        import json
334|
335|        recognizer = vosk.KaldiRecognizer(self._offline_model,
16000)
336|
337|        wav_io = io.BytesIO(audio_bytes)
338|        with wave.open(wav_io, 'rb') as wf:
339|            while True:
```

```
                data = wf.readframes(4000)
                if len(data) == 0:
                    break
                recognizer.AcceptWaveform(data)

        result_json = json.loads(recognizer.FinalResult())
        return RecognitionResult(
            text=result_json.get("text", ""),
            confidence=result_json.get("confidence", 0.0),
            language=lang_info["name"],
            is_final=True,
            engine_used="vosk",
            duration_ms=0.0,
        )

    def _recognize_sherpa(self, audio_bytes: bytes, lang_info:
Dict) ->
    RecognitionResult:
        import sherpa_onnx
        import numpy as np
        import io
        import wave

        wav_io = io.BytesIO(audio_bytes)
        with wave.open(wav_io, 'rb') as wf:
            frames = wf.readframes(wf.getnframes())
            samples = np.frombuffer(frames,
dtype=np.int16).astype(np.fl
    oat32) / 32768.0

        recognizer = sherpa_onnx.OnlineRecognizer.from_pretrained()
        recognizer.accept_waveform(16000, samples)
        result = recognizer.get_result()

        return RecognitionResult(
            text=result.text if hasattr(result, 'text') else "",
            confidence=0.95,
            language=lang_info["name"],
            is_final=True,
            engine_used="sherpa_onnx",
            duration_ms=0.0,
        )

    def hash_audio(self, audio_bytes: bytes) -> str:
```

```
        return hashlib.sha256(audio_bytes).hexdigest()

    def encrypt_audio(self, audio_bytes: bytes, key:
Optional[bytes] = N
    one) -> Tuple[bytes, bytes]:
        from cryptography.fernet import Fernet
        if key is None:
            key = Fernet.generate_key()
        f = Fernet(key)
        encrypted = f.encrypt(audio_bytes)
        return encrypted, key

    def cleanup_session(self, session_id: str) -> None:
        with self._session_lock:
            if session_id in self._sessions:
                del self._sessions[session_id]
                logger.info(f"Session cleaned: {session_id}")

    def cleanup_expired(self, max_age_sec: float = 1800.0) -> int:
        now = time.time()
        expired = []
        with self._session_lock:
            for sid, session in list(self._sessions.items()):
                if now - session.created_at > max_age_sec:
                    expired.append(sid)
            for sid in expired:
                del self._sessions[sid]
        if expired:
            logger.info(f"Cleaned {len(expired)} expired
sessions")
        return len(expired)


_speech_service_instance: Optional[SpeechService] = None

def get_speech_service() -> SpeechService:
    global _speech_service_instance
    if _speech_service_instance is None:
        _speech_service_instance = SpeechService()
    return _speech_service_instance
```

55. src/services/user_store.py

```
import os
import sqlite3
```

```
import threading
from datetime import datetime

from werkzeug.security import check_password_hash,
generate_password_has
    h

_SERVICE_DIR = os.path.dirname(os.path.abspath(__file__))
_SRC_DIR = os.path.dirname(_SERVICE_DIR)
_PROJECT_ROOT = os.path.dirname(_SRC_DIR)
_DEFAULT_DB_PATH = os.path.join(_PROJECT_ROOT, 'data', 'users.db')


class UserStore:
    def __init__(self, db_path=None):
        self.db_path = db_path or _DEFAULT_DB_PATH
        self._init_lock = threading.Lock()
        self._ensure_schema()

    def _connect(self):
        conn = sqlite3.connect(self.db_path, timeout=10)
        conn.row_factory = sqlite3.Row
        conn.execute('PRAGMA journal_mode=WAL')
        conn.execute('PRAGMA foreign_keys=ON')
        return conn

    def _ensure_schema(self):
        with self._init_lock:
            os.makedirs(os.path.dirname(self.db_path),
exist_ok=True)
            with self._connect() as conn:
                conn.execute(
                    '''
                    CREATE TABLE IF NOT EXISTS users (
                        id INTEGER PRIMARY KEY AUTOINCREMENT,
                        username TEXT NOT NULL UNIQUE COLLATE
NOCASE,
                        password_hash TEXT NOT NULL,
                        created_at TEXT NOT NULL,
                        last_login TEXT
                    )
                    '''
                )
                row = conn.execute(
```

```
                    'SELECT id FROM users WHERE username = ?
COLLATE NOC
    ASE',
                    ('yumo',)
                ).fetchone()
                if row is None:
                    conn.execute(
                        'INSERT INTO users (username,
password_hash, cre
    ated_at) VALUES (?, ?, ?)',
                        ('yumo', generate_password_hash('123456',
method
    ='pbkdf2:sha256'), self._now())
                    )

    @staticmethod
    def _now():
        return datetime.now().strftime('%Y-%m-%d %H:%M:%S')

    def get_by_username(self, username):
        with self._connect() as conn:
            return conn.execute(
                'SELECT id, username, password_hash, created_at,
last_lo
    gin '
                'FROM users WHERE username = ? COLLATE NOCASE',
                (username,)
            ).fetchone()

    def get_by_id(self, user_id):
        with self._connect() as conn:
            return conn.execute(
                'SELECT id, username, created_at, last_login FROM
users
    WHERE id = ?',
                (user_id,)
            ).fetchone()

    def create_user(self, username, password):
        password_hash = generate_password_hash(password,
method='pbkdf2:
    sha256')
        try:
            with self._connect() as conn:
```

```
                cursor = conn.execute(
                    'INSERT INTO users (username, password_hash,
created
   |    _at) VALUES (?, ?, ?)',
                    (username, password_hash, self._now())
                )
                user_id = cursor.lastrowid
        except sqlite3.IntegrityError as exc:
            raise ValueError('用户名已存在') from exc
        return self.get_by_id(user_id)

    def change_password(self, user_id, current_password,
new_password):
        with self._connect() as conn:
            row = conn.execute(
                'SELECT password_hash FROM users WHERE id = ?',
                (user_id,)
            ).fetchone()
            if row is None or not
check_password_hash(row['password_hash
   |    '], current_password):
                return False
            conn.execute(
                'UPDATE users SET password_hash = ? WHERE id = ?',
                (generate_password_hash(new_password,
method='pbkdf2:sha
   |    256'), user_id)
            )
        return True

    def verify_user(self, username, password):
        row = self.get_by_username(username)
        if row is None:
            return None
        if not check_password_hash(row['password_hash'],
password):
            return None
        with self._connect() as conn:
            conn.execute(
                'UPDATE users SET last_login = ? WHERE id = ?',
                (self._now(), row['id'])
            )
        return {'id': row['id'], 'username': row['username']}

```

```
111|
112|_store = None
113|_store_lock = threading.Lock()
114|
115|
116|def get_user_store():
117|    global _store
118|    if _store is None:
119|        with _store_lock:
120|            if _store is None:
121|                _store = UserStore()
122|    return _store
```

56. src/services/weight_tuner.py

```
  1|"""DAMO-ETS 权重自动寻优
  2|
  3|在固定随机种子下枚举若干组代表性权重方案，逐组跑一次仿真并记录
  4|能耗、温度、PUE、服务率与 deadline 违约，最后按写死的择优规则选出一组。
  5|
  6|择优规则：先筛出任务服务率不低于阈值的方案，再在其中取平均能耗最低者；
  7|若没有方案满足服务率阈值，则退化为取服务率最高的一组。
  8|"""
  9|import time
 10|from typing import Any, Dict, List, Optional
 11|
 12|from src.algorithms.damo_ets import DAMOETSScheduler
 13|from src.simulation import DataCenterSimulator
 14|from src.utils.logger import setup_logger
 15|
 16|logger = setup_logger("weight_tuner")
 17|
 18|SERVICE_RATE_FLOOR = 0.98
 19|SELECTION_RULE = "服务率 >= 98% 的前提下，能耗与温度归一化后等权综合最优"
 20|
 21|DEFAULT_SCALE = {"num_servers": 40, "simulation_hours": 8, "
seed": 42}
 22|
 23|CANDIDATES: List[Dict[str, Any]] = [
 24|    {"name": "默认权重", "weights": {"alpha_deadline": 0.35, "beta_energ
   |    y": 0.25, "gamma_thermal": 0.20, "delta_heterogeneity": 0.15, "
epsil
   |    on_migration": 0.05}},
 25|    {"name": "温度优先", "weights": {"alpha_deadline": 0.25, "beta_energ
```

```
   |    y": 0.20, "gamma_thermal": 0.35, "delta_heterogeneity": 0.15, "
epsil
   |    on_migration": 0.05}},
 26|    {"name": "强温度优先", "weights": {"alpha_deadline": 0.20, "beta_ene
   |    rgy": 0.20, "gamma_thermal": 0.45, "delta_heterogeneity":
0.10, "eps
   |    ilon_migration": 0.05}},
 27|    {"name": "能耗优先", "weights": {"alpha_deadline": 0.20, "beta_energ
   |    y": 0.45, "gamma_thermal": 0.20, "delta_heterogeneity": 0.10, "
epsil
   |    on_migration": 0.05}},
 28|    {"name": "违约优先", "weights": {"alpha_deadline": 0.50, "beta_energ
   |    y": 0.20, "gamma_thermal": 0.15, "delta_heterogeneity": 0.10, "
epsil
   |    on_migration": 0.05}},
 29|    {"name": "均衡方案", "weights": {"alpha_deadline": 0.30, "beta_energ
   |    y": 0.25, "gamma_thermal": 0.25, "delta_heterogeneity": 0.15, "
epsil
   |    on_migration": 0.05}},
 30|    {"name": "异构匹配优先", "weights": {"alpha_deadline": 0.25, "beta_e
   |    nergy": 0.20, "gamma_thermal": 0.20, "delta_heterogeneity":
0.30, "e
   |    psilon_migration": 0.05}},
 31|    {"name": "强能耗优先", "weights": {"alpha_deadline": 0.15, "beta_ene
   |    rgy": 0.55, "gamma_thermal": 0.20, "delta_heterogeneity":
0.05, "eps
   |    ilon_migration": 0.05}},
 32|]
 33|
 34|WEIGHT_KEYS = ["alpha_deadline", "beta_energy", "gamma_thermal", "
delta_
   |    heterogeneity", "epsilon_migration"]
 35|
 36|
 37|def _evaluate(weights: Dict[str, Any], num_servers: int, hours:
int, see
   |    d: int) -> Dict[str, Any]:
 38|    simulator = DataCenterSimulator(num_servers=num_servers,
simulation_
   |    hours=hours, seed=seed)
 39|    simulator.damo_ets_scheduler =
DAMOETSScheduler(simulator.servers, c
   |    onfig=dict(weights), seed=seed)
 40|    frame = simulator.run_simulation("damo_ets")
```

```
    stats = simulator.get_statistics(frame)["damo_ets"]
    return {
        "avg_energy": round(stats["avg_energy"], 1),
        "max_temp": round(stats["max_temp"], 2),
        "avg_pue": round(stats["avg_pue"], 3),
        "service_rate": round(stats.get("service_rate", 1.0), 4),
        "dropped_tasks": int(stats.get("dropped_tasks", 0)),
        "deadline_violations": int(stats.get("deadline_violations",
0)),
    }


def score_row(row: Dict[str, Any], pool: List[Dict[str, Any]]) ->
float:
    energies = [item["metrics"]["avg_energy"] for item in pool]
    temps = [item["metrics"]["max_temp"] for item in pool]
    e_low, e_high = min(energies), max(energies)
    t_low, t_high = min(temps), max(temps)
    energy = (row["metrics"]["avg_energy"] - e_low) / (e_high -
e_low) i
    f e_high > e_low else 0.0
    thermal = (row["metrics"]["max_temp"] - t_low) / (t_high -
t_low) if
     t_high > t_low else 0.0
    return round(0.5 * energy + 0.5 * thermal, 4)


def pick_best(rows: List[Dict[str, Any]]) -> Dict[str, Any]:
    qualified = [row for row in rows if
row["metrics"]["service_rate"] >
    = SERVICE_RATE_FLOOR]
    pool = qualified if qualified else rows
    best = min(pool, key=lambda row: score_row(row, pool))
    result = dict(best)
    result["score"] = score_row(best, pool)
    result["qualified"] = bool(qualified)
    return result


def run_sweep(num_servers: Optional[int] = None, hours:
Optional[int] =
    None,
              seed: Optional[int] = None, candidates:
Optional[List[Dict
```

```
   |    [str, Any]]] = None) -> Dict[str, Any]:
 74|    scale = dict(DEFAULT_SCALE)
 75|    if num_servers:
 76|        scale["num_servers"] = int(num_servers)
 77|    if hours:
 78|        scale["simulation_hours"] = int(hours)
 79|    if seed is not None:
 80|        scale["seed"] = int(seed)
 81|
 82|    plans = candidates or CANDIDATES
 83|    started = time.perf_counter()
 84|    rows: List[Dict[str, Any]] = []
 85|    for plan in plans:
 86|        metrics = _evaluate(plan["weights"],
scale["num_servers"], scale
   |    ["simulation_hours"], scale["seed"])
 87|        rows.append({"name": plan["name"], "weights":
dict(plan["weights
   |    "]), "metrics": metrics})
 88|        logger.info("寻优: %s -> 能耗 %.1f W, 温度 %.2f C, 服务率 %.2f%%
   |    " % (
 89|            plan["name"], metrics["avg_energy"],
metrics["max_temp"], me
   |    trics["service_rate"] * 100))
 90|
 91|    best = pick_best(rows)
 92|    baseline = next((row for row in rows if row["name"] == "默认权重"),
   |    rows[0])
 93|    return {
 94|        "rule": SELECTION_RULE,
 95|        "service_rate_floor": SERVICE_RATE_FLOOR,
 96|        "scale": scale,
 97|        "candidates": len(rows),
 98|        "elapsed_sec": round(time.perf_counter() - started, 2),
 99|        "rows": rows,
100|        "best": best,
101|        "baseline": baseline,
102|    }
103|
104|
105|def diff_weights(weights: Dict[str, Any], base: Dict[str, Any])
-> List[
   |    Dict[str, Any]]:
106|    changes = []
```

```
    for key in WEIGHT_KEYS:
        old = float(base.get(key, 0))
        new = float(weights.get(key, 0))
        if abs(new - old) >= 0.001:
            changes.append({"key": key, "from": old, "to": new, "
delta":
     round(new - old, 3)})
    changes.sort(key=lambda item: abs(item["delta"]),
reverse=True)
    return changes
```

57. src/simulation/__init__.py

```
from .datacenter_simulator import DataCenterSimulator,
ServerSnapshot, D
    EFAULT_ALGORITHMS
from .task_generator import TaskGenerator, TaskSpec
from .metrics import (
    SERVICE_RATE_FLOOR,
    RANKING_RULE,
    summarize,
    rank_algorithms,
    count_deadline_violations,
    safe_number,
)

__all__ = [
    'DataCenterSimulator',
    'ServerSnapshot',
    'DEFAULT_ALGORITHMS',
    'TaskGenerator',
    'TaskSpec',
    'SERVICE_RATE_FLOOR',
    'RANKING_RULE',
    'summarize',
    'rank_algorithms',
    'count_deadline_violations',
    'safe_number',
]
```

58. src/simulation/datacenter_simulator.py - 数据中心仿真器

```
import inspect
import numpy as np
import pandas as pd
from dataclasses import dataclass
```

```
from typing import Any, Dict, List, Optional, Tuple

from src.models import Server, EnergyModel, TemperatureModel, Task
from src.algorithms import (
    GreedyScheduler,
    GeneticScheduler,
    RoundRobinScheduler,
    DAMOETSScheduler,
    BaseScheduler,
    PLUGIN_ALGORITHMS,
    PluginSchedulerAdapter,
    is_plugin_algorithm,
)
from src.simulation.metrics import (
    SERVICE_RATE_FLOOR,
    count_deadline_violations,
    rank_algorithms,
    safe_number,
    summarize,
)
from src.simulation.task_generator import TaskGenerator, TaskSpec
from src.utils.config import Config
from src.utils.logger import setup_logger


DEFAULT_ALGORITHMS = ['greedy', 'genetic', 'round_robin', '
damo_ets']


@dataclass
class ServerSnapshot:
    server_id: int
    server_type: str
    max_load: float
    max_temp: float
    base_power: float
    max_power: float
    current_temp: float
    current_load: float
    idle_power_factor: float = 0.6
    dissipation_coefficient: float = 8.0
    thermal_inertia: float = 0.25


```

```
def _build_servers(
    num_servers: int,
    server_types_cfg: Optional[List[Dict[str, Any]]],
    idle_power_factor: float = 0.6,
    dissipation_coefficient: float = 8.0,
    thermal_inertia: float = 0.25,
) -> List[Server]:
    """Build a heterogeneous server list using ``server_types`` proporti
    ons."""
    thermal = dict(
        idle_power_factor=idle_power_factor,
        dissipation_coefficient=dissipation_coefficient,
        thermal_inertia=thermal_inertia,
    )
    if not server_types_cfg:
        return [Server(server_id=i, **thermal) for i in range(num_server
    s)]

    proportions = [float(t.get("proportion", 0)) for t in server_types_c
    fg]
    total = sum(proportions)
    if total <= 0:
        return [Server(server_id=i, **thermal) for i in range(num_server
    s)]

    servers: List[Server] = []
    sid = 0
    allocated = 0
    for idx, t in enumerate(server_types_cfg):
        if idx == len(server_types_cfg) - 1:
            count = num_servers - allocated
        else:
            count = int(round(num_servers * proportions[idx] / total))
            allocated += count
        for _ in range(count):
            if sid >= num_servers:
                break
            servers.append(Server(
                server_id=sid,
```

```
                max_load=float(t.get("max_load", 100)),
                max_temp=float(t.get("max_temp", 85)),
                base_power=float(t.get("base_power", 100)),
                max_power=float(t.get("max_power", 300)),
                server_type=str(t.get("name", "standard")),
                **thermal,
            ))
            sid += 1
        if sid >= num_servers:
            break

    while len(servers) < num_servers:
        servers.append(Server(server_id=len(servers), **thermal))
    return servers


class DataCenterSimulator:
    """Hour-by-hour data centre simulation with controlled algorithm com
    parison."""

    def __init__(
        self,
        num_servers: Optional[int] = None,
        simulation_hours: Optional[int] = None,
        seed: Optional[int] = None,
        config: Optional[Config] = None,
        data_source_override: Optional[Dict[str, Any]] = None,
    ) -> None:
        self.logger = setup_logger("datacenter_sim")

        cfg_data: Dict[str, Any] = {}
        try:
            cfg = config or Config()
            cfg_data = cfg.to_dict()
        except Exception:
            cfg_data = {}

        if data_source_override:
            merged_source = dict(cfg_data.get("data_source", {}) or {})
            merged_source.update(data_source_override)
            cfg_data["data_source"] = merged_source

```

```
        sim_cfg = cfg_data.get("simulation", {}) or {}
        self.num_servers: int = int(
            num_servers if num_servers is not None else
sim_cfg.get("num
    _servers", 10)
        )
        self.simulation_hours: int = int(
            simulation_hours if simulation_hours is not None
            else sim_cfg.get("simulation_hours", 24)
        )
        if seed is None:
            yaml_seed = sim_cfg.get("random_seed")
            seed = int(yaml_seed) if yaml_seed is not None else
None
        self.seed = seed

        server_types_cfg = cfg_data.get("server_types")
        if not isinstance(server_types_cfg, list):
            server_types_cfg = None

        task_gen_cfg = cfg_data.get("task_generation", {}) or {}
        task_types_cfg = task_gen_cfg.get("task_types")
        if not isinstance(task_types_cfg, list):
            task_types_cfg = None

        self.sla_slack: float = float(sim_cfg.get("sla_slack",
1.5))
        completion_cfg = cfg_data.get("task_completion", {}) or {}
        self.completion_mode: str =
str(completion_cfg.get("mode", "serv
    ice_based"))
        self.completion_probability: float =
float(completion_cfg.get("c
    ompletion_probability", 0.3))

        cooling_cfg = cfg_data.get("cooling", {}) or {}
        self.cooling_power_coefficient: float =
float(cooling_cfg.get("p
    ower_coefficient", 500))
        self.cooling_target_temp: float =
float(cooling_cfg.get("target_
    temp", 22))

        energy_cfg = cfg_data.get("energy", {}) or {}
```

```
156|        self.idle_power_factor: float =
float(energy_cfg.get("idle_power
   |    _factor", 0.6))
157|        self.dissipation_coefficient: float =
float(cooling_cfg.get("dis
   |    sipation_coefficient", 8.0))
158|        self.thermal_inertia: float =
float(cooling_cfg.get("thermal_ine
   |    rtia", 0.25))
159|        self.fan_target_temp: float =
float(cooling_cfg.get("fan_target_
   |    temp", 35))
160|
161|        self.servers: List[Server] = _build_servers(
162|            self.num_servers,
163|            server_types_cfg,
164|            self.idle_power_factor,
165|            self.dissipation_coefficient,
166|            self.thermal_inertia,
167|        )
168|        self.task_generator: TaskGenerator = TaskGenerator(
169|            base_rate=int(task_gen_cfg.get("base_rate", 10)),
170|            peak_rate=int(task_gen_cfg.get("peak_rate", 50)),
171|            peak_hours=tuple(task_gen_cfg.get("peak_hours", (9,
18))),
172|            task_types=task_types_cfg,
173|            seed=seed,
174|            sla_slack=self.sla_slack,
175|        )
176|        self.energy_model: EnergyModel =
EnergyModel(cfg_data.get("energ
   |    y"))
177|        self.temp_model: TemperatureModel =
TemperatureModel(fan_target_
   |    temp=self.fan_target_temp)
178|        self._build_schedulers()
179|
180|        self.results: Dict[str, List[Any]] = {
181|            'time': [],
182|            'total_load': [],
183|            'avg_temperature': [],
184|            'peak_temperature': [],
185|            'overheated_servers': [],
186|            'ambient_temperature': [],
```

```
            'total_energy': [],
            'pue': [],
            'cooling_power': [],
            'fan_speed': [],
            'algorithm': [],
            'deadline_violations': [],
            'assigned': [],
            'dropped': [],
            'offered_tasks': [],
            'offered_load': [],
            'served_load': [],
            'co2_kg': [],
        }

        self._stream: Optional[List[List[TaskSpec]]] = None
        self._seen_tasks: List[Task] = []
        self.scenario_digest: Optional[str] = None
        self.skipped_algorithms: List[str] = []
        self.raw_results: Optional[pd.DataFrame] = None
        self.statistics: Dict[str, Dict[str, Any]] = {}
        self.ranking: Dict[str, Any] = {}

        self.catalog = None
        self.ambient_series: Optional[List[float]] = None
        self.site_province: Optional[str] = None
        self.emission_region: Optional[str] = None
        self.emission_factor: Optional[float] = None
        self.trace_stream: Optional[List[List[TaskSpec]]] = None
        self.data_source_cfg: Dict[str, Any] =
cfg_data.get("data_source
    ", {}) or {}
        self.ambient_offset: int =
int(self.data_source_cfg.get("ambient
    _start_hour", 0) or 0)
        self._load_catalog_data()

        self.run_metadata: Dict[str, Any] = {
            "data_mode": "catalog" if self.ambient_series else "
syntheti
    c",
            "ambient_source": "catalog" if self.ambient_series
else "syn
    thetic-sine",
            "sla_slack": self.sla_slack,
```

```
223|            "completion_mode": self.completion_mode,
224|            "service_rate_floor": SERVICE_RATE_FLOOR,
225|            "emission_source": "catalog" if self.emission_factor
else No
   |    ne,
226|            "site_id": self.data_source_cfg.get("site_id"),
227|            "site_province": self.site_province,
228|            "emission_region": self.emission_region,
229|        }
230|
232|    def _load_catalog_data(self) -> None:
233|        mode = str(self.data_source_cfg.get("mode", "
synthetic")).lower(
   |    )
234|        if mode != "catalog":
235|            return
236|        try:
237|            from src.data.catalog import Catalog,
region_for_province
238|        except Exception as exc:
239|            self.logger.warning("数据目录模块不可用，回退到合成数据：%s",
   |     exc)
240|            return
241|
242|        try:
243|            catalog = Catalog(self.data_source_cfg.get("catalog_db"
))
244|        except Exception as exc:
245|            self.logger.warning("打开数据目录失败，回退到合成数据：%s", e
   |    xc)
246|            return
247|
248|        if not catalog.available():
249|            self.logger.warning(
250|                "未找到可用的数据目录 %s，回退到合成数据（演示前请先执行
t
   |    ools/etl 下的导入脚本）",
251|                catalog.db_path,
252|            )
253|            return
254|
255|        self.catalog = catalog
256|        site_id = self.data_source_cfg.get("site_id")
```

```
257|        self.site_province = catalog.city_province(site_id) if
site_id e
   |    lse None
258|        self.ambient_series = catalog.ambient_series(site_id)
259|
260|        self.trace_stream = catalog.task_stream(
261|            self.data_source_cfg.get("trace_id"),
262|            self.simulation_hours,
263|        )
264|        self.emission_region = region_for_province(self.site_provi
nce)
265|        self.emission_factor = catalog.emission_factor(self.emissi
on_reg
   |    ion)
266|
267|        if self.ambient_series:
268|            self.logger.info("环境温度来自数据目录，共 %d 个小时值", len(
   |    self.ambient_series))
269|        if self.trace_stream:
270|            self.logger.info("任务流来自数据目录，共 %d 个小时", len(self
   |    .trace_stream))
271|
272|    def build_scenario(self, hours: Optional[int] = None) ->
List[List[T
   |    askSpec]]:
273|        """Freeze the task stream for one scenario.
274|
275|        Generated once and replayed by every algorithm, so the
compariso
   |    n is
276|        controlled: all schedulers face the identical demand
sequence.
277|        """
278|        hours = self.simulation_hours if hours is None else hours
279|        if self.trace_stream:
280|            source = self.trace_stream
281|            self._stream = [list(source[h % len(source)]) for h
in range
   |    (hours)]
282|            if hours > len(source):
283|                self.run_metadata["scenario_cycle_hours"] =
len(source)
284|        else:
285|            self._stream = self.task_generator.build_stream(hours)
```

```
286|        self.scenario_digest = TaskGenerator.stream_digest(self._s
tream)
287|        self.run_metadata["scenario_digest"] =
self.scenario_digest
288|        self.run_metadata["scenario_tasks"] = int(sum(len(h) for
h in se
   |    lf._stream))
289|        self.run_metadata["scenario_source"] = "catalog" if
self.trace_s
   |    tream else "synthetic"
290|        return self._stream
291|
292|    def describe_data_sources(self) -> Dict[str, Any]:
293|        sources = dict(self.run_metadata)
294|        if self.catalog is not None:
295|            sources["provenance"] = self.catalog.provenance()
296|        return sources
297|
299|    def _build_schedulers(self) -> None:
300|        greedy_cfg: Dict[str, Any] = {}
301|        try:
302|            greedy_cfg = Config().get("greedy", {}) or {}
303|        except Exception:
304|            greedy_cfg = {}
305|        self.greedy_scheduler: GreedyScheduler = GreedyScheduler(
306|            self.servers,
307|            temperature_weight=float(greedy_cfg.get("temperature_w
eight"
   |    , 0.3)),
308|        )
309|        self.genetic_scheduler: GeneticScheduler =
GeneticScheduler(self
   |    .servers, seed=self.seed)
310|        self.round_robin_scheduler: RoundRobinScheduler =
RoundRobinSche
   |    duler(self.servers)
311|        self.damo_ets_scheduler: DAMOETSScheduler =
DAMOETSScheduler(sel
   |    f.servers, seed=self.seed)
312|        self.temp_model = TemperatureModel(
313|            fan_target_temp=getattr(self, "fan_target_temp", 35)
314|        )
315|
316|    def create_snapshot(self) -> List[ServerSnapshot]:
```

```
        return [ServerSnapshot(
            server_id=s.server_id,
            server_type=s.server_type,
            max_load=s.max_load,
            max_temp=s.max_temp,
            base_power=s.base_power,
            max_power=s.max_power,
            current_temp=s.current_temp,
            current_load=s.current_load,
            idle_power_factor=s.idle_power_factor,
            dissipation_coefficient=s.dissipation_coefficient,
            thermal_inertia=s.thermal_inertia,
        ) for s in self.servers]

    def restore_from_snapshot(self, snapshot:
List[ServerSnapshot]) -> N
    one:
        self.servers = [Server(
            server_id=snap.server_id,
            max_load=snap.max_load,
            max_temp=snap.max_temp,
            base_power=snap.base_power,
            max_power=snap.max_power,
            initial_temp=snap.current_temp,
            server_type=snap.server_type,
            idle_power_factor=snap.idle_power_factor,
            dissipation_coefficient=snap.dissipation_coefficient,
            thermal_inertia=snap.thermal_inertia,
        ) for snap in snapshot]
        for i, snap in enumerate(snapshot):
            self.servers[i].current_load = snap.current_load
        self._build_schedulers()

    def _update_schedulers(self) -> None:
        self._build_schedulers()

    def _resolve_scheduler(self, algorithm: str) -> BaseScheduler:
        if algorithm == 'greedy':
            return self.greedy_scheduler
        if algorithm == 'genetic':
            return self.genetic_scheduler
        if algorithm == 'round_robin':
            return self.round_robin_scheduler
        if algorithm == 'damo_ets':
```

```
            return self.damo_ets_scheduler
        if is_plugin_algorithm(algorithm):
            spec = PLUGIN_ALGORITHMS[algorithm]
            return PluginSchedulerAdapter(self.servers,
str(spec["plugin
   |    "]), spec.get("config"))
        raise ValueError(f"Unknown algorithm: {algorithm}")

    def _ambient_at(self, hour: int, series:
Optional[List[float]] = Non
   |    e) -> float:
        series = self.ambient_series if series is None else series
        if series:
            return float(series[(self.ambient_offset + hour) %
len(serie
   |    s)])
        return float(15 + 10 * np.sin((hour - 6) * np.pi / 12))

    def _release_finished_tasks(self, hour: int, rng:
np.random.Generato
   |    r) -> None:
        for server in self.servers:
            for task in server.tasks[:]:
                if self.completion_mode == "service_based":
                    task.service_hour(1.0)
                    if task.is_service_complete():
                        server.remove_task(task)
                        task.completed_time = hour
                elif rng.random() < self.completion_probability:
                    server.remove_task(task)
                    task.completed_time = hour

    def run_simulation(
        self,
        algorithm: str = 'greedy',
        task_stream: Optional[List[List[TaskSpec]]] = None,
    ) -> pd.DataFrame:
        self.logger.info(f"Starting simulation with {algorithm}
algorith
   |    m")
        scheduler = self._resolve_scheduler(algorithm)

        if hasattr(scheduler, "reset"):
            scheduler.reset()
```

```
394|
395|        stream = self._stream if task_stream is None else
task_stream
396|        if stream is None:
397|            stream = self.build_scenario()
398|        scenario = self.task_generator.materialize_stream(stream)
399|        supports_hour = 'current_hour' in
inspect.signature(scheduler.sc
   |    hedule_tasks).parameters
400|
401|        rng = np.random.default_rng(self.seed)
402|        self.results = {k: [] for k in self.results}
403|        self._seen_tasks = []
404|
405|        for hour in range(self.simulation_hours):
406|            ambient_temp = self._ambient_at(hour)
407|            if hour < len(scenario):
408|                new_tasks = scenario[hour]
409|            else:
410|                new_tasks = self.task_generator.generate_tasks(hou
r)
411|            self._seen_tasks.extend(new_tasks)
412|
413|            if supports_hour:
414|                placements = scheduler.schedule_tasks(new_tasks,
current
   |    _hour=hour)  # type: ignore[call-arg]
415|            else:
416|                placements = scheduler.schedule_tasks(new_tasks)
417|
418|            assigned = sum(1 for _task, _server in placements if
_server
   |     is not None)
419|            dropped = len(placements) - assigned
420|
421|            hottest = max((s.current_temp for s in self.servers),
defaul
   |    t=ambient_temp)
422|            fan_speed = self.temp_model.adjust_fan_speed(
423|                self.fan_target_temp, reference_temp=hottest
424|            )
425|            cooling_throughput = fan_speed *
self.cooling_power_coeffici
   |    ent
```

```
            self.temp_model.update_room_temperature(
                self.servers, ambient_temp, cooling_throughput
            )

            for server in self.servers:
                server.update_temperature(ambient_temp, fan_speed)

            # Measure while the work is still resident: releasing first would
            # report zero load (and base-only power) for every task that fits
            # inside one hour, which is exactly what a sub-hourly trace produces.
            total_load = sum(s.current_load for s in self.servers)
            served_load = sum(task.load for task, server in placements if server is not None)
            temperatures = [s.current_temp for s in self.servers] or [ambient_temp]
            avg_temp = float(np.mean(temperatures))
            peak_temp = float(np.max(temperatures))
            overheated = sum(1 for s in self.servers if s.is_overheated())
            cooling_power = self.energy_model.calculate_cooling_energy(
                self.servers, ambient_temp, self.cooling_target_temp, hottest
            )
            total_energy = self.energy_model.calculate_total_energy(
                self.servers, ambient_temp, self.cooling_target_temp, hottest
            )
            pue = self.energy_model.get_pue(
                self.servers, ambient_temp, self.cooling_target_temp, hottest
```

```
            )

            self._release_finished_tasks(hour, rng)

            self.results['time'].append(hour)
            self.results['total_load'].append(total_load)
            self.results['avg_temperature'].append(avg_temp)
            self.results['peak_temperature'].append(peak_temp)
            self.results['overheated_servers'].append(overheated)
            self.results['ambient_temperature'].append(ambient_tem
p)
            self.results['total_energy'].append(total_energy)
            self.results['pue'].append(pue)
            self.results['cooling_power'].append(cooling_power)
            self.results['fan_speed'].append(fan_speed)
            self.results['algorithm'].append(algorithm)
            self.results['deadline_violations'].append(
                count_deadline_violations(self._seen_tasks, hour)
            )
            self.results['assigned'].append(assigned)
            self.results['dropped'].append(dropped)
            self.results['offered_tasks'].append(len(new_tasks))
            self.results['offered_load'].append(sum(t.load for t
in new_
    tasks))
            self.results['served_load'].append(served_load)
            self.results['co2_kg'].append(
                total_energy * self.emission_factor / 1000.0 if
self.emi
    ssion_factor else None
            )

        self.logger.info(f"Simulation completed with {algorithm}
algorit
    hm")
        frame = pd.DataFrame(self.results)
        if not self.emission_factor and 'co2_kg' in frame.columns:
            frame = frame.drop(columns=['co2_kg'])
        return frame

    def run_comparison(self, algorithms: Optional[List[str]] =
None) ->
    pd.DataFrame:
        if algorithms is None:
```

```
            algorithms = list(DEFAULT_ALGORITHMS)

        self.results = {k: [] for k in self.results}
        if self._stream is None:
            self.build_scenario()

        baseline_snapshot = self.create_snapshot()

        dfs = []
        skipped: List[str] = []
        for algo in algorithms:
            self.restore_from_snapshot(baseline_snapshot)
            try:
                df = self.run_simulation(algo)
            except Exception as exc:
                self.logger.error(f"算法 {algo} 执行失败，已跳过：{exc}"
    )
                skipped.append(algo)
                self.results = {k: [] for k in self.results}
                continue
            dfs.append(df)
            self.results = {k: [] for k in self.results}

        self.skipped_algorithms = skipped
        if not dfs:
            raise RuntimeError("所有算法均执行失败，请检查插件依赖与配置
    ")

        self.raw_results = pd.concat(dfs, ignore_index=True)
        self.statistics = summarize(self.raw_results)
        self.ranking = rank_algorithms(self.statistics)
        return self.raw_results

    def reset_servers(self) -> None:
        self.servers = [Server(i) for i in
range(self.num_servers)]
        self._update_schedulers()

    def get_statistics(self, df: Optional[pd.DataFrame] = None)
-> Dict[
    str, Dict[str, Any]]:
        if df is None:
            if self.statistics:
                return self.statistics
```

```
            if self.raw_results is not None:
                df = self.raw_results
            else:
                raise ValueError("尚无仿真结果，请先运行 run_comparison 或 run_simulation")
        return summarize(df)

    def get_ranking(self, df: Optional[pd.DataFrame] = None) -> Dict[str, Any]:
        stats = self.get_statistics(df)
        self.ranking = rank_algorithms(stats)
        return self.ranking

    def _first_hour_column(self, df: pd.DataFrame, column: str) -> Dict[str, Optional[float]]:
        values: Dict[str, Optional[float]] = {}
        if df is None or df.empty or column not in df.columns:
            return values
        for algo in df["algorithm"].unique():
            algo_df = df[df["algorithm"] == algo].sort_values("time")
            values[algo] = safe_number(algo_df.iloc[0][column])
        return values

    def first_hour_loads(self, df: pd.DataFrame) -> Dict[str, Optional[float]]:
        """Served load at hour 0; differs only by each scheduler's own choices."""
        return self._first_hour_column(df, "total_load")

    def first_hour_offered(self, df: pd.DataFrame) -> Dict[str, Optional[float]]:
        """Offered demand at hour 0. Equal values prove a controlled run."""
        offered = self._first_hour_column(df, "offered_load")
        if offered:
            return offered
        return self.first_hour_loads(df)
```

```
554|
555|    def control_signature(self, df: pd.DataFrame) -> Dict[str,
Optional[
   |    float]]:
556|        return self.first_hour_offered(df)
557|
558|    def is_controlled(self, df: pd.DataFrame) -> bool:
559|        values = list(self.control_signature(df).values())
560|        return len(values) > 0 and len(set(values)) == 1
```

59. src/simulation/distributed_simulator.py

```
  1|from typing import List, Dict, Any, Tuple, Optional
  2|from concurrent.futures import ProcessPoolExecutor, as_completed
  3|from multiprocessing import cpu_count
  4|import numpy as np
  5|from src.models.server import Server
  6|from src.models.task import Task
  7|from src.models.energy_model import EnergyModel
  8|from src.models.temperature_model import TemperatureModel
  9|from src.algorithms.greedy_algorithm import GreedyScheduler
 10|from src.algorithms.genetic_algorithm import GeneticAlgorithm
 11|from src.algorithms.round_robin import RoundRobinScheduler
 12|from src.utils.logger import setup_logger
 13|from src.utils.config import Config
 14|
 15|logger = setup_logger("distributed_simulator")
 16|config = Config()
 17|
 18|class DistributedSimulator:
 19|    def __init__(self, num_servers: int = 100, parallel_workers:
int = N
   |    one):
 20|        self.num_servers = num_servers
 21|        self.parallel_workers = parallel_workers or min(8,
cpu_count())
 22|        self.servers: List[Server] = []
 23|        self.tasks: List[Task] = []
 24|        self.energy_model = EnergyModel()
 25|        self.temp_model = TemperatureModel()
 26|
 27|        self._initialize_servers()
 28|
 29|    def _initialize_servers(self):
 30|        server_types = config.server_types
```

```
        for i in range(self.num_servers):
            rand = np.random.random()
            cum_prob = 0.0
            server_type = server_types[0]

            for st in server_types:
                cum_prob += st['proportion']
                if rand < cum_prob:
                    server_type = st
                    break

            self.servers.append(Server(
                server_id=i,
                max_load=server_type['max_load'],
                max_temp=server_type['max_temp'],
                base_power=server_type['base_power'],
                max_power=server_type['max_power'],
                efficiency=server_type.get('efficiency', 0.85)
            ))

    def _generate_tasks_for_hour(self, hour: int) -> List[Task]:
        tasks = []
        task_types = config.task_generation.get('task_types', [])

        base_rate = config.task_generation.get('base_rate', 10)
        peak_rate = config.task_generation.get('peak_rate', 50)
        peak_hours = config.task_generation.get('peak_hours', [9,
18])

        if peak_hours[0] <= hour < peak_hours[1]:
            task_count = int(np.random.poisson(peak_rate))
        else:
            task_count = int(np.random.poisson(base_rate))

        for i in range(task_count):
            rand = np.random.random()
            cum_prob = 0.0
            task_type = task_types[0] if task_types else None

            for tt in task_types:
                cum_prob += tt['proportion']
                if rand < cum_prob:
                    task_type = tt
                    break
```

```
 74|
 75|            if task_type:
 76|                load = np.random.randint(task_type['load_range'][0]
, tas
   |    k_type['load_range'][1] + 1)
 77|                duration = np.random.randint(task_type['duration_r
ange']
   |    [0], task_type['duration_range'][1] + 1)
 78|                priority = np.random.randint(1, 6)
 79|            else:
 80|                load_range = config.task_generation.get('load_rang
e', [5
   |    , 30])
 81|                load = np.random.randint(load_range[0],
load_range[1] +
   |    1)
 82|                duration = np.random.randint(1, 6)
 83|                priority = np.random.randint(1, 6)
 84|
 85|            tasks.append(Task(
 86|                task_id=f"task_{hour}_{i}",
 87|                load=load,
 88|                priority=priority,
 89|                deadline=hour + duration,
 90|                created_time=hour
 91|            ))
 92|
 93|        return tasks
 94|
 95|    def _simulate_single_hour(self, hour: int, server_snapshot:
List[Dic
   |    t[str, float]],
 96|                                pending_tasks: List[Task],
algorithm: str)
   |     -> Dict[str, Any]:
 97|        new_tasks = self._generate_tasks_for_hour(hour)
 98|        all_tasks = pending_tasks + new_tasks
 99|
100|        servers = [Server(
101|            server_id=i,
102|            max_load=server_snapshot[i]['max_load'],
103|            max_temp=server_snapshot[i]['max_temp'],
104|            base_power=server_snapshot[i]['base_power'],
105|            max_power=server_snapshot[i]['max_power'],
```

```
            current_temp=server_snapshot[i]['current_temp'],
            current_load=server_snapshot[i]['current_load']
        ) for i in range(len(server_snapshot))]

        if algorithm == 'greedy':
            scheduler = GreedyScheduler(servers)
            assignments = scheduler.schedule_tasks(all_tasks)
        elif algorithm == 'genetic':
            scheduler = GeneticAlgorithm(servers, all_tasks)
            assignments = scheduler.schedule()
        elif algorithm == 'round_robin':
            scheduler = RoundRobinScheduler(servers)
            assignments = scheduler.schedule_tasks(all_tasks)
        else:
            assignments = []

        for task, server in assignments:
            server.assign_task(task)

        for server in servers:
            server.update_temperature()

        completed_tasks = [t for t in all_tasks if
t.completed_time == h
   |    our]
        remaining_tasks = [t for t in all_tasks if
t.completed_time is N
   |    one]

        total_load = sum(s.current_load for s in servers)
        avg_temp = sum(s.current_temp for s in servers) /
len(servers)
        total_energy = sum(self.energy_model.calculate_power(s)
for s in
   |     servers)
        pue = self.energy_model.calculate_pue(total_energy,
avg_temp)

        return {
            'hour': hour,
            'algorithm': algorithm,
            'total_load': total_load,
            'avg_temperature': avg_temp,
            'total_energy': total_energy,
```

```
            'pue': pue,
            'completed_tasks': len(completed_tasks),
            'remaining_tasks': len(remaining_tasks),
            'server_snapshot': [{
                'max_load': s.max_load,
                'max_temp': s.max_temp,
                'base_power': s.base_power,
                'max_power': s.max_power,
                'current_temp': s.current_temp,
                'current_load': s.current_load
            } for s in servers],
            'remaining_tasks_data': [{
                'task_id': t.task_id,
                'load': t.load,
                'priority': t.priority,
                'deadline': t.deadline,
                'created_time': t.created_time
            } for t in remaining_tasks]
        }

    def run_distributed_simulation(self, hours: int = 24,
algorithm: str
     = 'greedy') -> List[Dict[str, Any]]:
        results = []
        server_snapshot = [{
            'max_load': s.max_load,
            'max_temp': s.max_temp,
            'base_power': s.base_power,
            'max_power': s.max_power,
            'current_temp': 25.0,
            'current_load': 0.0
        } for s in self.servers]
        pending_tasks: List[Task] = []

        logger.info(f"Starting distributed simulation with
{self.num_ser
    vers} servers, {hours} hours, algorithm: {algorithm}")

        for hour in range(hours):
            result = self._simulate_single_hour(hour,
server_snapshot, p
    ending_tasks, algorithm)
            results.append(result)

```

```
            server_snapshot = result['server_snapshot']
            pending_tasks = [Task(**t) for t in
result['remaining_tasks_
    data']]

            if hour % 6 == 0:
                logger.info(f"Completed hour {hour}/{hours}")

        logger.info(f"Distributed simulation completed. Total
records: {
    len(results)}")
        return results

    def run_parallel_comparison(self, algorithms: List[str],
hours: int
    = 24) -> Dict[str, List[Dict[str, Any]]]:
        results = {}

        def run_algorithm(algorithm):
            simulator = DistributedSimulator(num_servers=self.num_
server
    s)
            return algorithm, simulator.run_distributed_simulation
(hours
    =hours, algorithm=algorithm)

        with ProcessPoolExecutor(max_workers=min(len(algorithms),
self.p
    arallel_workers)) as executor:
            futures = {executor.submit(run_algorithm, algo): algo
for al
    go in algorithms}

            for future in as_completed(futures):
                algo, algo_results = future.result()
                results[algo] = algo_results
                logger.info(f"Completed {algo} algorithm")

        return results

    def run_large_scale_test(self, server_counts: List[int],
hours: int
    = 24) -> Dict[str, Any]:
        results = {}
```

```
208|
209|        for count in server_counts:
210|            logger.info(f"Testing with {count} servers")
211|            simulator = DistributedSimulator(num_servers=count)
212|            result = simulator.run_distributed_simulation(hours=ho
urs, a
   |    lgorithm='greedy')
213|
214|            avg_pue = sum(r['pue'] for r in result) / len(result)
215|            avg_energy = sum(r['total_energy'] for r in result) /
len(re
   |    sult)
216|            avg_temp = sum(r['avg_temperature'] for r in result)
/ len(r
   |    esult)
217|
218|            results[count] = {
219|                'avg_pue': avg_pue,
220|                'avg_energy': avg_energy,
221|                'avg_temperature': avg_temp,
222|                'total_hours': hours,
223|                'server_count': count
224|            }
225|
226|        return results
```

60. src/simulation/metrics.py

```
  1|"""Shared metric definitions for algorithm comparison.
  2|
  3|Everything that ranks algorithms lives here so that the CLI, the
experim
   |    ent
  4|scripts, the Web API and the frontend all apply the same rule:
  5|
  6|    1. drop every algorithm whose service rate is below the floor,
  7|    2. rank the rest by energy per unit of actually served load.
  8|
  9|Ranking by raw energy alone rewards an algorithm for serving less
work.
 10|"""
 11|
 12|from typing import Any, Dict, Iterable, List, Optional
 13|
 14|import pandas as pd
```

```
15|
16|SERVICE_RATE_FLOOR = 0.98
17|
18|RANKING_RULE = "服务率 >= %.0f%% 的前提下，按单位服务负载能耗升序排名" %
  |    (
19|    SERVICE_RATE_FLOOR * 100.0,
20|)
21|
22|
23|def safe_number(value: Any) -> Optional[float]:
24|    """Convert to float, mapping NaN/inf/None to None so JSON
stays vali
  |    d."""
25|    try:
26|        number = float(value)
27|    except (TypeError, ValueError):
28|        return None
29|    if number != number or number in (float("inf"),
float("-inf")):
30|        return None
31|    return number
32|
33|
34|def count_deadline_violations(tasks: Iterable[Any], last_hour:
int) -> i
  |    nt:
35|    missed = 0
36|    for task in tasks:
37|        if task.missed_deadline(last_hour):
38|            missed += 1
39|    return missed
40|
41|
42|def _numeric(df: pd.DataFrame, name: str) -> Optional[pd.Series]:
43|    """Coerce a column to numbers, or return None when it is
absent."""
44|    if name not in df.columns:
45|        return None
46|    return pd.to_numeric(df[name], errors="coerce")
47|
48|
49|def _first_present(df: pd.DataFrame, names: Iterable[str]) ->
Optional[p
  |    d.Series]:
```

```
    for name in names:
        series = _numeric(df, name)
        if series is not None:
            return series
    return None


def summarize(df: pd.DataFrame) -> Dict[str, Dict[str, Any]]:
    """Per-algorithm summary used by every consumer of the
results."""
    stats: Dict[str, Dict[str, Any]] = {}
    if df is None or df.empty or "algorithm" not in df.columns:
        return stats

    for algo in df["algorithm"].unique():
        algo_df = df[df["algorithm"] == algo]
        entry: Dict[str, Any] = {"records": int(len(algo_df))}

        energy = _numeric(algo_df, "total_energy")
        entry["avg_energy"] = safe_number(energy.mean()) if
energy is no
    t None else None
        entry["total_energy"] = safe_number(energy.sum()) if
energy is n
    ot None else None

        pue = _numeric(algo_df, "pue")
        entry["avg_pue"] = safe_number(pue.mean()) if pue is not
None el
    se None

        load = _numeric(algo_df, "total_load")
        entry["total_load"] = safe_number(load.sum()) if load is
not Non
    e else None

        peak = _first_present(algo_df, ("peak_temperature", "
avg_tempera
    ture"))
        if peak is None:
            entry["max_temp"] = None
            entry["p95_temp"] = None
        else:
            peak = peak.dropna()
```

```
            entry["max_temp"] = safe_number(peak.max()) if
len(peak) els
    e None
            entry["p95_temp"] = safe_number(peak.quantile(0.95))
if len(
    peak) else None

        avg_temp = _numeric(algo_df, "avg_temperature")
        entry["avg_temp"] = safe_number(avg_temp.mean()) if
avg_temp is
    not None else None

        assigned_series = _numeric(algo_df, "assigned")
        dropped_series = _numeric(algo_df, "dropped")
        assigned = int(assigned_series.sum()) if assigned_series
is not
    None else 0
        dropped = int(dropped_series.sum()) if dropped_series is
not Non
    e else 0
        served = assigned + dropped
        entry["assigned_tasks"] = assigned
        entry["dropped_tasks"] = dropped
        entry["service_rate"] = round(assigned / served, 4) if
served el
    se 1.0

        violations = _numeric(algo_df, "deadline_violations")
        entry["deadline_violations"] = int(violations.max()) if
violatio
    ns is not None else 0
        entry["deadline_violation_rate"] = (
            round(entry["deadline_violations"] / served, 4) if
served el
    se 0.0
        )

        co2 = _numeric(algo_df, "co2_kg")
        if co2 is not None:
            entry["co2_kg"] = safe_number(co2.sum())

        overheated = _numeric(algo_df, "overheated_servers")
        if overheated is not None:
```

```
            entry["overheated_hours"] = int((overheated > 0).sum())

        total_energy = entry["total_energy"]
        served = _numeric(algo_df, "served_load")
        if served is not None:
            served_sum = float(served.sum())
        else:
            served_sum = entry["total_load"] or 0.0
        entry["served_load"] = safe_number(served_sum)
        if total_energy is not None and served_sum:
            entry["energy_per_served_load"] = round(total_energy / served_sum, 4)
        else:
            entry["energy_per_served_load"] = None

        stats[algo] = entry

    return stats


def rank_algorithms(
    stats: Dict[str, Dict[str, Any]],
    floor: float = SERVICE_RATE_FLOOR,
) -> Dict[str, Any]:
    """Apply the documented ranking rule and return the full decision trace."""
    eligible: List[Dict[str, Any]] = []
    excluded: List[Dict[str, Any]] = []

    for algo, metrics in stats.items():
        rate = metrics.get("service_rate")
        rate = 1.0 if rate is None else float(rate)
        intensity = metrics.get("energy_per_served_load")
        record = {
            "algorithm": algo,
            "service_rate": round(rate, 4),
            "energy_per_served_load": intensity,
            "max_temp": metrics.get("max_temp"),
        }
        if rate < floor:
            record["reason"] = "服务率低于 %.0f%%，不参与能耗排名" % (flo
```

```
    |    or * 100.0)
149|            excluded.append(record)
150|        elif intensity is None:
151|            record["reason"] = "缺少单位服务负载能耗，无法参与排名"
152|            excluded.append(record)
153|        else:
154|            eligible.append(record)
155|
156|    eligible.sort(
157|        key=lambda item: (
158|            item["energy_per_served_load"],
159|            item["max_temp"] if item["max_temp"] is not None else
float(
    |    "inf"),
160|            item["algorithm"],
161|        )
162|    )
163|
164|    for position, record in enumerate(eligible, start=1):
165|        record["rank"] = position
166|
167|    best = eligible[0]["algorithm"] if eligible else None
168|    best_metrics = None
169|    if best is not None:
170|        best_metrics = {
171|            "algorithm": best,
172|            "energy_per_served_load":
eligible[0]["energy_per_served_loa
    |    d"],
173|            "service_rate": eligible[0]["service_rate"],
174|            "avg_pue": stats[best].get("avg_pue"),
175|            "max_temp": stats[best].get("max_temp"),
176|        }
177|
178|    return {
179|        "rule": RANKING_RULE,
180|        "service_rate_floor": floor,
181|        "eligible": eligible,
182|        "excluded": excluded,
183|        "best": best,
184|        "best_metrics": best_metrics,
185|        "note": (
186|            "没有算法满足服务率门槛，本轮不给出能耗排名"
187|            if not eligible
```

```
            else "排名基于单位服务负载能耗，已排除服务率不足的算法"
        ),
    }
```

61. src/simulation/task_generator.py

```
import hashlib
import json
import numpy as np
from typing import List, Dict, Any, Optional, Tuple
from src.models import Task

TaskSpec = Tuple[int, int, str, int, int]


class TaskGenerator:
    DEFAULT_TASK_TYPES: List[Dict[str, Any]] = [
        {"name": "cpu_bound",    "proportion": 0.4, "load_range": [10, 50], "duration_range": [1, 5]},
        {"name": "gpu_bound",    "proportion": 0.3, "load_range": [20, 80], "duration_range": [2, 10]},
        {"name": "memory_bound", "proportion": 0.2, "load_range": [5, 25],  "duration_range": [3, 8]},
        {"name": "io_bound",     "proportion": 0.1, "load_range": [3, 15],  "duration_range": [1, 3]},
    ]

    def __init__(
        self,
        base_rate: int = 10,
        peak_rate: int = 50,
        peak_hours: tuple = (9, 18),
        task_types: Optional[List[Dict[str, Any]]] = None,
        seed: Optional[int] = None,
        sla_slack: float = 1.5,
    ) -> None:
        self.base_rate: int = base_rate
        self.peak_rate: int = peak_rate
        self.peak_hours: tuple = peak_hours
        self.task_types: List[Dict[str, Any]] = task_types or self.DEFAU
```

```
    |    LT_TASK_TYPES
 31|        self.sla_slack: float = float(sla_slack)
 32|        self.rng = np.random.default_rng(seed)
 33|        self.np_rng = np.random.default_rng(seed)
 34|
 35|    def get_rate_at_hour(self, hour: int) -> int:
 36|        start, end = self.peak_hours
 37|        if start <= hour < end:
 38|            return self.peak_rate
 39|        return self.base_rate
 40|
 41|    def _sample_workload_class(self) -> str:
 42|        names = [t["name"] for t in self.task_types]
 43|        weights = [float(t.get("proportion", 0)) for t in
self.task_type
    |    s]
 44|        total = sum(weights)
 45|        if total <= 0:
 46|            return names[0] if names else "cpu_bound"
 47|        probs = [w / total for w in weights]
 48|        return str(self.rng.choice(names, p=probs))
 49|
 50|    def _range_for(self, workload_class: str, key: str) -> tuple:
 51|        for t in self.task_types:
 52|            if t.get("name") == workload_class:
 53|                rng = t.get(key)
 54|                if isinstance(rng, (list, tuple)) and len(rng) ==
2:
 55|                    return int(rng[0]), int(rng[1])
 56|        return 5, 30
 57|
 58|    def _deadline_offset(self, duration: int) -> int:
 59|        """Deadline relative to arrival: duration scaled by the
SLA slac
    |    k factor.
 60|
 61|        ``sla_slack=1.0`` means zero queueing tolerance, so any
task tha
    |    t is not
 62|        served immediately misses its deadline.
 63|        """
 64|        scaled = int(round(duration * self.sla_slack))
 65|        return max(duration, scaled, 1)
 66|
```

```
    def generate_tasks(self, current_time: int) -> List[Task]:
        return [self._materialize(spec, current_time) for spec in
self.g
    enerate_specs(current_time)]

    def generate_specs(self, current_time: int) -> List[TaskSpec]:
        hour = current_time % 24
        rate = self.get_rate_at_hour(hour)
        num_tasks = int(self.rng.poisson(rate))
        specs: List[TaskSpec] = []

        for _ in range(num_tasks):
            workload_class = self._sample_workload_class()
            lo, hi = self._range_for(workload_class, "load_range")
            load = int(self.rng.integers(lo, hi + 1))
            priority = int(self.rng.integers(1, 6))

            d_lo, d_hi = self._range_for(workload_class, "
duration_range
    ")
            duration = int(self.rng.integers(d_lo, d_hi + 1))
            specs.append(
                (load, priority, workload_class, duration,
self._deadlin
    e_offset(duration))
            )

        return specs

    def build_stream(self, hours: int) -> List[List[TaskSpec]]:
        """Generate the whole scenario once.

        Every algorithm replays this exact stream, which is what
makes t
    he
        cross-algorithm comparison controlled instead of four
separate r
    uns.
        """
        return [self.generate_specs(hour) for hour in
range(hours)]

    @staticmethod
    def _materialize(spec: TaskSpec, arrival_hour: int) -> Task:
```

```
        load, priority, workload_class, duration, sla_offset =
spec
        task = Task(
            load=load,
            priority=priority,
            deadline=arrival_hour + sla_offset,
            arrival_time=arrival_hour,
            workload_class=workload_class,
            duration=duration,
        )
        task.created_time = arrival_hour
        return task

    def materialize_stream(self, stream: List[List[TaskSpec]]) ->
List[L
    ist[Task]]:
        """Build fresh Task objects for one algorithm run.

        Tasks carry mutable state (load on a server, start and
completio
    n
        times), so each run needs its own instances even though
the stre
    am
        values are shared.
        """
        return [
            [self._materialize(spec, hour) for spec in hour_specs]
            for hour, hour_specs in enumerate(stream)
        ]

    @staticmethod
    def stream_digest(stream: List[List[TaskSpec]]) -> str:
        payload = json.dumps(stream, separators=(",", ":"),
ensure_ascii
    =True)
        return hashlib.sha256(payload.encode("utf-8")).hexdigest()
[:16]

    def generate_daily_profile(self) -> List[Dict]:
        profile = []
        for hour in range(24):
            rate = self.get_rate_at_hour(hour)
            profile.append({
```

```
                'hour': hour,
                'rate': rate,
                'is_peak': self.peak_hours[0] <= hour <
self.peak_hours[
    |    1]
            })
        return profile
```

62. src/testing/__init__.py

```
"""
仿真模拟测试环境

提供完整的测试基础设施:
- Mock服务:  网络/硬件/协议/流量模拟
- 测试编排:  自动化测试流程管理
- 兼容性:    跨版本/跨平台测试
- 压力测试:  CPU/内存/IO/并发
- 安全扫描:  依赖/端口/SAST/注入
- 环境管理:  快照/回滚/重置
- 风险评估:  综合风险报告
"""

from .mock_services import (
    NetworkSimulator, HardwareProfileSimulator,
    ProtocolMockServer, TrafficGenerator,
    NetworkCondition, NetworkRegion, HardwareProfile, TrafficStats
)
from .test_orchestrator import (
    TestOrchestrator, TestSuite, TestResult,
    TestStatus, TestSeverity
)
from .compatibility_tester import (
    CompatibilityTester, CompatibilityCheck
)
from .stress_tester import (
    StressTester, StressMetrics
)
from .security_scanner import (
    SecurityScanner, Vulnerability,
    SeverityLevel, ScanType
)
from .environment_manager import (
    EnvironmentManager, Snapshot, EnvironmentState
)
```

```
from .risk_assessor import (
    RiskAssessor, RiskItem, RiskReport, RiskLevel
)
from .security_report import (
    generate_security_report, print_report_summary, SecurityReport
)
from .test_runner import run_full_test_suite

__all__ = [
    # Mock Services
    'NetworkSimulator', 'HardwareProfileSimulator',
    'ProtocolMockServer', 'TrafficGenerator',
    'NetworkCondition', 'NetworkRegion', 'HardwareProfile', 'TrafficStats',
    # Orchestrator
    'TestOrchestrator', 'TestSuite', 'TestResult',
    'TestStatus', 'TestSeverity',
    # Compatibility
    'CompatibilityTester', 'CompatibilityCheck',
    # Stress
    'StressTester', 'StressMetrics',
    # Security
    'SecurityScanner', 'Vulnerability', 'SeverityLevel', 'ScanType',
    # Environment
    'EnvironmentManager', 'Snapshot', 'EnvironmentState',
    # Risk
    'RiskAssessor', 'RiskItem', 'RiskReport', 'RiskLevel',
    # Report
    'generate_security_report', 'print_report_summary', 'SecurityReport',
    # Runner
    'run_full_test_suite'
]
```

63. src/testing/compatibility_tester.py

```
"""
兼容性测试模块

支持多维度的兼容性测试:
- 跨Python版本 (3.9/3.10/3.11/3.12)
- 跨操作系统 (openEuler/CentOS/Ubuntu/Windows)
```

```
- 依赖版本矩阵
- 硬件平台 (x86_64/arm64)
- API向后兼容

参考项目: tox, nox, pytest-matrix
"""

import sys
import platform
import subprocess
import importlib
from importlib.metadata import version, PackageNotFoundError
from packaging.version import parse as parse_version
from typing import Dict, Any, List, Optional, Tuple
from dataclasses import dataclass
from src.utils.logger import setup_logger
from src.utils.config import Config

logger = setup_logger("compatibility_tester")
config = Config()

@dataclass
class CompatibilityCheck:
    name: str
    dimension: str
    expected: str
    actual: str
    compatible: bool
    details: str = ""

class CompatibilityTester:
    """
    兼容性测试器

    验证系统在不同环境下的兼容性。
    支持矩阵化的兼容性验证。
    """

    def __init__(self):
        self.checks: List[CompatibilityCheck] = []

    def check_python_version(self, min_version: str = "3.9",
max_version
    : str = "3.12") -> CompatibilityCheck:
```

```
        current = f"{sys.version_info.major}.{sys.version_info.min
or}"
        compatible = tuple(map(int, current.split('.'))) >=
tuple(map(in
    t, min_version.split('.')))
        check = CompatibilityCheck(
            name="Python版本检查",
            dimension="python_version",
            expected=f">={min_version}",
            actual=current,
            compatible=compatible
        )
        self.checks.append(check)
        return check

    def check_os_compatibility(self, supported_os: List[str] =
None) ->
    CompatibilityCheck:
        if supported_os is None:
            supported_os = ["Linux", "Windows", "Darwin"]
        current = platform.system()
        compatible = current in supported_os
        check = CompatibilityCheck(
            name="操作系统兼容",
            dimension="os",
            expected=str(supported_os),
            actual=current,
            compatible=compatible
        )
        self.checks.append(check)
        return check

    def check_platform_arch(self, supported: List[str] = None) ->
Compat
    ibilityCheck:
        if supported is None:
            supported = ["x86_64", "AMD64", "arm64", "aarch64"]
        current = platform.machine()
        compatible = current.lower() in [s.lower() for s in
supported]
        check = CompatibilityCheck(
            name="硬件架构兼容",
            dimension="arch",
            expected=str(supported),
```

```
            actual=current,
            compatible=compatible
        )
        self.checks.append(check)
        return check

    def check_dependency_version(self, package: str, min_version: str, max_version: str = None) -> CompatibilityCheck:
        try:
            current = version(package)
            compatible = parse_version(current) >= parse_version(min_version)
            if max_version and compatible:
                compatible = parse_version(current) <= parse_version(max_version)
        except PackageNotFoundError:
            current = "NOT_INSTALLED"
            compatible = False

        check = CompatibilityCheck(
            name=f"依赖检查: {package}",
            dimension="dependency",
            expected=f">={min_version}" + (f", <={max_version}" if max_version else ""),
            actual=current,
            compatible=compatible
        )
        self.checks.append(check)
        return check

    def check_all_dependencies(self, dep_matrix: Dict[str, List[str]] = None) -> List[CompatibilityCheck]:
        if dep_matrix is None:
            dep_matrix = {
                'numpy': ['1.24.0'], 'scipy': ['1.10.0'],
                'pandas': ['2.0.0'], 'matplotlib': ['3.7.0'],
                'flask': ['2.3.0'], 'pyyaml': ['6.0'],
                'requests': ['2.31.0'], 'psutil': ['5.9.0']
            }
```

```
119|
120|        results = []
121|        for pkg, versions in dep_matrix.items():
122|            min_ver = versions[0]
123|            check = self.check_dependency_version(pkg, min_ver)
124|            results.append(check)
125|        return results
126|
127|    def check_api_compatibility(self, module_name: str,
required_functio
   |    ns: List[str]) -> List[CompatibilityCheck]:
128|        results = []
129|        try:
130|            mod = importlib.import_module(module_name)
131|            for func_name in required_functions:
132|                exists = hasattr(mod, func_name)
133|                results.append(CompatibilityCheck(
134|                  name=f"API检查: {module_name}.{func_name}",
135|                    dimension="api",
136|                    expected="exists",
137|                    actual="exists" if exists else "not_found",
138|                    compatible=exists
139|                ))
140|        except ImportError:
141|            results.append(CompatibilityCheck(
142|               name=f"模块导入: {module_name}",
143|                dimension="api",
144|                expected="importable",
145|                actual="not_found",
146|                compatible=False
147|            ))
148|        self.checks.extend(results)
149|        return results
150|
151|    def check_import_all_modules(self) ->
List[CompatibilityCheck]:
152|        modules = [
153|            'src.algorithms.greedy_algorithm',
154|            'src.algorithms.genetic_algorithm',
155|            'src.algorithms.round_robin',
156|            'src.models.server',
157|            'src.models.task',
158|            'src.simulation.datacenter_simulator',
159|            'src.plugin.base',
```

```
            'src.plugin.manager',
            'src.plugin.simulation_plugin',
            'src.utils.config',
            'src.utils.logger',
            'src.sdk.client',
        ]

        results = []
        for mod_name in modules:
            try:
                importlib.import_module(mod_name)
                results.append(CompatibilityCheck(
                  name=f"导入: {mod_name}",
                    dimension="module_import",
                    expected="importable",
                    actual="importable",
                    compatible=True
                ))
            except ImportError as e:
                results.append(CompatibilityCheck(
                  name=f"导入: {mod_name}",
                    dimension="module_import",
                    expected="importable",
                    actual=f"failed: {str(e)[:80]}",
                    compatible=False
                ))
        self.checks.extend(results)
        return results

    def get_summary(self) -> Dict[str, Any]:
        total = len(self.checks)
        passed = sum(1 for c in self.checks if c.compatible)
        failed = total - passed

        by_dimension = {}
        for c in self.checks:
            if c.dimension not in by_dimension:
                by_dimension[c.dimension] = {'total': 0, '
passed': 0}
            by_dimension[c.dimension]['total'] += 1
            if c.compatible:
                by_dimension[c.dimension]['passed'] += 1

        return {
```

```
            'total_checks': total,
            'passed': passed,
            'failed': failed,
            'compatibility_score': passed / max(total, 1) * 100,
            'by_dimension': by_dimension
        }
```

64. src/testing/environment_manager.py

```
"""
环境重置与恢复模块

实现测试环境的快速重置与恢复机制:
- 状态快照 (Snapshot)
- 快速回滚 (Rollback)
- 自动清理 (Auto-cleanup)
- 环境隔离 (Isolation)

参考项目: Docker Snapshot, Vagrant, Terraform
"""

import os
import json
import time
import shutil
import threading
from typing import Dict, Any, List, Optional, Tuple
from dataclasses import dataclass, field
from datetime import datetime
from enum import Enum
from src.utils.logger import setup_logger
from src.utils.config import Config

logger = setup_logger("environment_manager")
config = Config()

class EnvironmentState(Enum):
    CLEAN = "clean"
    INITIALIZED = "initialized"
    DIRTY = "dirty"
    RESTORING = "restoring"
    SNAPSHOTTING = "snapshotting"

@dataclass
class Snapshot:
```

```
    snapshot_id: str
    timestamp: str
    state: Dict[str, Any]
    description: str
    size_bytes: int = 0

class EnvironmentManager:
    """
    环境管理器

    提供测试环境快照/回滚能力:
    1. 创建快照 - 保存当前完整状态
    2. 恢复快照 - 一键回滚到历史状态
    3. 清理环境 - 重置到初始状态
    4. 状态对比 - 比较环境变更
    """

    def __init__(self, snapshot_dir: str = "test_snapshots"):
        self.snapshot_dir = snapshot_dir
        self.max_snapshots = 10
        self.current_state = EnvironmentState.CLEAN
        self._snapshots: List[Snapshot] = []
        self._state_data: Dict[str, Any] = {}
        self._lock = threading.Lock()

        os.makedirs(snapshot_dir, exist_ok=True)

    def save_state(self, key: str, value: Any) -> None:
        with self._lock:
            self._state_data[key] = value
            self.current_state = EnvironmentState.DIRTY

    def get_state(self, key: str, default: Any = None) -> Any:
        return self._state_data.get(key, default)

    def create_snapshot(self, description: str = "") -> Snapshot:
        timestamp = datetime.now()
        snapshot_id = f"snapshot_{timestamp.strftime('%Y%m%d_%H%M%
S_%f')
    }"

        self.current_state = EnvironmentState.SNAPSHOTTING

        snapshot_data = {
```

```
            'state': dict(self._state_data),
            'meta': {
                'description': description,
                'created_at': timestamp.isoformat(),
                'snapshot_id': snapshot_id
            }
        }

        filepath = os.path.join(self.snapshot_dir,
f"{snapshot_id}.json"
    )
        with open(filepath, 'w', encoding='utf-8') as f:
            json.dump(snapshot_data, f, indent=2, default=str,
ensure_as
    cii=False)

        size = os.path.getsize(filepath)
        snapshot = Snapshot(
            snapshot_id=snapshot_id,
            timestamp=timestamp.isoformat(),
            state=dict(self._state_data),
            description=description,
            size_bytes=size
        )

        with self._lock:
            self._snapshots.append(snapshot)
            if len(self._snapshots) > self.max_snapshots:
                oldest = self._snapshots.pop(0)
                old_path = os.path.join(self.snapshot_dir,
f"{oldest.sna
    pshot_id}.json")
                if os.path.exists(old_path):
                    os.remove(old_path)

        self.current_state = EnvironmentState.INITIALIZED
        logger.info(f"Snapshot created: {snapshot_id}")
        return snapshot

    def restore_snapshot(self, snapshot_id: str = None) -> bool:
        self.current_state = EnvironmentState.RESTORING

        if snapshot_id is None and self._snapshots:
            snapshot_id = self._snapshots[-1].snapshot_id
```

```
117|
118|        filepath = os.path.join(self.snapshot_dir,
f"{snapshot_id}.json"
   |    )
119|        if not os.path.exists(filepath):
120|            logger.error(f"Snapshot not found: {snapshot_id}")
121|            return False
122|
123|        try:
124|            with open(filepath, 'r', encoding='utf-8') as f:
125|                snapshot_data = json.load(f)
126|
127|            with self._lock:
128|                self._state_data = dict(snapshot_data['state'])
129|
130|            logger.info(f"Restored snapshot: {snapshot_id}")
131|            self.current_state = EnvironmentState.INITIALIZED
132|            return True
133|        except Exception as e:
134|            logger.error(f"Restore failed: {e}")
135|            self.current_state = EnvironmentState.DIRTY
136|            return False
137|
138|    def reset_to_clean(self) -> None:
139|        with self._lock:
140|            self._state_data.clear()
141|
142|        self.current_state = EnvironmentState.CLEAN
143|        logger.info("Environment reset to clean state")
144|
145|    def cleanup_snapshots(self, keep_last: int = 5) -> int:
146|        removed = 0
147|        with self._lock:
148|            while len(self._snapshots) > keep_last:
149|                oldest = self._snapshots.pop(0)
150|                path = os.path.join(self.snapshot_dir,
f"{oldest.snapsho
   |    t_id}.json")
151|                if os.path.exists(path):
152|                    os.remove(path)
153|                    removed += 1
154|
155|        logger.info(f"Cleaned up {removed} old snapshots")
156|        return removed
```

```
157|
158|    def list_snapshots(self) -> List[Dict[str, Any]]:
159|        return [{
160|            'id': s.snapshot_id,
161|            'timestamp': s.timestamp,
162|            'description': s.description,
163|            'size_kb': s.size_bytes / 1024
164|        } for s in self._snapshots]
165|
166|    def compare_snapshots(self, id1: str, id2: str) -> Dict[str,
Any]:
167|        file1 = os.path.join(self.snapshot_dir, f"{id1}.json")
168|        file2 = os.path.join(self.snapshot_dir, f"{id2}.json")
169|
170|        if not os.path.exists(file1) or not os.path.exists(file2):
171|            return {'error': 'One or both snapshots not found'}
172|
173|        with open(file1, 'r') as f:
174|            data1 = json.load(f)['state']
175|        with open(file2, 'r') as f:
176|            data2 = json.load(f)['state']
177|
178|        keys1, keys2 = set(data1.keys()), set(data2.keys())
179|        added = keys2 - keys1
180|        removed = keys1 - keys2
181|        common = keys1 & keys2
182|
183|        changed = [k for k in common if data1[k] != data2[k]]
184|
185|        return {
186|            'added_keys': list(added),
187|            'removed_keys': list(removed),
188|            'changed_keys': changed,
189|            'unchanged_keys': list(common - set(changed))
190|        }
```

65. src/testing/mock_services.py

```
1|"""
2|仿真模拟测试环境 - Mock服务组件
3|
4|提供可配置的模拟服务组件，实现对各类接口和协议的模拟支持:
5|- 网络拓扑模拟（延迟/带宽/丢包/抖动）
6|- 硬件配置模拟（CPU/RAM/Disk/NIC）
7|- 协议Mock（HTTP/gRPC/WebSocket）
```

```
- 数据流量模拟（正常/峰值/突发/DDoS）
"""

import random
import time
import json
import threading
import queue
from typing import Dict, Any, List, Optional, Callable, Tuple
from dataclasses import dataclass, field
from enum import Enum
from datetime import datetime
from src.utils.logger import setup_logger
from src.utils.config import Config

logger = setup_logger("mock_services")
config = Config()

class NetworkCondition(Enum):
    PERFECT = "perfect"
    NORMAL = "normal"
    DEGRADED = "degraded"
    CONGESTED = "congested"
    DISCONNECTED = "disconnected"

@dataclass
class NetworkRegion:
    name: str
    latency_ms: float
    bandwidth_mbps: float
    packet_loss: float
    jitter_ms: float

@dataclass
class HardwareProfile:
    name: str
    cpu_cores: int
    memory_gb: int
    disk_gb: int
    nic_speed_mbps: int
    gpu: Optional[str]
    npu: Optional[str]
    os: str
    cpu_usage: float = 0.0
```

```
    memory_usage: float = 0.0
    disk_usage: float = 0.0

@dataclass
class TrafficStats:
    requests_total: int = 0
    requests_success: int = 0
    requests_failed: int = 0
    bytes_sent: int = 0
    bytes_received: int = 0
    avg_response_ms: float = 0.0
    max_response_ms: float = 0.0
    min_response_ms: float = float('inf')
    error_rate: float = 0.0

class NetworkSimulator:
    """
    网络拓扑模拟器

    模拟Spine-Leaf数据中心网络拓扑:
           [Spine1]       [Spine2]
           /     \       /     \
      [Leaf1]  [Leaf2] [Leaf3] [Leaf4]
         |        |       |       |
      [Server] [Server] [Server] [Server]

    参考项目: Mininet, NS-3
    """

    def __init__(self, regions: List[Dict] = None):
        self.regions: List[NetworkRegion] = []
        self.condition = NetworkCondition.NORMAL
        self._latency_multiplier = 1.0
        self._packet_loss_multiplier = 1.0
        self._blocked_regions: set = set()

        if regions:
            for r in regions:
                self.regions.append(NetworkRegion(**r))
        else:
            cfg_regions = config.network.get('regions', []) if
hasattr(c
    onfig, 'network') else []
            for r in cfg_regions:
```

```
                self.regions.append(NetworkRegion(**r))

    def set_condition(self, condition: NetworkCondition) -> None:
        self.condition = condition
        multipliers = {
            NetworkCondition.PERFECT: (0.5, 0.0),
            NetworkCondition.NORMAL: (1.0, 1.0),
            NetworkCondition.DEGRADED: (2.0, 5.0),
            NetworkCondition.CONGESTED: (5.0, 10.0),
            NetworkCondition.DISCONNECTED: (1000.0, 1.0),
        }
        self._latency_multiplier, self._packet_loss_multiplier =
multipl
    iers[condition]
        logger.info(f"Network condition set to {condition.value}")

    def get_latency(self, region_name: str) -> float:
        if region_name in self._blocked_regions:
            return 9999.0
        for r in self.regions:
            if r.name == region_name:
                base = r.latency_ms + random.gauss(0, r.jitter_ms)
                return max(0, base * self._latency_multiplier)
        return 1.0 * self._latency_multiplier

    def simulates_packet_loss(self, region_name: str) -> bool:
        if region_name in self._blocked_regions:
            return True
        for r in self.regions:
            if r.name == region_name:
                return random.random() < r.packet_loss *
self._packet_lo
    ss_multiplier
        return False

    def get_bandwidth(self, region_name: str) -> float:
        for r in self.regions:
            if r.name == region_name:
                if self.condition == NetworkCondition.CONGESTED:
                    return r.bandwidth_mbps * 0.1
                return r.bandwidth_mbps
        return 1000.0

    def block_region(self, region_name: str) -> None:
```

```
        self._blocked_regions.add(region_name)
        logger.warning(f"Region blocked: {region_name}")

    def unblock_region(self, region_name: str) -> None:
        self._blocked_regions.discard(region_name)

    def simulate_transfer(self, data_size_bytes: int,
region_name: str)
    -> Tuple[float, bool]:
        bw = self.get_bandwidth(region_name) * 1e6 / 8
        transfer_time = data_size_bytes / max(bw, 1)

        latency = self.get_latency(region_name) / 1000
        total_time = transfer_time + latency * 2

        lost = self.simulates_packet_loss(region_name)

        return total_time, not lost


class HardwareProfileSimulator:
    """
    硬件配置模拟器

    模拟不同硬件平台的计算能力:
    - 鲲鹏920 + 昇腾310 (国产化)
    - 标准x86 + NVIDIA A100
    - 高性能GPU节点 (H100)
    - 边缘计算节点

    参考项目: NVIDIA DCGM, Prometheus Node Exporter
    """

    def __init__(self, profiles: List[Dict] = None):
        self.profiles: List[HardwareProfile] = []
        if profiles:
            for p in profiles:
                self.profiles.append(HardwareProfile(**p))
        else:
            cfg_profiles = config.hardware_profiles if
hasattr(config, '
    hardware_profiles') else []
            for p in cfg_profiles:
                self.profiles.append(HardwareProfile(**p))
```

```
174|
175|    def get_profile(self, name: str) -> Optional[HardwareProfile]:
176|        for p in self.profiles:
177|            if p.name == name:
178|                return p
179|        return None
180|
181|    def simulate_cpu_load(self, profile: HardwareProfile,
load_pct: floa
   |    t) -> float:
182|        profile.cpu_usage = min(100.0, max(0.0, load_pct))
183|        return profile.cpu_cores * (1.0 - profile.cpu_usage /
100.0)
184|
185|    def simulate_memory_usage(self, profile: HardwareProfile,
usage_gb:
   |    float) -> Tuple[float, bool]:
186|        profile.memory_usage = usage_gb
187|        pct = usage_gb / profile.memory_gb * 100.0
188|        return pct, pct < 90.0
189|
190|    def simulate_disk_io(self, profile: HardwareProfile,
read_mbps: floa
   |    t, write_mbps: float) -> float:
191|        total = read_mbps + write_mbps
192|        return min(100.0, total / 1000.0 * 100.0)
193|
194|    def estimate_task_capacity(self, profile: HardwareProfile) ->
int:
195|        base = profile.cpu_cores * 2
196|        if profile.gpu:
197|            base *= 3
198|        if profile.npu:
199|            base *= 2
200|        return int(base)
201|
202|    def get_all_status(self) -> List[Dict[str, Any]]:
203|        return [
204|            {
205|                'name': p.name, 'cpu_usage': p.cpu_usage,
206|                'memory_usage': p.memory_usage, 'disk_usage':
p.disk_usa
   |    ge,
```

```
                'available_cores': p.cpu_cores * (1 - p.cpu_usage
/ 100)
    ,
                'available_memory': p.memory_gb * (1 -
p.memory_usage /
    100)
            } for p in self.profiles
        ]


class ProtocolMockServer:
    """
    协议模拟服务器

    模拟各类API接口行为:
    - HTTP REST API (正常/延迟/错误响应)
    - WebSocket (实时推送)
    - 自定义协议

    参考项目: WireMock, Mockoon, Postman Mock Server
    """

    def __init__(self, base_url: str = "http://localhost:5000"):
        self.base_url = base_url
        self._mocks: Dict[str, Dict[str, Any]] = {}
        self._response_delay = 0.0
        self._error_rate = 0.0
        self._request_log: List[Dict] = []
        self._callbacks: Dict[str, Callable] = {}

    def register_mock(self, path: str, method: str = "GET",
                      response: Any = None, status_code: int =
200,
                      headers: Dict = None, delay_sec: float =
0.0) -> N
    one:
        key = f"{method}:{path}"
        self._mocks[key] = {
            'response': response or {},
            'status_code': status_code,
            'headers': headers or {'Content-Type': '
application/json'},
            'delay': delay_sec
        }
```

```

    def set_error_injection(self, rate: float) -> None:
        self._error_rate = max(0.0, min(1.0, rate))

    def set_response_delay(self, delay_sec: float) -> None:
        self._response_delay = delay_sec

    def mock_request(self, method: str, path: str, data: Any =
None) ->
    Dict[str, Any]:
        key = f"{method}:{path}"
        self._request_log.append({
            'method': method, 'path': path, 'data': data,
            'timestamp': datetime.now().isoformat()
        })

        if random.random() < self._error_rate:
            return {'status_code': 500, 'body': {'error': '
Injected serv
    er error'}}

        mock = self._mocks.get(key)
        if not mock:
            return {'status_code': 404, 'body': {'error': f'No
mock for
    {key}'}}

        if mock['delay'] > 0 or self._response_delay > 0:
            time.sleep(max(mock['delay'], self._response_delay))

        return {
            'status_code': mock['status_code'],
            'body': mock['response']() if
callable(mock['response']) els
    e mock['response'],
            'headers': mock['headers']
        }

    def register_callback(self, path: str, callback: Callable) ->
None:
        self._callbacks[path] = callback

    def get_request_log(self, limit: int = 100) -> List[Dict]:
        return self._request_log[-limit:]
```

```
278|
279|    def clear_request_log(self) -> None:
280|        self._request_log.clear()
281|
282|
283|class TrafficGenerator:
284|    """
285|    数据流量生成器
286|
287|    模拟不同负载场景:
288|    - 正常负载（日常运行）
289|    - 潮汐峰值（工作时间高峰）
290|    - 突发流量（瞬时高并发）
291|    - 持续高压（长时间高负载）
292|    - DDoS模拟（攻击流量）
293|
294|    参考项目: Apache JMeter, Locust, wrk2
295|    """
296|
297|    def __init__(self, patterns: List[Dict] = None):
298|        self.patterns: List[Dict] = patterns or []
299|        self._active = False
300|        self._threads: List[threading.Thread] = []
301|        self.stats = TrafficStats()
302|
303|    def load_config_patterns(self) -> None:
304|        cfg_patterns = config.traffic_patterns if hasattr(config, '
traff
   |    ic_patterns') else []
305|        for p in cfg_patterns:
306|            self.patterns.append(p)
307|
308|    def generate_traffic(self, pattern_name: str, target_api: str,
309|                          request_func: Callable) -> TrafficStats:
310|        pattern = None
311|        for p in self.patterns:
312|            if p['name'] == pattern_name:
313|                pattern = p
314|                break
315|
316|        if not pattern:
317|            raise ValueError(f"Unknown pattern: {pattern_name}")
318|
319|        rps = pattern['requests_per_sec']
```

```
        duration = pattern['duration_minutes'] * 60
        stats = TrafficStats()

        start_time = time.time()
        request_count = 0

        while time.time() - start_time < duration:
            batch = min(rps, 100)
            for _ in range(batch):
                req_start = time.time()
                try:
                    result = request_func()
                    stats.requests_total += 1
                    if result:
                        stats.requests_success += 1
                    else:
                        stats.requests_failed += 1
                except Exception:
                    stats.requests_failed += 1

                elapsed = (time.time() - req_start) * 1000
                stats.avg_response_ms = (stats.avg_response_ms *
request
    |    _count + elapsed) / (request_count + 1)
                stats.max_response_ms =
max(stats.max_response_ms, elaps
    |    ed)
                stats.min_response_ms =
min(stats.min_response_ms, elaps
    |    ed)
                request_count += 1

            time.sleep(1.0 / max(rps / batch, 1))

        stats.error_rate = stats.requests_failed /
max(stats.requests_to
    |    tal, 1)
        self.stats = stats
        return stats

    def generate_burst(self, num_requests: int, target_api: str,
                       request_func: Callable) -> TrafficStats:
        stats = TrafficStats()
        start = time.time()
```

```
356|
357|        for i in range(num_requests):
358|            req_start = time.time()
359|            try:
360|                request_func()
361|                stats.requests_success += 1
362|            except Exception:
363|                stats.requests_failed += 1
364|            stats.requests_total += 1
365|
366|            elapsed = (time.time() - req_start) * 1000
367|            stats.max_response_ms = max(stats.max_response_ms,
elapsed)
368|            stats.min_response_ms = min(stats.min_response_ms,
elapsed)
369|
370|        elapsed_total = (time.time() - start)
371|        if elapsed_total > 0:
372|            stats.avg_response_ms = (elapsed_total /
num_requests) * 100
   |    0
373|        stats.error_rate = stats.requests_failed /
max(stats.requests_to
   |    tal, 1)
374|        self.stats = stats
375|        return stats
```

66. src/testing/risk_assessor.py

```
  1|"""
  2|风险评估模块
  3|
  4|对测试结果进行综合风险评估:
  5|- 漏洞严重性评估 (CVSS)
  6|- 风险矩阵生成
  7|- 影响范围分析
  8|- 修复优先级排序
  9|
 10|参考项目: NIST CVSS Calculator, OWASP Risk Rating
 11|"""
 12|
 13|from typing import Dict, Any, List, Optional, Tuple
 14|from dataclasses import dataclass, field
 15|from enum import Enum
 16|from datetime import datetime
```

```
from src.utils.logger import setup_logger
from src.utils.config import Config

logger = setup_logger("risk_assessor")
config = Config()

class RiskLevel(Enum):
    CRITICAL = "critical"
    HIGH = "high"
    MEDIUM = "medium"
    LOW = "low"
    ACCEPTABLE = "acceptable"

@dataclass
class RiskItem:
    category: str
    description: str
    likelihood: float  # 0.0-1.0
    impact: float      # 0.0-1.0
    risk_score: float   # likelihood * impact
    risk_level: RiskLevel
    affected_components: List[str]
    mitigation: str

@dataclass
class RiskReport:
    report_id: str
    timestamp: str = field(default_factory=lambda:
datetime.now().isofor
    mat())
    overall_risk: RiskLevel = RiskLevel.LOW
    risk_items: List[RiskItem] = field(default_factory=list)
    safety_score: float = 100.0
    recommendations: List[str] = field(default_factory=list)

class RiskAssessor:
    """
    风险评估器

    综合评估系统安全风险:
    1. 漏洞风险等级评定
    2. 影响范围分析
    3. 风险矩阵可视化
    4. 修复建议排序
```

```
    """
    def __init__(self):
        self.risks: List[RiskItem] = []
        self._cvss_threshold = 7.0
    def assess_from_security_scan(self, vulnerabilities:
List[Any]) -> L
    ist[RiskItem]:
        severity_to_likelihood = {
            'critical': 0.9,
            'high': 0.7,
            'medium': 0.4,
            'low': 0.2,
            'info': 0.1
        }
        severity_to_impact = {
            'critical': 0.95,
            'high': 0.8,
            'medium': 0.5,
            'low': 0.3,
            'info': 0.1
        }
        for vuln in vulnerabilities:
            severity = getattr(vuln, 'severity', None)
            if severity:
                sev_str = severity.value if hasattr(severity, '
value') e
    lse str(severity)
                likelihood = severity_to_likelihood.get(sev_str,
0.5)
                impact = severity_to_impact.get(sev_str, 0.5)
            else:
                likelihood = 0.5
                impact = 0.5
            score = likelihood * impact
            level = self._score_to_level(score)
            risk = RiskItem(
                category=getattr(vuln, 'scan_type', '
unknown').value if
```

```
   |    hasattr(vuln.scan_type, 'value') else str(getattr(vuln, '
scan_type',
   |     'unknown')),
 96|                description=getattr(vuln, 'title', 'Unknown'),
 97|                likelihood=likelihood,
 98|                impact=impact,
 99|                risk_score=score,
100|                risk_level=level,
101|                affected_components=[getattr(vuln, '
affected_component',
   |     'unknown')],
102|                mitigation=getattr(vuln, 'remediation', 'No
mitigation a
   |    vailable')
103|            )
104|            self.risks.append(risk)
105|
106|        return self.risks
107|
108|    def assess_from_stress_test(self, metrics) -> RiskItem:
109|        violations = getattr(metrics, 'threshold_violations', [])
110|        error_rate = 1.0 - getattr(metrics, 'success_rate', 1.0)
111|        peak_cpu = getattr(metrics, 'peak_cpu_pct', 0)
112|
113|        if violations:
114|            likelihood = min(1.0, error_rate * 10 + peak_cpu /
200)
115|            impact = 0.7
116|            score = likelihood * impact
117|            level = self._score_to_level(score)
118|        else:
119|            return RiskItem(
120|                category='performance',
121|               description='系统性能在可接受范围内',
122|                likelihood=0.1, impact=0.2, risk_score=0.02,
123|                risk_level=RiskLevel.ACCEPTABLE,
124|                affected_components=['simulation_engine'],
125|               mitigation='无需处理'
126|            )
127|
128|        risk = RiskItem(
129|            category='performance',
130|           description=f"系统超阈值: {len(violations)}项",
131|            likelihood=likelihood,
```

```
            impact=impact,
            risk_score=score,
            risk_level=level,
            affected_components=['simulation_engine', '
api_server'],
            mitigation='优化算法性能，增加资源分配'
        )
        self.risks.append(risk)
        return risk

    def assess_from_compatibility(self, compat_summary: Dict[str,
Any])
    -> RiskItem:
        score_pct = compat_summary.get('compatibility_score', 100)

        if score_pct >= 95:
            level = RiskLevel.ACCEPTABLE
            likelihood, impact = 0.05, 0.1
        elif score_pct >= 80:
            level = RiskLevel.LOW
            likelihood, impact = 0.2, 0.3
        elif score_pct >= 60:
            level = RiskLevel.MEDIUM
            likelihood, impact = 0.4, 0.5
        else:
            level = RiskLevel.HIGH
            likelihood, impact = 0.7, 0.8

        score = likelihood * impact
        risk = RiskItem(
            category='compatibility',
            description=f"兼容性评分: {score_pct:.1f}%",
            likelihood=likelihood,
            impact=impact,
            risk_score=score,
            risk_level=level,
            affected_components=['all_modules'],
            mitigation='修复兼容性问题，更新依赖版本'
        )
        self.risks.append(risk)
        return risk

    def _score_to_level(self, score: float) -> RiskLevel:
        if score >= 0.8:
```

```
            return RiskLevel.CRITICAL
        elif score >= 0.6:
            return RiskLevel.HIGH
        elif score >= 0.3:
            return RiskLevel.MEDIUM
        elif score >= 0.1:
            return RiskLevel.LOW
        return RiskLevel.ACCEPTABLE

    def generate_report(self) -> RiskReport:
        if not self.risks:
            return RiskReport(
                report_id=f"RISK-{datetime.now().strftime('%Y%m%d%H%M')}",
                overall_risk=RiskLevel.LOW,
                safety_score=100.0
            )

        by_level = {level: 0 for level in RiskLevel}
        total_score = 0.0

        for r in self.risks:
            by_level[r.risk_level] += 1
            total_score += r.risk_score

        if by_level[RiskLevel.CRITICAL] > 0:
            overall = RiskLevel.CRITICAL
        elif by_level[RiskLevel.HIGH] > 0:
            overall = RiskLevel.HIGH
        elif by_level[RiskLevel.MEDIUM] > 2:
            overall = RiskLevel.MEDIUM
        elif by_level[RiskLevel.MEDIUM] > 0:
            overall = RiskLevel.LOW
        else:
            overall = RiskLevel.ACCEPTABLE

        safety_score = max(0.0, 100.0 - total_score * 100 / max(len(self.risks), 1))

        recommendations = []
        for r in self.risks:
```

```
212|            if r.risk_level in (RiskLevel.CRITICAL,
RiskLevel.HIGH):
213|                recommendations.insert(0, f"[紧急] {r.description}: {r.m
   |    itigation}")
214|            elif r.risk_level == RiskLevel.MEDIUM:
215|                recommendations.append(f"[建议] {r.description}: {r.miti
   |    gation}")
216|
217|        report = RiskReport(
218|            report_id=f"RISK-{datetime.now().strftime('%Y%m%d%H%M')
}",
219|            overall_risk=overall,
220|            risk_items=self.risks,
221|            safety_score=safety_score,
222|            recommendations=recommendations
223|        )
224|
225|        logger.info(f"Risk report generated: {overall.value},
safety_sco
   |    re={safety_score:.1f}")
226|        return report
```

67. src/testing/security_report.py

```
  1|"""
  2|安全评估报告生成器
  3|
  4|生成符合行业标准的安全评估报告:
  5|- NIST CVSS评分
  6|- OWASP Top 10对照
  7|- 修复优先级排序
  8|"""
  9|
 10|import json
 11|import os
 12|from typing import Dict, Any, List, Optional
 13|from datetime import datetime
 14|from dataclasses import dataclass
 15|
 16|@dataclass
 17|class SecurityReport:
 18|    report_id: str
 19|    timestamp: str
 20|    scan_summary: Dict[str, Any]
 21|    vulnerability_list: List[Dict[str, Any]]
```

```
    risk_assessment: Dict[str, Any]
    compliance_checklist: Dict[str, Any]
    recommendations: List[Dict[str, Any]]


def generate_security_report(
    vulnerabilities: List[Any],
    scan_summary: Dict[str, Any],
    risk_assessment: Dict[str, Any],
    output_path: str = None
) -> str:
    """
    生成安全评估报告

    Args:
        vulnerabilities: 漏洞列表
        scan_summary: 扫描摘要
        risk_assessment: 风险评估
        output_path: 输出路径

    Returns:
        报告文件路径
    """

    report = {
        'meta': {
            'report_id': f"SEC-{datetime.now().strftime('%Y%m%d-%H
%M%S')
    }",
            'timestamp': datetime.now().isoformat(),
            'generator': 'DataCenter-Scheduler Security Scanner',
            'version': '1.0.0'
        },
        'executive_summary': {
            'total_vulnerabilities': scan_summary.get('total', 0),
            'critical': scan_summary.get('critical_count', 0),
            'high': scan_summary.get('high_count', 0),
            'medium': scan_summary.get('by_severity',
{}).get('medium',
    0),
            'low': scan_summary.get('by_severity', {}).get('low',
0),
            'risk_level': risk_assessment.get('overall_risk', '
unknown')
```

```
    |    ,
 60|            'safety_score': risk_assessment.get('safety_score',
0),
 61|            'needs_immediate_action':
scan_summary.get('critical_count',
    |     0) > 0
 62|        },
 63|        'vulnerability_details': [],
 64|        'scan_coverage': {
 65|            'dependency_check': scan_summary.get('by_type',
{}).get('dep
    |    endency_check', 0) > 0,
 66|            'port_scan': scan_summary.get('by_type',
{}).get('port_scan'
    |    , 0) > 0,
 67|            'code_sast': scan_summary.get('by_type',
{}).get('code_sast'
    |    , 0) > 0,
 68|            'api_fuzzing': scan_summary.get('by_type',
{}).get('api_fuzz
    |    ing', 0) > 0,
 69|            'injection_test': scan_summary.get('by_type',
{}).get('injec
    |    tion_test', 0) > 0,
 70|            'auth_bypass': scan_summary.get('by_type',
{}).get('auth_byp
    |    ass', 0) > 0
 71|        },
 72|        'owasp_top10_checklist': {
 73|            'A01_Broken_Access_Control':
scan_summary.get('by_type', {})
    |    .get('auth_bypass', 0) > 0,
 74|            'A02_Cryptographic_Failures': False,
 75|            'A03_Injection': scan_summary.get('by_type',
{}).get('inject
    |    ion_test', 0) > 0,
 76|            'A05_Security_Misconfiguration':
scan_summary.get('by_type',
    |     {}).get('port_scan', 0) > 0,
 77|            'A06_Vulnerable_Components':
scan_summary.get('by_type', {})
    |    .get('dependency_check', 0) > 0,
 78|            'A08_Software_Integrity_Failures':
scan_summary.get('by_type
```

```
    |    ', {}).get('code_sast', 0) > 0,
 79|        },
 80|        'recommendations': risk_assessment.get('recommendations',
[]),
 81|        'remediation_plan': [
 82|            {
 83|                'priority': 'P0-紧急',
 84|                'condition': '存在CRITICAL级别漏洞',
 85|                'action': '立即停止部署，修复所有严重漏洞',
 86|                'timeline': '24小时内'
 87|            },
 88|            {
 89|                'priority': 'P1-高',
 90|                'condition': '存在HIGH级别漏洞',
 91|                'action': '制定修复计划，在下一版本中修复',
 92|                'timeline': '1周内'
 93|            },
 94|            {
 95|                'priority': 'P2-中',
 96|                'condition': '存在MEDIUM级别漏洞',
 97|                'action': '纳入迭代修复计划',
 98|                'timeline': '2周内'
 99|            },
100|            {
101|                'priority': 'P3-低',
102|                'condition': '安全评分<80',
103|                'action': '持续优化安全配置',
104|                'timeline': '持续'
105|            }
106|        ],
107|        'sign_off': {
108|            'scanner': 'DataCenter Scheduler Security Scanner
v1.0',
109|            'scanned_by': 'Automated Pipeline',
110|            'review_required': scan_summary.get('critical_count',
0) > 0
    |     or scan_summary.get('high_count', 0) > 2
111|        }
112|    }
113|
114|    # 填充漏洞详情
115|    for vuln in vulnerabilities:
116|        report['vulnerability_details'].append({
117|            'id': getattr(vuln, 'vuln_id', ''),
```

```
            'title': getattr(vuln, 'title', ''),
            'description': getattr(vuln, 'description', ''),
            'severity': getattr(vuln, 'severity', '
unknown').value if ha
    sattr(getattr(vuln, 'severity', None), 'value') else
str(getattr(vul
    n, 'severity', 'unknown')),
            'cve_id': getattr(vuln, 'cve_id', None),
            'cvss_score': getattr(vuln, 'cvss_score', None),
            'affected_component': getattr(vuln, '
affected_component', ''
    ),
            'remediation': getattr(vuln, 'remediation', '')
        })

    # 写入文件
    if output_path is None:
        os.makedirs('test_reports', exist_ok=True)
        output_path = f"test_reports/security_report_{report['meta'
]['re
    port_id']}.json"

    with open(output_path, 'w', encoding='utf-8') as f:
        json.dump(report, f, indent=2, ensure_ascii=False,
default=str)

    return output_path


def print_report_summary(report_data: Dict[str, Any]) -> None:
    """打印报告摘要"""
    import logging
    logger = logging.getLogger("security_report")

    exec_summary = report_data.get('executive_summary', {})
    sign_off = report_data.get('sign_off', {})
    recs = report_data.get('recommendations', [])

    logger.info("=" * 60)
    logger.info("  安全评估报告")
    logger.info("=" * 60)
    logger.info(f"  报告ID:     {report_data.get('meta', {}).get('report
    _id', 'N/A')}")
    logger.info(f"  扫描时间:   {report_data.get('meta', {}).get('timest
```

```
    |    amp', 'N/A')}")
152|    logger.info(f"  漏洞总数:   {exec_summary.get('total_vulnerabilities
    |    ', 0)}")
153|    logger.info(f"  严重/高:    {exec_summary.get('critical', 0)}/{exec_
    |    summary.get('high', 0)}")
154|    logger.info(f"  风险等级:   {exec_summary.get('risk_level', 'unknown
    |    ')}")
155|    logger.info(f"  安全评分:   {exec_summary.get('safety_score', 0):.1f
    |    }/100")
156|    logger.info(f"  需审查:     {'是   ' if sign_off.get('review_required
    |    ') else '否   '}")
157|
158|    if recs:
159|        logger.info("  修复建议:")
160|        for rec in recs[:5]:
161|            logger.info(f"    - {rec}")
162|
163|    logger.info("=" * 60)
```

68. src/testing/security_scanner.py

```
  1|"""
  2|安全漏洞扫描与渗透测试模块
  3|
  4|集成安全漏洞扫描和风险评估能力:
  5|- 依赖漏洞检测 (CVE/NVD)
  6|- 端口扫描 (开放端口检测)
  7|- 代码静态分析 (SAST)
  8|- API模糊测试 (Fuzzing)
  9|- 注入攻击测试 (SQL/XSS/Command)
 10|- 认证绕过测试
 11|- OWASP Top 10 检测
 12|
 13|参考项目: Bandit, OWASP ZAP, SQLMap, Snyk
 14|"""
 15|
 16|import os
 17|import re
 18|import json
 19|import hashlib
 20|import socket
 21|import ssl
 22|from typing import Dict, Any, List, Optional, Set, Tuple
 23|from dataclasses import dataclass, field
 24|from enum import Enum
```

```
from datetime import datetime
from src.utils.logger import setup_logger
from src.utils.config import Config

logger = setup_logger("security_scanner")
config = Config()

class SeverityLevel(Enum):
    CRITICAL = "critical"
    HIGH = "high"
    MEDIUM = "medium"
    LOW = "low"
    INFO = "info"

class ScanType(Enum):
    DEPENDENCY = "dependency_check"
    PORT_SCAN = "port_scan"
    SAST = "code_sast"
    FUZZING = "api_fuzzing"
    INJECTION = "injection_test"
    AUTH = "auth_bypass"

@dataclass
class Vulnerability:
    vuln_id: str
    title: str
    description: str
    severity: SeverityLevel
    cve_id: Optional[str]
    cvss_score: Optional[float]
    affected_component: str
    remediation: str
    scan_type: ScanType
    found_at: str = field(default_factory=lambda:
datetime.now().isoform
    at())

class SecurityScanner:
    """
    安全扫描器

    多维度安全检测:
    1. 依赖检查 - CVE漏洞库匹配
    2. 端口扫描 - 检测未授权开放端口
```

```
    3. 代码审计 - 硬编码密钥/不安全函数
    4. API模糊测试 - 异常输入处理
    5. 注入测试 - SQL/XSS/命令注入
    6. 认证测试 - 绕过/弱口令
    """

    def __init__(self):
        self.vulnerabilities: List[Vulnerability] = []
        self._scanned_files: Set[str] = set()
        self._allowed_ports = set()
        self._blocked_ports = set()
        self._load_port_config()

    def _load_port_config(self):
        try:
            self._allowed_ports =
set(config.security.get('allowed_ports
    ', [5000, 8000]))
            self._blocked_ports =
set(config.security.get('blocked_ports
    ', [22, 3306]))
        except Exception:
            self._allowed_ports = {5000, 8000, 8080}
            self._blocked_ports = {22, 3306, 5432, 6379}

    def scan_dependencies(self) -> List[Vulnerability]:
        import pkg_resources
        vulns = []

        known_vulnerable = {
            'pyyaml': {
                'version': '<6.0.1',
                'cve': 'CVE-2020-14343',
                'cvss': 7.5,
                'desc': 'YAML解析器存在不安全反序列化漏洞'
            },
            'requests': {
                'version': '<2.31.0',
                'cve': 'CVE-2023-32681',
                'cvss': 6.1,
                'desc': '代理授权头信息泄漏'
            },
            'urllib3': {
                'version': '<1.26.18',
```

```
                'cve': 'CVE-2023-45803',
                'cvss': 4.2,
                'desc': '重定向后授权信息泄漏'
            },
            'werkzeug': {
                'version': '<2.3.8',
                'cve': 'CVE-2023-46136',
                'cvss': 7.5,
                'desc': '多部分表单数据DoS漏洞'
            }
        }

        for dist in pkg_resources.working_set:
            pkg_name = dist.project_name.lower()
            if pkg_name in known_vulnerable:
                info = known_vulnerable[pkg_name]
                if pkg_resources.parse_version(dist.version) < pkg_resources.parse_version(info['version'].lstrip('<=').lstrip('<')):
                    v = Vulnerability(
                        vuln_id=f"DEP-{pkg_name.upper()}",
                        title=f"依赖漏洞: {pkg_name}",
                        description=info['desc'],
                        severity=SeverityLevel.HIGH if info['cvss'] >= 7 else SeverityLevel.MEDIUM,
                        cve_id=info['cve'],
                        cvss_score=info['cvss'],
                        affected_component=f"{pkg_name}=={dist.version}",
                        remediation=f"升级至 {pkg_name}>={info['version']}",
                        scan_type=ScanType.DEPENDENCY
                    )
                    vulns.append(v)

        self.vulnerabilities.extend(vulns)
        logger.info(f"Dependency scan found {len(vulns)} vulnerabilities")
        return vulns

```

```
    def scan_ports(self, host: str = 'localhost', ports:
List[int] = Non
    e) -> List[Vulnerability]:
        if ports is None:
            ports = list(range(1, 1025))

        vulns = []
        open_ports = []

        for port in ports:
            sock = socket.socket(socket.AF_INET,
socket.SOCK_STREAM)
            sock.settimeout(0.1)
            result = sock.connect_ex((host, port))
            if result == 0:
                open_ports.append(port)
            sock.close()

        for port in open_ports:
            if port in self._blocked_ports:
                vulns.append(Vulnerability(
                    vuln_id=f"PORT-{port}",
                  title=f"敏感端口开放: {port}",
                  description=f"端口 {port} 应被防火墙阻止但当前开放",
                    severity=SeverityLevel.HIGH,
                    cve_id=None, cvss_score=None,
                    affected_component=f"{host}:{port}",
                  remediation=f"在防火墙中阻止端口 {port}",
                    scan_type=ScanType.PORT_SCAN
                ))

        self.vulnerabilities.extend(vulns)
        logger.info(f"Port scan: {len(open_ports)} open,
{len(vulns)} vu
    lnerabilities")
        return vulns

    def scan_code_sast(self, src_dir: str = 'src') ->
List[Vulnerability
    ]:
        vulns = []
        dangerous_patterns = {
            'hardcoded_secret': (
```

```
                r'(?:password|secret|api_key|token|auth)\s*=\s*["\'
][^"\
    '\\n]{8,}["\']',
                SeverityLevel.CRITICAL,
                '硬编码密钥检测'
            ),
            'eval_usage': (
                r'\beval\s*\(',
                SeverityLevel.HIGH,
                '危险的eval()调用'
            ),
            'exec_usage': (
                r'\bexec\s*\(',
                SeverityLevel.HIGH,
                '危险的exec()调用'
            ),
            'sql_injection_risk': (
                r'(?:execute|cursor\.execute)\s*\(\s*["\'].*%(?:s|
d)',
                SeverityLevel.HIGH,
                '潜在SQL注入风险'
            ),
            'debug_mode': (
                r'debug\s*=\s*True',
                SeverityLevel.MEDIUM,
                '调试模式开启'
            ),
            'insecure_deserialization': (
                r'\bpickle\.(?:loads|load)\(',
                SeverityLevel.HIGH,
                '不安全的反序列化'
            ),
            'ssl_verify_disabled': (
                r'verify\s*=\s*False',
                SeverityLevel.MEDIUM,
                'SSL证书验证禁用'
            )
        }

        for root, dirs, files in os.walk(src_dir):
            dirs[:] = [d for d in dirs if d not in
('__pycache__', '.git
    ', 'venv', 'node_modules')]
            for file in files:
```

```
                if not file.endswith('.py'):
                    continue
                filepath = os.path.join(root, file)
                if filepath in self._scanned_files:
                    continue
                self._scanned_files.add(filepath)

                try:
                    with open(filepath, 'r', encoding='utf-8') as
f:
                        content = f.read()
                        for pattern_name, (pattern, severity,
title) in
   |    dangerous_patterns.items():
                            matches = re.finditer(pattern,
content, re.I
   |    GNORECASE)
                            for match in matches:
                                line_no =
content[:match.start()].count(
   |    '\n') + 1
                                vulns.append(Vulnerability(

vuln_id=f"SAST-{hashlib.md5(f'{filep
   |    ath}:{line_no}'.encode()).hexdigest()[:8]}",
                                    title=title,
                                    description=f"在 {filepath}:{line_no
   |    } 发现 {pattern_name}",
                                    severity=severity,
                                    cve_id=None, cvss_score=None,

affected_component=f"{filepath}:{lin
   |    e_no}",
                                    remediation=f"检查并修复 {filepath}
   |    第{line_no}行",
                                    scan_type=ScanType.SAST
                                ))
                except Exception as e:
                    logger.warning(f"SAST scan failed for
{filepath}: {e
   |    }")

        self.vulnerabilities.extend(vulns)
        logger.info(f"SAST scan found {len(vulns)} issues")
```

```
        return vulns

    def test_injection(self, api_endpoint: str, method: str = 'POST') -> List[Vulnerability]:
        import requests
        import urllib.parse

        vulns = []
        payloads = {
            'SQL注入': [
                "' OR '1'='1",
                "1; DROP TABLE simulations",
                "' UNION SELECT * FROM users --",
                "1' AND '1'='1"
            ],
            'XSS注入': [
                "<script>alert(1)</script>",
                "javascript:alert(document.cookie)",
                "<img src=x onerror=alert(1)>",
                "<body onload=alert(1)>"
            ],
            '命令注入': [
                "; ls -la",
                "| cat /etc/passwd",
                "$(whoami)",
                "`id`"
            ],
            '路径穿越': [
                "../../../etc/passwd",
                "..\\..\\..\\windows\\system32",
                "/etc/passwd%00.html"
            ]
        }

        for category, payload_list in payloads.items():
            for payload in payload_list[:2]:
                try:
                    if method.upper() == 'GET':
                        resp = requests.get(f"{api_endpoint}?input={urllib.parse.quote(payload)}", timeout=10)
                    else:
                        resp = requests.post(api_endpoint,
```

```
                            json={'data': payload},
                            headers={'Content-Type': '
application/json'}
    ,
                            timeout=10
                        )

                    if resp.status_code >= 500:
                        vulns.append(Vulnerability(
                            vuln_id=f"INJ-{hashlib.md5(f'{category
}:{pay
    load}'.encode()).hexdigest()[:8]}",
                            title=f"{category}漏洞",
                            description=f"服务端返回 {resp.status_code},
    替代预期错误",
                            severity=SeverityLevel.MEDIUM if '
XSS' in ca
    tegory else SeverityLevel.HIGH,
                            cve_id=None, cvss_score=None,
                            affected_component=api_endpoint,
                            remediation=f"对 {category} 类输入实施严格验
    证和转义",
                            scan_type=ScanType.INJECTION
                        ))
                except requests.exceptions.ConnectionError:
                    pass
                except Exception as e:
                    logger.warning(f"Injection test failed: {e}")

        self.vulnerabilities.extend(vulns)
        logger.info(f"Injection test found {len(vulns)}
vulnerabilities"
    )
        return vulns

    def test_auth_bypass(self, api_endpoint: str) ->
List[Vulnerability]
    :
        import requests
        vulns = []

        tests = [
            ('无认证头', {}),
            ('空Authorization', {'Authorization': ''}),
```

```
            ('无效Bearer', {'Authorization': 'Bearer invalid_token_12345'}),
            ('Basic编码注入', {'Authorization': 'Basic YWRtaW46J29yJzEnPScxJw=='}),
        ]

        for test_name, headers in tests:
            try:
                resp = requests.get(api_endpoint, headers=headers, timeout=10)
                if resp.status_code == 200:
                    vulns.append(Vulnerability(
                        vuln_id=f"AUTH-{test_name.replace(' ', '_')}",
                        title=f"认证绕过: {test_name}",
                        description=f"使用 {test_name} 可访问受保护资源",
                        severity=SeverityLevel.CRITICAL,
                        cve_id=None, cvss_score=9.0,
                        affected_component=api_endpoint,
                        remediation="实施严格的认证验证",
                        scan_type=ScanType.AUTH
                    ))
            except Exception:
                pass

        self.vulnerabilities.extend(vulns)
        return vulns

    def scan_all(self, src_dir: str = 'src', api_endpoint: str = 'http://localhost:5000/api/v1') -> List[Vulnerability]:
        logger.info("Starting full security scan...")
        self.scan_dependencies()
        self.scan_code_sast(src_dir)
        self.scan_ports()
        self.test_injection(api_endpoint)
        self.test_auth_bypass(f"{api_endpoint}/metrics")
        logger.info(f"Full scan complete: {len(self.vulnerabilities)} total vulnerabilities")
        return self.vulnerabilities

```

```
    def get_summary(self) -> Dict[str, Any]:
        by_severity = {s: 0 for s in SeverityLevel}
        by_type = {s: 0 for s in ScanType}

        for v in self.vulnerabilities:
            by_severity[v.severity] += 1
            by_type[v.scan_type] += 1

        return {
            'total': len(self.vulnerabilities),
            'by_severity': {s.value: c for s, c in
by_severity.items()},
            'by_type': {t.value: c for t, c in by_type.items()},
            'critical_count': by_severity[SeverityLevel.CRITICAL],
            'high_count': by_severity[SeverityLevel.HIGH],
            'needs_immediate_action':
by_severity[SeverityLevel.CRITICAL
    ] > 0
        }
```

69. src/testing/stress_tester.py

```
"""
压力测试模块

模拟不同负载条件下的系统表现:
- CPU压力测试（多线程计算密集）
- 内存压力测试（大数据分配）
- 磁盘IO压力测试（密集读写）
- 网络压力测试（高并发请求）
- 时间窗口测试（持续高压）

参考项目: Apache JMeter, Locust, stress-ng
"""

import time
import threading
import random
import numpy as np
from typing import Dict, Any, List, Optional, Callable, Tuple
from dataclasses import dataclass, field
from enum import Enum
from concurrent.futures import ThreadPoolExecutor,
ProcessPoolExecutor,
    as_completed
```

```
from src.utils.logger import setup_logger
from src.utils.config import Config
import psutil

logger = setup_logger("stress_tester")
config = Config()

@dataclass
class StressMetrics:
    test_name: str
    duration_sec: float
    peak_cpu_pct: float
    avg_cpu_pct: float
    peak_memory_mb: float
    avg_memory_mb: float
    peak_disk_io_mbps: float
    peak_network_mbps: float
    total_requests: int
    success_rate: float
    avg_response_ms: float
    p50_response_ms: float
    p95_response_ms: float
    p99_response_ms: float
    error_count: int
    timeout_count: int
    threshold_violations: List[str] = field(default_factory=list)

class StressTester:
    """
    压力测试器

    系统性地模拟各种负载场景，评估系统极限:
    1. 阶梯增压（逐步提高负载找到拐点）
    2. 恒压测试（长时间保持高负载）
    3. 突发测试（瞬时高并发）
    4. 混合负载（多类型任务混合）
    """

    def __init__(self, target_func: Callable = None):
        self.target_func = target_func
        self.metrics_history: List[StressMetrics] = []
        self.thresholds = self._load_thresholds()

    def _load_thresholds(self) -> Dict:
```

```
        try:
            return config.stress_thresholds
        except Exception:
            return {
                'cpu': {'warning': 70, 'critical': 85, 'max': 95},
                'memory': {'warning': 75, 'critical': 90, 'max':
95},
                'response_time': {'warning_ms': 200, '
critical_ms': 500,
    'timeout_ms': 5000},
                'error_rate': {'warning': 0.01, 'critical': 0.05, '
max':
    0.10}
            }

    def _monitor_system(self, duration_sec: float) -> Dict[str,
List[flo
    at]]:
        cpu_samples = []
        mem_samples = []
        interval = 0.5
        for _ in range(int(duration_sec / interval)):
            cpu_samples.append(psutil.cpu_percent(interval=0.1))
            mem = psutil.virtual_memory()
            mem_samples.append(mem.used / (1024 * 1024))
        return {'cpu': cpu_samples, 'memory': mem_samples}

    def run_cpu_stress(self, num_threads: int = 4, duration_sec:
int = 3
    0) -> StressMetrics:
        logger.info(f"Starting CPU stress test: {num_threads}
threads, {
    duration_sec}s")

        def cpu_intensive_work():
            for _ in range(1000000):
                np.random.rand(100, 100).dot(np.random.rand(100,
100))

        start = time.time()
        responses = []

        monitor_thread = threading.Thread(target=self._monitor_sys
tem, a
```

```
    |    rgs=(duration_sec,))
 97|        monitor_thread.start()
 98|
 99|        with ThreadPoolExecutor(max_workers=num_threads) as
executor:
100|            futures = [executor.submit(cpu_intensive_work) for _
in rang
    |    e(num_threads * 10)]
101|            for future in as_completed(futures):
102|                pass
103|
104|        monitor_thread.join(timeout=5)
105|        elapsed = time.time() - start
106|
107|        metrics = StressMetrics(
108|            test_name="CPU压力测试",
109|            duration_sec=elapsed,
110|            peak_cpu_pct=psutil.cpu_percent(),
111|            avg_cpu_pct=psutil.cpu_percent(),
112|            peak_memory_mb=psutil.virtual_memory().used / (1024 *
1024),
113|            avg_memory_mb=psutil.virtual_memory().used / (1024 *
1024),
114|            peak_disk_io_mbps=0, peak_network_mbps=0,
115|            total_requests=num_threads * 10,
116|            success_rate=1.0,
117|            avg_response_ms=0, p50_response_ms=0,
118|            p95_response_ms=0, p99_response_ms=0,
119|            error_count=0, timeout_count=0
120|        )
121|        self.metrics_history.append(metrics)
122|        return metrics
123|
124|    def run_concurrency_stress(self, concurrency_levels:
List[int],
125|                              request_func: Callable,
126|                              requests_per_level: int = 100) ->
List[St
    |    ressMetrics]:
127|        results = []
128|        for concurrency in concurrency_levels:
129|            logger.info(f"Concurrency stress test: {concurrency}
concurr
    |    ent")
```

```
            responses = []
            errors = 0
            timeouts = 0
            start = time.time()

            def worker():
                try:
                    result = request_func()
                    responses.append(time.time())
                    return result
                except Exception:
                    nonlocal errors
                    errors += 1
                    return None

            with ThreadPoolExecutor(max_workers=concurrency) as
executor
   |    :
                futures = [executor.submit(worker) for _ in
range(reques
   |    ts_per_level)]
                for future in as_completed(futures):
                    try:
                        future.result(timeout=5)
                    except Exception:
                        timeouts += 1

            elapsed = time.time() - start
            if len(responses) >= 2:
                latencies = [t - start for t in sorted(responses)]
                metrics = StressMetrics(
                  test_name=f"并发压力: {concurrency}",
                    duration_sec=elapsed,
                    peak_cpu_pct=psutil.cpu_percent(),
                    avg_cpu_pct=psutil.cpu_percent(),
                    peak_memory_mb=psutil.virtual_memory().used /
(1024
   |    * 1024),
                    avg_memory_mb=psutil.virtual_memory().used /
(1024 *
   |     1024),
                    peak_disk_io_mbps=0, peak_network_mbps=0,
                    total_requests=requests_per_level,
```

```
165|                    success_rate=(requests_per_level - errors) /
request
   |    s_per_level,
166|                    avg_response_ms=np.mean(latencies) * 1000,
167|                    p50_response_ms=np.percentile(latencies, 50)
* 1000,
168|                    p95_response_ms=np.percentile(latencies, 95)
* 1000,
169|                    p99_response_ms=np.percentile(latencies, 99)
* 1000,
170|                    error_count=errors, timeout_count=timeouts
171|                )
172|                results.append(metrics)
173|                self.metrics_history.append(metrics)
174|
175|        return results
176|
177|    def run_endurance_test(self, request_func: Callable,
duration_minute
   |    s: int = 5) -> StressMetrics:
178|        logger.info(f"Starting endurance test: {duration_minutes}
minute
   |    s")
179|        start = time.time()
180|        end = start + duration_minutes * 60
181|        total = 0
182|        errors = 0
183|
184|        while time.time() < end:
185|            try:
186|                request_func()
187|                total += 1
188|            except Exception:
189|                errors += 1
190|
191|        elapsed = time.time() - start
192|        metrics = StressMetrics(
193|            test_name="持续负载测试",
194|            duration_sec=elapsed,
195|            peak_cpu_pct=psutil.cpu_percent(),
196|            avg_cpu_pct=psutil.cpu_percent(),
197|            peak_memory_mb=psutil.virtual_memory().used / (1024 *
1024),
```

```
            avg_memory_mb=psutil.virtual_memory().used / (1024 *
1024),
            peak_disk_io_mbps=0, peak_network_mbps=0,
            total_requests=total,
            success_rate=(total - errors) / max(total, 1),
            avg_response_ms=elapsed / max(total, 1) * 1000,
            p50_response_ms=0, p95_response_ms=0,
p99_response_ms=0,
            error_count=errors, timeout_count=0
        )
        self.metrics_history.append(metrics)
        return metrics

    def check_thresholds(self, metrics: StressMetrics) ->
List[str]:
        violations = []

        thresholds = self.thresholds
        if metrics.peak_cpu_pct > thresholds.get('cpu',
{}).get('critica
    l', 85):
            violations.append(f"CPU超临界阈值: {metrics.peak_cpu_pct:.1f
    }% > {thresholds['cpu']['critical']}%")
        elif metrics.peak_cpu_pct > thresholds.get('cpu',
{}).get('warni
    ng', 70):
            violations.append(f"CPU超警告阈值: {metrics.peak_cpu_pct:.1f
    }% > {thresholds['cpu']['warning']}%")

        if (1.0 - metrics.success_rate) >
thresholds.get('error_rate', {
    }).get('critical', 0.05):
            violations.append(f"错误率超临界阈值: {(1.0-metrics.success_
    rate)*100:.1f}%")

        if metrics.avg_response_ms >
thresholds.get('response_time', {})
    .get('critical_ms', 500):
            violations.append(f"响应时间超临界阈值: {metrics.avg_respons
    e_ms:.0f}ms")

        metrics.threshold_violations = violations
        return violations

```

```
    def get_summary(self) -> Dict[str, Any]:
        if not self.metrics_history:
            return {'tests': 0}

        all_peak_cpu = [m.peak_cpu_pct for m in
self.metrics_history]
        all_error_rates = [1.0 - m.success_rate for m in
self.metrics_hi
    story]

        return {
            'total_tests': len(self.metrics_history),
            'max_cpu_pct': max(all_peak_cpu),
            'avg_cpu_pct': np.mean(all_peak_cpu),
            'max_error_rate': max(all_error_rates),
            'avg_error_rate': np.mean(all_error_rates),
            'total_violations': sum(len(m.threshold_violations)
for m in
     self.metrics_history),
            'violations': [m.threshold_violations for m in
self.metrics_
    history if m.threshold_violations]
        }
```

70. src/testing/test_orchestrator.py

```
"""
自动化测试引擎

建立自动化测试流程，支持:
- 兼容性测试 (跨版本/跨平台/跨依赖)
- 压力测试 (CPU/内存/IO/网络/并发)
- 安全渗透测试 (注入/模糊/认证绕过)
- 回归测试

参考项目: pytest, Selenium Grid, Locust
"""

import time
import json
import threading
import subprocess
import os
from typing import Dict, Any, List, Optional, Callable, Tuple
from dataclasses import dataclass, field
```

```
from enum import Enum
from datetime import datetime
from concurrent.futures import ThreadPoolExecutor, as_completed
from src.utils.logger import setup_logger
from src.utils.config import Config

logger = setup_logger("test_orchestrator")
config = Config()

class TestStatus(Enum):
    PENDING = "pending"
    RUNNING = "running"
    PASSED = "passed"
    FAILED = "failed"
    SKIPPED = "skipped"
    ERROR = "error"

class TestSeverity(Enum):
    CRITICAL = "critical"
    HIGH = "high"
    MEDIUM = "medium"
    LOW = "low"

@dataclass
class TestResult:
    test_id: str
    test_name: str
    category: str
    status: TestStatus
    severity: TestSeverity
    duration_sec: float
    message: str = ""
    details: Dict[str, Any] = field(default_factory=dict)
    timestamp: str = field(default_factory=lambda:
datetime.now().isofor
    mat())

@dataclass
class TestSuite:
    name: str
    description: str
    tests: List[Callable] = field(default_factory=list)
    setup: Optional[Callable] = None
    teardown: Optional[Callable] = None
```

```
    timeout_sec: int = 300


class TestOrchestrator:
    """
    测试编排引擎

    管理测试套件的执行流程:
    1. 环境准备 (Setup)
    2. 测试执行 (并发执行)
    3. 结果收集 (Result Collection)
    4. 环境清理 (Teardown)
    5. 报告生成 (Report)
    """

    def __init__(self, parallel_workers: int = 4):
        self.parallel_workers = parallel_workers
        self._suites: Dict[str, TestSuite] = {}
        self._results: List[TestResult] = []
        self._running = False
        self._hooks: Dict[str, List[Callable]] = {
            'before_all': [], 'after_all': [],
            'before_each': [], 'after_each': [],
            'on_failure': []
        }

    def register_suite(self, suite: TestSuite) -> None:
        self._suites[suite.name] = suite

    def add_hook(self, event: str, callback: Callable) -> None:
        if event in self._hooks:
            self._hooks[event].append(callback)

    def run_all(self) -> List[TestResult]:
        self._running = True
        self._results = []

        for hook in self._hooks['before_all']:
            hook()

        for name, suite in self._suites.items():
            suite_results = self._run_suite(suite)
            self._results.extend(suite_results)

```

```
        for hook in self._hooks['after_all']:
            hook()

        self._running = False
        return self._results

    def _run_suite(self, suite: TestSuite) -> List[TestResult]:
        results = []
        logger.info(f"Running suite: {suite.name}")

        if suite.setup:
            try:
                suite.setup()
            except Exception as e:
                logger.error(f"Suite setup failed: {e}")
                return [TestResult(
                    test_id=f"{suite.name}_setup",
                    test_name="Suite Setup",
                    category="setup",
                    status=TestStatus.ERROR,
                    severity=TestSeverity.CRITICAL,
                    duration_sec=0,
                    message=str(e)
                )]

        with ThreadPoolExecutor(max_workers=self.parallel_workers)
 as ex
   |    ecutor:
            futures = {}
            for test in suite.tests:
                future = executor.submit(self._run_single_test,
test, su
   |    ite)
                futures[future] = test

            for future in as_completed(futures):
                try:
                    result = future.result(timeout=suite.timeout_s
ec)
                    results.append(result)
                except Exception as e:
                    test = futures[future]
                    results.append(TestResult(
```

```
                        test_id=getattr(test, '__name__', '
unknown'),
                        test_name=getattr(test, '__name__', '
unknown'),
                        category=suite.name,
                        status=TestStatus.ERROR,
                        severity=TestSeverity.HIGH,
                        duration_sec=suite.timeout_sec,
                        message=str(e)
                    ))

        if suite.teardown:
            try:
                suite.teardown()
            except Exception as e:
                logger.error(f"Suite teardown failed: {e}")

        return results

    def _run_single_test(self, test_func: Callable, suite:
TestSuite) ->
     TestResult:
        name = getattr(test_func, '__name__', 'unknown')
        start = time.time()

        for hook in self._hooks['before_each']:
            hook()

        try:
            result = test_func()
            status = TestStatus.PASSED if result else
TestStatus.FAILED
            message = "Test passed" if result else "Test
assertion faile
    d"
        except Exception as e:
            status = TestStatus.ERROR
            message = str(e)
            for hook in self._hooks['on_failure']:
                hook(name, str(e))

        for hook in self._hooks['after_each']:
            hook()

```

```
        elapsed = time.time() - start
        return TestResult(
            test_id=name, test_name=name, category=suite.name,
            status=status, severity=TestSeverity.MEDIUM,
            duration_sec=elapsed, message=message
        )

    def get_summary(self) -> Dict[str, Any]:
        total = len(self._results)
        passed = sum(1 for r in self._results if r.status ==
TestStatus.
    PASSED)
        failed = sum(1 for r in self._results if r.status ==
TestStatus.
    FAILED)
        errors = sum(1 for r in self._results if r.status ==
TestStatus.
    ERROR)
        skipped = sum(1 for r in self._results if r.status ==
TestStatus
    .SKIPPED)

        return {
            'total': total,
            'passed': passed,
            'failed': failed,
            'errors': errors,
            'skipped': skipped,
            'pass_rate': passed / max(total, 1) * 100,
            'total_duration_sec': sum(r.duration_sec for r in
self._resu
    lts)
        }

    def get_results_by_severity(self, severity: TestSeverity) ->
List[Te
    stResult]:
        return [r for r in self._results if r.severity ==
severity]
```

71. src/testing/test_runner.py

```
"""
测试环境一键运行器

```

```
集成所有测试模块，提供统一的测试入口:
1. 环境检测 + 初始化
2. 兼容性测试 (跨版本/跨平台)
3. 压力测试 (CPU/内存/并发)
4. 安全扫描 (依赖/端口/SAST/注入)
5. 风险评估 + 报告生成
"""

import sys
import json
import time
import os
from datetime import datetime
from typing import Dict, Any, List
from src.utils.logger import setup_logger
from src.utils.config import Config
from src.testing import (
    CompatibilityTester, StressTester, SecurityScanner,
    RiskAssessor, EnvironmentManager
)

logger = setup_logger("test_runner")
config = Config()

def run_full_test_suite(output_file: str = None) -> Dict[str,
Any]:
    """
    运行完整测试套件

    流程:
    1. 环境初始化 + 快照
    2. 兼容性测试
    3. 压力测试
    4. 安全扫描
    5. 风险评估
    6. 报告生成
    7. 环境清理
    """

    start_time = time.time()
    results = {
        'run_id': datetime.now().strftime('%Y%m%d_%H%M%S'),
        'timestamp': datetime.now().isoformat(),
        'summary': {},
```

```
        'compatibility': {},
        'stress': {},
        'security': {},
        'risk': None,
        'duration_sec': 0
    }

    logger.info("=" * 60)
    logger.info("开始仿真模拟测试环境 - 完整测试套件")
    logger.info("=" * 60)

    env_mgr = EnvironmentManager()
    env_mgr.save_state('test_start', datetime.now().isoformat())
    env_mgr.create_snapshot("测试执行前快照")

    try:
        # 1. 兼容性测试
        logger.info("\n[1/5] 兼容性测试...")
        ct = CompatibilityTester()
        ct.check_python_version("3.9", "3.12")
        ct.check_os_compatibility()
        ct.check_platform_arch()
        ct.check_all_dependencies()
        ct.check_import_all_modules()
        results['compatibility'] = ct.get_summary()
        logger.info(f"  兼容性评分: {results['compatibility']['compatibi
    lity_score']:.1f}%")

        # 2. 压力测试
        logger.info("\n[2/5] 压力测试...")
        st = StressTester()
        cpu_metrics = st.run_cpu_stress(num_threads=2,
duration_sec=5)
        st.check_thresholds(cpu_metrics)
        results['stress'] = st.get_summary()
        logger.info(f"  最高CPU使用率: {results['stress']['max_cpu_pct']
    :.1f}%")

        # 3. 安全扫描
        logger.info("\n[3/5] 安全扫描...")
        ss = SecurityScanner()
        ss.scan_dependencies()
        ss.scan_code_sast('src')
        ss.scan_ports('localhost', [5000, 5432, 6379, 8080])
```

```
        results['security'] = ss.get_summary()
        logger.info(f"  发现漏洞数: {results['security']['total']}")
        logger.info(f"  严重: {results['security']['critical_count']} / 高: {results['security']['high_count']}")

        # 4. 风险评估
        logger.info("\n[4/5] 风险评估...")
        ra = RiskAssessor()
        ra.assess_from_security_scan(ss.vulnerabilities)
        ra.assess_from_stress_test(cpu_metrics)
        report = ra.generate_report()
        results['risk'] = {
            'overall_risk': report.overall_risk.value,
            'safety_score': report.safety_score,
            'recommendations': report.recommendations
        }
        logger.info(f"  整体风险等级: {report.overall_risk.value}")
        logger.info(f"  安全评分: {report.safety_score:.1f}/100")

        # 5. 汇总
        logger.info("\n[5/5] 生成测试报告...")
        results['summary'] = {
            'compatibility_score': results['compatibility']['compatibility_score'],
            'stress_tests_passed': results['stress']['total_tests'],
            'security_vulnerabilities': results['security']['total'],
            'critical_vulnerabilities': results['security']['critical_count'],
            'risk_level': results['risk']['overall_risk'],
            'safety_score': results['risk']['safety_score']
        }

        results['duration_sec'] = time.time() - start_time
        logger.info(f"\n测试完成! 耗时: {results['duration_sec']:.1f}s")

        # 生成报告文件
        if output_file is None:
            os.makedirs('test_reports', exist_ok=True)
            output_file = f"test_reports/test_report_{results['run_id']}
```

```
    |    .json"
124|
125|        with open(output_file, 'w', encoding='utf-8') as f:
126|            json.dump(results, f, indent=2, ensure_ascii=False,
default=
    |    str)
127|
128|        logger.info(f"报告已保存至: {output_file}")
129|
130|    except Exception as e:
131|        logger.error(f"测试执行异常: {e}", exc_info=True)
132|        results['error'] = str(e)
133|
134|    finally:
135|        env_mgr.create_snapshot("测试执行后快照")
136|        env_mgr.cleanup_snapshots(keep_last=5)
137|
138|    _print_report(results)
139|    return results
140|
141|
142|def _print_report(results: Dict[str, Any]) -> None:
143|    logger.info("=" * 60)
144|    logger.info("  测试环境执行报告")
145|    logger.info("=" * 60)
146|    logger.info(f"  执行ID:      {results['run_id']}")
147|    logger.info(f"  耗时:        {results['duration_sec']:.1f}s")
148|    logger.info(f"  兼容性评分:  {results['compatibility'].get('compatib
    |    ility_score', 0):.1f}%")
149|    logger.info(f"  安全漏洞:    {results['security'].get('total', 0)} (
    |    {results['security'].get('critical_count', 0)} 严重)")
150|    logger.info(f"  风险等级:    {results['risk'].get('overall_risk', 'N
    |    /A')}")
151|    logger.info(f"  安全评分:    {results['risk'].get('safety_score', 0)
    |    :.1f}/100")
152|    logger.info("=" * 60)
153|
154|    if results.get('summary', {}).get('risk_level') == 'critical':
155|        logger.warning("  发现严重风险！请立即处理以下问题:")
156|        for rec in results['risk'].get('recommendations', []):
157|            logger.warning(f"    - {rec}")
158|    elif results.get('summary', {}).get('safety_score', 100) >=
80:
159|        logger.info("  系统安全状况良好")
```

```
160|
161|
162|if __name__ == '__main__':
163|    run_full_test_suite()
```

72. src/utils/__init__.py

```
  1|from .data_utils import (
  2|    save_simulation_data,
  3|    load_simulation_data,
  4|    generate_synthetic_data,
  5|    export_results_to_excel,
  6|    save_run_metadata,
  7|    load_run_metadata
  8|)
  9|from .config import Config
 10|from .logger import setup_logger
 11|from .validation import (
 12|    validate_int,
 13|    validate_str,
 14|    validate_algorithms,
 15|    validate_simulation_params,
 16|    sanitize_filename,
 17|    security_headers,
 18|    RateLimiter,
 19|    ValidationError
 20|)
 21|
 22|__all__ = [
 23|    'save_simulation_data',
 24|    'load_simulation_data',
 25|    'generate_synthetic_data',
 26|    'export_results_to_excel',
 27|    'save_run_metadata',
 28|    'load_run_metadata',
 29|    'Config',
 30|    'setup_logger',
 31|    'validate_int',
 32|    'validate_str',
 33|    'validate_algorithms',
 34|    'validate_simulation_params',
 35|    'sanitize_filename',
 36|    'security_headers',
 37|    'RateLimiter',
 38|    'ValidationError'
```

```
]
```

73. src/utils/cache.py

```
"""
缓存模块

提供内存缓存支持，用于优化频繁的数据读取操作:
- 基于 TTL (Time-To-Live) 的缓存过期机制
- 线程安全的缓存操作
- 支持手动失效和自动清理
"""

import threading
import time
from typing import Any, Dict, Optional, Callable
from functools import wraps
from src.utils.logger import setup_logger

logger = setup_logger("cache")


class CacheEntry:
    """缓存条目"""

    def __init__(self, value: Any, ttl_sec: int):
        self.value = value
        self.created_at = time.time()
        self.ttl_sec = ttl_sec

    def is_expired(self) -> bool:
        return time.time() - self.created_at > self.ttl_sec


class Cache:
    """
    线程安全的内存缓存

    使用示例:
        cache = Cache()
        cache.set("key", value, ttl_sec=300)
        value = cache.get("key")
    """

    def __init__(self):
```

```
        self._store: Dict[str, CacheEntry] = {}
        self._lock = threading.RLock()

    def get(self, key: str) -> Optional[Any]:
        """获取缓存值，过期返回 None"""
        with self._lock:
            entry = self._store.get(key)
            if entry is None:
                return None
            if entry.is_expired():
                del self._store[key]
                logger.debug(f"Cache expired for key: {key}")
                return None
            logger.debug(f"Cache hit for key: {key}")
            return entry.value

    def set(self, key: str, value: Any, ttl_sec: int = 300) ->
None:
        """设置缓存值，默认 TTL 5 分钟"""
        with self._lock:
            self._store[key] = CacheEntry(value, ttl_sec)
            logger.debug(f"Cache set for key: {key}
(ttl={ttl_sec}s)")

    def delete(self, key: str) -> bool:
        """删除缓存条目"""
        with self._lock:
            if key in self._store:
                del self._store[key]
                logger.debug(f"Cache deleted for key: {key}")
                return True
            return False

    def clear(self) -> None:
        """清空所有缓存"""
        with self._lock:
            self._store.clear()
            logger.info("Cache cleared")

    def cleanup_expired(self) -> int:
        """清理过期条目，返回清理数量"""
        with self._lock:
            expired_keys = [k for k, v in self._store.items() if
v.is_ex
```

```
    |    pired()]
 83|            for k in expired_keys:
 84|                del self._store[k]
 85|            if expired_keys:
 86|                logger.debug(f"Cleaned up {len(expired_keys)}
expired ca
    |    che entries")
 87|            return len(expired_keys)
 88|
 89|    def get_stats(self) -> Dict[str, Any]:
 90|        """获取缓存统计信息"""
 91|        with self._lock:
 92|            total = len(self._store)
 93|            expired = sum(1 for v in self._store.values() if
v.is_expire
    |    d())
 94|            return {
 95|                'total_entries': total,
 96|                'expired_entries': expired,
 97|                'active_entries': total - expired
 98|            }
 99|
100|
101|# 全局缓存实例
102|_global_cache = Cache()
103|
104|
105|def get_cache() -> Cache:
106|    """获取全局缓存实例"""
107|    return _global_cache
108|
109|
110|def cached(ttl_sec: int = 300, key_func: Optional[Callable] =
None):
111|    """
112|    缓存装饰器
113|
114|    Args:
115|        ttl_sec: 缓存过期时间（秒）
116|        key_func: 自定义缓存键生成函数，接收原函数参数，返回字符串键
117|
118|    使用示例:
119|        @cached(ttl_sec=60)
120|        def expensive_function(x, y):
```

```
            return x + y
    """
    def decorator(func: Callable) -> Callable:
        @wraps(func)
        def wrapper(*args, **kwargs):
            cache = get_cache()

            if key_func:
                cache_key = key_func(*args, **kwargs)
            else:
               # 默认使用函数名和参数生成键
                key_parts = [func.__module__, func.__name__]
                if args:
                    key_parts.append(str(args))
                if kwargs:
                    key_parts.append(str(sorted(kwargs.items())))
                cache_key = "|".join(key_parts)

            cached_value = cache.get(cache_key)
            if cached_value is not None:
                return cached_value

            result = func(*args, **kwargs)
            cache.set(cache_key, result, ttl_sec)
            return result
        return wrapper
    return decorator
```

74. src/utils/config.py

```
import yaml
import platform
import psutil
import os
from pathlib import Path
from typing import Any, Dict, Optional, List

class Config:
    """支持环境感知的配置管理器"""

    _instance: Optional['Config'] = None

    def __new__(cls, config_path: Optional[str] = None, env:
Optional[st
    r] = None) -> 'Config':
```

```
        if cls._instance is None:
            cls._instance = super().__new__(cls)
            cls._instance._loaded = False
        return cls._instance

    def __init__(self, config_path: Optional[str] = None, env: Optional[
    str] = None):
        if not self._loaded:
            self._load(config_path, env)

    def _load(self, config_path: Optional[str], env: Optional[str]) -> N
    one:
        env_config: Optional[Path] = None

        if config_path is None:
            config_dir = Path(__file__).parent.parent.parent / "config"
            base_config = config_dir / "default.yaml"

            env = env or os.environ.get('DCS_ENV', 'development')
            env_config = config_dir / f"{env}.yaml"
        else:
            base_config = Path(config_path)
            env = env or os.environ.get('DCS_ENV')
            if env:
                env_config = base_config.parent / f"{env}.yaml"

        with open(base_config, 'r', encoding='utf-8') as f:
            loaded = yaml.safe_load(f)
        self._data: Dict[str, Any] = loaded if isinstance(loaded, dict)
    else {}

        if env_config is not None:
            try:
                with open(env_config, 'r', encoding='utf-8') as f:
                    env_data = yaml.safe_load(f)
            except FileNotFoundError:
                env_data = None
            if isinstance(env_data, dict):
                self._deep_merge(self._data, env_data)

```

```
        self._apply_environment_overrides()
        self._detected_env = self._detect_runtime_env()
        self._loaded = True

    def _deep_merge(self, base: dict, override: dict) -> None:
        for key, value in override.items():
            if key in base and isinstance(base[key], dict) and
isinstanc
    e(value, dict):
                self._deep_merge(base[key], value)
            else:
                base[key] = value

    def _apply_environment_overrides(self) -> None:
        env_map = {
            'DCS_NUM_SERVERS': ('simulation', 'num_servers', int),
            'DCS_SIM_HOURS': ('simulation', 'simulation_hours',
int),
            'DCS_PARALLEL_WORKERS': ('simulation', '
parallel_workers', i
    nt),
            'DCS_DATA_DIR': ('output', 'data_dir', str),
            'DCS_REPORTS_DIR': ('output', 'reports_dir', str),
            'DCS_ENABLE_VIZ': ('output', 'enable_visualization',
lambda
    x: x.lower() == 'true'),
            'DCS_POPULATION_SIZE': ('genetic', 'population_size',
int),
            'DCS_GENERATIONS': ('genetic', 'generations', int),
            'DCS_AMBIENT_TEMP': ('environment', '
ambient_temp_base', flo
    at),
            'DCS_API_URL': ('api', 'api_url', str),
        }

        for env_var, (section, key, converter) in env_map.items():
            value = os.environ.get(env_var)
            if value is not None:
                try:
                    if section not in self._data:
                        self._data[section] = {}
                    self._data[section][key] = converter(value)
                except (ValueError, TypeError):
                    pass
```

```
 85|
 86|    def _detect_runtime_env(self) -> Dict[str, Any]:
 87|        info = {
 88|            'os': platform.system(),
 89|            'os_release': platform.release(),
 90|            'arch': platform.machine(),
 91|            'python_version': platform.python_version(),
 92|            'is_docker': os.path.exists('/.dockerversion'),
 93|            'is_k8s': bool(os.environ.get('KUBERNETES_SERVICE_HOST'
)),
 94|        }
 95|
 96|        try:
 97|            info['cpu_physical'] = psutil.cpu_count(logical=False)
 98|            info['cpu_logical'] = psutil.cpu_count(logical=True)
 99|        except Exception:
100|            info['cpu_physical'] = 1
101|            info['cpu_logical'] = 1
102|
103|        try:
104|            mem = psutil.virtual_memory()
105|            info['total_memory_gb'] = round(mem.total / (1024 **
3), 1)
106|            info['available_memory_gb'] = round(mem.available /
(1024 **
   |     3), 1)
107|        except Exception:
108|            info['total_memory_gb'] = 0
109|
110|        try:
111|            import torch
112|            info['has_cuda'] = torch.cuda.is_available()
113|            info['cuda_count'] = torch.cuda.device_count() if
info['has_
   |    cuda'] else 0
114|        except ImportError:
115|            info['has_cuda'] = False
116|            info['cuda_count'] = 0
117|
118|        try:
119|            import torch_npu
120|            info['has_ascend'] = torch_npu.npu.is_available()
121|        except ImportError:
```

```
            info['has_ascend'] =
os.path.exists('/usr/local/Ascend')

        return info

    def get_runtime_info(self) -> Dict[str, Any]:
        return self._detected_env

    def get_optimal_workers(self) -> int:
        cores = self._detected_env.get('cpu_physical', 4)
        if cores >= 16:
            return 8
        elif cores >= 8:
            return 4
        elif cores >= 4:
            return 2
        return 1

    def __getattr__(self, key: str) -> Any:
        if key.startswith('_'):
            raise AttributeError(key)
        data = self.__dict__.get('_data')
        if not isinstance(data, dict) or key not in data:
            raise AttributeError(key)
        val = data[key]
        if isinstance(val, dict):
            class SectionAccessor:
                def __init__(self, data):
                    self._data = data

                def __getattr__(self, k):
                    return self._data.get(k)

                def get(self, k, default=None):
                    return self._data.get(k, default)

                def __contains__(self, k):
                    return k in self._data

                def __repr__(self):
                    return str(self._data)

            return SectionAccessor(val)
        return val
```

```
165|
166|    def get(self, key: str, default: Any = None) -> Any:
167|        return self._data.get(key, default)
168|
169|    def set(self, key: str, value: Any) -> None:
170|        self._data[key] = value
171|
172|    def __contains__(self, key: str) -> bool:
173|        return key in self._data
174|
175|    def to_dict(self) -> Dict[str, Any]:
176|        return self._data.copy()
177|
178|    def reload(self, config_path: Optional[str] = None, env:
Optional[st
   |    r] = None) -> None:
179|        self._loaded = False
180|        Config._instance = None
181|        self._load(config_path, env)
```

75. src/utils/data_utils.py

```
  1|import json
  2|import os
  3|from typing import Any, Dict, Optional
  4|
  5|import numpy as np
  6|import pandas as pd
  7|
  8|from src.utils.logger import setup_logger
  9|
 10|logger = setup_logger("data_utils")
 11|
 12|DATA_DIR = os.path.join(os.path.dirname(__file__), '..', '..', '
data')
 13|METADATA_FILE = 'run_metadata.json'
 14|
 15|
 16|def _data_dir() -> str:
 17|    path = os.path.abspath(DATA_DIR)
 18|    os.makedirs(path, exist_ok=True)
 19|    return path
 20|
 21|
```

```
def save_simulation_data(df: pd.DataFrame, filename: str = 'simulation_results.csv') -> str:
    try:
        filepath = os.path.join(_data_dir(), filename)
        df.to_csv(filepath, index=False)
        logger.info(f"Simulation data saved to {filepath}")
        return filepath
    except Exception as e:
        logger.error(f"Failed to save simulation data: {str(e)}")
        raise


def save_run_metadata(metadata: Dict[str, Any], filename: str = METADATA_FILE) -> str:
    """Persist what produced the current results: data source, seeds, rule."""
    try:
        filepath = os.path.join(_data_dir(), filename)
        with open(filepath, 'w', encoding='utf-8') as f:
            json.dump(metadata, f, ensure_ascii=False, indent=2, default=str)
        logger.info(f"Run metadata saved to {filepath}")
        return filepath
    except Exception as e:
        logger.error(f"Failed to save run metadata: {str(e)}")
        raise


def load_run_metadata(filename: str = METADATA_FILE) -> Optional[Dict[str, Any]]:
    try:
        filepath = os.path.join(_data_dir(), filename)
        if not os.path.exists(filepath):
            return None
        with open(filepath, 'r', encoding='utf-8') as f:
            return json.load(f)
    except Exception as e:
        logger.warning(f"Failed to load run metadata: {str(e)}")
        return None
```

```
 56|
 57|
 58|def load_simulation_data(filename: str = '
simulation_results.csv') -> Op
   |    tional[pd.DataFrame]:
 59|    try:
 60|        filepath = os.path.join(_data_dir(), filename)
 61|        if os.path.exists(filepath):
 62|            logger.info(f"Loading simulation data from
{filepath}")
 63|            return pd.read_csv(filepath)
 64|        logger.warning(f"Simulation data file not found:
{filepath}")
 65|        return None
 66|    except Exception as e:
 67|        logger.error(f"Failed to load simulation data: {str(e)}")
 68|        raise
 69|
 70|
 71|def generate_synthetic_data(hours: int = 24, num_servers: int =
10) -> p
   |    d.DataFrame:
 72|    try:
 73|        time = np.arange(hours)
 74|        ambient_temp = 15 + 10 * np.sin((time - 6) * np.pi / 12)
 75|
 76|        data = []
 77|        for t in time:
 78|            for server_id in range(num_servers):
 79|                load = np.random.randint(0, 100)
 80|                temp = 25 + load * 0.3 + np.random.normal(0, 2)
 81|                power = 100 + load * 2
 82|
 83|                data.append({
 84|                    'time': t,
 85|                    'server_id': server_id,
 86|                    'load': load,
 87|                    'temperature': temp,
 88|                    'power': power,
 89|                    'ambient_temp': ambient_temp[t]
 90|                })
 91|
 92|        logger.info(f"Generated synthetic data with {len(data)}
records"
```

```
    )
        return pd.DataFrame(data)
    except Exception as e:
        logger.error(f"Failed to generate synthetic data: {str(e)}")
        raise


def export_results_to_excel(df: pd.DataFrame, filename: str = 'simulation_results.xlsx') -> str:
    try:
        filepath = os.path.join(_data_dir(), filename)

        from src.simulation.metrics import summarize

        summary_rows = []
        for algo, metrics in summarize(df).items():
            summary_rows.append({'algorithm': algo, **metrics})
        summary = pd.DataFrame(summary_rows)

        with pd.ExcelWriter(filepath) as writer:
            df.to_excel(writer, sheet_name='Results', index=False)
            if not summary.empty:
                summary.to_excel(writer, sheet_name='Summary', index=False)

        logger.info(f"Results exported to {filepath}")
        return filepath
    except Exception as e:
        logger.error(f"Failed to export results to Excel: {str(e)}")
        raise
```

76. src/utils/download_stats.py

```
"""
应用下载统计模块

提供下载次数记录与查询功能:
- 多维度统计（按平台/应用/日期）
- 持久化存储（JSON文件）
- 并发安全（文件锁）
"""
```

```
 9|
10|import json
11|import os
12|import threading
13|from datetime import datetime
14|from typing import Dict, Any, List, Optional
15|from src.utils.logger import setup_logger
16|
17|logger = setup_logger("download_stats")
18|
19|class DownloadStats:
20|    """下载统计管理器"""
21|
22|    def __init__(self, data_dir: str = "data"):
23|        self.data_dir = data_dir
24|        self.stats_file = os.path.join(data_dir, "
download_stats.json")
25|        self._lock = threading.Lock()
26|        os.makedirs(data_dir, exist_ok=True)
27|        self._ensure_stats_file()
28|
29|    def _ensure_stats_file(self) -> None:
30|        if not os.path.exists(self.stats_file):
31|            self._write_stats({
32|                'version': '1.0',
33|                'created_at': datetime.now().isoformat(),
34|                'records': [],
35|                'total': 0,
36|                'by_platform': {},
37|                'by_app_type': {},
38|                'by_date': {}
39|            })
40|
41|    def _read_stats(self) -> Dict[str, Any]:
42|        try:
43|            with open(self.stats_file, 'r', encoding='utf-8') as
f:
44|                return json.load(f)
45|        except (FileNotFoundError, json.JSONDecodeError):
46|            return {
47|                'version': '1.0',
48|                'records': [],
49|                'total': 0,
50|                'by_platform': {},
```

```
                'by_app_type': {},
                'by_date': {}
            }

    def _write_stats(self, data: Dict[str, Any]) -> None:
        with open(self.stats_file, 'w', encoding='utf-8') as f:
            json.dump(data, f, indent=2, ensure_ascii=False)

    def record_download(
        self, platform: str, app_type: str, filename: str,
        size_mb: float, user_agent: str = "", ip_address: str = ""
    ) -> Dict[str, Any]:
        with self._lock:
            stats = self._read_stats()

            record = {
                'platform': platform,
                'app_type': app_type,
                'filename': filename,
                'size_mb': size_mb,
                'user_agent': user_agent[:200],
                'ip_address': ip_address,
                'timestamp': datetime.now().isoformat()
            }
            stats['records'].append(record)

            stats['total'] = stats.get('total', 0) + 1

            stats['by_platform'][platform] =
stats['by_platform'].get(pl
    atform, 0) + 1
            stats['by_app_type'][app_type] =
stats['by_app_type'].get(ap
    p_type, 0) + 1

            today = datetime.now().strftime('%Y-%m-%d')
            stats['by_date'][today] = stats['by_date'].get(today,
0) + 1

            if len(stats['records']) > 10000:
                stats['records'] = stats['records'][-5000:]

            self._write_stats(stats)
```

```
            logger.info(f"Download recorded: {platform}/{app_type}/{filename}")

            return record

    def get_summary(self) -> Dict[str, Any]:
        stats = self._read_stats()
        return {
            'success': True,
            'total': stats.get('total', 0),
            'by_platform': stats.get('by_platform', {}),
            'by_app_type': stats.get('by_app_type', {}),
            'by_date': dict(sorted(stats.get('by_date', {}).items())[-7:]),
            'recent': stats['records'][-10:] if stats.get('records') else []
        }

    def get_by_platform(self, platform: str) -> int:
        stats = self._read_stats()
        return stats.get('by_platform', {}).get(platform, 0)

    def reset_stats(self) -> bool:
        with self._lock:
            self._write_stats({
                'version': '1.0',
                'created_at': datetime.now().isoformat(),
                'records': [],
                'total': 0,
                'by_platform': {},
                'by_app_type': {},
                'by_date': {}
            })
            logger.warning("Download stats reset")
            return True

_download_stats_instance: Optional[DownloadStats] = None

def get_download_stats() -> DownloadStats:
    global _download_stats_instance
    if _download_stats_instance is None:
```

```
127|        _download_stats_instance = DownloadStats()
128|    return _download_stats_instance
```

77. src/utils/logger.py

```
  1|import logging
  2|import sys
  3|from pathlib import Path
  4|from typing import Optional
  5|from logging.handlers import RotatingFileHandler
  6|
  7|
  8|class _ResilientStream:
  9|    """Console stream that degrades unencodable characters
instead of ra
   |    ising.
 10|
 11|    A Chinese Windows console defaults to GBK, where markers such
as   /
   |       /
 12|    → do not exist. Without this wrapper a single log line can
abort a w
   |    hole
 13|    CLI command with UnicodeEncodeError.
 14|    """
 15|
 16|    def __init__(self, stream) -> None:
 17|        self._stream = stream
 18|
 19|    def write(self, text: str):
 20|        try:
 21|            return self._stream.write(text)
 22|        except UnicodeEncodeError:
 23|            encoding = getattr(self._stream, 'encoding', None) or '
ascii
   |    '
 24|            safe = text.encode(encoding, '
replace').decode(encoding, 're
   |    place')
 25|            return self._stream.write(safe)
 26|
 27|    def flush(self) -> None:
 28|        try:
 29|            self._stream.flush()
 30|        except Exception:
```

```
            pass

    def isatty(self) -> bool:
        return bool(getattr(self._stream, 'isatty', lambda:
False)())


def _console_stream():
    for stream in (sys.stdout, sys.stderr):
        reconfigure = getattr(stream, 'reconfigure', None)
        if callable(reconfigure):
            try:
                reconfigure(encoding='utf-8', errors='replace')
            except Exception:
                pass
    return _ResilientStream(sys.stdout)


def setup_logger(name: str = "datacenter_sim", level: int =
logging.INFO
    ) -> logging.Logger:
    logger = logging.getLogger(name)
    logger.setLevel(level)

    if not logger.handlers:
        ch = logging.StreamHandler(_console_stream())
        ch.setLevel(level)
        ch.setFormatter(logging.Formatter(
            '[%(asctime)s] [%(levelname)s] %(message)s',
            datefmt='%H:%M:%S'
        ))
        logger.addHandler(ch)

        log_dir = Path(__file__).parent.parent.parent / "logs"
        log_dir.mkdir(exist_ok=True)

       # 使用 RotatingFileHandler 实现日志轮转，单个文件最大 10MB，保留
5
     个备份
        fh = RotatingFileHandler(
            log_dir / "simulation.log",
            maxBytes=10 * 1024 * 1024,
            backupCount=5,
            encoding='utf-8'
```

```
        )
        fh.setLevel(logging.DEBUG)
        fh.setFormatter(logging.Formatter(
            '%(asctime)s [%(levelname)s] [%(name)s] %(message)s'
        ))
        logger.addHandler(fh)

    return logger
```

78. src/utils/validation.py

```
"""
项目级安全输入验证模块

提供统一的输入验证、清洗和安全防护:
- 类型验证与转换
- 范围约束检查
- SQL/XSS/命令注入防护
- 安全HTTP头生成
- 速率限制辅助

参考: OWASP Input Validation Cheat Sheet, Flask-Security
"""
import re
import os
import hashlib
import secrets
from typing import Any, Dict, List, Optional, Tuple, TypeVar,
Union

T = TypeVar('T')

class ValidationError(ValueError):
    pass

def validate_int(value: Any, min_val: int = None, max_val: int =
None,
                 default: int = None, name: str = "value") -> int:
    if value is None:
        if default is not None:
            return default
        raise ValidationError(f"{name} 不能为空")
    try:
        v = int(value)
    except (ValueError, TypeError):
```

```
        raise ValidationError(f"{name} 必须是整数，收到: {value}")
    if min_val is not None and v < min_val:
        raise ValidationError(f"{name} 不能小于 {min_val}，当前: {v}")
    if max_val is not None and v > max_val:
        raise ValidationError(f"{name} 不能大于 {max_val}，当前: {v}")
    return v

def validate_float(value: Any, min_val: float = None, max_val: float = None,
                   default: float = None, name: str = "value") -> float:
    if value is None:
        if default is not None:
            return default
        raise ValidationError(f"{name} 不能为空")
    try:
        v = float(value)
    except (ValueError, TypeError):
        raise ValidationError(f"{name} 必须是数字，收到: {value}")
    if min_val is not None and v < min_val:
        raise ValidationError(f"{name} 不能小于 {min_val}，当前: {v}")
    if max_val is not None and v > max_val:
        raise ValidationError(f"{name} 不能大于 {max_val}，当前: {v}")
    return v

def validate_str(value: Any, min_len: int = 0, max_len: int = 256,
                 pattern: str = None, allowed_chars: str = None,
                 default: str = "", name: str = "value") -> str:
    if value is None:
        if default is not None:
            return default
        raise ValidationError(f"{name} 不能为空")
    v = str(value)
    if len(v) < min_len:
        raise ValidationError(f"{name} 长度不能小于 {min_len}")
    if len(v) > max_len:
        raise ValidationError(f"{name} 长度不能超过 {max_len}")
    if pattern and not re.match(pattern, v):
        raise ValidationError(f"{name} 格式无效: {v}")
    if allowed_chars:
        for ch in v:
            if ch not in allowed_chars:
                raise ValidationError(f"{name} 包含非法字符: '{ch}'")
```

```
    return v

def validate_algorithms(algorithms: Any, allowed: List[str] = None) -> List[str]:
    if allowed is None:
        allowed = ['greedy', 'genetic', 'round_robin', 'damo_ets', 'nsga2', 'moead']
    if algorithms is None:
        return ['greedy', 'genetic', 'round_robin', 'damo_ets']
    if isinstance(algorithms, str):
        algorithms = [a.strip() for a in algorithms.split(',')]
    if not isinstance(algorithms, list):
        raise ValidationError("algorithms 必须是列表或逗号分隔字符串")
    result = []
    for algo in algorithms:
        algo = str(algo).strip().lower()
        if algo not in allowed:
            raise ValidationError(f"不支持的算法: '{algo}', 支持: {allowed}")
        result.append(algo)
    return result or ['greedy']

def sanitize_filename(filename: str, max_len: int = 128) -> str:
    dangerous = r'[<>:"/\\|?*\';&$`!#]'
    sanitized = re.sub(dangerous, '_', filename)
    path_traversal = r'(\.\./|\.\.\\)'
    sanitized = re.sub(path_traversal, '', sanitized)
    sanitized = sanitized.strip('. ')
    if len(sanitized) > max_len:
        name, ext = os.path.splitext(sanitized)
        sanitized = name[:max_len - len(ext)] + ext
    if not sanitized:
        sanitized = 'download'
    return sanitized

SQL_INJECTION_PATTERNS = re.compile(
    r"(UNION\s+SELECT|DROP\s+TABLE|ALTER\s+TABLE|INSERT\s+INTO|"
    r"DELETE\s+FROM|--\s|\bOR\b\s+['\"]?\d['\"]?\s*=\s*['\"]?\d)",
    re.IGNORECASE
)

XSS_PATTERNS = re.compile(
```

```
    r"(<script|javascript:|onerror\s*=|onload\s*=|"
    r"<img[^>]+onerror|<svg|eval\s*\()",
    re.IGNORECASE
)

def detect_sql_injection(value: str) -> bool:
    return bool(SQL_INJECTION_PATTERNS.search(str(value)))

def detect_xss(value: str) -> bool:
    return bool(XSS_PATTERNS.search(str(value)))

def sanitize_html(value: str) -> str:
    return (value
            .replace('&', '&amp;')
            .replace('<', '&lt;')
            .replace('>', '&gt;')
            .replace('"', '&quot;')
            .replace("'", '&#x27;'))

def validate_simulation_params(data: Dict[str, Any]) -> Dict[str,
Any]:
    return {
        'num_servers': validate_int(
            data.get('num_servers'), min_val=1, max_val=1000,
default=10
    ,
            name='服务器数量'),
        'simulation_hours': validate_int(
            data.get('simulation_hours'), min_val=1, max_val=720,
defaul
    t=24,
            name='仿真时长'),
        'parallel_workers': validate_int(
            data.get('parallel_workers'), min_val=1, max_val=32,
default
    =4,
            name='并行工作器'),
        'run_mode': validate_str(
            data.get('run_mode'), max_len=32,
            pattern=r'^(standard|distributed|large_scale)$',
default='st
    andard',
            name='运行模式'),
        'server_type': validate_str(
```

```
            data.get('server_type'), max_len=32,
            pattern=r'^(mixed|standard|high_performance|gpu_server)$', d
    efault='mixed',
            name='服务器类型'),
        'algorithms': validate_algorithms(data.get('algorithms')),
        'visualization': bool(data.get('visualization', True)),
        'save_results': bool(data.get('save_results', True)),
    }

def security_headers() -> Dict[str, str]:
    return {
        'X-Content-Type-Options': 'nosniff',
        'X-Frame-Options': 'DENY',
        'X-XSS-Protection': '1; mode=block',
        'Referrer-Policy': 'strict-origin-when-cross-origin',
        'Permissions-Policy': 'camera=(), microphone=(self), geolocation
    =()',
        'Strict-Transport-Security': 'max-age=31536000; includeSubDomain
    s',
        'Content-Security-Policy': (
            "default-src 'self'; "
            "script-src 'self' 'unsafe-inline' https://cdn.jsdelivr.net
    https://cdnjs.cloudflare.com; "
            "style-src 'self' 'unsafe-inline' https://cdn.jsdelivr.net h
    ttps://cdnjs.cloudflare.com; "
            "img-src 'self' data:; "
            "font-src 'self' https://cdnjs.cloudflare.com; "
            "connect-src 'self'; "
            "frame-ancestors 'none'; "
            "form-action 'self'"
        ),
    }

def generate_csrf_token() -> str:
    return secrets.token_hex(32)

def hash_sha256(data: bytes) -> str:
    return hashlib.sha256(data).hexdigest()

```

```
class RateLimiter:
    """简易令牌桶速率限制器"""

    def __init__(self, max_requests: int = 60, window_sec: float = 60.0):
        self.max_requests = max_requests
        self.window_sec = window_sec
        self._clients: Dict[str, List[float]] = {}
        import threading
        self._lock = threading.Lock()

    def _cleanup(self, now: float) -> None:
        for ip in list(self._clients.keys()):
            self._clients[ip] = [t for t in self._clients[ip] if now - t < self.window_sec]
            if not self._clients[ip]:
                del self._clients[ip]

    def is_allowed(self, client_ip: str) -> bool:
        import time
        now = time.time()
        with self._lock:
            self._cleanup(now)
            if client_ip not in self._clients:
                self._clients[client_ip] = []
            if len(self._clients[client_ip]) >= self.max_requests:
                return False
            self._clients[client_ip].append(now)
            return True

    def remaining(self, client_ip: str) -> int:
        import time
        now = time.time()
        with self._lock:
            self._cleanup(now)
            return max(0, self.max_requests - len(self._clients.get(client_ip, [])))
```

79. src/visualization/__init__.py

```
from .plot_utils import (
    plot_energy_comparison,
```

```
    plot_temperature_trend,
    plot_load_distribution,
    plot_pue_comparison,
    plot_cooling_power,
    plot_service_rate,
    plot_summary_stats
)

__all__ = [
    'plot_energy_comparison',
    'plot_temperature_trend',
    'plot_load_distribution',
    'plot_pue_comparison',
    'plot_cooling_power',
    'plot_service_rate',
    'plot_summary_stats'
]
```

80. src/visualization/plot_utils.py

```
import matplotlib
matplotlib.use('Agg')

import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
from typing import Optional, Sequence, Tuple

PALETTE = ['#2f6feb', '#e8833a', '#3fa66a', '#b4545b', '#7a6bd6', '#0f8b8d', '#8a8f98']
TEMP_REFERENCE_C = 35.0
PUE_REFERENCE = 1.6


def _render(fig: plt.Figure, save_path: Optional[str]) -> plt.Figure:
    """Save and close. Closing matters inside a long-running web process."""
    if save_path:
        fig.savefig(save_path, dpi=300, bbox_inches='tight')
    plt.close(fig)
    return fig

```

```
 21|
 22|def _apply_style(style: str) -> None:
 23|    try:
 24|        plt.style.use(style)
 25|    except Exception:
 26|        plt.style.use('default')
 27|
 28|
 29|def plot_energy_comparison(
 30|    df: pd.DataFrame,
 31|    save_path: Optional[str] = None,
 32|    figsize: Tuple[int, int] = (12, 6),
 33|    style: str = 'seaborn-v0_8-darkgrid'
 34|) -> plt.Figure:
 35|    _apply_style(style)
 36|    fig, ax = plt.subplots(figsize=figsize)
 37|    sns.lineplot(data=df, x='time', y='total_energy',
hue='algorithm', m
   |    arker='o', ax=ax)
 38|    ax.set_title('Energy Consumption Comparison')
 39|    ax.set_xlabel('Time (hours)')
 40|    ax.set_ylabel('Total Energy (W)')
 41|    ax.legend(title='Algorithm')
 42|    ax.grid(True)
 43|    return _render(fig, save_path)
 44|
 45|
 46|def plot_temperature_trend(
 47|    df: pd.DataFrame,
 48|    save_path: Optional[str] = None,
 49|    figsize: Tuple[int, int] = (12, 6),
 50|    style: str = 'seaborn-v0_8-darkgrid',
 51|    reference_c: Optional[float] = TEMP_REFERENCE_C,
 52|    peak: bool = False
 53|) -> plt.Figure:
 54|    _apply_style(style)
 55|    y = 'peak_temperature' if peak and 'peak_temperature' in
df.columns
   |    else 'avg_temperature'
 56|    fig, ax = plt.subplots(figsize=figsize)
 57|    sns.lineplot(data=df, x='time', y=y, hue='algorithm',
marker='s', ax
   |    =ax)
 58|    if 'ambient_temperature' in df.columns:
```

```
        ambient = df.groupby('time')['ambient_temperature'].first()
.rese
    t_index()
        ax.plot(ambient['time'], ambient['ambient_temperature'],
               color='#6b7280', linestyle=':', linewidth=1.4, label='环
    境温度')
    if reference_c is not None:
        ax.axhline(y=reference_c, color='red', linestyle='--',
                   label=f'参考线 ({reference_c:g}° C)')
    ax.set_title('Temperature Trend Comparison')
    ax.set_xlabel('Time (hours)')
    ax.set_ylabel('Temperature (°C)')
    ax.legend()
    ax.grid(True)
    return _render(fig, save_path)


def plot_load_distribution(
    df: pd.DataFrame,
    save_path: Optional[str] = None,
    figsize: Tuple[int, int] = (12, 6),
    style: str = 'seaborn-v0_8-darkgrid'
) -> plt.Figure:
    _apply_style(style)
    fig, ax = plt.subplots(figsize=figsize)
    sns.lineplot(data=df, x='time', y='total_load',
hue='algorithm', mar
    ker='^', ax=ax)
    ax.set_title('Total Load Distribution')
    ax.set_xlabel('Time (hours)')
    ax.set_ylabel('Total Load')
    ax.legend(title='Algorithm')
    ax.grid(True)
    return _render(fig, save_path)


def plot_pue_comparison(
    df: pd.DataFrame,
    save_path: Optional[str] = None,
    figsize: Tuple[int, int] = (12, 6),
    style: str = 'seaborn-v0_8-darkgrid',
    reference: Optional[float] = PUE_REFERENCE
) -> plt.Figure:
    _apply_style(style)
```

```
    fig, ax = plt.subplots(figsize=figsize)
    sns.lineplot(data=df, x='time', y='pue', hue='algorithm',
marker='d'
    , ax=ax)
    if reference is not None:
        ax.axhline(y=reference, color='green', linestyle='--',
                   label=f'参考线 (PUE {reference:g})')
    ax.set_title('PUE Comparison')
    ax.set_xlabel('Time (hours)')
    ax.set_ylabel('PUE')
    ax.legend()
    ax.grid(True)
    return _render(fig, save_path)


def plot_cooling_power(
    df: pd.DataFrame,
    save_path: Optional[str] = None,
    figsize: Tuple[int, int] = (12, 6),
    style: str = 'seaborn-v0_8-darkgrid'
) -> plt.Figure:
    _apply_style(style)
    fig, ax = plt.subplots(figsize=figsize)
    sns.lineplot(data=df, x='time', y='cooling_power',
hue='algorithm',
    marker='*', ax=ax)
    ax.set_title('Cooling Power Consumption')
    ax.set_xlabel('Time (hours)')
    ax.set_ylabel('Cooling Power (W)')
    ax.legend(title='Algorithm')
    ax.grid(True)
    return _render(fig, save_path)


def plot_service_rate(
    df: pd.DataFrame,
    save_path: Optional[str] = None,
    figsize: Tuple[int, int] = (12, 6),
    style: str = 'seaborn-v0_8-darkgrid'
) -> plt.Figure:
    _apply_style(style)
    fig, ax = plt.subplots(figsize=figsize)
    hourly = df.copy()
```

```
    hourly['service_rate'] = hourly['assigned'] /
(hourly['assigned'] +
    hourly['dropped']).replace(0, pd.NA)
    sns.lineplot(data=hourly, x='time', y='service_rate',
hue='algorithm
    ', marker='o', ax=ax)
    ax.set_ylim(0, 1.05)
    ax.set_title('Task Service Rate')
    ax.set_xlabel('Time (hours)')
    ax.set_ylabel('Service Rate')
    ax.legend(title='Algorithm')
    ax.grid(True)
    return _render(fig, save_path)


def plot_summary_stats(
    stats: dict,
    save_path: Optional[str] = None,
    figsize: Tuple[int, int] = (14, 10),
    style: str = 'seaborn-v0_8-darkgrid'
) -> plt.Figure:
    _apply_style(style)
    algorithms = list(stats.keys())
    metrics = ['avg_energy', 'max_temp', 'avg_pue', '
service_rate']
    metric_names = ['Avg Energy (W)', 'Peak Temp (°C)', 'Avg
PUE', 'Serv
    ice Rate']
    colors: Sequence[str] = [PALETTE[i % len(PALETTE)] for i in
range(le
    n(algorithms))]

    fig, axes = plt.subplots(2, 2, figsize=figsize)

    for i, (metric, name) in enumerate(zip(metrics,
metric_names)):
        row, col = i // 2, i % 2
        values = [stats[algo].get(metric) or 0 for algo in
algorithms]
        axes[row, col].bar(algorithms, values, color=colors)
        axes[row, col].set_title(name)
        axes[row, col].grid(True)

    plt.tight_layout()
```

```
170|    return _render(fig, save_path)
```

81. tools/etl/build_catalog.py

```
  1|"""Build data/catalog.db from offline snapshots in data/raw.
  2|
  3|Each dataset is a CSV plus a sidecar JSON describing where it
came from:
  4|
  5|    data/raw/ambient/<name>.csv
  6|    data/raw/ambient/<name>.json
  7|
  8|The JSON sidecar carries publisher / url / license /
retrieved_at, which
   |     is what
  9|makes the numbers in a report traceable. A dataset without a
sidecar is
   |    refused,
 10|so "where did this come from" can never be answered with a shrug.
 11|
 12|Usage:
 13|    python tools/etl/build_catalog.py --list
 14|    python tools/etl/build_catalog.py --db data/catalog.db
 15|    python tools/etl/build_catalog.py --kind trace --scale-keep
0.35 --s
   |    la-slack 1.5
 16|"""
 17|
 18|import argparse
 19|import csv
 20|import hashlib
 21|import io
 22|import json
 23|import os
 24|import sqlite3
 25|import sys
 26|import time
 27|from typing import Any, Dict, List, Optional, Tuple
 28|
 29|HERE = os.path.dirname(os.path.abspath(__file__))
 30|PROJECT_ROOT = os.path.abspath(os.path.join(HERE, "..", ".."))
 31|RAW_ROOT = os.path.join(PROJECT_ROOT, "data", "raw")
 32|
 33|
 34|def set_raw_root(path):
```

```
    global RAW_ROOT
    RAW_ROOT = os.path.abspath(path)
    return RAW_ROOT
SCHEMA_PATH = os.path.join(HERE, "schema.sql")

KINDS = ("ambient", "trace", "standards", "emission", "server_type", "schools", "climate")

REQUIRED_COLUMNS = {
    "ambient": ("ts", "temp_c"),
    "trace": ("load_units",),
    "standards": ("standard_id", "pue_limit"),
    "emission": ("region", "year", "kg_co2_per_kwh"),
    "server_type": ("server_type",),
    "schools": ("school_name", "city"),
    "climate": ("city", "date", "tmax_c", "tmin_c", "tmean_c"),
}


def sha256_of(path: str) -> str:
    digest = hashlib.sha256()
    with open(path, "rb") as f:
        for chunk in iter(lambda: f.read(65536), b""):
            digest.update(chunk)
    return digest.hexdigest()


def read_csv(path: str) -> List[Dict[str, str]]:
    with io.open(path, "r", encoding="utf-8-sig", newline="") as f:
        return list(csv.DictReader(f))


def read_sidecar(csv_path: str) -> Optional[Dict[str, Any]]:
    meta_path = os.path.splitext(csv_path)[0] + ".json"
    if not os.path.exists(meta_path):
        return None
    with io.open(meta_path, "r", encoding="utf-8") as f:
        return json.load(f)


def dataset_files(kind: str) -> List[str]:
    folder = os.path.join(RAW_ROOT, kind)
```

```
    if not os.path.isdir(folder):
        return []
    return sorted(
        os.path.join(folder, name)
        for name in os.listdir(folder)
        if name.lower().endswith(".csv")
    )


def list_datasets() -> None:
    print("数据目录快照 (data/raw):")
    found = False
    for kind in KINDS:
        for path in dataset_files(kind):
            found = True
            meta = read_sidecar(path)
            marker = "有出处" if meta else "缺少 sidecar JSON (不会导入)"
            print("  [%s] %s  <- %s" % (kind, os.path.basename(path), ma
rker))
    if not found:
        print("  (空) 请按 tools/etl/README.md 的格式放入 CSV 与 sidecar
 JSON")


def connect(db_path: str) -> sqlite3.Connection:
    os.makedirs(os.path.dirname(db_path), exist_ok=True)
    with io.open(SCHEMA_PATH, "r", encoding="utf-8") as f:
        schema = f.read()
    conn = sqlite3.connect(db_path)
    conn.executescript(schema)
    conn.commit()
    return conn


def register_source(conn: sqlite3.Connection, source_id: str, kind: str,
                    csv_path: str, meta: Dict[str, Any]) -> None:
    conn.execute(
        """INSERT OR REPLACE INTO meta_data_source
           (source_id, name, publisher, url, license, retrieved_at, sha2
56, etl_script, note)
```

```
            VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)""",
        (
            source_id,
            meta.get("name") or os.path.basename(csv_path),
            meta.get("publisher", ""),
            meta.get("url", ""),
            meta.get("license", ""),
            meta.get("retrieved_at", ""),
            sha256_of(csv_path),
            "tools/etl/build_catalog.py",
            meta.get("note", ""),
        ),
    )


def register_version(conn: sqlite3.Connection, source_id: str,
kind: str
   |    ,
                     table_name: str, row_count: int, csv_path:
str,
                     hash_key: str, span: Tuple[str, str]) -> str:
    version_id = "%s:%s:%s" % (kind, source_id, hash_key[:12])
    conn.execute(
        """INSERT OR REPLACE INTO meta_dataset_version
           (version_id, source_id, table_name, row_count, min_ts,
max_ts
   |    , hash, created_at)
           VALUES (?, ?, ?, ?, ?, ?, ?, ?)""",
        (version_id, source_id, table_name, row_count, span[0],
span[1],
         hash_key, time.strftime("%Y-%m-%d %H:%M:%S")),
    )
    return version_id


def _num(value: Any, cast=float, default=None):
    try:
        return cast(value)
    except (TypeError, ValueError):
        return default


def import_ambient(conn, csv_path, meta, source_id) -> int:
    rows = read_csv(csv_path)
```

```
    site_id = str(meta.get("site_id") or "default")
    payload = []
    timestamps = []
    for row in rows:
        ts = str(row.get("ts") or row.get("time") or "")
        temp = _num(row.get("temp_c") if row.get("temp_c") is not
None e
    lse row.get("temperature"))
        if ts == "" or temp is None:
            continue
        timestamps.append(ts)
        payload.append((site_id, ts, temp,
                        _num(row.get("humidity")),
_num(row.get("wind_ms
    ")),
                        source_id, ""))
    if not payload:
        return 0
    conn.execute(
        "DELETE FROM dim_ambient_hourly WHERE site_id = ? AND
source_id
    = ?",
        (site_id, source_id),
    )
    version_id = register_version(conn, source_id, "ambient", "
dim_ambie
    nt_hourly",
                                  len(payload), csv_path,
sha256_of(csv_p
    ath),
                                  (min(timestamps),
max(timestamps)))
    conn.executemany(
        """INSERT OR REPLACE INTO dim_ambient_hourly
           (site_id, ts, temp_c, humidity, wind_ms, source_id,
version_i
    d)
           VALUES (?, ?, ?, ?, ?, ?, ?)""",
        [(p[0], p[1], p[2], p[3], p[4], p[5], version_id) for p
in paylo
    ad],
    )
    return len(payload)

```

```

def _hour_of(row: Dict[str, Any], base_ts: Optional[float]) ->
Optional[
    int]:
    if row.get("hour") not in (None, ""):
        return int(float(row["hour"]))
    if row.get("arrival_hour") not in (None, ""):
        return int(float(row["arrival_hour"]))
    if row.get("ts") not in (None, "") and base_ts is not None:
        try:
            return int((float(row["ts"]) - base_ts) // 3600)
        except (TypeError, ValueError):
            return None
    return None


def import_trace(conn, csv_path, meta, source_id, scale_keep:
float,
                 sla_slack: float, seed: int) -> int:
    import random

    rows = read_csv(csv_path)
    if not rows:
        return 0

    base_ts = None
    if "ts" in rows[0] and "hour" not in rows[0] and "
arrival_hour" not
    in rows[0]:
        values = [_num(r.get("ts")) for r in rows]
        values = [v for v in values if v is not None]
        base_ts = min(values) if values else None

    if scale_keep < 1.0:
        rng = random.Random(seed)
        rows = [row for row in rows if rng.random() < scale_keep]

    trace_id = str(meta.get("trace_id") or
os.path.splitext(os.path.base
    name(csv_path))[0])
    payload = []
    seq = 0
    for row in rows:
        hour = _hour_of(row, base_ts)
```

```
        load = _num(row.get("load_units") or row.get("load"), int)
        if hour is None or load is None or hour < 0:
            continue
        duration = _num(row.get("duration_h") or
row.get("duration"), in
    t, 1) or 1
        offset = _num(row.get("deadline_offset_h"), int)
        if offset is None:
            offset = max(duration, int(round(duration *
sla_slack)), 1)
        payload.append((
            trace_id, seq, hour, int(load),
            _num(row.get("priority"), int, 3),
            str(row.get("workload_class") or "cpu_bound"),
            int(duration), int(offset), source_id,
        ))
        seq += 1

    if not payload:
        return 0

    conn.execute("DELETE FROM fact_task_arrivals WHERE trace_id = ?
", (t
    race_id,))
    hours = [p[2] for p in payload]
    register_version(conn, source_id, "trace", "
fact_task_arrivals", len
    (payload),
                     csv_path, sha256_of(csv_path),
(str(min(hours)), st
    r(max(hours))))
    conn.executemany(
        """INSERT OR REPLACE INTO fact_task_arrivals
           (trace_id, seq, hour, load_units, priority,
workload_class,
            duration_h, deadline_offset_h, source_id)
           VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)""",
        payload,
    )
    return len(payload)


def import_standards(conn, csv_path, meta, source_id) -> int:
    rows = read_csv(csv_path)
```

```
    payload = []
    for row in rows:
        limit = _num(row.get("pue_limit"))
        if limit is None:
            continue
        payload.append((str(row.get("standard_id") or
meta.get("name") o
    r ""),
                        str(row.get("grade") or ""), limit,
                        str(row.get("applies_to") or ""),
source_id))
    if payload:
        conn.execute("DELETE FROM dim_pue_benchmark WHERE
source_id = ?"
    , (source_id,))
        register_version(conn, source_id, "standards", "
dim_pue_benchmar
    k", len(payload),
                         csv_path, sha256_of(csv_path), ("", ""))
        conn.executemany(
            """INSERT INTO dim_pue_benchmark
               (standard_id, grade, pue_limit, applies_to,
source_id)
               VALUES (?, ?, ?, ?, ?)""",
            payload,
        )
    return len(payload)


def import_emission(conn, csv_path, meta, source_id) -> int:
    rows = read_csv(csv_path)
    payload = []
    for row in rows:
        region = str(row.get("region") or "")
        year = _num(row.get("year"), int)
        factor = _num(row.get("kg_co2_per_kwh"))
        if not region or year is None or factor is None:
            continue
        payload.append((region, year, factor, source_id))
    if payload:
        conn.execute("DELETE FROM dim_emission_factor WHERE
source_id =
    ?", (source_id,))
```

```
        register_version(conn, source_id, "emission", "dim_emission_fact
    or", len(payload),
                         csv_path, sha256_of(csv_path), ("", ""))
        conn.executemany(
            """INSERT INTO dim_emission_factor
               (region, year, kg_co2_per_kwh, source_id) VALUES (?, ?, ?
    , ?)""",
            payload,
        )
    return len(payload)


def import_server_type(conn, csv_path, meta, source_id) -> int:
    rows = read_csv(csv_path)
    payload = []
    for row in rows:
        name = str(row.get("server_type") or row.get("name") or "")
        if not name:
            continue
        payload.append((name, _num(row.get("max_load"), float, 100.0),
                        _num(row.get("max_temp"), float, 85.0),
                        _num(row.get("base_power"), float, 100.0),
                        _num(row.get("max_power"), float, 300.0),
                        _num(row.get("efficiency"), float, 0.9), source_
    id))
    if payload:
        conn.execute("DELETE FROM dim_server_type WHERE source_id = ?",
    (source_id,))
        register_version(conn, source_id, "server_type", "dim_server_typ
    e", len(payload),
                         csv_path, sha256_of(csv_path), ("", ""))
        conn.executemany(
            """INSERT OR REPLACE INTO dim_server_type
               (server_type, max_load, max_temp, base_power, max_power,
    efficiency, source_id)
               VALUES (?, ?, ?, ?, ?, ?, ?)""",
```

```
            payload,
        )
    return len(payload)


def import_schools(conn, csv_path, meta, source_id) -> int:
    rows = read_csv(csv_path)
    payload = []
    for row in rows:
        name = str(row.get("school_name") or "").strip()
        city = str(row.get("city") or "").strip()
        if not name or not city:
            continue
        payload.append((name, str(row.get("province") or ""),
city,
                        str(row.get("level") or ""),
str(row.get("depart
   |    ment") or ""),
                        source_id))
    if payload:
        conn.execute("DELETE FROM dim_school WHERE source_id = ?
", (sour
   |    ce_id,))
        register_version(conn, source_id, "schools", "
dim_school", len(p
   |    ayload),
                         csv_path, sha256_of(csv_path), ("", ""))
        conn.executemany(
            """INSERT INTO dim_school
               (school_name, province, city, level, department,
source_i
   |    d)
               VALUES (?, ?, ?, ?, ?, ?)""",
            payload,
        )
    return len(payload)


def import_climate(conn, csv_path, meta, source_id) -> int:
    rows = read_csv(csv_path)
    payload = []
    dates = []
    for row in rows:
        city = str(row.get("city") or "").strip()
```

```
        date = str(row.get("date") or "").strip()
        tmax = _num(row.get("tmax_c"))
        tmin = _num(row.get("tmin_c"))
        tmean = _num(row.get("tmean_c"))
        if not city or not date or tmax is None or tmin is None
or tmean
    |     is None:
            continue
        dates.append(date)
        payload.append((city, date, tmax, tmin, tmean, source_id))
    if payload:
        conn.execute("DELETE FROM dim_city_daily WHERE source_id
= ?", (
    |    source_id,))
        register_version(conn, source_id, "climate", "
dim_city_daily", l
    |    en(payload),
                         csv_path, sha256_of(csv_path),
                         (min(dates), max(dates)))
        conn.executemany(
            """INSERT OR REPLACE INTO dim_city_daily
               (city, date, tmax_c, tmin_c, tmean_c, source_id)
               VALUES (?, ?, ?, ?, ?, ?)""",
            payload,
        )
    return len(payload)


IMPORTERS = {
    "ambient": import_ambient,
    "schools": import_schools,
    "climate": import_climate,
    "standards": import_standards,
    "emission": import_emission,
    "server_type": import_server_type,
}


def load_kind(conn, kind: str, args) -> int:
    total = 0
    for csv_path in dataset_files(kind):
        meta = read_sidecar(csv_path)
        basename = os.path.splitext(os.path.basename(csv_path))[0]
        if meta is None:
```

```
            print("  跳过 %s: 缺少出处 sidecar (%s.json)" % (basename, b
    asename))
             continue
        missing = [c for c in REQUIRED_COLUMNS.get(kind, ()) if c
not in
     (read_csv(csv_path)[0] if read_csv(csv_path) else {})]
        if missing:
            print("  跳过 %s: 缺少列 %s" % (basename, ", ".join(missing)
    ))
             continue
        source_id = str(meta.get("source_id") or "%s:%s" % (kind,
basena
    me))
        register_source(conn, source_id, kind, csv_path, meta)
        if kind == "trace":
            count = import_trace(conn, csv_path, meta, source_id,
                                 args.scale_keep, args.sla_slack,
args.s
    eed)
        else:
            count = IMPORTERS[kind](conn, csv_path, meta,
source_id)
        conn.commit()
        print("  导入 [%s] %s -> %d 行 (source_id=%s)" % (kind, basename
    , count, source_id))
        total += count
    return total


def main() -> int:
    parser = argparse.ArgumentParser(description="Build the
provenance-t
    agged data catalog")
    parser.add_argument("--db", default=os.path.join(PROJECT_ROOT,
 "data
    ", "catalog.db"))
    parser.add_argument("--kind", default=None,
choices=list(KINDS),
                        help="只导入某一类数据，默认全部")
    parser.add_argument("--scale-keep", type=float, default=1.0,
                        help="trace 降采样保留比例 (0, 1]")
    parser.add_argument("--sla-slack", type=float, default=1.5,
                        help="缺失 deadline 时使用的 SLA 松弛系数")
    parser.add_argument("--seed", type=int, default=42, help="降采样随机
```

```
    种子")
    parser.add_argument("--list", action="store_true", help="只列出可用快照")
    parser.add_argument("--raw-root", default=RAW_ROOT,
                        help="快照根目录，默认 data/raw")
    args = parser.parse_args()
    set_raw_root(args.raw_root)

    if args.list:
        list_datasets()
        return 0

    conn = connect(args.db)
    kinds = [args.kind] if args.kind else list(KINDS)
    print("构建数据目录: %s" % args.db)
    total = 0
    for kind in kinds:
        total += load_kind(conn, kind, args)
    conn.commit()

    tables = [row[0] for row in conn.execute(
        "SELECT name FROM sqlite_master WHERE type='table' ORDER BY name
    ")]
    print("完成: 共写入 %d 行, 表: %s" % (total, ", ".join(tables)))
    if total == 0:
        print("\n提示: data/raw 下暂无可用快照。数据获取方式见 tools/etl/README.md ")
    conn.close()
    return 0


if __name__ == "__main__":
    sys.exit(main())
```

82. tools/etl/import_schools.py

```
"""Import the official MOE school list and per-city climate.

Source: Ministry of Education, "全国高等学校名单" (2025), published as .xls.
The list gives school name, supervising authority, city and level. Coordinates
```

```
  5|are resolved with Open-Meteo geocoding, then daily 2025
temperature is f
   |    etched
  6|from the ERA5 archive. Everything is cached under data/raw so
re-runs ar
   |    e free.
  7|
  8|Usage:
  9|    python tools/etl/import_schools.py --xls <file.xls>
 10|    python tools/etl/import_schools.py --xls <file.xls>
--max-cities 10
 11|"""
 12|
 13|import argparse
 14|import csv
 15|import io
 16|import json
 17|import os
 18|import sys
 19|import time
 20|import urllib.parse
 21|import urllib.request
 22|
 23|import xlrd
 24|
 25|HERE = os.path.dirname(os.path.abspath(__file__))
 26|PROJECT_ROOT = os.path.abspath(os.path.join(HERE, "..", ".."))
 27|RAW = os.path.join(PROJECT_ROOT, "data", "raw")
 28|SCHOOL_DIR = os.path.join(RAW, "schools")
 29|CLIMATE_DIR = os.path.join(RAW, "climate")
 30|CACHE_COORDS = os.path.join(SCHOOL_DIR, "city_coords.json")
 31|
 32|UA = "codex-campus-dc-import/1.0 (competition demo)"
 33|DELAY = 0.35
 34|
 35|
 36|def http_json(url, retries=4):
 37|    req = urllib.request.Request(url, headers={"User-Agent": UA})
 38|    for attempt in range(retries):
 39|        try:
 40|            with urllib.request.urlopen(req, timeout=40) as resp:
 41|                return json.loads(resp.read().decode("utf-8"))
 42|        except Exception as exc:
 43|            if attempt == retries - 1:
```

```
                raise
            time.sleep(2 * (attempt + 1))
    return None


def parse_schools(xls_path):
    wb = xlrd.open_workbook(xls_path)
    schools = []
    for sheet_index in range(wb.nsheets):
        sh = wb.sheet_by_index(sheet_index)
        province = ""
        for r in range(sh.nrows):
            cells = [str(sh.cell_value(r, c)).strip() for c in
range(sh.
    ncols)]
            if not any(cells):
                continue
            joined = " ".join(cells)
            if "（" in joined and "所）" in joined and not cells[1]:
                province = joined.split(" ")[0].strip()
                continue
            if len(cells) < 4:
                continue
            name = cells[1]
            if not name or name in ("学校名称",):
                continue
            city = ""
            level = ""
            dept = cells[3] if len(cells) > 3 else ""
            if sh.ncols >= 7:
                city = cells[4]
                level = cells[5]
            else:
                city = province
                level = "成人高校"
            if city and name and not name.startswith("附件"):
                schools.append({
                    "school_name": name,
                    "province": province,
                    "city": city,
                    "level": level or "未知",
                    "department": dept,
                    "sheet": sh.name,
                })
```

```
    return schools


def load_cache():
    if os.path.exists(CACHE_COORDS):
        with io.open(CACHE_COORDS, "r", encoding="utf-8") as f:
            return json.load(f)
    return {}


def save_cache(cache):
    with io.open(CACHE_COORDS, "w", encoding="utf-8") as f:
        json.dump(cache, f, ensure_ascii=False, indent=2)


def geocode(name, cache):
    """Resolve a Chinese city name with Nominatim (administrative level
    only)."""
    if name in cache:
        return cache[name]
    candidates = [name]
    for suffix in ("市", "地区", "盟", "自治州", "自治县"):
        if name.endswith(suffix) and len(name) > len(suffix):
            candidates.append(name[: -len(suffix)])
    found = None
    for candidate in candidates:
        url = ("https://nominatim.openstreetmap.org/search?q="
               + urllib.parse.quote(candidate)
               + "&format=jsonv2&limit=3&countrycodes=cn&featurety
pe=cit
    y")
        try:
            payload = http_json(url)
        except Exception:
            payload = None
        if isinstance(payload, list) and payload:
            item = payload[0]
            found = {
                "name": item.get("display_name",
candidate).split(",")[0
    ],
                "latitude": float(item.get("lat")),
                "longitude": float(item.get("lon")),
```

```
                "admin1": item.get("display_name", "
").split(",")[1].str
    ip()
                          if "," in item.get("display_name", "")
else ""
    ,
               "country": "中国",
            }
            break
        time.sleep(1.1)
    cache[name] = found
    time.sleep(1.1)
    return found


def fetch_climate(city, coords):
    path = os.path.join(CLIMATE_DIR, city + ".json")
    if os.path.exists(path):
        with io.open(path, "r", encoding="utf-8") as f:
            return json.load(f)
    url = ("https://archive-api.open-meteo.com/v1/archive?latitude
={lat}
    &longitude={lon}"
           "&start_date=2025-01-01&end_date=2025-12-31"
           "&daily=temperature_2m_max,temperature_2m_min,temperatu
re_2m_
    mean"
           "&timezone=Asia%2FShanghai").format(lat=coords["latitud
e"], l
    on=coords["longitude"])
    payload = http_json(url)
    payload["_url"] = url
    with io.open(path, "w", encoding="utf-8") as f:
        json.dump(payload, f, ensure_ascii=False)
    time.sleep(DELAY)
    return payload


def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--xls", required=True)
    parser.add_argument("--max-cities", type=int, default=0)
    args = parser.parse_args()

```

```
    os.makedirs(SCHOOL_DIR, exist_ok=True)
    os.makedirs(CLIMATE_DIR, exist_ok=True)

    schools = parse_schools(args.xls)
    print("schools:", len(schools))
    cities = sorted({s["city"] for s in schools if s["city"]})
    if args.max_cities:
        cities = cities[: args.max_cities]
    print("cities:", len(cities))

    cache = load_cache()
    resolved = {}
    for index, city in enumerate(cities, 1):
        coords = geocode(city, cache)
        if coords:
            resolved[city] = coords
        if index % 25 == 0:
            save_cache(cache)
            print("  geocoded %d/%d" % (index, len(cities)),
flush=True)
    save_cache(cache)
    print("geocoded:", len(resolved), "/", len(cities))

    ok = 0
    for index, city in enumerate(sorted(resolved), 1):
        try:
            fetch_climate(city, resolved[city])
            ok += 1
        except Exception as exc:
            print("  climate failed for %s: %s" % (city, exc))
        if index % 25 == 0:
            print("  climate %d/%d" % (index, len(resolved)),
flush=True
    )
    print("climate fetched:", ok, "/", len(resolved))

    with io.open(os.path.join(SCHOOL_DIR, "schools.csv"), "w",
encoding=
    "utf-8", newline="") as f:
        writer = csv.DictWriter(f, fieldnames=["school_name", "
province"
    , "city", "level", "department"])
        writer.writeheader()
        for s in schools:
```

```
            writer.writerow({k: s[k] for k in writer.fieldnames})

    rows = []
    for city in sorted(resolved):
        path = os.path.join(CLIMATE_DIR, city + ".json")
        if not os.path.exists(path):
            continue
        with io.open(path, "r", encoding="utf-8") as f:
            data = json.load(f)
        daily = data.get("daily") or {}
        for date, tmax, tmin, tmean in zip(
                daily.get("time", []),
                daily.get("temperature_2m_max", []),
                daily.get("temperature_2m_min", []),
                daily.get("temperature_2m_mean", [])):
            if tmax is None or tmin is None or tmean is None:
                continue
            rows.append((city, date, tmax, tmin, tmean))
    with io.open(os.path.join(CLIMATE_DIR, "city_daily.csv"), "w", encod
    ing="utf-8", newline="") as f:
        f.write("city,date,tmax_c,tmin_c,tmean_c\n")
        for r in rows:
            f.write("%s,%s,%.1f,%.1f,%.1f\n" % r)
    print("daily climate rows:", len(rows))

    meta = {
        "source_id": "moe-school-list-2025",
        "name": "全国高等学校名单（2025）",
        "publisher": "中华人民共和国教育部",
        "url": "https://www.moe.gov.cn/jyb_xxgk/s5743/s5744/A03/202506/t
    20250627_1195683.html",
        "license": "政府公开信息",
        "retrieved_at": "2026-10-17",
        "note": "含普通高校与成人高校；所在地为地级市；经纬度由 Nominatim（
    OpenStreetMap，ODbL）行政区级解析",
    }
    with io.open(os.path.join(SCHOOL_DIR, "schools.json"), "w", encoding
    ="utf-8") as f:
        json.dump(meta, f, ensure_ascii=False, indent=2)

```

```
    climate_meta = {
        "source_id": "open-meteo-era5-cities-2025",
        "name": "中国各城市 2025 年逐日气温（ERA5 再分析）",
        "publisher": "Open-Meteo / ECMWF ERA5",
        "url": "https://archive-api.open-meteo.com/v1/archive",
        "license": "CC BY 4.0 Open-Meteo  ERA5 © ECMWF",
        "retrieved_at": "2026-10-17",
        "note": "逐日最高/最低/平均气温；仿真时按标准日变化曲线展开为逐时值",
    }
    with io.open(os.path.join(CLIMATE_DIR, "city_daily.json"), "w", encoding="utf-8") as f:
        json.dump(climate_meta, f, ensure_ascii=False, indent=2)

    print("done")


if __name__ == "__main__":
    main()
```

83. tools/test_ai_chat.py

```
"""End-to-end checks for the floating assistant's chat endpoint.

Run with the web service up:
    python tools/test_ai_chat.py
"""

import io
import json
import os
import sys
import time
from typing import Any, Dict, List

import requests

PROJECT_ROOT = os.path.abspath(os.path.join(os.path.dirname(__file__), "..."))
if PROJECT_ROOT not in sys.path:
    sys.path.insert(0, PROJECT_ROOT)

if hasattr(sys.stdout, "reconfigure"):
```

```
    sys.stdout.reconfigure(encoding="utf-8", errors="replace")

BASE = "http://127.0.0.1:5000"
INTERNAL_FIELDS = ("max_temp", "avg_pue", "energy_per_served_load", "service_rate")
CODE_IDS = ("round_robin", "damo_ets", "greedy", "genetic", "nsga2", "mohead")

RESULTS: List[Dict[str, Any]] = []
TRANSCRIPT: List[Dict[str, Any]] = []


def check(name: str, ok: bool, detail: str = "") -> None:
    RESULTS.append({"name": name, "ok": bool(ok), "detail": detail})


def chat(messages: List[Dict[str, str]]) -> Dict[str, Any]:
    started = time.time()
    response = requests.post(BASE + "/api/v1/ai/chat",
                             json={"messages": messages}, timeout=180)
    response.raise_for_status()
    payload = response.json()
    payload["_elapsed"] = round(time.time() - started, 2)
    return payload


def hygiene(reply: str) -> None:
    lowered = reply.lower()
    found_fields = [w for w in INTERNAL_FIELDS if w in lowered]
    found_ids = [w for w in CODE_IDS if w in lowered]
    check("无内部字段名", not found_fields, ", ".join(found_fields))
    check("无代码标识符", not found_ids, ", ".join(found_ids))


def main() -> int:
    print("=" * 64)
    print("AI 助手对话测试")
    print("=" * 64)

```

```
    status = requests.get(BASE + "/api/v1/ai/status",
timeout=30).json()
    info = status.get("status", {})
    print("\n[0] 服务状态")
    print("  available=%s model=%s key_source=%s" % (
        info.get("available"), info.get("model"),
info.get("key_source")
    ))
    check("模型可用", bool(info.get("available")))
    check("Key 来自文件", info.get("key_source") == "file", str(info.get
    ("key_source")))

    metrics = requests.get(BASE + "/api/v1/metrics",
timeout=30).json()
    stats = metrics.get("data", {})
    ranking = metrics.get("ranking", {})
    best = ranking.get("best")
    best_metrics = stats.get(best, {}) if best else {}
    print("\n[1] 基准数据（/api/v1/metrics）")
    for algo, m in stats.items():
        print("  %-11s rate=%.4f W/load=%-7s peak=%-6s pue=%-7s
co2=%s"
    % (
            algo, float(m.get("service_rate") or 0),
m.get("energy_per_s
    erved_load"),
            m.get("max_temp"), m.get("avg_pue"), m.get("co2_kg")))
    print("  best=%s" % best)

    print("\n[2] 单轮问答")
    site = (metrics.get("data_source", {}) or {}).get("site_id")
or ""
    cases = [
        ("排名/最省电", "当前排名如何？哪个算法最省电？",
         [best.replace("_", "-").upper()] if best else []),
        ("温度/PUE", "各算法的峰值温度和 PUE 是多少？", []),
        ("服务率", "任务服务率怎么样？有没有任务被丢弃？", []),
        ("碳排", "当前碳排是多少？", []),
        ("数据来源", "这些数据是哪来的？可靠吗？", ["ERA5", "阿里云", "生
    态环境部"]),
        ("使用方法", "怎么运行一次仿真？", []),
        ("数据更新", "当前数据更新了吗？", [site] if site else []),
        ("最佳时段", "什么时候性价比最高？", ["小时"]),
        ("低温配置", "温度低于60度怎么配置？", ["60"]),
```

```
91|        ("配置试算", "如果服务器增加到24台会怎样？", ["24"]),
92|    ]
93|
94|    for label, question, must_contain in cases:
95|        payload = chat([{"role": "user", "content": question}])
96|        reply = payload.get("reply", "")
97|        print("\n  [%s] %s" % (label, question))
98|        print("    source=%s  %.2fs" % (payload.get("source"),
payload.g
  |    et("_elapsed", 0)))
99|        print("    " + reply.replace("\n", "\n    "))
100|        TRANSCRIPT.append({
101|            "label": label, "question": question, "reply": reply,
102|            "source": payload.get("source"), "elapsed":
payload.get("_el
  |    apsed"),
103|        })
104|        check("%s: 有回复" % label, bool(reply.strip()), "empty")
105|        check("%s: 走大模型" % label, payload.get("source") == "deepseek
  |    ",
106|              str(payload.get("source")))
107|        check("%s: 返回数据快照" % label,
108|              bool((payload.get("snapshot") or
{}).get("generated_at")),
109|              str(payload.get("snapshot")))
110|        check("%s: 无内部字段名" % label,
111|              not any(word in reply.lower() for word in
INTERNAL_FIELDS)
  |    ,
112|              ", ".join(word for word in INTERNAL_FIELDS if word
in repl
  |    y.lower()))
113|        if label == "配置试算":
114|            run = payload.get("suggested_run") or {}
115|           check("配置试算: 返回建议配置",
116|                  (run.get("params") or {}).get("num_servers") ==
24,
117|                  str(run))
118|        for token in must_contain:
119|            if token:
120|                check("%s: 提到 %s" % (label, token), token.lower() in r
  |    eply.lower(), token)
121|
122|    print("\n[3] 数值准确性")
```

```
    payload = chat([{"role": "user", "content": "排名第一的算法，单位负载能耗、峰值温度、PUE 各是多少？"}])
    reply = payload.get("reply", "")
    print("    " + reply.replace("\n", "\n    "))
    if best_metrics:
        intensity = float(best_metrics.get("energy_per_served_load") or 0)
        peak = float(best_metrics.get("max_temp") or 0)
        pue = float(best_metrics.get("avg_pue") or 0)
        check("能耗数值一致", (str(round(intensity, 2)) in reply) or (str(round(intensity, 1)) in reply),
              "expect %.4f" % intensity)
        check("峰值温度一致", str(round(peak, 1)) in reply or str(int(peak)) in reply,
              "expect %.2f" % peak)
        check("PUE 一致", str(round(pue, 3)) in reply, "expect %.4f" % pue)

    print("\n[4] 多轮上下文")
    history = [{"role": "user", "content": "当前排名第一的是谁？"}]
    first = chat(history)
    print("  Q1: 当前排名第一的是谁？")
    print("    " + first.get("reply", "").replace("\n", "\n    "))
    history.append({"role": "assistant", "content": first.get("reply", "")})
    history.append({"role": "user", "content": "它的峰值温度是多少？"})
    second = chat(history)
    print("  Q2: 它的峰值温度是多少？")
    print("    " + second.get("reply", "").replace("\n", "\n    "))
    check("多轮指代正确", "DAMO" in second.get("reply", "").upper(),
          second.get("reply", "")[:80])

    print("\n[5] 格式规范")
    for payload in (first, second):
        hygiene(payload.get("reply", ""))

    print("\n[6] 本地兜底（模型不可用时）")
    try:
        from src.services.ai_service import rule_chat
        fallback = rule_chat("当前排名如何？", {
            "stats": stats,
```

```
158|            "ranking": ranking,
159|            "data_source": metrics.get("data_source", {}),
160|        })
161|        print("    " + fallback.replace("\n", "\n    "))
162|        check("兜底有回复", bool(fallback.strip()))
163|        check("兜底无内部字段名",
164|              not any(w in fallback.lower() for w in
INTERNAL_FIELDS))
165|    except Exception as exc:
166|        check("兜底可调用", False, str(exc))
167|
168|    passed = sum(1 for item in RESULTS if item["ok"])
169|    report_path = os.path.join(PROJECT_ROOT, "AI对话测试报告.md")
170|    lines = ["# AI 助手对话测试报告", "",
171|            "- 模型：%s" % info.get("model"),
172|            "- Key 来源：%s" % info.get("key_source"),
173|            "- 结果：**%d / %d 项通过**" % (passed, len(RESULTS)), ""]
174|    lines.append("## 基准数据")
175|    lines.append("")
176|    lines.append("| 算法 | 服务率 | 单位负载能耗 | 峰值温度 | PUE |")
177|    lines.append("|---|---|---|---|---|")
178|    for algo, m in stats.items():
179|        lines.append("| %s | %.4f | %s | %s | %.4f |" % (
180|            algo, float(m.get("service_rate") or 0),
m.get("energy_per_s
   |    erved_load"),
181|            m.get("max_temp"), float(m.get("avg_pue") or 0)))
182|    lines.append("")
183|    lines.append("## 对话记录")
184|    for item in TRANSCRIPT:
185|        lines.append("")
186|        lines.append("### %s" % item["label"])
187|        lines.append("")
188|        lines.append("**问：** %s" % item["question"])
189|        lines.append("")
190|        lines.append("**答（%s，%.2fs）：**" % (item["source"],
item["elapse
   |    d"] or 0))
191|        lines.append("")
192|        lines.append(item["reply"].replace("\n", "\n\n"))
193|    lines.append("")
194|    lines.append("## 检查项")
195|    lines.append("")
196|    for item in RESULTS:
```

```
197|        lines.append("- %s %s%s" % (" " if item["ok"] else " ",
item["na
   |    me"],
198|                                     (" %s " % item["detail"]) if
not ite
   |    m["ok"] else ""))
199|    with io.open(report_path, "w", encoding="utf-8") as f:
200|        f.write("\n".join(lines) + "\n")
201|    print("\n报告已写入: %s" % report_path)
202|    print("\n" + "=" * 64)
203|    print("汇总: %d / %d 项通过" % (passed, len(RESULTS)))
204|    for item in RESULTS:
205|        if not item["ok"]:
206|            print("  FAIL: %s (%s)" % (item["name"],
item["detail"]))
207|    print("=" * 64)
208|    return 0 if passed == len(RESULTS) else 1
209|
210|
211|if __name__ == "__main__":
212|    sys.exit(main())
```

84. tools/validate_data.py

```
  1|"""Data quality checks: are the numbers in the same ballpark as
reality?
  2|
  3|This is not a correctness proof. It answers one question: do the
values
   |    the
  4|simulator uses look like the real world (temperature, tariff
shape, work
   |    load
  5|mix, PUE, service rate), or has something drifted?
  6|
  7|Usage:
  8|    python tools/validate_data.py
  9|    python tools/validate_data.py --report ../../数据质量校验报告.md
 10|"""
 11|
 12|import argparse
 13|import io
 14|import os
 15|import sqlite3
 16|import statistics
```

```
import sys
from collections import Counter

HERE = os.path.dirname(os.path.abspath(__file__))
PROJECT_ROOT = os.path.abspath(os.path.join(HERE, ".."))
DB = os.path.join(PROJECT_ROOT, "data", "catalog.db")
CSV = os.path.join(PROJECT_ROOT, "data", "simulation_results.csv")

ROWS = []


def record(section, name, actual, expected, ok, note=""):
    ROWS.append({
        "section": section,
        "name": name,
        "actual": actual,
        "expected": expected,
        "status": "PASS" if ok else "WARN",
        "note": note,
    })


def between(value, low, high):
    return low <= value <= high


# Approximate climatological reference ranges (degC). Wide on purpose.
CITY_NOTES = {
    "上海市": "2025 为暖年，年均较常年高约 1.2℃；夏季极端 40.2℃",
    "拉萨市": "ERA5 网格点在高原地形有暖偏差，1 月较站点常年高约 4℃",
}

CITY_CLIMATE = {
    "哈尔滨市": {"annual": (2, 8), "jan": (-26, -10), "jul": (20, 30)},
    "北京市": {"annual": (11, 16), "jan": (-8, 2), "jul": (24, 30)},
    "上海市": {"annual": (15, 18), "jan": (2, 8), "jul": (27, 32)},
    "广州市": {"annual": (21, 24), "jan": (12, 19), "jul": (27, 32)},
    "三亚市": {"annual": (24, 27), "jan": (18, 23), "jul": (27, 31)},
    "昆明市": {"annual": (13, 17), "jan": (6, 11), "jul": (18, 23)},
    "乌鲁木齐市": {"annual": (4, 10), "jan": (-20, -8), "jul": (21, 28)},
    "拉萨市": {"annual": (6, 11), "jan": (-6, 3), "jul": (13, 19)},
}
```

```
59|
60|
61|def check_climate(conn):
62|    print("\n[1] 城市气温 vs 气候参考区间")
63|    print("  %-8s %7s %7s %7s   %s" % ("城市", "年均", "1月", "7月", "结
  |    论"))
64|    for city, ref in CITY_CLIMATE.items():
65|        rows = conn.execute(
66|            "SELECT date, tmean_c FROM dim_city_daily WHERE
city=? ORDER
  |     BY date", (city,)
67|        ).fetchall()
68|        if not rows:
69|           record("气温", city, "无数据", "应有 365 天", False)
70|           print("  %-8s %7s" % (city, "无数据"))
71|            continue
72|        annual = statistics.mean(t for _, t in rows)
73|        jan = statistics.mean(t for d, t in rows if d[5:7] == "
01")
74|        jul = statistics.mean(t for d, t in rows if d[5:7] == "
07")
75|        ok = (between(annual, *ref["annual"]) and between(jan,
*ref["jan
  |    "])
76|              and between(jul, *ref["jul"]))
77|        note = CITY_NOTES.get(city, "")
78|       record("气温", city + " 年均", round(annual, 1), "%s~%s" % ref["
  |    annual"],
79|               between(annual, *ref["annual"]), note)
80|       record("气温", city + " 1月", round(jan, 1), "%s~%s" % ref["jan"
  |    ],
81|               between(jan, *ref["jan"]), note)
82|       record("气温", city + " 7月", round(jul, 1), "%s~%s" % ref["jul"
  |    ],
83|               between(jul, *ref["jul"]))
84|        print("  %-8s %6.1f  %6.1f  %6.1f    %s" % (city, annual,
jan, j
  |    ul,
85|                                                    "PASS" if ok
else "
  |    WARN"))
86|
87|
88|def check_trace(conn):
```

```
    print("\n[3] 任务流分布")
    rows = conn.execute(
        "SELECT hour, load_units, duration_h, priority,
workload_class "
        "FROM fact_task_arrivals WHERE
trace_id='alibaba-gpu-v2025-d019-
    168h' ORDER BY hour"
    ).fetchall()
    if not rows:
        record("任务流", "数据", "缺失", "应为 16797 条", False)
        print("  无数据")
        return
    per_hour = Counter(r[0] for r in rows)
    counts = [per_hour.get(h, 0) for h in range(168)]
    loads = sorted(r[1] for r in rows)
    durs = Counter(r[2] for r in rows)
    mean = statistics.mean(counts)
    ratio = max(counts) / max(1, min(counts))
    q = lambda v, x: v[int((len(v) - 1) * x)]

    checks = [
        ("任务总数", len(rows), "15000~18000", 15000 <= len(rows) <= 180
    00),
        ("每小时均值", round(mean, 1), "80~120", 80 <= mean <= 120),
        ("峰谷比", round(ratio, 1), "2.5~6.0", 2.5 <= ratio <= 6.0),
        ("负载 p50", q(loads, .5), "20~60", 20 <= q(loads, .5) <= 60),
        ("负载 p99", q(loads, .99), "60~80", 60 <= q(loads, .99) <= 80),
        ("优先级种类", len(set(r[3] for r in rows)), "1（原始轨迹无此字段
     ", True),
        ("负载类别数", len(set(r[4] for r in rows)), "1（原始轨迹无此字段
     ", True),
    ]
    for name, actual, expected, ok in checks:
        record("任务流", name, actual, expected, ok)
        print("  %-12s %-22s 期望 %-24s %s" % (name, actual, expected, "
    PASS" if ok else "WARN"))
    one_hour = durs.get(1, 0) / len(rows)
    note_ok = True
    record("任务流", "1 小时任务占比", "%.1f%%" % (one_hour * 100),
           "高占比属预期（小时粒度）", note_ok, "原始 batch 任务 99.9% 不
    足 1 小时")
    print("  1 小时任务占比 %.1f%%（小时粒度下的预期结果）" % (one_hour
*
    100))
```

```
123|
124|
125|def check_carbon(conn):
126|    print("\n[4] 碳因子")
127|    value = conn.execute(
128|        "SELECT kg_co2_per_kwh FROM dim_emission_factor ORDER BY
rowid L
   |    IMIT 1"
129|    ).fetchone()
130|    actual = value[0] if value else None
131|    ok = actual is not None and 0.1 <= actual <= 0.9
132|    record("碳因子", "全国电力平均", actual, "0.1~0.9 kgCO2/kWh", ok)
133|    print("  全国电力平均 %s kgCO2/kWh  期望 0.1~0.9  %s" % (actual, "PA
   |    SS" if ok else "WARN"))
134|
135|
136|def check_simulation():
137|    print("\n[5] 仿真输出")
138|    if not os.path.exists(CSV):
139|        record("仿真", "结果文件", "缺失", "应存在", False)
140|        print("  未找到 simulation_results.csv")
141|        return
142|    with io.open(CSV, "r", encoding="utf-8") as f:
143|        header = f.readline().strip().split(",")
144|        rows = [dict(zip(header, line.strip().split(","))) for
line in f
   |     if line.strip()]
145|    if not rows:
146|        record("仿真", "结果行", 0, ">0", False)
147|        return
148|    pue = [float(r["pue"]) for r in rows if r.get("pue")]
149|    temps = [float(r["peak_temperature"]) for r in rows if
r.get("peak_t
   |    emperature")]
150|    assigned = sum(int(float(r.get("assigned") or 0)) for r in
rows)
151|    dropped = sum(int(float(r.get("dropped") or 0)) for r in rows)
152|    served = sum(float(r.get("served_load") or 0) for r in rows)
153|    energy = sum(float(r["total_energy"]) for r in rows)
154|    service = assigned / (assigned + dropped) if assigned +
dropped else
   |     0
155|    intensity = energy / served if served else 0
156|
```

```
    checks = [
        ("PUE 范围", "%.2f~%.2f" % (min(pue), max(pue)), "1.05~1.80", all(1.05 <= p <= 1.80 for p in pue)),
        ("峰值温度", "%.1f℃" % max(temps), "30~85℃", 30 <= max(temps) <=
 85),
        ("任务服务率", "%.2f%%" % (service * 100), "80%~100%", 0.80 <= service <= 1.0),
        ("单位负载能耗", "%.2f W/负载" % intensity, "2~15", 2 <= intensity <= 15),
    ]
    for name, actual, expected, ok in checks:
        record("仿真", name, actual, expected, ok)
        print("  %-12s %-18s 期望 %-12s %s" % (name, actual, expected, "PASS" if ok else "WARN"))


def check_consistency(conn):
    print("\n[6] 跨模块一致性")
    bj = conn.execute(
        "SELECT COUNT(*) FROM dim_ambient_hourly WHERE site_id='北京市'"
    ).fetchone()[0]
    series = conn.execute(
        "SELECT AVG(tmean_c) FROM dim_city_daily WHERE city='北京市'"
    ).fetchone()[0]
    record("一致性", "北京逐时行数", bj, "8760", bj == 8760)
    record("一致性", "北京逐日年均", round(series, 1), "11~16", 11 <= series <= 16)
    print("  北京逐时行数 %d（期望 8760）  %s" % (bj, "PASS" if bj == 8760
     else "WARN"))
    print("  北京逐日年均 %.1f℃（期望 11~16）  %s" % (series, "PASS" if 11
     <= series <= 16 else "WARN"))


def write_report(path):
    lines = ["# 数据质量校验报告", "", "> 自动生成，用于确认各数据源与真实世界的量级一致。", ""]
    lines.append("| 分类 | 检查项 | 实际值 | 参考区间 | 结论 | 说明 |")
    lines.append("|---|---|---|---|---|---|")
    for r in ROWS:
        lines.append("| %s | %s | %s | %s | %s | %s |" % (
            r["section"], r["name"], r["actual"], r["expected"],
```

```
            r["status"], r["note"]))
    passed = sum(1 for r in ROWS if r["status"] == "PASS")
    lines.append("")
    lines.append("**汇总：%d / %d 项通过。**" % (passed, len(ROWS)))
    with io.open(path, "w", encoding="utf-8") as f:
        f.write("\n".join(lines) + "\n")
    print("\n报告已写入: %s" % path)


def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--report",
default=os.path.join(PROJECT_ROOT, "
    数据质量校验报告.md"))
    args = parser.parse_args()

    if not os.path.exists(DB):
        print("数据目录不存在: %s" % DB)
        return 1
    conn = sqlite3.connect(DB)
    check_climate(conn)
    check_trace(conn)
    check_carbon(conn)
    conn.close()
    check_simulation()
    conn = sqlite3.connect(DB)
    check_consistency(conn)
    conn.close()
    write_report(os.path.abspath(args.report))

    passed = sum(1 for r in ROWS if r["status"] == "PASS")
    print("\n汇总: %d / %d 项通过" % (passed, len(ROWS)))
    return 0


if __name__ == "__main__":
    sys.exit(main())
```

85. tests/__init__.py

```
"""Marker file for the tests root package."""
```

86. tests/algorithms/__init__.py

```
"""Marker file for the algorithms test package."""
```

87. tests/algorithms/test_damo_ets.py

```
  1|"""Unit tests for DAMOETSScheduler."""
  2|
  3|import sys
  4|import os
  5|
  6|# Make the project importable when running from the tests
directory
  7|PROJECT_ROOT = os.path.abspath(os.path.join(os.path.dirname(__file
__), "
   |    "..", ".."))
  8|if PROJECT_ROOT not in sys.path:
  9|    sys.path.insert(0, PROJECT_ROOT)
 10|
 11|from src.algorithms import DAMOETSScheduler, GreedyScheduler
 12|from src.models import Server, Task
 13|
 14|
 15|def _make_servers(count: int = 4) -> list:
 16|    servers = []
 17|    for i in range(count):
 18|        servers.append(Server(
 19|            server_id=i,
 20|            max_load=100,
 21|            max_temp=85,
 22|            base_power=100,
 23|            max_power=300,
 24|            initial_temp=25,
 25|            server_type="standard" if i % 2 == 0 else "
gpu_server",
 26|        ))
 27|    return servers
 28|
 29|
 30|def _make_task(load: int = 10, deadline=None, wclass: str = "
cpu_bound",
   |     priority: int = 1) -> Task:
 31|    return Task(load=load, priority=priority, deadline=deadline,
workloa
   |    d_class=wclass)
 32|
 33|
 35|# Tests
 37|
```

```
def test_empty_tasks_returns_empty():
    scheduler = DAMOETSScheduler(_make_servers(), seed=42)
    assert scheduler.schedule_tasks([]) == []
    assert scheduler.get_deadline_violations() == 0


def test_edf_ordering():
    """Tasks with earlier deadlines should be assigned first; non-deadli
    ne tasks last."""
    servers = _make_servers(4)
    scheduler = DAMOETSScheduler(servers, seed=1)

    late = _make_task(load=10, deadline=10, wclass="cpu_bound")
    early = _make_task(load=10, deadline=2, wclass="cpu_bound")
    no_dl = _make_task(load=10, deadline=None, wclass="cpu_bound")

    results = scheduler.schedule_tasks([late, early, no_dl], current_hou
    r=0)
    assigned = [t for t, s in results if s is not None]
    assert assigned[0] is early
    assert assigned[1] is late
    # no_dl is processed last (no deadline group), still assigned
    assert no_dl in assigned


def test_heterogeneous_match_prefers_gpu_for_gpu_bound():
    """GPU-bound tasks should preferentially route to gpu_server."""
    servers = _make_servers(4)  # 0,2 = standard; 1,3 = gpu_server
    scheduler = DAMOETSScheduler(servers, seed=0)

    gpu_task = _make_task(load=20, deadline=5, wclass="gpu_bound", prior
    ity=5)
    results = scheduler.schedule_tasks([gpu_task], current_hour=0)
    chosen = results[0][1]
    assert chosen is not None
    assert chosen.server_type == "gpu_server"


def test_thermal_safety_rejects_overheating_assignment():
```

```
    """If all servers would overheat, the task should be
unassigned."""
    servers = _make_servers(2)
    for s in servers:
        s.current_temp = 84  # Just below max (85 * 0.95 = 80.75)
-> alr
    eady unsafe
        s.current_load = 50
    scheduler = DAMOETSScheduler(servers, seed=0)

    task = _make_task(load=30, deadline=5, wclass="cpu_bound",
priority=
    5)
    results = scheduler.schedule_tasks([task], current_hour=0)
    assert results[0][1] is None


def test_deadline_violation_counter():
    """Tasks whose deadline is already in the past at assignment
hour ar
    e counted."""
    servers = _make_servers(4)
    scheduler = DAMOETSScheduler(servers, seed=0)

    # current_hour=10, but the task's deadline is 5 — already
missed
    expired = _make_task(load=10, deadline=5, wclass="cpu_bound")
    future = _make_task(load=10, deadline=15, wclass="cpu_bound")

    results = scheduler.schedule_tasks([expired, future],
current_hour=1
    0)
    violations = scheduler.get_deadline_violations()
    assert violations == 1
    # Both should be assigned (the counter is about deadline
breach, not
     assignment failure)
    assert all(server is not None for _, server in results)


def test_reproducible_with_seed():
    """Same seed + same inputs -> same assignments and same
violation co
    unt."""
```

```
    servers_a = _make_servers(4)
    servers_b = _make_servers(4)
    scheduler_a = DAMOETSScheduler(servers_a, seed=123)
    scheduler_b = DAMOETSScheduler(servers_b, seed=123)

    def _build_tasks():
        return [
            _make_task(
                load=15,
                deadline=i + 3,
                wclass="gpu_bound" if i % 2 == 0 else "cpu_bound",
            )
            for i in range(6)
        ]

    tasks_a = _build_tasks()
    tasks_b = _build_tasks()

    res_a = scheduler_a.schedule_tasks(tasks_a, current_hour=0)
    res_b = scheduler_b.schedule_tasks(tasks_b, current_hour=0)

    # Compare by (deadline, workload_class, server_id) — not task_id (uu
    id)
    def _signature(results):
        return [
            (t.deadline, t.workload_class, s.server_id if s else None)
            for t, s in results
        ]

    assert _signature(res_a) == _signature(res_b)
    assert scheduler_a.get_deadline_violations() == scheduler_b.get_dead
    line_violations()


def test_damo_ets_lower_or_equal_energy_than_greedy_under_heavy_de
adline
    _load():
    """DAMO-ETS prioritises deadline-urgency so it should be at least as
     energy-efficient
```

```
    as Greedy when both have enough capacity, measured by avg total assi
    gned load per task."""
    servers_a = _make_servers(6)
    servers_b = _make_servers(6)

    damo = DAMOETSScheduler(servers_a, seed=0)
    greedy = GreedyScheduler(servers_b)

    tasks = []
    for i in range(10):
        tasks.append(_make_task(
            load=20,
            deadline=i + 2,
            wclass="gpu_bound" if i % 3 == 0 else "cpu_bound",
            priority=3,
        ))

    damo_res = damo.schedule_tasks(tasks, current_hour=0)
    greedy_res = greedy.schedule_tasks(tasks)

    damo_assigned = sum(1 for _, s in damo_res if s is not None)
    greedy_assigned = sum(1 for _, s in greedy_res if s is not None)

    # Both should assign the majority; DAMO-ETS should be at least as go
    od
    assert damo_assigned >= greedy_assigned - 1


def test_get_name_returns_damo_ets():
    scheduler = DAMOETSScheduler(_make_servers(2), seed=0)
    assert scheduler.get_name() == "damo_ets"


def test_reset_clears_violation_counter():
    scheduler = DAMOETSScheduler(_make_servers(2), seed=0)
    expired = _make_task(load=5, deadline=1, wclass="cpu_bound")
    scheduler.schedule_tasks([expired], current_hour=5)
    assert scheduler.get_deadline_violations() >= 1
    scheduler.reset()
    assert scheduler.get_deadline_violations() == 0
```

88. tests/algorithms/test_genetic_reproducibility.py

```
"""Reproducibility tests for the GeneticScheduler."""

import sys
import os

PROJECT_ROOT = os.path.abspath(os.path.join(os.path.dirname(__file__), "
    ..", ".."))
if PROJECT_ROOT not in sys.path:
    sys.path.insert(0, PROJECT_ROOT)

from src.algorithms import GeneticScheduler, GeneticAlgorithm
from src.models import Server, Task


def _make_servers(count: int = 6) -> list:
    return [Server(server_id=i) for i in range(count)]


def _make_tasks(count: int = 8) -> list:
    return [
        Task(
            load=10 + (i % 3) * 5,
            priority=(i % 5) + 1,
            workload_class="cpu_bound" if i % 2 == 0 else "
gpu_bound",
        )
        for i in range(count)
    ]


def test_genetic_algorithm_reproducible_with_seed():
    """GeneticAlgorithm.evolve() with the same seed returns the
same bes
    t assignment."""
    tasks = _make_tasks(8)
    servers_a = _make_servers(6)
    servers_b = _make_servers(6)

    ga_a = GeneticAlgorithm(servers_a, tasks, seed=42)
    ga_b = GeneticAlgorithm(servers_b, tasks, seed=42)

    best_a, fit_a = ga_a.evolve()
```

```
    best_b, fit_b = ga_b.evolve()

    assert best_a == best_b, "Best individuals must match with same seed"
    assert abs(fit_a - fit_b) < 1e-9


def test_genetic_scheduler_reproducible_via_full_run():
    """GeneticScheduler.schedule_tasks returns the same set of (task, server_id) pairs."""
    tasks = _make_tasks(8)
    servers_a = _make_servers(6)
    servers_b = _make_servers(6)

    s1 = GeneticScheduler(servers_a, seed=7)
    s2 = GeneticScheduler(servers_b, seed=7)

    res1 = s1.schedule_tasks(tasks)
    res2 = s2.schedule_tasks(tasks)

    pairs1 = [(t.task_id, s.server_id if s else None) for t, s in res1]
    pairs2 = [(t.task_id, s.server_id if s else None) for t, s in res2]
    assert pairs1 == pairs2


def test_genetic_different_seed_produces_different_results():
    """Sanity check: different seeds should usually give different results."""
    tasks = _make_tasks(10)
    s1 = GeneticScheduler(_make_servers(8), seed=1)
    s2 = GeneticScheduler(_make_servers(8), seed=2)
    res1 = s1.schedule_tasks(tasks)
    res2 = s2.schedule_tasks(tasks)
    pairs1 = [(t.task_id, s.server_id if s else None) for t, s in res1]
    pairs2 = [(t.task_id, s.server_id if s else None) for t, s in res2]
    assert pairs1 != pairs2
```

89. tests/data/__init__.py

```
  1|'''Data catalog tests.'''
```

90. tests/data/test_catalog_pipeline.py

```
  1|"""End-to-end test of the offline data catalog pipeline.
  2|
  3|Builds a catalog from fixture snapshots in a temporary directory,
then r
   |    uns the
  4|simulator in catalog mode and checks that the data actually
drives the r
   |    esults
  5|and that provenance is recorded.
  6|"""
  7|
  8|import csv
  9|import io
 10|import json
 11|import os
 12|import subprocess
 13|import sys
 14|
 15|PROJECT_ROOT = os.path.abspath(os.path.join(os.path.dirname(__file
__), "
   |    ..", ".."))
 16|if PROJECT_ROOT not in sys.path:
 17|    sys.path.insert(0, PROJECT_ROOT)
 18|
 19|from src.data.catalog import Catalog
 20|from src.simulation import DataCenterSimulator
 21|
 22|
 23|class _StubConfig:
 24|    def __init__(self, data_source):
 25|        self._data = {
 26|            "simulation": {"random_seed": 42, "sla_slack": 1.5},
 27|            "task_generation": {"base_rate": 4, "peak_rate": 10, "
peak_h
   |    ours": [9, 18]},
 28|            "task_completion": {"mode": "service_based", "
completion_pro
   |    bability": 0.3},
 29|            "cooling": {"power_coefficient": 500, "target_temp":
22, "fa
```

```
   |    n_target_temp": 35,
 30|                        "dissipation_coefficient": 8.0, "
thermal_inertia
   |    ": 0.25},
 31|            "energy": {"idle_power_factor": 0.6, "
part_load_penalty": 0.
   |    5,
 32|                       "temp_penalty_coefficient": 1.0, "
temp_reference"
   |    : 35},
 33|            "genetic": {"population_size": 8, "generations": 4, "
enable_
   |    parallel": False},
 34|            "greedy": {"temperature_weight": 0.3},
 35|            "data_source": data_source,
 36|        }
 37|
 38|    def to_dict(self):
 39|        return self._data
 40|
 41|
 42|def _write_dataset(raw_root, kind, name, rows, meta):
 43|    folder = os.path.join(raw_root, kind)
 44|    os.makedirs(folder, exist_ok=True)
 45|    csv_path = os.path.join(folder, name + ".csv")
 46|    with io.open(csv_path, "w", encoding="utf-8", newline="") as
f:
 47|        writer = csv.DictWriter(f,
fieldnames=list(rows[0].keys()))
 48|        writer.writeheader()
 49|        writer.writerows(rows)
 50|    with io.open(os.path.splitext(csv_path)[0] + ".json", "w",
encoding=
   |    "utf-8") as f:
 51|        json.dump(meta, f, ensure_ascii=False)
 52|    return csv_path
 53|
 54|
 55|def _ambient_rows(hours=48):
 56|    rows = []
 57|    for hour in range(hours):
 58|        ts = "2026-03-01T%02d:00:00" % (hour % 24)
 59|        temp = 26.0 + 8.0 * ((hour % 24) / 24.0)
```

```
 60|        rows.append({"ts": ts, "temp_c": round(temp, 2), "
humidity": 45,
   |     "wind_ms": 2.0})
 61|    return rows
 62|
 63|
 64|def _build(raw_root, db_path, extra_args=()):
 65|    cmd = [sys.executable, os.path.join("tools", "etl", "
build_catalog.p
   |    y"),
 66|           "--raw-root", str(raw_root), "--db", str(db_path)]
 67|    cmd.extend(extra_args)
 68|    return subprocess.run(cmd, cwd=PROJECT_ROOT,
capture_output=True, te
   |    xt=True)
 69|
 70|
 71|def test_catalog_builds_and_feeds_the_simulator(tmp_path):
 72|    raw_root = tmp_path / "raw"
 73|    db_path = tmp_path / "catalog.db"
 74|
 75|    _write_dataset(str(raw_root), "ambient", "campus",
_ambient_rows(48)
   |    , {
 76|        "source_id": "test-ambient",
 77|        "name": "测试用逐时气温",
 78|        "publisher": "Test Publisher",
 79|        "url": "https://example.invalid/test",
 80|        "license": "test-only",
 81|        "retrieved_at": "2026-10-15",
 82|        "site_id": "campus-x",
 83|    })
 84|    outcome = _build(str(raw_root), str(db_path))
 85|    assert outcome.returncode == 0, outcome.stderr
 86|    assert "测试用逐时气温" in outcome.stdout or "ambient" in outcome.st
   |    dout
 87|
 88|    catalog = Catalog(str(db_path))
 89|    assert catalog.available()
 90|
 91|    series = catalog.ambient_series("campus-x")
 92|    assert series is not None and len(series) == 48
 93|    assert series[0] == 26.0
 94|
```

```
    provenance = catalog.provenance()
    ids = {row["source_id"] for row in provenance}
    assert "test-ambient" in ids
    ambient_meta = [row for row in provenance if row["source_id"]
== "te
    st-ambient"][0]
    assert ambient_meta["publisher"] == "Test Publisher"
    assert ambient_meta["url"] == "https://example.invalid/test"
    assert ambient_meta["sha256"]

    versions = catalog.versions()
    assert versions and all(row["row_count"] for row in versions)

    simulator = DataCenterSimulator(
        num_servers=16, simulation_hours=24, seed=42,
        config=_StubConfig({
            "mode": "catalog",
            "catalog_db": str(db_path),
            "site_id": "campus-x",
        }),
    )
    assert simulator.ambient_series is not None
    assert simulator.run_metadata["data_mode"] == "catalog"

    df = simulator.run_comparison(algorithms=["greedy", "
round_robin"])
    assert df["ambient_temperature"].nunique() > 1
    assert df["pue"].nunique() > 1


def test_catalog_without_database_falls_back_to_synthetic(tmp_path)
:
    simulator = DataCenterSimulator(
        num_servers=8, simulation_hours=4, seed=1,
        config=_StubConfig({"mode": "catalog", "catalog_db":
str(tmp_pat
    h / "missing.db")}),
    )
    assert simulator.ambient_series is None
    assert simulator.run_metadata["ambient_source"] == "
synthetic-sine"
    df = simulator.run_comparison(algorithms=["greedy"])
    assert not df.empty

```

```
132|
133|def test_dataset_without_provenance_is_refused(tmp_path):
134|    raw_root = tmp_path / "raw"
135|    folder = os.path.join(str(raw_root), "ambient")
136|    os.makedirs(folder, exist_ok=True)
137|    csv_path = os.path.join(folder, "no_meta.csv")
138|    with io.open(csv_path, "w", encoding="utf-8", newline="") as
f:
139|        writer = csv.DictWriter(f, fieldnames=["ts", "temp_c"])
140|        writer.writeheader()
141|        writer.writerow({"ts": "2026-01-01T00:00:00", "temp_c":
1.0})
142|
143|    db_path = tmp_path / "catalog.db"
144|    outcome = _build(str(raw_root), str(db_path))
145|    assert outcome.returncode == 0
146|
147|    catalog = Catalog(str(db_path))
148|    assert catalog.ambient_series() is None
149|    assert catalog.provenance() == []
150|
151|
152|def test_trace_snapshot_builds_a_task_stream(tmp_path):
153|    raw_root = tmp_path / "raw"
154|    db_path = tmp_path / "catalog.db"
155|    rows = []
156|    for hour in range(12):
157|        for i in range(5):
158|            rows.append({
159|                "hour": hour,
160|                "load_units": 10 + i * 3,
161|                "priority": 3,
162|                "workload_class": "cpu_bound" if i % 2 == 0 else "
gpu_bo
   |    und",
163|                "duration_h": 2,
164|            })
165|    _write_dataset(str(raw_root), "trace", "cluster", rows, {
166|        "source_id": "test-trace",
167|        "name": "测试用任务轨迹（派生演示数据）",
168|        "publisher": "Test Publisher",
169|        "url": "https://example.invalid/test-trace",
170|        "license": "test-only",
171|        "retrieved_at": "2026-10-15",
```

```
172|        "trace_id": "cluster-x",
173|    })
174|
175|    outcome = _build(str(raw_root), str(db_path), ("--sla-slack", "
1.5")
   |    )
176|    assert outcome.returncode == 0, outcome.stderr
177|
178|    catalog = Catalog(str(db_path))
179|    stream = catalog.task_stream("cluster-x")
180|    assert stream is not None
181|    assert len(stream) == 12
182|    assert len(stream[0]) == 5
183|    load, priority, workload_class, duration, offset =
stream[0][0]
184|    assert load >= 10 and priority == 3 and duration == 2 and
offset >=
   |    2
185|
186|    simulator = DataCenterSimulator(
187|        num_servers=8, simulation_hours=12, seed=5,
188|        config=_StubConfig({"mode": "catalog", "catalog_db":
str(db_path
   |    ),
189|                            "trace_id": "cluster-x"}),
190|    )
191|    assert simulator.trace_stream is not None
192|    df = simulator.run_comparison(algorithms=["greedy", "
damo_ets"])
193|    assert simulator.run_metadata["scenario_source"] == "catalog"
194|    assert df["offered_load"].max() > 0
```

91. tests/plugins/__init__.py

```
1|"""GitHub Skills 插件单元测试包"""
```

92. tests/plugins/test_pymoo_plugin.py

```
1|"""
2|NSGA2 / MOEAD 插件单元测试
3|
4|覆盖:
5|    1. manifest 字段完整性
6|    2. initialize 成功路径
7|    3. execute 调度结果结构
8|    4. 依赖缺失降级（mock）
```

```
  9|
 10|运行: pytest tests/plugins/test_pymoo_plugin.py -v
 11|"""
 12|
 13|import os
 14|import sys
 15|import unittest
 16|
 17|# Make project importable
 18|PROJECT_ROOT = os.path.abspath(os.path.join(os.path.dirname(__file
__), "
   |    "..", ".."))
 19|if PROJECT_ROOT not in sys.path:
 20|    sys.path.insert(0, PROJECT_ROOT)
 21|
 22|from src.plugin.protocol import PluginManifest, PluginResult  #
noqa: E4
   |    02
 23|from src.plugins._deps import has_pymoo  # noqa: E402
 24|
 25|
 26|class TestNSGA2Manifest(unittest.TestCase):
 27|    """测试 NSGA2 manifest 完整性"""
 28|
 29|    def setUp(self):
 30|        if not has_pymoo():
 31|            self.skipTest("pymoo 未安装，跳过")
 32|        from src.plugins.nsga2_scheduler import
NSGA2SchedulerPlugin
 33|        self.plugin = NSGA2SchedulerPlugin()
 34|        self.manifest = self.plugin.get_manifest()
 35|
 36|    def test_manifest_is_plugin_manifest(self):
 37|        self.assertIsInstance(self.manifest, PluginManifest)
 38|
 39|    def test_manifest_required_fields(self):
 40|        self.assertEqual(self.manifest.name, "
NSGA2SchedulerPlugin")
 41|        self.assertEqual(self.manifest.version, "1.0.0")
 42|        self.assertIn("schedule",
self.manifest.supported_operations)
 43|
 44|    def test_manifest_input_schema_keys(self):
 45|        schema = self.manifest.input_schema
```

```
        for key in ["pop_size", "n_gen", "selection", "seed"]:
            self.assertIn(key, schema, f"缺少 input_schema 字段: {key}")


class TestNSGA2Initialize(unittest.TestCase):
    """测试 NSGA2 initialize 路径"""

    def setUp(self):
        if not has_pymoo():
            self.skipTest("pymoo 未安装，跳过")
        from src.plugins.nsga2_scheduler import
NSGA2SchedulerPlugin
        self.plugin = NSGA2SchedulerPlugin()

    def test_initialize_default_config(self):
        ok = self.plugin.initialize({})
        self.assertTrue(ok, "默认配置应能成功初始化")

    def test_initialize_invalid_selection(self):
        ok = self.plugin.initialize({"selection": "INVALID"})
        self.assertFalse(ok, "非法 selection 应失败")


class TestNSGA2Execute(unittest.TestCase):
    """测试 NSGA2 execute 调度"""

    def setUp(self):
        if not has_pymoo():
            self.skipTest("pymoo 未安装，跳过")
        from src.plugins.nsga2_scheduler import
NSGA2SchedulerPlugin
        self.plugin = NSGA2SchedulerPlugin()
        self.plugin.initialize({"pop_size": 20, "n_gen": 30, "
seed": 42}
    )

        # 构造仿真器 + 小任务集
        from src.simulation import DataCenterSimulator
        sim = DataCenterSimulator(num_servers=5,
simulation_hours=2, see
    d=42)
        self.servers = sim.servers
        from src.models import Task
        self.tasks = [
```

```
            Task(load=5, priority=1, deadline=10,
workload_class="cpu_bo
    und"),
            Task(load=3, priority=2, deadline=5,
workload_class="memory_
    bound"),
            Task(load=7, priority=1, deadline=None,
workload_class="io_b
    ound"),
        ]

    def test_execute_returns_valid_assignments(self):
        result = self.plugin.execute(tasks=self.tasks,
servers=self.serv
    ers)
        self.assertIsInstance(result, PluginResult)
        self.assertTrue(result.success, f"NSGA2 应成功: {result.message}
    ")
        self.assertIn("assignments", result.data)
        self.assertEqual(len(result.data["assignments"]),
len(self.tasks
    ))

    def test_execute_missing_params(self):
        result = self.plugin.execute()
        self.assertFalse(result.success)
        self.assertEqual(result.error_code, "MISSING_PARAM")


class TestDependencyMissingFallback(unittest.TestCase):
    """测试依赖缺失时优雅降级"""

    def test_dep_check_returns_dict(self):
        from src.plugins._deps import get_dependency_status
        status = get_dependency_status()
        self.assertIsInstance(status, dict)
        for key in ["gymnasium", "stable_baselines3", "pymoo"]:
            self.assertIn(key, status)
            self.assertIn("available", status[key])
            self.assertIn("install_hint", status[key])


class TestMOEADManifest(unittest.TestCase):
    """测试 MOEAD manifest 完整性（与 NSGA2 镜像）"""
```

```
117|
118|    def setUp(self):
119|        if not has_pymoo():
120|            self.skipTest("pymoo 未安装，跳过")
121|        from src.plugins.moead_scheduler import
MOEADSchedulerPlugin
122|        self.plugin = MOEADSchedulerPlugin()
123|
124|    def test_manifest_name_and_ops(self):
125|        m = self.plugin.get_manifest()
126|        self.assertEqual(m.name, "MOEADSchedulerPlugin")
127|        self.assertIn("schedule", m.supported_operations)
128|
129|    def test_initialize_default(self):
130|        ok = self.plugin.initialize({})
131|        self.assertTrue(ok)
132|
133|
134|if __name__ == "__main__":
135|    unittest.main()
```

93. tests/plugins/test_rl_ppo_plugin.py

```
  1|"""
  2|Stable Baselines3 RLSchedulerPlugin 单元测试
  3|
  4|覆盖:
  5|    1. manifest 字段完整性
  6|    2. initialize 路径（成功/失败）
  7|    3. 模型路径不存在时降级
  8|    4. 依赖缺失降级
  9|
 10|运行: pytest tests/plugins/test_rl_ppo_plugin.py -v
 11|"""
 12|
 13|import os
 14|import sys
 15|import unittest
 16|from unittest.mock import patch, MagicMock
 17|
 18|PROJECT_ROOT = os.path.abspath(os.path.join(os.path.dirname(__file
__), "
   |    ..", ".."))
 19|if PROJECT_ROOT not in sys.path:
 20|    sys.path.insert(0, PROJECT_ROOT)
```

```
 21|
 22|from src.plugin.protocol import PluginResult  # noqa: E402
 23|from src.plugins._deps import has_gymnasium, has_sb3  # noqa: E402
 24|
 25|
 26|class TestRLManifest(unittest.TestCase):
 27|    """测试 RL manifest 完整性"""
 28|
 29|    def setUp(self):
 30|        from src.plugins.rl_ppo_scheduler import RLSchedulerPlugin
 31|        self.plugin = RLSchedulerPlugin()
 32|        self.manifest = self.plugin.get_manifest()
 33|
 34|    def test_manifest_name(self):
 35|        self.assertEqual(self.manifest.name, "RLSchedulerPlugin")
 36|        self.assertEqual(self.manifest.version, "1.0.0")
 37|
 38|    def test_manifest_has_train_and_predict(self):
 39|        ops = self.manifest.supported_operations
 40|        self.assertIn("train", ops)
 41|        self.assertIn("predict", ops)
 42|
 43|    def test_manifest_input_schema(self):
 44|        schema = self.manifest.input_schema
 45|        for key in ["algorithms", "train", "total_timesteps", "
model_pat
   |    h"]:
 46|            self.assertIn(key, schema)
 47|
 48|
 49|class TestRLInitialize(unittest.TestCase):
 50|    """测试 initialize 路径"""
 51|
 52|    def setUp(self):
 53|        from src.plugins.rl_ppo_scheduler import RLSchedulerPlugin
 54|        self.plugin = RLSchedulerPlugin()
 55|
 56|    def test_initialize_dependency_missing(self):
 57|        """依赖缺失时 initialize 返回 False（不被异常）"""
 58|        # 模拟 gymnasium/sb3 不可用
 59|        with patch("src.plugins.rl_ppo_scheduler.has_gymnasium",
return_
   |    value=False), \
```

```
            patch("src.plugins.rl_ppo_scheduler.has_sb3",
return_value=
    False):
            ok = self.plugin.initialize({"num_servers": 10})
            self.assertFalse(ok, "依赖缺失时 initialize 应返回 False")

    def test_initialize_default_config(self):
        """SB3 + gym 可用时默认配置应能成功"""
        if not (has_gymnasium() and has_sb3()):
            self.skipTest("stable-baselines3/gymnasium 未安装，跳过")
        ok = self.plugin.initialize({})
        self.assertTrue(ok)

    def test_initialize_invalid_servers(self):
        if not (has_gymnasium() and has_sb3()):
            self.skipTest("依赖未安装，跳过")
        ok = self.plugin.initialize({"num_servers": 0})
        self.assertFalse(ok, "num_servers=0 应失败")


class TestRLExecuteErrorCodes(unittest.TestCase):
    """测试 execute 在异常情况下的错误码"""

    def setUp(self):
        from src.plugins.rl_ppo_scheduler import RLSchedulerPlugin
        self.plugin = RLSchedulerPlugin()

    def test_execute_dependency_missing(self):
        """依赖缺失时 execute 返回 PluginResult.fail(DEPENDENCY_MISSING)
    """
        with patch("src.plugins.rl_ppo_scheduler.has_gymnasium",
return_
    value=False), \
            patch("src.plugins.rl_ppo_scheduler.has_sb3",
return_value=
    False):
            # 即使 initialize 返回 False，execute 也要返回失败结果而非抛
    异常
            self.plugin.initialize({})
            result = self.plugin.execute(algorithms=["rl_ppo"],
train=Fa
    lse)
            self.assertIsInstance(result, PluginResult)
            self.assertFalse(result.success)
```

```
 94|            self.assertEqual(result.error_code, "
DEPENDENCY_MISSING")
 95|
 96|    def test_load_model_not_found(self):
 97|        """模型文件不存在时返回 MODEL_NOT_FOUND"""
 98|        if not (has_gymnasium() and has_sb3()):
 99|            self.skipTest("依赖未安装，跳过")
100|        self.plugin.initialize({"num_servers": 5, "model_path": "
/nonexi
   |    stent/ppo.zip"})
101|        # execute 推理模式，无 model_path → 失败
102|        result = self.plugin.execute(train=False, tasks=[],
servers=[])
103|        # 推理无 tasks 时直接调用 _load_model，model_path 不存在 →
MODEL_
   |    NOT_FOUND
104|        # 如果传了空 tasks 列表但有 model_path 仍走 _load_model
105|        self.assertIsInstance(result, PluginResult)
106|
107|
108|class TestPluginDiscovery(unittest.TestCase):
109|    """测试插件自动发现机制"""
110|
111|    def test_discover_plugins_finds_github_skill_plugins(self):
112|        from src.plugin import PluginManager
113|        pm = PluginManager()
114|        n = pm.discover_plugins(plugin_dir="src/plugins")
115|        self.assertGreaterEqual(n, 0, "discover_plugins 不应抛异常")
116|        # 即便 0 个也允许（依赖缺失时所有插件都返回 False，discover 会跳
过
   |
117|        # 实际有 pymoo/gym 装上时应该有 3+
118|
119|
120|if __name__ == "__main__":
121|    unittest.main()
```

94. tests/simulation/__init__.py

```
  1|'''Controlled-comparison tests.'''
```

95. tests/simulation/test_controlled_comparison.py

```
  1|"""Tests for the controlled comparison guarantees.
  2|
```

```
  3|These lock in the properties that make the algorithm numbers
comparable:
  4|one task stream per scenario, an SLA metric that can actually
move, and
   |    a
  5|ranking rule that refuses to reward an algorithm for serving less
work.
  6|"""
  7|
  8|import os
  9|import sys
 10|
 11|PROJECT_ROOT = os.path.abspath(os.path.join(os.path.dirname(__file
__), "
   |    "..", ".."))
 12|if PROJECT_ROOT not in sys.path:
 13|    sys.path.insert(0, PROJECT_ROOT)
 14|
 15|import pandas as pd
 16|
 17|from src.simulation import DataCenterSimulator
 18|from src.simulation.metrics import SERVICE_RATE_FLOOR,
rank_algorithms,
   |    summarize, safe_number
 19|from src.simulation.task_generator import TaskGenerator
 20|
 21|ALGORITHMS = ["greedy", "genetic", "round_robin", "damo_ets"]
 22|
 23|
 24|class _StubConfig:
 25|    """Minimal config object so tests do not depend on the YAML
files.""
   |    "
 26|
 27|    def __init__(self, overrides=None):
 28|        self._data = {
 29|            "simulation": {"random_seed": 42, "sla_slack": 1.5},
 30|            "task_generation": {"base_rate": 6, "peak_rate": 20, "
peak_h
   |    ours": [9, 18]},
 31|            "task_completion": {"mode": "service_based", "
completion_pro
   |    bability": 0.3},
```

```
            "cooling": {"power_coefficient": 500, "target_temp":
22, "fa
    n_target_temp": 35,
                        "dissipation_coefficient": 8.0, "
thermal_inertia
    ": 0.25},
            "energy": {"idle_power_factor": 0.6, "
part_load_penalty": 0.
    5},
            "genetic": {"population_size": 12, "generations": 8, "
enable
    _parallel": False},
            "greedy": {"temperature_weight": 0.3},
            "data_source": {"mode": "synthetic"},
        }
        if overrides:
            for section, values in overrides.items():
                self._data.setdefault(section, {}).update(values)

    def to_dict(self):
        return self._data


def _simulator(servers=40, hours=8, seed=42, overrides=None):
    return DataCenterSimulator(
        num_servers=servers,
        simulation_hours=hours,
        seed=seed,
        config=_StubConfig(overrides),
    )


def test_all_algorithms_see_the_same_offered_demand():
    """The control guarantee is about demand, not about what each
algori
    thm served."""
    sim = _simulator()
    df = sim.run_comparison(algorithms=ALGORITHMS)
    offered = sim.control_signature(df)
    assert set(offered) == set(ALGORITHMS)
    assert len(set(offered.values())) == 1, offered
    assert sim.is_controlled(df)


```

```
def test_scenario_stream_is_reproducible():
    first = TaskGenerator(seed=7, sla_slack=1.5).build_stream(6)
    second = TaskGenerator(seed=7, sla_slack=1.5).build_stream(6)
    assert first == second
    assert TaskGenerator.stream_digest(first) ==
TaskGenerator.stream_di
    gest(second)


def test_materialised_tasks_are_independent_instances():
    generator = TaskGenerator(seed=3)
    stream = generator.build_stream(3)
    run_a = generator.materialize_stream(stream)
    run_b = generator.materialize_stream(stream)
    assert run_a[0][0] is not run_b[0][0]
    run_a[0][0].completed_time = 99
    assert run_b[0][0].completed_time is None


def test_deadline_violations_are_reachable():
    tight = _simulator(servers=12, hours=12,
overrides={"simulation": {"
    sla_slack": 1.0}})
    df = tight.run_comparison(algorithms=ALGORITHMS)
    stats = tight.get_statistics(df)
    assert any(entry["deadline_violations"] > 0 for entry in
stats.value
    s())

    loose = _simulator(servers=60, hours=12,
overrides={"simulation": {"
    sla_slack": 4.0}})
    df_loose = loose.run_comparison(algorithms=ALGORITHMS)
    loose_stats = loose.get_statistics(df_loose)
    assert all(entry["deadline_violation_rate"] <= 1.0 for entry
in loos
    e_stats.values())


def test_summary_reports_service_rate_and_energy_intensity():
    sim = _simulator(servers=60)
    df = sim.run_comparison(algorithms=["greedy", "round_robin"])
    stats = sim.get_statistics(df)
    for entry in stats.values():
```

```
        assert 0.0 <= entry["service_rate"] <= 1.0
        assert entry["energy_per_served_load"] is not None
        assert entry["energy_per_served_load"] > 0
        assert entry["assigned_tasks"] + entry["dropped_tasks"] >
0


def test_ranking_excludes_algorithms_below_the_service_floor():
    stats = {
        "honest": {"service_rate": 0.99, "
energy_per_served_load": 5.0,
    "max_temp": 40.0},
        "slacker": {"service_rate": 0.40, "
energy_per_served_load": 3.0,
     "max_temp": 35.0},
    }
    ranking = rank_algorithms(stats)
    assert ranking["best"] == "honest"
    assert [item["algorithm"] for item in ranking["excluded"]] ==
["slac
    ker"]
    assert "服务率" in ranking["excluded"][0]["reason"]


def test_ranking_orders_by_energy_intensity():
    stats = {
        "a": {"service_rate": 1.0, "energy_per_served_load": 7.0, "
max_t
    emp": 40.0},
        "b": {"service_rate": 1.0, "energy_per_served_load": 5.0, "
max_t
    emp": 42.0},
    }
    ranking = rank_algorithms(stats)
    assert ranking["best"] == "b"
    assert [item["algorithm"] for item in ranking["eligible"]] ==
["b",
    "a"]


def test_ranking_reports_no_winner_when_nothing_qualifies():
    stats = {"a": {"service_rate": 0.5, "energy_per_served_load":
5.0, "
    max_temp": 40.0}}
```

```
    ranking = rank_algorithms(stats)
    assert ranking["best"] is None
    assert "不给出能耗排名" in ranking["note"]


def test_cooling_cost_rises_with_the_hot_aisle():
    sim = _simulator(servers=10, hours=4)
    for server in sim.servers:
        server.current_load = server.max_load
    cool = sim.energy_model.calculate_cooling_energy(sim.servers, 25.0,
    22.0, peak_temp=30.0)
    hot = sim.energy_model.calculate_cooling_energy(sim.servers, 25.0, 2
    2.0, peak_temp=50.0)
    assert hot > cool


def test_algorithms_differ_in_energy_intensity():
    """Scheduling choices must be able to move the energy figure."""
    sim = _simulator(servers=24, hours=12)
    df = sim.run_comparison(algorithms=["greedy", "round_robin", "damo_e
    ts"])
    stats = sim.get_statistics(df)
    intensities = [
        entry["energy_per_served_load"] for entry in stats.values()
        if entry["energy_per_served_load"] is not None
    ]
    assert len(intensities) >= 2
    assert len(set(intensities)) > 1, intensities


def test_safe_number_maps_non_numbers_to_none():
    assert safe_number(float("nan")) is None
    assert safe_number(None) is None
    assert safe_number("abc") is None
    assert safe_number(2.5) == 2.5


def test_summarize_tolerates_missing_columns():
    df = pd.DataFrame([
```

```
165|        {"time": 0, "algorithm": "x", "total_energy": 100.0, "
total_load
   |    ": 10.0},
166|        {"time": 1, "algorithm": "x", "total_energy": 120.0, "
total_load
   |    ": 12.0},
167|    ])
168|    stats = summarize(df)
169|    assert stats["x"]["avg_energy"] == 110.0
170|    assert stats["x"]["service_rate"] == 1.0
171|    assert stats["x"]["deadline_violations"] == 0
172|
173|
174|def test_pue_responds_to_ambient_temperature():
175|    sim = _simulator(servers=10, hours=6)
176|    warm = sim.energy_model.get_pue(sim.servers, 30.0, 22.0)
177|    cool = sim.energy_model.get_pue(sim.servers, 10.0, 22.0)
178|    assert warm is not None and cool is not None
179|    assert warm > cool
180|
181|
182|def test_pue_is_none_without_it_load():
183|    sim = _simulator(servers=0, hours=2)
184|    assert sim.energy_model.get_pue([], 25.0, 22.0) is None
185|
186|
187|def test_config_missing_key_raises_attribute_error():
188|    from src.utils.config import Config
189|    config = Config()
190|    try:
191|        config.definitely_not_a_key
192|    except AttributeError:
193|        return
194|    raise AssertionError("Config 未知键应当抛 AttributeError, 否则 hasat
   |    tr 永远为真")
195|
196|
197|def test_service_floor_constant_is_documented():
198|    assert 0.0 < SERVICE_RATE_FLOOR <= 1.0
```

96. tests/test_ai_chat.py

```
  1|"""Tests for the floating assistant's chat backend."""
  2|
  3|import os
```

```
import sys

PROJECT_ROOT = os.path.abspath(os.path.join(os.path.dirname(__file__), "
    ..."))
if PROJECT_ROOT not in sys.path:
    sys.path.insert(0, PROJECT_ROOT)

import pandas as pd

from src.services.ai_service import rule_chat
from src.services.simulation_insights import build_insights
from src.simulation.metrics import rank_algorithms, summarize


def _context():
    stats = {
        "damo_ets": {
            "service_rate": 0.9912, "energy_per_served_load": 5.2333,
            "max_temp": 54.4, "p95_temp": 48.0, "avg_pue": 1.4638,
            "deadline_violations": 3,
        },
        "greedy": {
            "service_rate": 1.0, "energy_per_served_load": 5.604,
            "max_temp": 56.6, "p95_temp": 50.0, "avg_pue": 1.4509,
            "deadline_violations": 0,
        },
    }
    return {
        "stats": stats,
        "ranking": rank_algorithms(stats),
       "data_source": {"site_id": "北京市", "emission_region": "华北"},
    }


def _insight_context():
    rows = []
    for algorithm, values in {
        "greedy": [(10.0, 50.0, 1.50), (7.0, 60.0, 1.45), (5.0, 70.0, 1.
    40)],
```

```
        "damo_ets": [(8.0, 50.0, 1.42), (6.0, 60.0, 1.38), (5.5, 65.0, 1
    .35)],
    }.items():
        for hour, (intensity, temperature, pue) in enumerate(values):
            rows.append({
                "time": hour,
                "algorithm": algorithm,
                "total_load": 10.0,
                "served_load": 10.0,
                "total_energy": intensity * 10.0,
                "pue": pue,
                "peak_temperature": temperature,
                "avg_temperature": temperature - 2,
                "ambient_temperature": 25.0,
                "fan_speed": 0.5,
                "cooling_power": 100.0,
                "assigned": 10,
                "dropped": 0,
                "overheated_servers": 0,
            })
    df = pd.DataFrame(rows)
    meta = {
        "site_id": "测试市",
        "site_province": "测试省",
        "emission_region": "华北",
         "num_servers": 10,
         "simulation_hours": 3,
         "controlled": True,
    }
    insights = build_insights(df, meta, generated_at="2026-10-18 10:00:0
    0")
    stats = summarize(df)
    return {
        "stats": stats,
        "ranking": rank_algorithms(stats),
        "insights": insights,
        "data_source": meta,
    }


def test_insights_pick_distinct_best_hours():
```

```
    insights = _insight_context()["insights"]
    first = insights["overall_best_hours"][0]
    assert first["hour"] == 2
    assert first["algorithm"] == "greedy"
    assert first["energy_per_load"] == 5.0
    assert insights["safe_best_hours"][0]["hour"] == 2


def test_insights_build_temperature_thresholds():
    insights = _insight_context()["insights"]
    threshold = next(item for item in
insights["temperature_thresholds"]
     if item["threshold_c"] == 55)
    assert threshold["hours_below"] == 1
    assert threshold["best_algorithm"] == "damo_ets"


def test_rule_chat_answers_best_time():
    reply = rule_chat("什么时候性价比最高？", _insight_context())
    assert "第 3 小时" in reply
    assert "贪心算法" in reply


def test_rule_chat_answers_temperature_configuration():
    reply = rule_chat("温度低于55度怎么配置？", _insight_context())
    assert "低于 55℃" in reply
    assert "DAMO-ETS" in reply
    assert "只作参考" not in reply


def test_rule_chat_reports_latest_snapshot():
    reply = rule_chat("当前数据更新了吗？", _insight_context())
    assert "测试市" in reply
    assert "2026-10-18 10:00:00" in reply


def test_rule_chat_handles_free_form_configuration():
    reply = rule_chat("如果把服务器增加到24台会怎样？", _insight_context(
    ))
    assert "方向性判断" in reply
    assert "服务器数量" in reply
    assert "重新运行仿真" in reply


```

```
def test_suggest_run_keeps_current_defaults():
    from src.api.ai_api import _suggest_run
    context = _insight_context()
    context["data_snapshot"] = context["insights"]["snapshot"]
    context["config"] = {"num_server_range": [1, 1000], "
fan_speed_range
    ": [0.2, 1.0]}
    run = _suggest_run("如果服务器增加到24台会怎样？", context)
    assert run is not None
    assert run["params"]["num_servers"] == 24
    assert run["params"]["simulation_hours"] == 3
    assert run["params"]["algorithms"] == ["greedy", "damo_ets"]


def test_suggest_run_for_algorithm_change():
    from src.api.ai_api import _suggest_run
    context = _insight_context()
    context["data_snapshot"] = context["insights"]["snapshot"]
    context["config"] = {"num_server_range": [1, 1000], "
fan_speed_range
    ": [0.2, 1.0]}
    run = _suggest_run("如果换成NSGA-II和MOEA-D一起跑呢？", context)
    assert run is not None
    assert run["params"]["algorithms"] == ["nsga2", "moead"]
    assert run["params"]["num_servers"] == 10


def test_auto_run_budget_blocks_large_requests(monkeypatch):
    from src.api import ai_api

    class IdleManager:
        def active_run(self):
            return None

    monkeypatch.setattr(ai_api, "get_run_manager", lambda:
IdleManager()
    )
    budget = ai_api._auto_run_budget({"num_servers": 500, "
simulation_ho
    urs": 24})
    assert budget["allowed"] is False
    assert "100" in budget["reason"]


```

```
def test_auto_run_budget_allows_small_requests(monkeypatch):
    from src.api import ai_api

    class IdleManager:
        def active_run(self):
            return None

    monkeypatch.setattr(ai_api, "get_run_manager", lambda:
IdleManager()
    )
    budget = ai_api._auto_run_budget({"num_servers": 24, "
simulation_hou
    rs": 12})
    assert budget["allowed"] is True


def test_suggest_run_resolves_school_to_city():
    from src.api.ai_api import _suggest_run
    context = _insight_context()
    context["data_snapshot"] = dict(context["insights"]["snapshot"]
)
    context["data_snapshot"]["site"] = "上海市"
    context["data_snapshot"]["school_name"] = None
    context["config"] = {"num_server_range": [1, 1000], "
fan_speed_range
    ": [0.2, 1.0]}
    run = _suggest_run("用北京大学试算，服务器增加到30台会怎样？",
context
    )
    assert run is not None
    assert run["params"]["school_name"] == "北京大学"
    assert run["params"]["site_id"] == "北京市"
    assert run["params"]["num_servers"] == 30


def test_suggest_run_allows_school_only_change():
    from src.api.ai_api import _suggest_run
    context = _insight_context()
    context["data_snapshot"] = dict(context["insights"]["snapshot"]
)
    context["data_snapshot"]["site"] = "上海市"
    context["data_snapshot"]["school_name"] = None
    context["config"] = {"num_server_range": [1, 1000], "
fan_speed_range
```

```
    |     ": [0.2, 1.0]}
191|     run = _suggest_run("用北京大学跑一下", context)
192|     assert run is not None
193|     assert run["params"]["school_name"] == "北京大学"
194|     assert run["params"]["site_id"] == "北京市"
195|
196|
197|def test_suggest_run_skips_identical_latest_config():
198|     from src.api.ai_api import _suggest_run
199|     context = _insight_context()
200|     context["data_snapshot"] = dict(context["insights"]["snapshot"]
)
201|     context["data_snapshot"].update({
202|         "site": "北京市",
203|         "school_name": "北京大学",
204|         "num_servers": 30,
205|         "simulation_hours": 24,
206|         "algorithms": ["greedy", "damo_ets"],
207|     })
208|     context["config"] = {"num_server_range": [1, 1000], "
fan_speed_range
    |     ": [0.2, 1.0]}
209|     run = _suggest_run("用北京大学试算，服务器增加到30台会怎样？",
context
    |     )
210|     assert run is None
211|
212|def test_rule_chat_names_the_best_algorithm():
213|     reply = rule_chat("哪个算法最省电？", _context())
214|     assert "DAMO-ETS" in reply
215|     assert "5.2333" in reply
216|
217|
218|def test_rule_chat_explains_data_source():
219|     reply = rule_chat("这些数据是哪来的？", _context())
220|     assert "ERA5" in reply or "阿里云" in reply
221|     assert "生态环境部" in reply
222|
223|
224|def test_rule_chat_without_results_asks_to_run():
225|     reply = rule_chat("排名", {"stats": {}, "ranking": {}, "data_source"
    |     : {}})
226|     assert "开始运行" in reply
227|
```

```
228|
229|def test_ranking_reason_distinguishes_missing_intensity():
230|    ranking = rank_algorithms({
231|        "x": {"service_rate": 1.0, "energy_per_served_load":
None, "max_
   |    temp": 50.0}
232|    })
233|    assert ranking["excluded"][0]["reason"] == "缺少单位服务负载能耗，无
   |    法参与排名"
234|
235|
236|def test_chat_endpoint_falls_back_when_model_unavailable(monkeypat
ch):
237|    import src.api.app  # noqa: F401  (registers the blueprint)
238|    from src.api import ai_api
239|    from src.api.app import app
240|
241|    class Broken:
242|        model = "none"
243|        key_source = "none"
244|
245|        def reload_key(self):
246|            pass
247|
248|        def chat_messages(self, *args, **kwargs):
249|            raise RuntimeError("no network")
250|
251|    monkeypatch.setattr(ai_api, "get_ai_service", lambda:
Broken())
252|    client = app.test_client()
253|    response = client.post(
254|        "/api/v1/ai/chat",
255|        json={"messages": [{"role": "user", "content": "当前排名如何？"}]
   |    },
256|    )
257|    assert response.status_code == 200
258|    payload = response.get_json()
259|    assert payload["success"] is True
260|    assert payload["source"] == "rule"
261|    assert payload["reply"]
```

97. tests/test_plan_optimizer.py

```
  1|import time
  2|
```

```
from src.services.plan_optimizer import PlanOptimizer,
_server_counts


def test_server_counts_include_bounds():
    assert _server_counts(10, 10) == [10]
    counts = _server_counts(10, 30)
    assert counts[0] == 10
    assert counts[-1] == 30
    assert len(counts) == len(set(counts))


def test_small_global_optimization_runs():
    optimizer = PlanOptimizer()
    job = optimizer.start({
        "server_min": 12,
        "server_max": 16,
        "simulation_hours": 2,
        "algorithms": ["greedy", "round_robin"],
        "target": "pue",
        "service_rate_floor": 0.98,
        "site_id": "北京市",
        "include_weights": False,
    })
    deadline = time.time() + 60
    current = optimizer.snapshot(job["job_id"])
    while current["status"] not in ("success", "failed") and
time.time()
    |     < deadline:
        time.sleep(0.2)
        current = optimizer.snapshot(job["job_id"])
    assert current["status"] == "success", current.get("error")
    result = current["result"]
    assert result["top"]
    assert len(result["top"]) <= 3
    assert result["best"]["metrics"]["service_rate"] >= 0.98
    assert all(plan["metrics"].get("avg_pue") is not None for
plan in re
    |    sult["top"])
```

98. tests/test_run_store.py

```
"""Tests for the persistent run registry."""

import pandas as pd
```

```
  4|
  5|from src.services import run_store
  6|
  7|
  8|def _isolate(tmp_path, monkeypatch):
  9|    monkeypatch.setattr(run_store, "DATA_DIR", str(tmp_path))
 10|    monkeypatch.setattr(run_store, "RUNS_DIR", str(tmp_path / "
runs"))
 11|    monkeypatch.setattr(run_store, "DB_PATH", str(tmp_path / "
runs.db"))
 12|    monkeypatch.setattr(run_store, "LEGACY_RESULTS", str(tmp_path
/ "sim
   |    ulation_results.csv"))
 13|    monkeypatch.setattr(run_store, "LEGACY_METADATA",
str(tmp_path / "ru
   |    n_metadata.json"))
 14|    run_store._init()
 15|
 16|
 17|def test_run_store_roundtrip(tmp_path, monkeypatch):
 18|    _isolate(tmp_path, monkeypatch)
 19|    run_id = run_store.create_run({
 20|        "num_servers": 6,
 21|        "simulation_hours": 2,
 22|        "algorithms": ["greedy"],
 23|       ″site_id″: ″北京市″,
 24|    }, origin="test")
 25|    df = pd.DataFrame([{
 26|        "time": 0,
 27|        "algorithm": "greedy",
 28|        "total_load": 10.0,
 29|        "served_load": 10.0,
 30|        "total_energy": 50.0,
 31|        "pue": 1.4,
 32|        "peak_temperature": 55.0,
 33|        "assigned": 10,
 34|        "dropped": 0,
 35|    }])
 36|    summary = {
 37|        "greedy": {
 38|            "service_rate": 1.0,
 39|            "energy_per_served_load": 5.0,
 40|            "avg_pue": 1.4,
 41|            "max_temp": 55.0,
```

```
 42|        }
 43|    }
 44|    run_store.save_results(run_id, df, {"run_id": run_id},
summary)
 45|    run_store.update_run(run_id, status="success", progress=100)
 46|    record = run_store.get_run(run_id)
 47|    assert record["status"] == "success"
 48|    assert record["summary"]["greedy"]["energy_per_served_load"]
== 5.0
 49|    assert run_store.list_runs(1)[0]["run_id"] == run_id
 50|    assert run_store.load_results(run_id).shape[0] == 1
 51|
 52|
 53|def test_compare_runs_uses_right_minus_left(tmp_path,
monkeypatch):
 54|    _isolate(tmp_path, monkeypatch)
 55|    left = run_store.create_run({"num_servers": 6, "
simulation_hours": 2
   |    , "algorithms": ["greedy"]})
 56|    right = run_store.create_run({"num_servers": 24, "
simulation_hours":
   |     2, "algorithms": ["greedy"]})
 57|    run_store.update_run(left, summary_json={
 58|        "greedy": {"service_rate": 0.5, "energy_per_served_load":
4.0, "
   |    avg_pue": 1.4, "max_temp": 60.0}
 59|    })
 60|    run_store.update_run(right, summary_json={
 61|        "greedy": {"service_rate": 0.8, "energy_per_served_load":
5.0, "
   |    avg_pue": 1.5, "max_temp": 55.0}
 62|    })
 63|    comparison = run_store.compare_runs(left, right)
 64|    item = comparison["metrics"][0]
 65|    assert item["energy_per_served_load"]["delta"] == 1.0
 66|    assert item["energy_per_served_load"]["percent"] == 25.0
 67|    assert item["service_rate"]["delta"] == 0.3
```